Kiến trúc sạch

Cẩm nang thực hành về cấu trúc
và thiết kế phần mềm

Robert C. Martin

Kiến trúc phần mềm tốt giúp giảm nỗ lực con người,
tăng năng suất và duy trì khả năng thay đổi trong thời gian dài.

Bản dịch tiếng Việt

Dùng cho mục đích học tập cá nhân

Nhiều tên gọi mà nhà sản xuất và người bán dùng để phân biệt sản phẩm của họ được xác nhận là nhãn hiệu. Khi những tên gọi đó xuất hiện trong sách này và nhà xuất bản biết có yêu cầu bảo hộ nhãn hiệu, chúng được in hoa chữ cái đầu hoặc viết hoa toàn bộ.

Các tác giả và nhà xuất bản đã cẩn trọng khi biên soạn cuốn sách này, nhưng không đưa ra bất kỳ bảo đảm nào, dù rõ ràng hay ngụ ý, và không chịu trách nhiệm về sai sót hoặc thiếu sót. Nhà xuất bản cũng không chịu trách nhiệm về thiệt hại ngẫu nhiên hay thiệt hại phát sinh từ việc sử dụng thông tin hoặc chương trình trong sách này.

Nhà xuất bản có chính sách giảm giá tốt khi đặt mua số lượng lớn hoặc đặt mua cho các đợt bán đặc biệt. Đơn hàng có thể bao gồm bản điện tử và/hoặc bìa cùng nội dung tùy chỉnh theo doanh nghiệp, mục tiêu đào tạo, định hướng tiếp thị và nhận diện thương hiệu của bạn. Để biết thêm thông tin, vui lòng liên hệ:

Bộ phận bán hàng doanh nghiệp và chính phủ Hoa Kỳ
(800) 382-3419
corpsales@pearsontechgroup.com

Khách hàng ngoài Hoa Kỳ vui lòng liên hệ bộ phận bán hàng quốc tế:

International Sales
international@pearsoned.com

Sách có danh mục tài liệu tham khảo và mục lục.
ISBN-13: 978-0-13-449416-6
ISBN-10: 0-13-449416-4

Bản quyền © 2018 Pearson Education, Inc.

Bảo lưu mọi quyền. In tại Hoa Kỳ. Ấn phẩm này được bảo hộ bản quyền. Trước khi sao chép, lưu trữ trong hệ thống truy xuất hoặc truyền tải dưới bất kỳ hình thức hay phương tiện nào, như điện tử, cơ học, photocopy, ghi âm hoặc cách thức tương tự, ngoài phạm vi được phép, phải được nhà xuất bản chấp thuận. Mọi yêu cầu cấp phép xin gửi đến:

Pearson Education, Inc
Rights and Contracts Department
501 Boylston Street, Suite 900
Boston, MA 02116
Fax: (617) 671-3447

ISBN-13: 978-0-13-449416-6
ISBN-10: 0-13-449416-4

In trên giấy tái chế tại Courier, Stoughton, Massachusetts, Hoa Kỳ.

Ấn bản đầu tiên, tháng 9 năm 2017

Bản quyền bản gốc thuộc Pearson Education. Bản dịch này chỉ dùng cho mục đích học tập cá nhân.

Mục lục

Lời tựa xii

Lời nói đầu ..... **xv**
Lời cảm ơn ..... **xvii**
Về tác giả ..... **xvii**
Giới thiệu ..... **xviii**

Chương 1: Thiết kế và kiến trúc là gì? **1**
Thiết kế và kiến trúc **2**
Mục tiêu? **2**
Nghiên cứu tình huống **3**
Dấu hiệu của một mớ hỗn độn 6
Góc nhìn của lãnh đạo 7
Chuyện gì đã xảy ra? 7
Kết luận **10**

Chương 2: Câu chuyện về hai giá trị **11**
Hành vi **12**
Kiến trúc **12**
Giá trị lớn hơn **13**
Ma trận Eisenhower **13**
Chiến đấu cho kiến trúc **14**
Phần II: Bắt đầu từ những viên gạch: Các mô hình lập trình 16

Chương 3: Tổng quan về các mô hình lập trình **17**
Lập trình có cấu trúc **18**
Lập trình hướng đối tượng **18**
Lập trình hàm **18**
Điều đáng suy ngẫm **19**
Kết luận **19**

Chương 4: Lập trình có cấu trúc **20**

Chứng minh **21**
Một tuyên ngôn có hại **22**
Phân tách chức năng **23**
Không có chứng minh hình thức **23**
Khoa học giải nguy **23**
Kiểm thử **24**
Kết luận **24**

Chương 5: Lập trình hướng đối tượng **25**
Đóng gói (Encapsulation)? **26**
Kế thừa (Inheritance)? **28**
Đa hình (Polymorphism)? **29**
Sức mạnh của đa hình 31
Đảo ngược phụ thuộc (Dependency Inversion) 32
Kết luận **34**

Chương 6: Lập trình hàm **35**
Bình phương các số nguyên (SQUARES OF INTEGERS) **36**
Bất biến và kiến trúc (IMMUTABILITY AND ARCHITECTURE) **37**
Tách biệt trạng thái có thể thay đổi (SEGREGATION OF MUTABILITY) **38**
Nguồn sự kiện (EVENT SOURCING) **39**
Kết luận (CONCLUSION) **40**
Phần III: Các nguyên tắc thiết kế 41

Chương 7: SRP: Nguyên tắc trách nhiệm đơn nhất **44**
Triệu chứng 1: Trùng lặp ngẫu nhiên (Accidental Duplication) **46**
Triệu chứng 2: Hợp nhất (Merges) **47**
Giải pháp (Solutions) **48**
Kết luận **50**

Chương 8: OCP: Nguyên tắc đóng-mở **51**
Một thử nghiệm tư duy **53**
Kiểm soát chiều **54**
Che giấu thông tin **55**
Kết luận **55**

Chương 9: LSP: Nguyên tắc thay thế Liskov **56**
Dẫn dắt việc sử dụng kế thừa **57**
Vấn đề vuông/hình chữ nhật **57**
LSP và kiến trúc **58**
Ví dụ vi phạm LSP **59**
Kết luận **60**

Chương 10: ISP: Nguyên tắc phân tách giao diện 61
ISP và ngôn ngữ 62
ISP và kiến trúc 63
Kết luận 63
Chương 11: DIP: Nguyên tắc đảo ngược phụ thuộc 64
Trừu tượng ổn định 65
Nhà máy (Factories) 66
Thành phần cụ thể (Concrete Components) 67
Kết luận 67
Phần IV: Nguyên tắc thành phần 68
Chương 12: Các thành phần 69
Lịch sử ngắn về các thành phần 70
Khả năng tái định vị 72
Các trình liên kết 73
Kết luận 74
Chương 13: Tính gắn kết của thành phần 75
Nguyên tắc tương đương giữa tái sử dụng và phát hành 76
Nguyên tắc Đóng chung 77
Sự tương tự với SRP 77
Nguyên tắc Tái sử dụng chung 78
Mối quan hệ với ISP 78
Biểu đồ căng thẳng về tính gắn kết của thành phần 79
Kết luận 80
Chương 14: Sự kết nối giữa các thành phần 81
Nguyên tắc phụ thuộc không chu trình (ADP) 82
Build hàng tuần 82
Loại bỏ các chu trình phụ thuộc 83
Tác động của chu trình trong biểu đồ phụ thuộc thành phần 85
Phá vỡ chu trình 85
Sự dao động (The Jitters) 87
Thiết kế từ trên xuống (Top-Down Design) 87
Nguyên tắc phụ thuộc ổn định (SDP) 88
Ổn định (Stability) 88
Các chỉ số Ổn định 89
Không phải thành phần nào cũng nên ổn định 91
Các thành phần trừu tượng (Abstract Components) 93
Nguyên tắc trừu tượng ổn định (SAP) 94
Chúng ta đặt chính sách cấp cao ở đâu? 94

Giới thiệu Nguyên tắc Trừu tượng Ổn định 94
Đo lường mức độ trừu tượng 95
Dãy chính (Main Sequence) 95
Tránh các vùng loại trừ 97
Khoảng cách từ Dãy chính 99
Kết luận 100
Phần V: Kiến trúc 101

Chương 15: Kiến trúc là gì? 102
Phát triển 104
Triển khai 104
Vận hành 105
Bảo trì 105
Giữ các lựa chọn mở 106
Độc lập thiết bị 107
Thư quảng cáo hàng loạt (Junk Mail) 108
Địa chỉ vật lý (Physical Addressing) 109
Kết luận 110
Chương 16: Tính độc lập 111
Trường hợp sử dụng (Use Cases) 112
Vận hành (Operation) 112
Phát triển (Development) 113
Triển khai (Deployment) 113
Giữ các lựa chọn mở (Leaving Options Open) 113
Tách biệt các lớp (Decoupling Layers) 114
Tách biệt các trường hợp sử dụng (Decoupling Use Cases) 114
Chế độ tách biệt (Decoupling Mode) 115
Khả năng phát triển độc lập (Independent Develop-ability) 115
Khả năng triển khai độc lập (Independent Deployability) 116
Trùng lặp (Duplication) 116
Các chế độ tách biệt (lại) (Decoupling Modes (Again)) 117
Kết luận (Conclusion) 118
Chương 17: Ranh giới: Vạch đường phân cách 119
Hai câu chuyện buồn 120
FitNesse 122
Vạch những đường nào, và vạch khi nào? 123
Còn đầu vào và đầu ra thì sao? 127
Kiến trúc plugin 128
Lý lẽ về plugin 129

Kết luận 131
Chương 18: Giải phẫu ranh giới 132
Vượt qua ranh giới 133
Khối nguyên khối đáng sợ 133
Các thành phần triển khai 135
Luồng 135
Tiến trình cục bộ 135
Dịch vụ 136
Kết luận 136
Chương 19: Chính sách và cấp độ 137
Cấp độ 138
Kết luận 141
Chương 20: Quy tắc nghiệp vụ 142
Entity 143
Ca sử dụng 144
Mô hình yêu cầu và phản hồi 146
Kết luận 146
Chương 21: Kiến trúc lên tiếng 147
Chủ đề của kiến trúc 149
Mục đích của kiến trúc 149
Thế còn web thì sao? 150
Framework là công cụ, không phải lối sống 150
Kiến trúc có thể kiểm thử 150
Kết luận 151
Chương 22: Kiến trúc sạch 152
Quy tắc phụ thuộc 154
Entity 154
Ca sử dụng 155
Bộ chuyển đổi giao diện 155
Framework và trình điều khiển 155
Chỉ có bốn vòng tròn? 156
Vượt qua ranh giới 156
Dữ liệu nào đi qua ranh giới? 157
Kịch bản điển hình 157
Kết luận 158
Chương 23: Presenter và đối tượng khiêm tốn 160
Mẫu Đối tượng Khiêm tốn 161

Presenter và View **161**
Kiểm thử và kiến trúc **162**
Cổng cơ sở dữ liệu **162**
Bộ ánh xạ dữ liệu **162**
Bộ lắng nghe dịch vụ **163**
Kết luận **163**

Chương 24: Ranh giới một phần 164

Bỏ qua bước cuối **166**
Ranh giới một chiều **166**
Mẫu Facade **167**
Kết luận **168**

Chương 25: Các lớp và ranh giới 169

Săn Wumpus **170**
Kiến trúc sạch? **171**
Giao cắt các luồng **174**
Tách các luồng **174**
Kết luận **176**

Chương 26: Thành phần Main 178

Chi tiết tối hậu **180**
Kết luận **183**

Chương 27: Dịch vụ: lớn và nhỏ 184

Kiến trúc dịch vụ? **185**
Lợi ích của dịch vụ? **185**
Ngụy biện về tính tách rời 185
Ngụy biện về phát triển và triển khai độc lập 186
Vấn đề mèo con 186
Đối tượng giải cứu tình hình **187**
Dịch vụ dựa trên thành phần **190**
Mối quan tâm cắt ngang **191**
Kết luận **191**

Chương 28: Ranh giới kiểm thử 192

Kiểm thử là thành phần của hệ thống **193**
Thiết kế để dễ kiểm thử **193**
API kiểm thử **194**
Phụ thuộc cấu trúc **194**
Bảo mật **195**
Kết luận **195**

Chương 29: Kiến trúc nhúng sạch 196
Bài kiểm tra App-titude 198
Nút thắt phần cứng đích 200
Kiến trúc nhúng sạch có thể kiểm thử 200
Các lớp 201
Phần cứng là chi tiết 203
Bộ xử lý là chi tiết 205
Hệ điều hành là chi tiết 207
Lập trình theo giao diện và khả năng thay thế 210
Chỉ thị biên dịch có điều kiện DRY 211
Kết luận 211
Phần VI: Chi tiết 212

Chương 30: Cơ sở dữ liệu chỉ là chi tiết 213
Cơ sở dữ liệu quan hệ 214
Vì sao hệ thống cơ sở dữ liệu phổ biến đến vậy? 214
Nếu không có đĩa thì sao? 215
Chi tiết 215
Còn hiệu năng thì sao? 216
Câu chuyện 216
Kết luận 217
Chương 31: Web chỉ là chi tiết 218
Con lắc vô tận 219
Kết luận 220
Chương 32: Framework chỉ là chi tiết 222
Tác giả framework 223
Mối ràng buộc bất cân xứng 223
Những rủi ro 224
Giải pháp 224
Bây giờ tôi tuyên bố hai bạn thành vợ chồng... 225
Kết luận 225
Chương 33: Nghiên cứu tình huống: Bán video 226
Sản phẩm 227
Phân tích ca sử dụng 227
Kiến trúc thành phần 229
Quản lý phụ thuộc 230
Kết luận 231
Chương 34: Chương còn thiếu 232

Chia gói theo lớp **234**
Chia gói theo tính năng **236**
Cổng và bộ chuyển đổi **238**
Chia gói theo thành phần **241**
Ma quỷ ẩn trong chi tiết triển khai **246**
Tổ chức và đóng gói **246**
Các chế độ tách rời khác **249**
Kết luận: lời khuyên còn thiếu **250**
Phần VII: Phụ lục 252

Phụ lục A: Khảo cổ kiến trúc **253**
Hệ thống kế toán công đoàn **255**
Trại trẻ mồ côi máy tính mini 260
Cắt laser **262**
Giám sát đúc nhôm **264**
4-TEL **265**
Máy tính khu vực dịch vụ **268**
Xác định nơi điều phối 268
Kiến trúc 269
Cuộc đại thiết kế lại trong mơ 270
Châu Âu 270
Kết luận về SAC **271**
Ngôn ngữ C **271**
C 272
BOSS 272
pCCU 272
DLU/DRU 274
Kiến trúc 275
VRS **275**
Đặt tên 276
Kiến trúc 276
Kết luận về VRS **277**
Lễ tân điện tử **277**
Sự sụp đổ của ER **278**
Hệ thống điều phối thợ **278**
Clear Communications **280**
Chuẩn bị **281**
Uncle Bob **281**
Cuộc điện thoại **282**
ROSE **282**

Các cuộc tranh luận tiếp diễn **283**
… Dù mang tên nào khác **283**
Kỳ thi đăng ký kiến trúc sư **284**
Kết luận **286**

Lời tựa

Khi nói về kiến trúc, chúng ta đang nói đến điều gì?

Cũng như mọi phép ẩn dụ, nhìn phần mềm qua lăng kính kiến trúc có thể giúp ta thấy rõ một số điều, đồng thời che khuất những điều khác. Nó có thể hứa nhiều hơn khả năng thực hiện, nhưng cũng có thể làm được nhiều hơn những gì đã hứa.

Sức hấp dẫn dễ thấy của kiến trúc nằm ở cấu trúc. Cấu trúc chi phối các mô hình và cuộc thảo luận về phát triển phần mềm: thành phần, lớp, hàm, mô-đun, tầng, dịch vụ lớn nhỏ. Nhưng cấu trúc tổng thể của nhiều hệ thống phần mềm lại khiến người ta khó tin hoặc khó hiểu. Có những sơ đồ doanh nghiệp kiểu Xô Viết được định sẵn thành mã cũ; những tháp Jenga chông chênh vươn lên đám mây; những tầng khảo cổ bị chôn trong dòng trượt của một mớ bòng bong. Cấu trúc phần mềm không hiển nhiên tuân theo trực giác của chúng ta như cấu trúc tòa nhà.

Tòa nhà có cấu trúc vật lý dễ nhận thấy, dù đặt trên đá hay bê tông, vươn cao thành vòm hay trải rộng, dù lớn hay nhỏ, lộng lẫy hay bình thường. Cấu trúc của chúng buộc phải tuân theo trọng lực và đặc tính vật liệu. Phần mềm thì ít phải bận tâm đến trọng lực, trừ khi ta dùng từ ấy để nói về mức độ nghiêm trọng. Vậy phần mềm được làm từ gì? Khác với tòa nhà làm bằng gạch, bê tông, gỗ, thép và kính, phần mềm được làm từ phần mềm. Những cấu trúc phần mềm lớn được ghép từ các thành phần nhỏ hơn, rồi những thành phần ấy lại được ghép từ các thành phần nhỏ hơn nữa. Cứ thế mãi, như những con rùa chồng lên nhau.

Khi nói về kiến trúc phần mềm, ta đang nói về một thứ có bản chất đệ quy và phân dạng, được khắc họa trong mã. Mọi thứ đều là chi tiết. Các cấp độ chi tiết đan xen cũng góp phần tạo nên kiến trúc tòa nhà, nhưng nói về quy mô vật lý trong phần mềm thì không có nhiều ý nghĩa. Phần mềm có cấu trúc, có nhiều cấu trúc và nhiều kiểu cấu trúc; sự đa dạng ấy vượt xa những gì ta thấy trong công trình xây dựng. Thậm chí, có thể lập luận khá thuyết phục rằng hoạt động thiết kế và sự tập trung vào thiết kế trong phần mềm còn nhiều hơn trong kiến trúc công trình. Theo nghĩa đó, gọi kiến trúc phần mềm là "kiến trúc" còn sát nghĩa hơn gọi kiến trúc tòa nhà như vậy!

Con người hiểu quy mô vật lý và thường tìm kiếm nó trong thế giới xung quanh. Dù sơ đồ PowerPoint trông hấp dẫn và dễ nhìn, những chiếc hộp trên sơ đồ không phải là kiến trúc của hệ thống phần mềm. Chúng chắc chắn thể hiện một góc nhìn về kiến trúc, nhưng nếu nhầm các hộp ấy với toàn cảnh, với chính kiến trúc, ta sẽ bỏ lỡ cả hai. Kiến trúc phần mềm không có một hình dạng cố định. Một cách trực quan hóa cụ thể là lựa chọn, không phải điều tất yếu. Lựa chọn ấy lại dựa trên nhiều lựa chọn khác: đưa gì vào, bỏ gì đi, nhấn

mạnh điều gì bằng hình dạng hoặc màu sắc, làm mờ điều gì bằng cách trình bày đồng đều hoặc lược bỏ. Không có góc nhìn nào tự nhiên hay vốn dĩ ưu việt hơn góc nhìn khác.

Dù nói về vật lý hay quy mô vật lý trong kiến trúc phần mềm không thật phù hợp, chúng ta vẫn quan tâm đến một số giới hạn vật lý. Tốc độ bộ xử lý và băng thông mạng có thể quyết định khắc nghiệt hiệu năng của hệ thống. Bộ nhớ và lưu trữ có thể giới hạn tham vọng của bất kỳ cơ sở mã nào. Phần mềm có thể được dệt từ những giấc mơ, nhưng nó vẫn vận hành trong thế giới vật chất.

> *"Đây là điều quái dị của tình yêu, thưa nàng: ý chí thì vô hạn, còn việc thực hiện lại bị giới hạn; khát vọng thì vô biên, nhưng hành động phải làm nô lệ cho giới hạn."*
>
> *(William Shakespeare)*

Thế giới vật chất là nơi chúng ta, công ty và nền kinh tế của mình tồn tại. Nhờ đó, ta có thêm một thước đo để hiểu kiến trúc phần mềm: những lực và đại lượng ít mang tính vật lý hơn, nhưng vẫn có thể dùng để trao đổi và suy luận.

> *"Kiến trúc thể hiện những quyết định thiết kế quan trọng định hình hệ thống; mức độ quan trọng được đo bằng chi phí thay đổi."*
>
> *(Grady Booch)*

Thời gian, tiền bạc và công sức giúp ta hình dung quy mô, phân biệt việc lớn với việc nhỏ, phần thuộc về kiến trúc với phần còn lại. Chúng cũng cho ta cách đánh giá kiến trúc có tốt hay không: kiến trúc tốt đáp ứng nhu cầu của người dùng, người phát triển và chủ sở hữu ở một thời điểm nhất định, đồng thời tiếp tục đáp ứng những nhu cầu ấy theo thời gian.

> *"Nếu bạn cho rằng kiến trúc tốt tốn kém, hãy thử dùng kiến trúc tồi."*
>
> *(Brian Foote và Joseph Yoder)*

Những thay đổi thường xảy ra trong quá trình phát triển hệ thống không nên trở thành các thay đổi tốn kém, khó thực hiện, phải lập dự án riêng để quản lý thay vì được đưa vào nhịp công việc hằng ngày và hằng tuần.

Từ đó nảy sinh một vấn đề liên quan đến vật lý, cũng chẳng nhỏ chút nào: du hành thời gian. Làm sao biết trước những thay đổi thường gặp để đưa ra các quyết định quan trọng dựa trên chúng? Làm sao giảm công sức và chi phí phát triển trong tương lai nếu không có quả cầu tiên tri hay cỗ máy thời gian?

> *"Kiến trúc gồm những quyết định mà ta ước mình có thể đưa ra đúng ngay từ đầu dự án, nhưng khả năng ta làm đúng chúng cũng chẳng cao hơn khả năng làm đúng bất kỳ quyết định nào khác."*
>
> *(Ralph Johnson)*

Hiểu quá khứ vốn đã khó; nắm bắt hiện tại thì luôn chông chênh; còn dự đoán tương lai chẳng phải chuyện đơn giản.

Đây là lúc con đường rẽ ra nhiều hướng.

Con đường tối tăm nhất dẫn đến quan niệm rằng kiến trúc vững chắc, ổn định phải dựa vào quyền uy và sự cứng nhắc. Nếu thay đổi tốn kém thì hãy loại bỏ thay đổi, dập tắt nguyên nhân hoặc đẩy chúng vào một mê cung quan liêu. Kiến trúc sư nắm quyền tuyệt đối, còn kiến trúc trở thành một xã hội phản địa đàng với các nhà phát triển và là nguồn bực bội triền miên cho tất cả mọi người.

Một con đường khác nồng nặc mùi khái quát hóa suy đoán. Lối đi ấy rải đầy phỏng đoán hard-code, vô số tham số, những ngôi mộ chứa mã chết và độ phức tạp ngẫu nhiên nhiều đến mức ngân sách bảo trì cũng không gánh nổi.

Con đường chúng tôi quan tâm nhất là con đường sạch nhất. Nó thừa nhận phần mềm vốn mềm dẻo và muốn giữ đặc tính ấy thành một thuộc tính hạng nhất của hệ thống. Nó chấp nhận rằng hiểu biết của chúng ta luôn không đầy đủ, đồng thời hiểu rằng con người vốn thường xuyên làm việc trong hoàn cảnh thiếu thông tin, và chúng ta làm việc ấy khá tốt. Con đường này phát huy sở trường thay vì khoét sâu điểm yếu. Chúng ta tạo ra sự vật rồi khám phá chúng. Chúng ta đặt câu hỏi và tiến hành thí nghiệm. Kiến trúc tốt hình thành khi ta xem việc xây dựng nó như một hành trình chứ không phải đích đến, như một quá trình tìm hiểu không ngừng chứ không phải một hiện vật bất động.

> *“Kiến trúc là một giả thuyết, cần được kiểm chứng bằng việc triển khai và đo lường.”*
>
> *(Tom Gilb)*

Bước đi trên con đường ấy đòi hỏi sự cẩn trọng và chú ý, suy nghĩ và quan sát, thực hành và nguyên tắc. Thoạt nghe có thể tưởng là chậm, nhưng mọi thứ phụ thuộc vào cách ta bước đi.

> *“Muốn đi nhanh, hãy làm cho tốt.”*
>
> *(Robert C. Martin)*

Chúc bạn tận hưởng hành trình.

(Kevlin Henney, tháng 5 năm 2017)

Lời nói đầu

Tên cuốn sách này là *Clean Architecture*. Đó là một cái tên đầy tham vọng. Có người còn cho rằng nó ngạo mạn. Vì sao tôi chọn tên ấy, và vì sao tôi viết cuốn sách này?

Tôi viết dòng mã đầu tiên năm 1964, khi mới 12 tuổi. Năm nay là 2016; như vậy tôi đã viết mã hơn nửa thế kỷ. Trong thời gian ấy, tôi học được một số điều về cách cấu trúc hệ thống phần mềm mà tôi nghĩ người khác cũng có thể thấy hữu ích.

Tôi học được những điều đó bằng cách xây dựng nhiều hệ thống, lớn có nhỏ có. Tôi từng làm hệ thống nhúng nhỏ và hệ thống xử lý theo lô lớn; hệ thống thời gian thực và hệ thống web; ứng dụng dòng lệnh, giao diện đồ họa, điều khiển quy trình, trò chơi, kế toán, viễn thông, công cụ thiết kế, ứng dụng vẽ và nhiều loại khác.

Tôi từng xây dựng ứng dụng đơn luồng, đa luồng, dùng một vài tiến trình nặng hoặc nhiều tiến trình nhẹ, đa bộ xử lý, cơ sở dữ liệu, toán học, hình học tính toán và nhiều loại khác nữa.

Tôi đã xây dựng rất nhiều ứng dụng, rất nhiều hệ thống. Nhìn lại tất cả chúng, tôi rút ra một điều khiến mình kinh ngạc:

Các quy tắc kiến trúc đều giống nhau!

Điều đó thật bất ngờ vì các hệ thống tôi từng xây dựng khác nhau đến mức cực đoan. Vì sao những hệ thống khác xa nhau như vậy lại tuân theo các quy tắc kiến trúc tương tự? Tôi kết luận rằng quy tắc kiến trúc phần mềm độc lập với mọi biến số khác.

Kết luận ấy càng đáng ngạc nhiên khi nhìn vào những thay đổi của phần cứng trong nửa thế kỷ đó. Tôi bắt đầu lập trình trên những cỗ máy lớn như tủ lạnh nhà bếp, có chu kỳ nửa megahertz, bộ nhớ lõi 4K, bộ nhớ đĩa 32K và giao diện teletype truyền được 10 ký tự mỗi giây. Còn tôi đang viết lời nói đầu này trên xe buýt trong chuyến đi qua Nam Phi. Tôi dùng MacBook có bốn lõi i7, mỗi lõi chạy ở tốc độ 2,8 gigahertz. Máy có RAM 16 gigabyte, SSD một terabyte và màn hình Retina 2880×1800 hiển thị được video độ phân giải rất cao. Chênh lệch năng lực tính toán thật choáng ngợp. Một phép phân tích hợp lý cho thấy chiếc MacBook này mạnh hơn ít nhất 10^{22} lần những máy tính đầu tiên tôi dùng cách đây nửa thế kỷ.

Hai mươi hai bậc độ lớn là một con số khổng lồ. Đó là số ångström từ Trái Đất đến Alpha-Centuri. Đó là số electron trong tiền lẻ trong túi hoặc ví của bạn. Và con số ấy, ít nhất là con số ấy, thể hiện mức tăng năng lực tính toán tôi đã trải qua trong đời mình.

Với mức tăng năng lực tính toán khổng lồ đó, phần mềm tôi viết thay đổi ra sao? Chắc chắn nó lớn hơn. Trước kia, tôi nghĩ 2.000 dòng mã là một chương trình lớn. Dù sao, đó là cả một hộp thẻ đục lỗ nặng 10 pound. Giờ đây, một chương trình chỉ thật sự lớn khi vượt quá 100.000 dòng.

Phần mềm cũng chạy hiệu quả hơn nhiều. Ngày nay, chúng ta làm được những việc mà thập niên 1960 khó lòng tưởng tượng. *The Forbin Project*, *The Moon Is a Harsh Mistress* và *2001: A Space Odyssey* đều từng hình dung tương lai hiện tại, nhưng đã dự đoán trật khá xa. Những tác phẩm ấy tưởng tượng ra các cỗ máy khổng lồ có ý thức. Thực tế, chúng ta có những cỗ máy nhỏ đến khó tin, nhưng vẫn chỉ là máy móc.

Còn một điều khác về phần mềm ngày nay so với phần mềm ngày trước: nó được tạo

nên từ cùng một thứ. Nó vẫn gồm các câu lệnh `if`, câu lệnh gán và vòng lặp `while`.

Bạn có thể phản đối rằng chúng ta đã có ngôn ngữ tốt hơn và mô hình lập trình ưu việt hơn. Dù sao thì ta cũng lập trình bằng Java, C# hoặc Ruby, dùng thiết kế hướng đối tượng. Đúng vậy, nhưng mã vẫn là sự kết hợp của trình tự, lựa chọn và lặp, đúng như hồi thập niên 1960 và 1950. Quan sát kỹ thực hành lập trình máy tính, ta sẽ nhận ra rất ít điều thay đổi trong 50 năm. Ngôn ngữ tốt hơn một chút. Công cụ tốt hơn rất nhiều. Nhưng các khối xây dựng cơ bản của chương trình máy tính vẫn nguyên như cũ.

Nếu đưa một lập trình viên từ năm 1966 đến năm 2016, đặt cô ấy[1] trước chiếc MacBook của tôi đang chạy IntelliJ và chỉ cho cô ấy Java, có lẽ cô ấy cần 24 giờ để hết choáng. Sau đó, cô ấy vẫn viết mã được. Java chẳng khác C nhiều, thậm chí cũng không khác Fortran bao nhiêu.

Nếu đưa bạn về năm 1966, chỉ cho bạn cách viết và sửa mã PDP-8 bằng cách đục băng giấy trên chiếc teletype 10 ký tự mỗi giây, có lẽ bạn cần 24 giờ để hết thất vọng. Sau đó, bạn vẫn viết mã được. Mã không thay đổi nhiều đến thế.

Bí mật nằm ở đây: mã không thay đổi là lý do khiến các quy tắc kiến trúc phần mềm nhất quán giữa nhiều loại hệ thống. Quy tắc kiến trúc phần mềm là quy tắc sắp xếp và lắp ghép các khối xây dựng chương trình. Vì những khối ấy phổ biến và không thay đổi, quy tắc sắp xếp chúng cũng phổ biến và bất biến.

Lập trình viên trẻ có thể cho rằng điều đó vô lý. Họ có thể khăng khăng rằng mọi thứ ngày nay đều mới và khác, rằng các quy tắc cũ đã lỗi thời. Nếu nghĩ vậy, họ đã nhầm nghiêm trọng. Các quy tắc vẫn vậy. Bất chấp ngôn ngữ, framework và mô hình mới, các quy tắc hiện nay vẫn giống thời Alan Turing viết mã máy đầu tiên vào năm 1946.

Có một điều đã thay đổi: trước kia, chúng ta chưa biết những quy tắc ấy là gì, nên cứ vi phạm chúng hết lần này đến lần khác. Sau nửa thế kỷ kinh nghiệm, giờ đây chúng ta đã hiểu được chúng.

Cuốn sách này nói về chính những quy tắc ấy, những quy tắc không đổi theo thời gian.

Đăng ký bản Clean Architecture của bạn trên trang InformIT để nhận các bản cập nhật và sửa lỗi khi chúng được phát hành. Để đăng ký, truy cập `informit.com/register` rồi đăng nhập hoặc tạo tài khoản. Nhập ISBN sản phẩm `9780134494166` và nhấn Submit. Mở thẻ Registered Products, tìm liên kết Access Bonus Content bên cạnh sản phẩm này, rồi truy cập tài liệu bổ sung theo liên kết đó.

---

1. Người lập trình năm 1966 rất có thể là nữ, vì thời ấy phụ nữ chiếm một phần lớn trong lực lượng lập trình viên.

Lời cảm ơn

Những người đã góp phần tạo nên cuốn sách này, không theo thứ tự cụ thể:

Chris Guzikowski, Chris Zahn, Matt Heuser, Jeff Overbey, Micah Martin, Justin Martin, Carl Hickman, James Grenning, Simon Brown, Kevlin Henney, Jason Gorman, Doug Bradbury, Colin Jones, Grady Booch, Kent Beck, Martin Fowler, Alistair Cockburn, James O. Coplien, Tim Conrad, Richard Lloyd, Ken Finder, Kris Iyer (CK), Mike Carew, Jerry Fitzpatrick, Jim Newkirk, Ed Thelen, Joe Mabel và Bill Degnan.

Còn rất nhiều người khác không thể kể hết.

Trong lần đọc soát cuối cuốn sách, khi đọc chương về Kiến trúc biết nói, nụ cười rạng rỡ và tiếng cười du dương của Jim Weirich vang lên trong tâm trí tôi. Chúc Jim lên đường bình an!

Về tác giả



Robert C. Martin (Uncle Bob) là lập trình viên từ năm 1970. Ông đồng sáng lập cleancoders.com, trang cung cấp khóa học video trực tuyến cho nhà phát triển phần mềm, và sáng lập Uncle Bob Consulting LLC, đơn vị cung cấp dịch vụ tư vấn phần mềm, đào tạo và phát triển kỹ năng cho các tập đoàn lớn trên toàn thế giới. Ông từng giữ chức Master Craftsman tại 8th Light, Inc., công ty tư vấn phần mềm ở Chicago. Ông đã đăng hàng chục bài viết trên các tạp chí chuyên ngành và thường xuyên diễn thuyết tại hội nghị, triển lãm quốc tế. Ông làm tổng biên tập *C++ Report* trong ba năm và là chủ tịch đầu tiên của Agile Alliance.

Martin là tác giả và biên tập viên của nhiều cuốn sách, trong đó có *The Clean Coder*, *Clean Code*, *UML for Java Programmers*, *Agile Software Development*, *Extreme Programming in Practice*, *More C++ Gems*, *Pattern Languages of Program Design 3* và *Designing Object Oriented C++ Applications Using the Booch Method*.

Giới thiệu

Không cần nhiều kiến thức hay kỹ năng để khiến một chương trình chạy được. Học sinh trung học vẫn làm điều đó mỗi ngày. Sinh viên đại học lập những công ty trị giá hàng tỷ đô-la bằng cách chắp vá vài dòng PHP hoặc Ruby. Ở các văn phòng làm việc kiểu ô bàn trên khắp thế giới, vô số lập trình viên mới vào nghề cày qua những tài liệu yêu cầu khổng lồ trong hệ thống theo dõi vấn đề cồng kềnh, cố ép hệ thống "hoạt động" bằng ý chí. Mã họ viết có thể chẳng đẹp, nhưng chạy được. Vì làm cho thứ gì đó chạy được một lần vốn không khó đến vậy.

Làm cho phần mềm đúng lại là chuyện khác. Điều đó khó. Nó cần kiến thức và kỹ năng mà phần lớn lập trình viên trẻ chưa tích lũy được. Nó cần suy nghĩ và sự thấu hiểu mà nhiều lập trình viên không dành thời gian rèn luyện. Nó đòi hỏi kỷ luật và sự tận tâm ở mức mà nhiều người chưa từng nghĩ mình sẽ cần. Trên hết, nó cần niềm đam mê nghề và mong muốn trở thành một người làm nghề chuyên nghiệp.

Khi làm phần mềm cho đúng, một điều kỳ diệu xảy ra: bạn không cần hàng đống lập trình viên để duy trì nó. Bạn không cần tài liệu yêu cầu khổng lồ hay hệ thống theo dõi vấn đề cồng kềnh. Bạn cũng không cần những văn phòng đầy người ở khắp nơi cùng lập trình suốt ngày đêm.

Phần mềm được làm tốt chỉ cần một phần nhỏ nguồn lực con người để xây dựng và bảo trì. Thay đổi đơn giản và nhanh chóng. Lỗi hiếm khi xuất hiện. Công sức được giảm thiểu, còn chức năng và khả năng thích ứng được tăng tối đa. Đúng là viễn cảnh này nghe hơi lý tưởng. Nhưng tôi từng thấy nó xảy ra. Tôi đã tham gia dự án có thiết kế và kiến trúc giúp việc viết mã, bảo trì trở nên dễ dàng. Tôi đã làm những dự án chỉ cần một phần nhỏ nhân lực dự kiến. Tôi từng làm hệ thống có tỷ lệ lỗi cực thấp. Tôi đã thấy kiến trúc phần mềm tốt có thể tạo ra khác biệt lớn cho hệ thống, dự án và nhóm. Tôi đã đến miền đất hứa.

Nhưng đừng chỉ tin lời tôi. Hãy nhìn vào trải nghiệm của chính bạn. Bạn đã từng gặp tình cảnh ngược lại chưa? Bạn có từng làm trên hệ thống liên kết và phụ thuộc chằng chịt đến mức mọi thay đổi, dù nhỏ đến đâu, cũng mất hàng tuần và đầy rủi ro? Bạn đã từng chịu cảnh mã tồi và thiết kế mục nát cản trở công việc chưa? Thiết kế của những hệ thống bạn làm có từng khiến tinh thần nhóm sa sút, khách hàng mất lòng tin, còn quản lý mất kiên nhẫn không? Bạn đã từng thấy nhóm, phòng ban, thậm chí cả công ty bị kéo sụp bởi cấu trúc phần mềm mục nát chưa? Bạn đã từng xuống địa ngục lập trình chưa?

Tôi đã từng. Và ở mức độ nào đó, phần lớn chúng ta cũng vậy. Vật lộn với thiết kế phần mềm tồi phổ biến hơn nhiều so với tận hưởng niềm vui làm việc trên một thiết kế tốt.

1

Thiết kế và kiến trúc là gì?



Thiết kế và kiến trúc

Trong nhiều năm, người ta đã nhầm lẫn về thiết kế và kiến trúc. Thiết kế là gì? Kiến trúc là gì? Hai khái niệm này khác nhau ra sao?

Một mục tiêu của cuốn sách này là gỡ bỏ mọi nhầm lẫn và định nghĩa dứt khoát thiết kế và kiến trúc là gì. Trước hết, tôi khẳng định rằng chúng chẳng khác nhau. Không khác chút nào.

Từ "kiến trúc" thường được dùng để chỉ những thứ ở cấp cao, tách rời khỏi các chi tiết cấp thấp; còn "thiết kế" thường gợi đến những cấu trúc và quyết định ở cấp thấp hơn. Nhưng cách dùng này vô lý nếu nhìn vào công việc của một kiến trúc sư thực thụ.

Hãy nghĩ đến kiến trúc sư đã thiết kế ngôi nhà mới của tôi. Ngôi nhà ấy có kiến trúc không? Dĩ nhiên là có. Kiến trúc của nó là gì? Là hình dáng, diện mạo bên ngoài, các mặt đứng, cách bố trí không gian và phòng ốc. Nhưng khi xem các bản vẽ kiến trúc sư tạo ra, tôi thấy vô số chi tiết cấp thấp. Tôi thấy vị trí của từng ổ cắm, công tắc đèn và bóng đèn. Tôi thấy công tắc nào điều khiển đèn nào. Tôi thấy vị trí lò sưởi, kích thước và vị trí bình nước nóng cùng máy bơm hố thu. Tôi thấy những bản vẽ chi tiết mô tả cách xây tường, mái nhà và móng.

Nói ngắn gọn, tôi thấy mọi chi tiết nhỏ hỗ trợ cho các quyết định cấp cao. Tôi cũng thấy các chi tiết cấp thấp và quyết định cấp cao đều là một phần trong toàn bộ thiết kế ngôi nhà.

Thiết kế phần mềm cũng vậy. Các chi tiết cấp thấp và cấu trúc cấp cao cùng thuộc về một tổng thể. Chúng kết thành một thể liên tục, định hình hệ thống. Không thể có cái này mà thiếu cái kia; thật ra, chẳng có đường ranh rõ ràng nào ngăn cách chúng. Các quyết định trải liên tục từ cấp cao nhất đến cấp thấp nhất.

Mục tiêu?

Mục tiêu của những quyết định ấy là gì? Mục tiêu của thiết kế phần mềm tốt là gì? Mục tiêu ấy chính là mô tả lý tưởng của tôi:

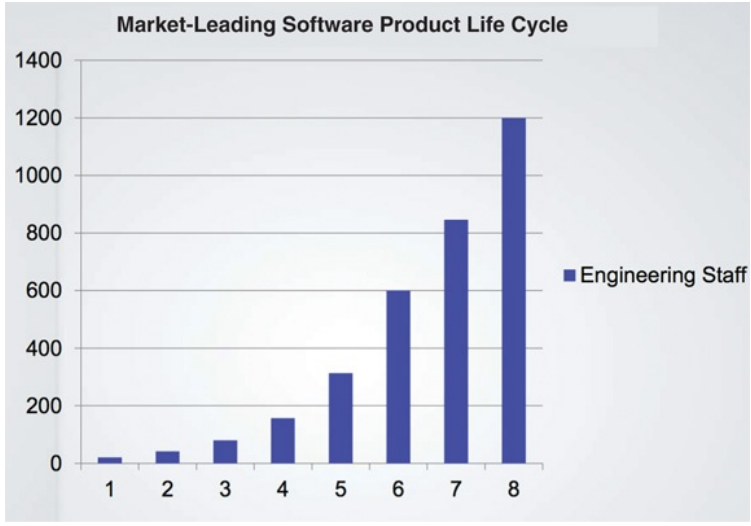
> *Mục tiêu của kiến trúc phần mềm là giảm thiểu nguồn lực con người cần thiết để xây dựng và bảo trì hệ thống theo yêu cầu.*

Chất lượng thiết kế được đo bằng nỗ lực cần bỏ ra để đáp ứng nhu cầu của khách hàng. Nếu nỗ lực ấy thấp và duy trì ở mức thấp suốt vòng đời hệ thống, thiết kế tốt. Nếu mỗi bản phát hành lại cần nhiều công sức hơn, thiết kế kém. Đơn giản vậy thôi.

Nghiên cứu tình huống

Hãy xem xét ví dụ sau. Tình huống này sử dụng dữ liệu thực từ một công ty muốn được giấu tên.

Trước tiên, hãy xem quy mô đội ngũ kỹ thuật tăng lên như thế nào. Tôi tin bạn sẽ đồng ý rằng xu hướng này rất đáng khích lệ. Mức tăng trưởng như Hình 1.1 cho thấy hẳn là công ty đã thành công đáng kể!



Hình 1.1

Tăng trưởng của đội ngũ kỹ thuật. Tái hiện với sự cho phép của Jason Gorman, từ một buổi thuyết trình.

Bây giờ, hãy xem năng suất của công ty trong cùng giai đoạn, tính đơn giản bằng số dòng mã (Hình 1.2).

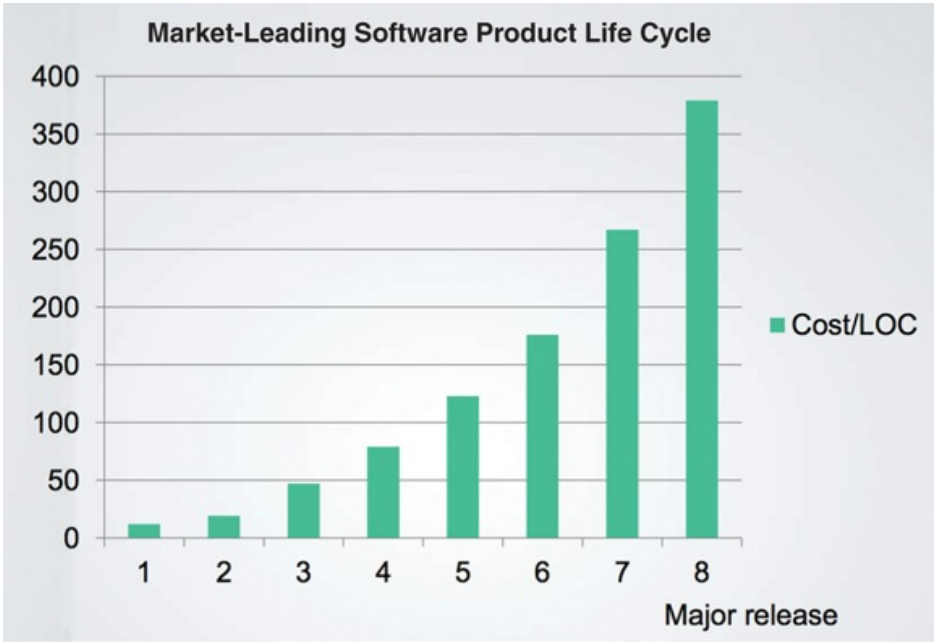


Hình 1.2

Năng suất trong cùng giai đoạn

Rõ ràng đã có chuyện không ổn. Mỗi bản phát hành đều có thêm nhiều nhà phát triển hỗ trợ, nhưng lượng mã tăng lên có vẻ đang tiến dần đến một giới hạn.

Đây mới là biểu đồ đáng sợ: Hình 1.3 cho thấy chi phí trên mỗi dòng mã thay đổi theo thời gian.



Hình 1.3

Chi phí trên mỗi dòng mã theo thời gian

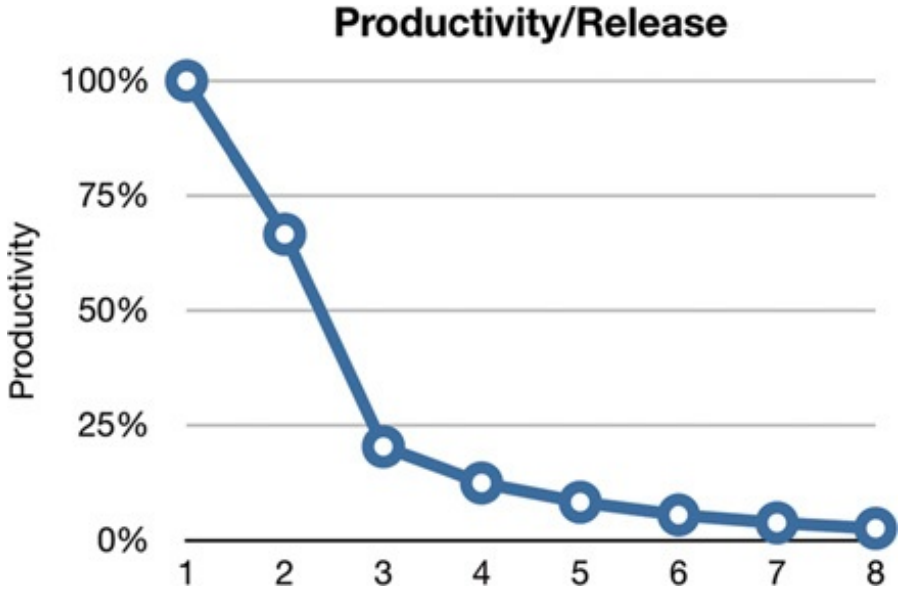
Những xu hướng này không thể kéo dài. Dù hiện tại công ty có lãi đến đâu, các đường cong ấy rồi sẽ bào mòn lợi nhuận của mô hình kinh doanh và đẩy công ty vào tình trạng đình trệ, thậm chí sụp đổ.

Điều gì gây ra sự thay đổi đáng kể về năng suất này? Vì sao chi phí tạo ra mã ở bản phát hành 8 lại cao gấp 40 lần bản phát hành 1?

Dấu hiệu của một mớ hỗn độn

Bạn đang nhìn vào dấu hiệu của một mớ hỗn độn. Khi hệ thống bị chắp vá trong vội vã, khi số lượng lập trình viên là yếu tố duy nhất quyết định sản lượng, và khi người ta hầu như chẳng nghĩ đến sự sạch sẽ của mã hay cấu trúc thiết kế, bạn có thể tin rằng đường cong ấy sẽ tiếp tục đi xuống đến tận cùng tồi tệ.

Hình 1.4 cho thấy đường cong này trông ra sao với các nhà phát triển. Ban đầu, năng suất của họ gần 100%, nhưng giảm dần sau mỗi bản phát hành. Đến bản phát hành thứ tư, có thể thấy năng suất sẽ chạm đáy khi tiến dần về không.



Hình 1.4

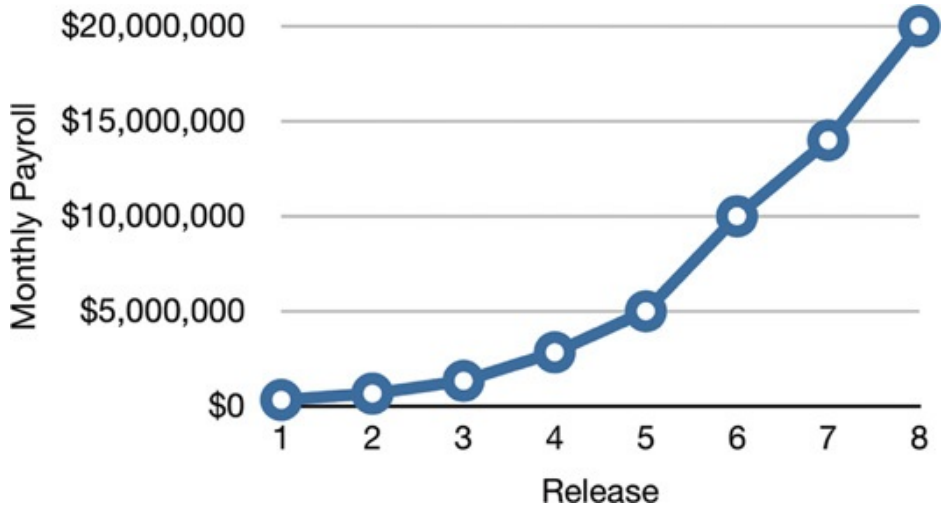
Năng suất qua từng bản phát hành

Từ góc nhìn của các nhà phát triển, tình cảnh này vô cùng bực bội vì ai cũng làm việc chăm chỉ. Không ai giảm nỗ lực.

Dù đã cố hết sức, làm thêm giờ và tận tâm, họ vẫn chẳng hoàn thành được bao nhiêu. Toàn bộ nỗ lực bị rút khỏi việc phát triển tính năng và phải dùng để xoay xở với mớ hỗn độn. Công việc của họ, nếu còn gọi là công việc, đã biến thành chuyển đống bừa bộn từ chỗ này sang chỗ khác, hết lần này đến lần khác, chỉ để thêm một tính năng nhỏ nhoi.

Góc nhìn của lãnh đạo

Nếu bạn nghĩ tình hình đó đã tệ, hãy hình dung lãnh đạo nhìn vào biểu đồ này sẽ thấy gì! Hình 1.5 mô tả quỹ lương phát triển hằng tháng trong cùng giai đoạn.



Hình 1.5

Quỹ lương phát triển hằng tháng theo bản phát hành

Bản phát hành 1 được bàn giao với quỹ lương hằng tháng chỉ vài trăm nghìn đô-la. Bản thứ hai tốn thêm vài trăm nghìn. Đến bản phát hành thứ tám, quỹ lương mỗi tháng đã lên tới 20 triệu đô-la và vẫn còn tăng.

Chỉ riêng biểu đồ này đã đủ đáng sợ. Rõ ràng đang có chuyện đáng báo động. Ta hy vọng doanh thu tăng nhanh hơn chi phí để bù đắp khoản chi đó. Nhưng nhìn theo cách nào thì đường cong này cũng đáng lo.

Giờ hãy so sánh đường cong ở Hình 1.5 với số dòng mã được viết trong mỗi bản phát hành ở Hình 1.2. Mấy trăm nghìn đô-la mỗi tháng ban đầu mua được rất nhiều chức năng, nhưng 20 triệu đô-la ở giai đoạn cuối gần như chẳng mua được gì! Bất kỳ giám đốc tài chính nào nhìn hai biểu đồ này cũng hiểu rằng cần hành động ngay để ngăn thảm họa.

Nhưng có thể làm gì? Chuyện gì đã xảy ra? Điều gì khiến năng suất giảm mạnh đến vậy? Ngoài việc giậm chân và nổi giận với các nhà phát triển, lãnh đạo còn có thể làm gì?

Chuyện gì đã xảy ra?

Gần 2.600 năm trước, Aesop kể câu chuyện Rùa và Thỏ. Bài học của câu chuyện được nhắc đi nhắc lại bằng nhiều cách:

- “Chậm mà chắc sẽ thắng cuộc đua.”
- “Cuộc đua không dành cho người nhanh, cũng như trận chiến không dành cho người mạnh.”

- "Càng vội vã, càng chậm chạp."

Câu chuyện cho thấy sự tự tin thái quá thật dại dột ra sao. Thỏ tin chắc mình nhanh hơn hẳn nên chẳng coi cuộc đua là nghiêm túc, rồi nằm ngủ trong lúc Rùa cán đích.

Các nhà phát triển ngày nay cũng đang trong một cuộc đua tương tự và thể hiện sự tự tin thái quá như vậy. Họ không ngủ, trái lại, phần lớn làm việc đến kiệt sức. Nhưng một phần trong đầu họ lại ngủ quên: phần hiểu rằng mã tốt, sạch và được thiết kế cẩn thận rất quan trọng.

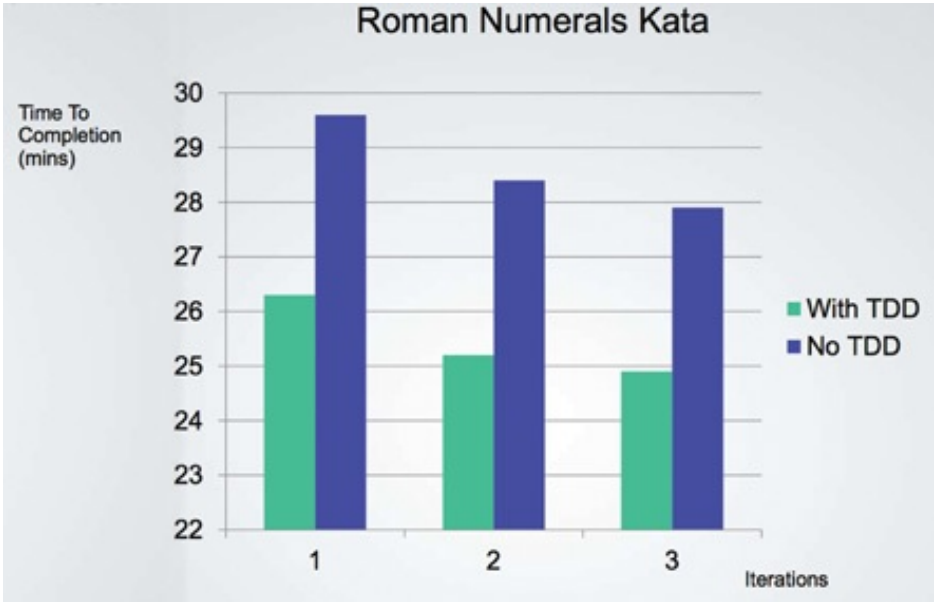
Họ tin vào một lời nói dối quen thuộc: "Cứ ra thị trường trước đã, sau này ta sẽ dọn dẹp mã!" Dĩ nhiên, việc dọn dẹp chẳng bao giờ diễn ra, vì áp lực thị trường không giảm đi. Ra thị trường trước chỉ có nghĩa là giờ đây bạn có cả đám đối thủ bám sát phía sau, và phải chạy nhanh hết sức để vượt lên.

Vì vậy, các nhà phát triển chẳng bao giờ đổi cách làm. Họ không thể quay lại dọn dẹp vì phải hoàn thành tính năng tiếp theo, rồi tính năng kế tiếp, rồi nữa. Mớ hỗn độn ngày càng lớn, còn năng suất tiếp tục tiến dần về không.

Thỏ quá tự tin vào tốc độ của mình; các nhà phát triển cũng quá tự tin rằng họ sẽ duy trì được năng suất. Mớ mã ngày càng rối, âm thầm bào mòn năng suất, chẳng bao giờ dừng lại. Nếu cứ để mặc, nó sẽ kéo năng suất xuống bằng không chỉ trong vài tháng.

Lời nói dối lớn hơn mà các nhà phát triển tin vào là viết mã cẩu thả giúp họ làm nhanh trong ngắn hạn, chỉ khiến họ chậm lại về lâu dài. Những người tin điều đó vừa quá tự tin vào khả năng chuyển từ tạo ra mớ hỗn độn sang dọn dẹp nó trong tương lai, vừa mắc một sai lầm thực tế đơn giản. Sự thật là làm việc trong cảnh hỗn độn luôn chậm hơn giữ cho mọi thứ gọn gàng, bất kể xét trong khoảng thời gian nào.

Hãy xem kết quả thí nghiệm đáng chú ý của Jason Gorman, được thể hiện ở Hình 1.6. Jason tiến hành thử nghiệm trong sáu ngày. Mỗi ngày, ông hoàn thành một chương trình đơn giản để chuyển số nguyên sang chữ số La Mã. Ông biết mình đã xong khi bộ kiểm thử chấp nhận được xác định trước đều chạy qua. Mỗi ngày, công việc mất chưa đến 30 phút. Trong ngày thứ nhất, thứ ba và thứ năm, Jason áp dụng một kỷ luật phát triển phần mềm có tên phát triển hướng kiểm thử (TDD). Ba ngày còn lại, ông viết mã mà không áp dụng kỷ luật đó.



Hình 1.6

Thời gian hoàn thành qua các lần lặp, có và không dùng TDD

Trước hết, hãy chú ý đến đường cong học tập trong Hình 1.6. Công việc những ngày sau hoàn thành nhanh hơn những ngày đầu. Cũng hãy để ý rằng các ngày dùng TDD hoàn thành nhanh hơn khoảng 10% so với các ngày không dùng TDD; ngay cả ngày dùng TDD chậm nhất cũng nhanh hơn ngày không dùng TDD nhanh nhất.

Một số người có thể nhìn kết quả này và cho rằng đó là điều phi thường. Nhưng với những ai không bị sự tự tin thái quá của Thỏ đánh lừa, kết quả ấy hoàn toàn dễ đoán, vì họ biết sự thật giản dị này về phát triển phần mềm:

Cách duy nhất để làm nhanh là làm cho tốt.

Đó là câu trả lời cho thế khó của lãnh đạo. Cách duy nhất để đảo ngược đà giảm năng suất và tăng chi phí là khiến các nhà phát triển thôi suy nghĩ như Thỏ tự phụ và bắt đầu chịu trách nhiệm về mớ hỗn độn họ đã tạo ra.

Các nhà phát triển có thể nghĩ giải pháp là bắt đầu lại từ đầu và thiết kế lại toàn bộ hệ thống. Nhưng đó lại là Thỏ đang lên tiếng. Chính sự tự tin thái quá từng dẫn đến mớ hỗn độn giờ lại bảo họ rằng chỉ cần được chạy lại cuộc đua, họ sẽ làm tốt hơn. Thực tế chẳng đẹp đẽ như vậy:

Sự tự tin thái quá sẽ khiến lần thiết kế lại tạo ra mớ hỗn độn y như dự án ban đầu.

Kết luận

Trong mọi trường hợp, lựa chọn tốt nhất là để tổ chức phát triển nhận ra và tránh sự tự tin thái quá của chính mình, đồng thời nghiêm túc xem xét chất lượng kiến trúc phần mềm.

Muốn coi trọng kiến trúc phần mềm, bạn cần biết kiến trúc phần mềm tốt là gì. Muốn xây dựng hệ thống có thiết kế và kiến trúc giúp giảm nỗ lực, tăng năng suất, bạn cần biết những thuộc tính nào của kiến trúc hệ thống sẽ dẫn đến kết quả đó.

Đó là nội dung cuốn sách này. Sách mô tả diện mạo của những kiến trúc và thiết kế phần mềm tốt, sạch, để các nhà phát triển có thể xây dựng hệ thống sinh lợi trong thời gian dài.

2

Câu chuyện về hai giá trị



Hành vi

Mọi hệ thống phần mềm mang lại hai giá trị khác nhau cho các bên liên quan: hành vi và cấu trúc. Các nhà phát triển phần mềm chịu trách nhiệm đảm bảo cả hai giá trị đó luôn cao. Thật đáng buồn, họ thường chỉ tập trung vào một bên mà bỏ qua bên kia. Còn đáng buồn hơn, họ thường tập trung vào giá trị kém hơn trong hai cái, khiến hệ thống phần mềm cuối cùng trở nên vô giá trị.

Giá trị đầu tiên của phần mềm là hành vi. Lập trình viên được thuê để khiến máy móc hoạt động theo cách giúp các bên liên quan kiếm được hoặc tiết kiệm tiền. Chúng ta làm điều đó bằng cách hỗ trợ các bên liên quan xây dựng bản đặc tả chức năng, hay tài liệu yêu cầu. Sau đó, chúng ta viết mã để máy móc đáp ứng những yêu cầu ấy.

Khi máy móc vi phạm các yêu cầu đó, lập trình viên lấy bộ gỡ lỗi ra và khắc phục sự cố.

Nhiều lập trình viên tin rằng đó là toàn bộ công việc của họ. Họ tin rằng việc của họ là khiến máy móc thực hiện các yêu cầu và sửa bất kỳ lỗi nào. Họ lầm to.

Kiến trúc

Giá trị thứ hai của phần mềm liên quan đến chính từ "software", một từ ghép từ "soft" và "ware". "Ware" có nghĩa là "sản phẩm"; còn "soft"… Chính ở đó nằm giá trị thứ hai.

Phần mềm được phát minh để trở nên "mềm". Nó được xem là cách giúp thay đổi hành vi máy móc dễ dàng. Nếu muốn hành vi máy móc khó thay đổi, hẳn chúng ta đã gọi nó là phần cứng.

Để thực hiện mục đích ấy, phần mềm phải mềm, tức là phải dễ thay đổi. Khi các bên liên quan đổi ý về một tính năng, thay đổi đó phải đơn giản và dễ thực hiện. Độ khó của thay đổi chỉ nên tỷ lệ thuận với phạm vi thay đổi, chứ không phụ thuộc vào hình dạng của nó.

Chính sự khác biệt giữa phạm vi và hình dạng thường làm chi phí phát triển phần mềm tăng. Vì thế chi phí có thể tăng vượt xa mức tương ứng với quy mô thay đổi được yêu cầu. Đó là lý do năm phát triển đầu tiên rẻ hơn năm thứ hai, còn năm thứ hai rẻ hơn năm thứ ba.

Từ góc nhìn của các bên liên quan, họ chỉ cung cấp một luồng các thay đổi có phạm vi tương đương nhau. Từ góc nhìn của nhà phát triển, các bên liên quan đang đưa cho họ một luồng các mảnh ghép xếp hình mà họ phải lắp vào một bức tranh ghép ngày càng phức tạp. Mỗi yêu cầu mới lại khó lắp hơn yêu cầu trước, vì hình dáng của hệ thống không khớp với hình dáng của yêu cầu.

Tôi dùng từ "hình dáng" (shape) theo một cách không thông dụng, nhưng nghĩ rằng ẩn dụ này phù hợp. Các nhà phát triển phần mềm thường cảm thấy như bị ép phải nhét chốt vuông vào lỗ tròn.

Vấn đề nằm ở kiến trúc hệ thống. Kiến trúc càng ưu tiên hình dạng này hơn hình dạng khác, các tính năng mới càng khó lắp vào cấu trúc đó. Vì vậy, trong chừng mực khả thi, kiến trúc nên không thiên về bất kỳ hình dạng cụ thể nào.

Giá trị lớn hơn

Chức năng hay kiến trúc? Hai giá trị này, cái nào lớn hơn? Việc hệ thống phần mềm hoạt động quan trọng hơn, hay việc nó dễ thay đổi quan trọng hơn?

Nếu hỏi các nhà quản lý kinh doanh, họ thường nói rằng việc hệ thống phần mềm hoạt động quan trọng hơn. Các nhà phát triển, ngược lại, thường đồng ý với thái độ này. Nhưng đó là thái độ sai. Tôi có thể chứng minh nó sai bằng công cụ logic đơn giản là xem xét các cực đoan.

- Nếu bạn cho tôi một chương trình chạy hoàn hảo nhưng không thể thay đổi, khi yêu cầu thay đổi thì chương trình sẽ không còn hoạt động và tôi cũng không thể sửa nó. Khi ấy, chương trình trở nên vô dụng.
- Nếu bạn cho tôi một chương trình không hoạt động nhưng dễ thay đổi, tôi có thể sửa nó và giữ cho nó hoạt động khi yêu cầu thay đổi. Nhờ vậy, chương trình tiếp tục hữu ích.

Bạn có thể không thấy lập luận này thuyết phục. Dù sao, không có chương trình nào hoàn toàn không thể thay đổi.

Tuy nhiên, có những hệ thống gần như không thể thay đổi vì chi phí thay đổi vượt quá lợi ích của nó. Nhiều hệ thống đạt đến ngưỡng ấy ở một số tính năng hoặc cấu hình.

Ma trận Eisenhower

Hãy xem ma trận tầm quan trọng so với sự cấp bách của Tổng thống Dwight D. Eisenhower (Hình 2.1). Về ma trận này, Eisenhower từng nói:

> *Tôi có hai loại vấn đề: cấp bách và quan trọng. Những việc cấp bách không quan trọng, và những việc quan trọng không bao giờ cấp bách.*[1]



Hình 2.1

1. Trích từ bài phát biểu tại Đại học Northwestern năm 1954.

Ma trận Eisenhower

Câu châm ngôn xưa này đúng ở nhiều điểm. Những việc cấp bách hiếm khi thật sự quan trọng, còn những việc quan trọng hiếm khi cấp bách.

Giá trị đầu tiên của phần mềm (hành vi) cấp bách nhưng không nhất thiết quan trọng.

Giá trị thứ hai của phần mềm (kiến trúc) quan trọng nhưng gần như không bao giờ cấp bách.

Tất nhiên, có những việc vừa cấp bách vừa quan trọng. Cũng có những việc không cấp bách và không quan trọng. Cuối cùng, chúng ta có thể sắp xếp bốn cặp này thành các mức ưu tiên:

1. Cấp bách và quan trọng
2. Không cấp bách và quan trọng
3. Cấp bách và không quan trọng
4. Không cấp bách và không quan trọng

Lưu ý rằng kiến trúc của mã (phần quan trọng) nằm ở hai vị trí đầu trong danh sách này, còn hành vi của mã chiếm vị trí thứ nhất và thứ ba.

Sai lầm mà các nhà quản lý kinh doanh và nhà phát triển thường mắc là nâng các mục ở vị trí 3 lên vị trí 1. Nói cách khác, họ không phân biệt được những tính năng cấp bách nhưng không quan trọng với những tính năng thực sự vừa cấp bách vừa quan trọng. Sự thất bại này dẫn đến việc bỏ qua kiến trúc quan trọng của hệ thống để ưu tiên các tính năng không quan trọng.

Bài toán khó cho các nhà phát triển phần mềm là các nhà quản lý kinh doanh không được trang bị để đánh giá tầm quan trọng của kiến trúc. Đó chính là thứ các nhà phát triển phần mềm được thuê để làm. Do đó, trách nhiệm của đội ngũ phát triển phần mềm là khẳng định tầm quan trọng của kiến trúc lên trên sự cấp bách của các tính năng.

Chiến đấu cho kiến trúc

Thực hiện trách nhiệm này có nghĩa là bước vào một cuộc chiến, hay nói đúng hơn là một cuộc đấu tranh. Thú thực, mọi việc vẫn diễn ra như vậy. Đội ngũ phát triển phải đấu tranh cho điều họ tin là tốt nhất cho công ty, cũng như đội ngũ quản lý, marketing, bán hàng và vận hành. Luôn có sự đấu tranh.

Các đội ngũ phát triển phần mềm hiệu quả đối mặt trực diện với cuộc đấu tranh đó. Họ không ngần ngại tranh luận với các bên liên quan khác như những người ngang hàng. Hãy nhớ rằng với tư cách nhà phát triển phần mềm, bạn cũng là một bên liên quan. Bạn có lợi ích gắn với phần mềm và cần bảo vệ lợi ích ấy. Đó là một phần trong vai trò và bổn phận của bạn, cũng là lý do quan trọng khiến bạn được tuyển dụng.

Thách thức này quan trọng gấp đôi nếu bạn là một kiến trúc sư phần mềm. Kiến trúc sư phần mềm, nhờ mô tả công việc của họ, tập trung nhiều hơn vào cấu trúc của hệ thống hơn là các tính năng và chức năng. Các kiến trúc sư tạo ra một kiến trúc cho phép các tính năng và chức năng đó dễ dàng phát triển, dễ dàng sửa đổi, và dễ dàng mở rộng.

Chỉ cần nhớ: Nếu kiến trúc đến sau cùng, hệ thống sẽ ngày càng tốn kém để phát triển,

và cuối cùng việc thay đổi sẽ trở nên gần như không thể đối với một phần hoặc toàn bộ hệ thống. Nếu để chuyện đó xảy ra, nghĩa là đội ngũ phát triển phần mềm không chiến đấu đủ mạnh cho những gì họ biết là cần thiết.

Phần II: Bắt đầu từ những viên gạch: Các mô hình lập trình

Kiến trúc phần mềm khởi đầu từ mã nguồn, và vì thế chúng ta sẽ bắt đầu cuộc thảo luận về kiến trúc bằng việc xem xét những gì chúng ta đã học được về mã từ khi mã lần đầu được viết.

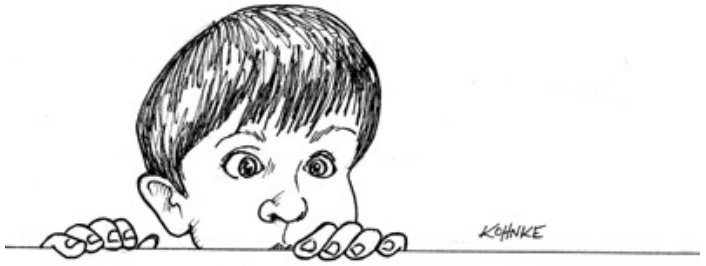
Năm 1938, Alan Turing đặt nền móng cho những gì sau này trở thành lập trình máy tính. Ông không phải người đầu tiên nghĩ ra máy có thể lập trình, nhưng ông là người đầu tiên hiểu rằng chương trình đơn giản là dữ liệu. Đến năm 1945, Turing đã viết các chương trình thật trên máy tính thật bằng mã mà chúng ta có thể nhận ra (nếu nhíu mắt đủ lâu). Những chương trình đó dùng vòng lặp, phân nhánh, gán giá trị, thủ tục con, ngăn xếp và các cấu trúc quen thuộc khác. Ngôn ngữ của Turing là nhị phân.

Từ đó đến nay, lập trình đã trải qua nhiều cuộc cách mạng. Cuộc cách mạng quen thuộc nhất là sự ra đời của các ngôn ngữ lập trình. Cuối thập niên 1940, các bộ hợp dịch (assembler) xuất hiện, giải phóng lập trình viên khỏi việc vất vả dịch chương trình sang mã nhị phân. Năm 1951, Grace Hopper phát minh ra A0, bộ biên dịch đầu tiên. Bà cũng đặt ra thuật ngữ *compiler*. Fortran ra đời năm 1953, một năm sau khi tôi sinh. Sau đó là một làn sóng ngôn ngữ lập trình nối tiếp không dứt: COBOL, PL/1, SNOBOL, C, Pascal, C++, Java và nhiều ngôn ngữ khác.

Một cuộc cách mạng khác, có lẽ quan trọng hơn, là sự ra đời của các mô hình lập trình (paradigm). Mô hình lập trình là cách viết chương trình, tương đối độc lập với ngôn ngữ. Nó cho biết nên dùng những cấu trúc lập trình nào và dùng khi nào. Đến nay có ba mô hình như vậy. Vì những lý do sẽ bàn sau, có lẽ không có mô hình nào khác.

3

Tổng quan về các mô hình lập trình





Ba mô hình lập trình (paradigm) trong chương tổng quan này là lập trình có cấu trúc, lập trình hướng đối tượng và lập trình hàm.

Lập trình có cấu trúc

Mô hình đầu tiên được áp dụng, dù không phải mô hình đầu tiên được phát minh, là lập trình có cấu trúc. Edsger Wybe Dijkstra khám phá ra mô hình này năm 1968. Ông chỉ ra rằng việc dùng các lệnh nhảy tùy tiện (câu lệnh `goto`) gây hại cho cấu trúc chương trình. Như sẽ thấy trong các chương sau, Dijkstra thay những lệnh nhảy đó bằng các cấu trúc quen thuộc hơn như `if/then/else` và `do/while/until`.

Chúng ta có thể tóm tắt mô hình lập trình có cấu trúc như sau:

> ***Lập trình có cấu trúc áp đặt kỷ luật lên việc chuyển giao quyền điều khiển trực tiếp.***

Lập trình hướng đối tượng

Mô hình thứ hai được áp dụng, nhưng thực ra đã được khám phá sớm hơn hai năm, vào năm 1966, bởi Ole Johan Dahl và Kristen Nygaard. Hai lập trình viên này nhận ra rằng khung ngăn xếp lời gọi hàm trong ngôn ngữ ALGOL có thể được chuyển sang heap, nhờ đó các biến cục bộ do hàm khai báo vẫn tồn tại lâu sau khi hàm trả về. Hàm trở thành hàm tạo của lớp, biến cục bộ trở thành biến thể hiện, còn hàm lồng nhau trở thành phương thức. Từ đó dẫn đến việc khám phá ra tính đa hình thông qua cách dùng con trỏ hàm có kỷ luật.

Chúng ta có thể tóm tắt mô hình lập trình hướng đối tượng như sau:

> ***Lập trình hướng đối tượng áp đặt kỷ luật lên việc chuyển giao quyền điều khiển gián tiếp.***

Lập trình hàm

Mô hình thứ ba chỉ mới bắt đầu được áp dụng gần đây, nhưng lại là mô hình đầu tiên được phát minh. Thực ra, nó ra đời trước cả lập trình máy tính. Lập trình hàm bắt nguồn trực tiếp từ công trình của Alonzo Church. Năm 1936, trong lúc theo đuổi cùng vấn đề toán học đã thu hút Alan Turing, Church phát minh ra λ-calculus (lambda calculus). λ-calculus là nền tảng của ngôn ngữ LISP, do John McCarthy phát minh năm 1958. Một khái niệm nền tảng của λ-calculus là tính bất biến (immutability), nghĩa là giá trị các ký hiệu không thay đổi. Do đó, ngôn ngữ hàm không có câu lệnh gán (assignment statement). Trên thực tế, hầu hết ngôn ngữ hàm đều có cách thay đổi giá trị biến, nhưng chỉ cho phép làm vậy theo những quy tắc rất chặt chẽ.

Chúng ta có thể tóm tắt mô hình lập trình hàm như sau:

> ***Lập trình hàm áp đặt kỷ luật lên việc gán giá trị.***

Điều đáng suy ngẫm

Hãy nhận ra quy luật mà tôi đã cố ý thiết lập khi giới thiệu ba mô hình lập trình này: Mỗi mô hình lập trình đều **loại bỏ** khả năng khỏi người lập trình. **Không cái nào thêm khả năng mới.** Mỗi cái đều áp đặt một loại kỷ luật hạn chế. Các mô hình lập trình nói cho chúng ta biết những gì **không được làm**, nhiều hơn là nói những gì **nên làm**.

Một cách nhìn khác về vấn đề này là nhận ra rằng mỗi mô hình lập trình lấy đi của chúng ta một thứ gì đó. Ba mô hình lập trình cùng nhau loại bỏ câu lệnh `goto`, con trỏ hàm và câu lệnh gán. Vậy còn lại gì để lấy đi nữa?

Có lẽ không. Do đó ba mô hình lập trình này rất có thể là ba mô hình duy nhất chúng ta sẽ thấy, ít nhất là ba mô hình mang tính "tiêu cực" như thế. Bằng chứng khác cho thấy không còn mô hình nào như vậy là chúng đã được khám phá hết trong khoảng mười năm từ 1958 đến 1968. Trong nhiều thập kỷ sau đó, không có mô hình mới nào được thêm vào.

Kết luận

Bài học lịch sử về các mô hình lập trình này có liên quan gì đến kiến trúc? **Mọi thứ.** Chúng ta dùng tính đa hình làm cơ chế để vượt qua ranh giới kiến trúc; dùng lập trình hàm để áp đặt kỷ luật lên vị trí và cách truy cập dữ liệu; và dùng lập trình có cấu trúc làm nền tảng thuật toán cho các mô-đun của mình.

Hãy nhận ra quy luật tôi cố ý đưa vào khi giới thiệu ba mô hình này: mô hình nào cũng tước bớt khả năng của lập trình viên. Không mô hình nào bổ sung khả năng mới. Mỗi mô hình đặt ra một kỷ luật nhằm hạn chế. Chúng chủ yếu cho ta biết không nên làm gì, hơn là nên làm gì.

4

Lập trình có cấu trúc



Edsger Wybe Dijkstra sinh tại Rotterdam năm 1930. Ông sống sót qua vụ ném bom Rotterdam trong Thế chiến thứ hai và thời kỳ Đức chiếm đóng Hà Lan. Năm 1948, ông tốt nghiệp trung học với điểm cao nhất có thể đạt được ở toán, vật lý, hóa học và sinh học. Tháng 3 năm 1952, khi 21 tuổi (chỉ chín tháng trước khi tôi chào đời), Dijkstra nhận việc tại Trung tâm Toán học Amsterdam với tư cách là lập trình viên đầu tiên của Hà Lan.

Năm 1955, sau ba năm làm lập trình viên, và vẫn còn là sinh viên, Dijkstra kết luận rằng thách thức trí tuệ của việc lập trình lớn hơn thách thức trí tuệ của vật lý lý thuyết. Kết quả là ông chọn lập trình làm nghề nghiệp dài hạn. Năm 1957, Dijkstra kết hôn với Maria

Debets. Lúc đó, bạn phải khai nghề nghiệp như một phần trong lễ cưới tại Hà Lan. Cơ quan Hà Lan từ chối chấp nhận "lập trình viên" là nghề của Dijkstra; họ chưa bao giờ nghe nói đến nghề ấy. Để làm hài lòng họ, Dijkstra chấp nhận "vật lý lý thuyết" làm chức danh công việc.

Là một phần của việc quyết định theo đuổi lập trình như một nghề nghiệp, Dijkstra đã bàn bạc với người quản lý của mình, Adriaan van Wijngaarden. Dijkstra lo ngại rằng chưa ai xác định ra một kỷ luật, hay một khoa học, của lập trình, và do đó ông sẽ không được coi trọng. Người quản lý trả lời rằng Dijkstra có thể là một trong những người sẽ khám phá ra những kỷ luật đó, từ đó biến phần mềm thành một khoa học.

Dijkstra bắt đầu sự nghiệp vào thời kỳ ống chân không, khi máy tính to lớn, mong manh, chậm chạp, thiếu tin cậy và (theo tiêu chuẩn ngày nay) cực kỳ hạn chế. Những năm đầu đó, chương trình được viết bằng nhị phân hoặc bằng hợp ngữ rất thô sơ. Đầu vào được đưa vào bằng băng giấy hoặc thẻ đục lỗ. Vòng lặp sửa mã, biên dịch rồi kiểm thử kéo dài hàng giờ, có khi hàng ngày.

Chính trong môi trường nguyên thủy ấy mà Dijkstra đã có những khám phá vĩ đại của mình.

Chứng minh

Vấn đề mà Dijkstra nhận ra sớm là lập trình rất khó, và lập trình viên không làm tốt lắm. Một chương trình có độ phức tạp vừa phải chứa quá nhiều chi tiết để não bộ con người có thể quản lý nếu không có sự giúp đỡ. Bỏ sót chỉ một chi tiết nhỏ dẫn đến những chương trình có vẻ như chạy được, nhưng lại thất bại theo những cách bất ngờ.

Giải pháp của Dijkstra là áp dụng kỷ luật chứng minh trong toán học. Ông hình dung việc xây dựng một hệ thống phân cấp kiểu Euclid gồm các tiên đề, định lý, hệ quả và bổ đề. Dijkstra nghĩ rằng lập trình viên có thể dùng hệ thống đó như các nhà toán học vẫn làm: dùng những cấu trúc đã được chứng minh, rồi ghép chúng bằng mã mà họ sẽ tự chứng minh là đúng.

Để khởi đầu, Dijkstra nhận ra mình phải minh họa kỹ thuật lập các chứng minh cơ bản cho những thuật toán đơn giản. Ông thấy việc đó khá khó khăn.

Trong quá trình nghiên cứu, Dijkstra phát hiện một số cách dùng câu lệnh `goto` ngăn cản việc chia đệ quy mô-đun thành những đơn vị ngày càng nhỏ. Điều đó khiến ta không thể áp dụng phương pháp chia để trị cần thiết để lập các chứng minh hợp lý.

Tuy nhiên, các cách dùng goto khác thì không gặp vấn đề này. Dijkstra nhận ra rằng những cách dùng "tốt" của goto này tương ứng với các cấu trúc điều khiển lựa chọn và lặp đơn giản như if/then/else và do/while. Các mô-đun chỉ dùng những loại cấu trúc điều khiển đó có thể được chia đệ quy thành các đơn vị có thể chứng minh.

Dijkstra biết rằng những cấu trúc điều khiển đó, khi kết hợp với thực thi tuần tự, là đặc biệt. Chúng đã được xác định hai năm trước đó bởi Böhm và Jacopini, những người đã chứng minh rằng tất cả chương trình đều có thể được xây dựng chỉ từ ba cấu trúc: tuần tự, lựa chọn và lặp.

Khám phá này thật đáng chú ý: chính những cấu trúc điều khiển giúp mô-đun có thể

chứng minh được cũng là tập hợp tối thiểu các cấu trúc điều khiển đủ để xây dựng mọi chương trình. Lập trình có cấu trúc ra đời từ đó.

Dijkstra chỉ ra rằng các câu lệnh tuần tự có thể được chứng minh đúng bằng liệt kê đơn giản. Kỹ thuật này theo dõi toán học các đầu vào của chuỗi tuần tự đến các đầu ra của chuỗi đó. Cách tiếp cận này không khác gì một chứng minh toán học thông thường.

Dijkstra giải quyết lựa chọn bằng cách áp dụng lại liệt kê. Mỗi đường đi qua cấu trúc lựa chọn đều được liệt kê. Nếu cả hai đường đi cuối cùng đều tạo ra kết quả toán học phù hợp, thì chứng minh là chắc chắn.

Lặp thì hơi khác. Để chứng minh một vòng lặp đúng, Dijkstra phải dùng quy nạp. Ông chứng minh trường hợp 1 bằng liệt kê. Sau đó chứng minh trường hợp rằng nếu N được giả định đúng, thì N + 1 cũng đúng, lại bằng liệt kê. Ông cũng chứng minh tiêu chí bắt đầu và kết thúc của vòng lặp bằng liệt kê.

Những chứng minh ấy tẻ nhạt và phức tạp, nhưng chúng vẫn là những chứng minh. Khi các phương pháp này được phát triển, dường như đã có thể xây dựng một hệ thống phân cấp các định lý kiểu Euclid.

Một tuyên ngôn có hại

Năm 1968, Dijkstra viết một thư gửi biên tập viên của CACM, được đăng trong số tháng 3. Tiêu đề của bức thư là “Go To Statement Considered Harmful” (Câu lệnh Go To được xem là có hại). Bài viết nêu rõ lập trường của ông về ba cấu trúc điều khiển.

Và thế giới lập trình bùng nổ. Lúc đó chúng ta chưa có Internet, nên mọi người không thể đăng những meme chê bai Dijkstra, và không thể công kích ông trực tuyến. Nhưng họ có thể, và họ đã làm, viết thư gửi biên tập viên của nhiều tạp chí xuất bản.

Những bức thư đó không nhất thiết lịch sự. Một số cực kỳ tiêu cực; những bức khác lại ủng hộ mạnh mẽ lập trường của ông. Và như vậy cuộc tranh luận bắt đầu, cuối cùng kéo dài khoảng một thập kỷ.

Cuối cùng, tranh luận cũng lắng xuống. Lý do rất đơn giản: Dijkstra đã thắng. Khi các ngôn ngữ máy tính phát triển, câu lệnh `goto` dần bị đẩy ra ngoài, cho đến khi gần như biến mất. Hầu hết ngôn ngữ hiện đại không có `goto`, và LISP thì chưa bao giờ có.

Ngày nay chúng ta đều lập trình có cấu trúc, dù không phải do lựa chọn. Ngôn ngữ của chúng ta đơn giản là không cho phép tùy tiện chuyển giao quyền điều khiển trực tiếp.

Một số người có thể xem các lệnh `break` có nhãn trong Java hoặc ngoại lệ là tương tự `goto`. Thực tế, những cấu trúc này không phải kiểu chuyển giao quyền điều khiển hoàn toàn không giới hạn từng có trong các ngôn ngữ cũ như Fortran hay COBOL. Ngay cả ngôn ngữ vẫn hỗ trợ từ khóa `goto` thường cũng giới hạn đích nhảy trong phạm vi hàm hiện tại.

Phân tách chức năng

Lập trình có cấu trúc cho phép các mô-đun được phân tách đệ quy thành các đơn vị có thể chứng minh, mà lần lượt có nghĩa là các mô-đun có thể được phân tách theo chức năng. Đó là, bạn có thể lấy một bài toán quy mô lớn và phân tách nó thành các hàm cấp cao. Mỗi hàm đó sau đó có thể được phân tách thành các hàm cấp thấp hơn, vô tận. Hơn nữa, mỗi hàm đã phân tách đó có thể được biểu diễn bằng các cấu trúc điều khiển bị hạn chế của lập trình có cấu trúc.

Trên nền tảng này, các phương pháp như phân tích có cấu trúc và thiết kế có cấu trúc trở nên phổ biến vào cuối thập niên 1970 và suốt thập niên 1980. Ed Yourdon, Larry Constantine, Tom DeMarco và Meilir Page-Jones đã quảng bá các kỹ thuật ấy. Nhờ áp dụng chúng, lập trình viên có thể chia các hệ thống lớn thành mô-đun và thành phần, rồi tiếp tục chia nhỏ thành những hàm có thể chứng minh.

Không có chứng minh hình thức

Nhưng các chứng minh ấy không bao giờ thành hiện thực. Hệ thống phân cấp định lý kiểu Euclid không được xây dựng. Lập trình viên nói chung cũng không thấy lợi ích của việc kiên nhẫn chứng minh hình thức từng hàm nhỏ một. Cuối cùng, giấc mơ của Dijkstra phai nhạt rồi tan biến. Ngày nay, ít lập trình viên tin rằng chứng minh hình thức là cách thích hợp để tạo ra phần mềm chất lượng cao.

Tất nhiên, chứng minh toán học kiểu Euclid chính thức không phải là chiến lược duy nhất để chứng minh điều gì đó đúng. Một chiến lược rất thành công khác là phương pháp khoa học.

Khoa học giải nguy

Khoa học khác căn bản với toán học ở chỗ các lý thuyết và định luật khoa học không thể được chứng minh là đúng. Tôi không thể chứng minh rằng định luật chuyển động thứ hai của Newton, $F = ma$, hay định luật vạn vật hấp dẫn, $F = Gm_1m_2/r^2$, là đúng. Tôi có thể trình bày các định luật này và đo đạc để xác nhận chúng đến nhiều chữ số thập phân, nhưng không thể chứng minh chúng theo nghĩa toán học. Dù tiến hành bao nhiêu thí nghiệm hay thu thập bao nhiêu bằng chứng thực nghiệm, vẫn luôn có khả năng một thí nghiệm nào đó cho thấy các định luật ấy sai.

Đó là bản chất của lý thuyết và định luật khoa học: chúng có thể bị bác bỏ, nhưng không thể chứng minh là đúng.

Vậy mà mỗi ngày chúng ta đều đặt mạng sống của mình vào những định luật ấy. Mỗi lần lên xe, bạn tin rằng $F = ma$ mô tả đáng tin cậy cách thế giới vận hành. Mỗi bước đi đều đặt sức khỏe và an toàn của bạn vào giả định rằng $F = Gm_1m_2/r^2$ là đúng.

Khoa học không chứng minh các mệnh đề là đúng; thay vào đó, nó cố chứng minh chúng là sai. Sau nhiều nỗ lực mà vẫn không bác bỏ được một mệnh đề, ta xem nó là đủ đúng cho mục đích của mình.

Tất nhiên, không phải mệnh đề nào cũng có thể chứng minh. Câu "Đây là một lời nói dối" không đúng cũng không sai. Đây là một trong những ví dụ đơn giản nhất về mệnh đề không thể chứng minh.

Cuối cùng, có thể nói toán học là ngành chứng minh các mệnh đề có thể chứng minh là đúng. Ngược lại, khoa học là ngành cố chứng minh các mệnh đề có thể chứng minh là sai.

Kiểm thử

Dijkstra từng nói: "Kiểm thử cho thấy lỗi có tồn tại, chứ không chứng minh được lỗi không tồn tại." Nói cách khác, một bài kiểm thử có thể chứng minh chương trình sai, nhưng không thể chứng minh chương trình đúng. Sau khi kiểm thử đủ nhiều, ta chỉ có thể xem chương trình là đủ đúng cho mục đích của mình.

Hàm ý của điều này rất đáng kể. Phát triển phần mềm không phải một hoạt động toán học, dù có vẻ như nó thao tác với các cấu trúc toán học. Thay vào đó, phát triển phần mềm giống khoa học hơn: ta xác nhận tính đúng đắn bằng cách cố hết sức mà vẫn không chứng minh được chương trình sai.

Chỉ có thể cố chứng minh một chương trình sai nếu chương trình đó có thể chứng minh được. Một chương trình không thể chứng minh được, chẳng hạn do dùng `goto` không kiểm soát, sẽ không thể được xem là đúng, cho dù ta kiểm thử bao nhiêu lần.

Lập trình có cấu trúc buộc chúng ta phân tách đệ quy một chương trình thành một tập hợp các hàm nhỏ có thể chứng minh. Sau đó chúng ta có thể dùng bài kiểm thử để thử chứng minh những hàm nhỏ có thể chứng minh đó là sai. Nếu những bài kiểm thử đó không thể chứng minh sự sai, thì chúng ta coi các hàm đó là đủ đúng cho mục đích của mình.

Kết luận

Chính khả năng chia chương trình thành những đơn vị có thể kiểm chứng và bác bỏ khiến lập trình có cấu trúc vẫn có giá trị ngày nay. Đó là lý do ngôn ngữ hiện đại thường không hỗ trợ câu lệnh `goto` không kiểm soát. Ở cấp độ kiến trúc, đó cũng là lý do ta vẫn xem phân tách chức năng là một trong những thực hành tốt nhất.

Ở mọi cấp độ, từ hàm nhỏ nhất đến thành phần lớn nhất, phần mềm vận hành như khoa học và vì vậy dựa trên khả năng bác bỏ. Kiến trúc sư phần mềm cố gắng định nghĩa các mô-đun, thành phần và dịch vụ có thể dễ dàng kiểm thử để phát hiện sai sót. Để làm vậy, họ dùng những kỷ luật hạn chế tương tự lập trình có cấu trúc, nhưng ở cấp độ cao hơn nhiều.

Chính những kỷ luật hạn chế ấy là những gì chúng ta sẽ nghiên cứu chi tiết ở các chương tiếp theo.

5

Lập trình hướng đối tượng



Như chúng ta sẽ thấy, nền tảng của kiến trúc tốt là hiểu và áp dụng các nguyên lý thiết kế hướng đối tượng (OO). Nhưng chính xác thì OO là gì?

Một câu trả lời cho câu hỏi này là "Sự kết hợp của dữ liệu và hàm." Mặc dù thường được trích dẫn, đây là một câu trả lời rất không thỏa mãn vì nó ngụ ý rằng `o.f()` có gì đó khác biệt so với `f(o)`. Điều này vô lý. Các lập trình viên đã truyền cấu trúc dữ liệu vào hàm từ lâu trước năm 1966, khi Dahl và Nygaard chuyển khung ngăn xếp gọi hàm lên heap và phát minh ra OO.

Một câu trả lời phổ biến khác là "Một cách mô hình hóa thế giới thực." Nói vậy vẫn lẩn tránh vấn đề. "Mô hình hóa thế giới thực" thực sự có nghĩa là gì, và tại sao ta lại muốn làm vậy? Có lẽ ý câu đó là OO giúp phần mềm dễ hiểu hơn vì gần với thế giới thực. Nhưng cách giải thích ấy vẫn lẩn tránh vấn đề và quá mơ hồ. Nó không cho ta biết OO là gì.

Một số người dựa vào ba từ thần thánh để giải thích bản chất của OO: đóng gói (encapsulation), kế thừa (inheritance), và đa hình (polymorphism). Ngụ ý là OO là sự phối trộn thích đáng của ba thứ này, hoặc ít nhất một ngôn ngữ OO phải hỗ trợ ba thứ này.

Hãy kiểm tra từng khái niệm này lần lượt.

Đóng gói (Encapsulation)?

Người ta xem đóng gói là một phần của định nghĩa OO vì các ngôn ngữ OO giúp đóng gói dữ liệu và hàm một cách dễ dàng, hiệu quả. Nhờ đó, có thể vạch ranh giới quanh một nhóm dữ liệu và hàm gắn kết với nhau. Bên ngoài ranh giới ấy, dữ liệu được ẩn đi và chỉ một số hàm được công khai. Ta thấy cách này được dùng với các thành viên dữ liệu `private` và hàm thành viên `public` của một lớp.

Ý tưởng này không hề riêng có ở OO. Thực tế, ngôn ngữ C đã có thể đóng gói hoàn toàn. Xét chương trình C đơn giản này:

point.h

```
struct Point;
struct Point* makePoint(double x, double y);
double distance (struct Point *p1, struct Point *p2);
```

point.c

```
#include "point.h"
#include <stdlib.h>
#include <math.h>

struct Point {
    double x,y;
};
struct Point* makepoint(double x, double y) {
    struct Point* p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p;
}

double distance(struct Point* p1, struct Point* p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx*dx+dy*dy);
}
```

Người dùng của `point.h` không có quyền truy cập nào vào các thành viên của `struct Point`. Chúng có thể gọi hàm `makePoint()` và hàm `distance()`, nhưng chúng hoàn

toàn không biết gì về việc triển khai của cấu trúc dữ liệu `Point` hoặc các hàm.

Đây là sự đóng gói hoàn hảo trong một ngôn ngữ không hướng đối tượng. Các lập trình viên C vẫn thường làm như vậy: khai báo trước các cấu trúc dữ liệu và hàm trong tệp header, rồi triển khai chúng trong các tệp mã nguồn. Người dùng không hề truy cập được các thành phần nằm trong những tệp triển khai đó.

Nhưng rồi C++ ra đời cùng với OO, và sự đóng gói hoàn hảo của C bị phá vỡ.

Bộ biên dịch C++, vì những lý do kỹ thuật,[1] cần các biến thành viên của một lớp phải được khai báo trong tệp header của lớp đó. Vì vậy chương trình Point của chúng ta thay đổi trông như sau:

point.h

```
class Point {
public:
    Point(double x, double y);
    double distance(const Point& p) const;

private:
    double x;
    double y;
};
```

point.cc

```
#include "point.h"
#include <math.h>

Point::Point(double x, double y)
: x(x), y(y)
{}

double Point::distance(const Point& p) const {
    double dx = x-p.x;
    double dy = y-p.y;
    return sqrt(dx*dx + dy*dy);
}
```

Khách hàng (client) của tệp header `point.h` biết về các biến thành viên `x` và `y`! Bộ biên dịch sẽ ngăn chặn việc truy cập vào chúng, nhưng client vẫn biết chúng tồn tại. Ví dụ, nếu tên của các biến thành viên đó thay đổi, tệp `point.cc` phải được biên dịch lại! Đóng gói đã bị phá vỡ.

Thật vậy, cách sửa chữa một phần cho sự đóng gói là đưa các từ khóa `public`, `private`, và `protected` vào ngôn ngữ. Tuy nhiên, đây là một thủ thuật (hack) cần thiết do nhu cầu kỹ thuật của bộ biên dịch phải thấy những biến đó trong tệp header.

Java và C# đơn giản là loại bỏ hoàn toàn sự tách biệt header/triển khai, làm suy yếu sự đóng gói thêm nữa. Trong các ngôn ngữ này, không thể tách biệt khai báo và định nghĩa của một lớp.

---

1. Bộ biên dịch C++ cần biết kích thước của các instance (thể hiện) của mỗi lớp.

Vì những lý do này, rất khó chấp nhận rằng OO phụ thuộc vào sự đóng gói mạnh. Thực tế, nhiều ngôn ngữ OO[2] có rất ít hoặc không có sự đóng gói được ép buộc.

OO dựa vào giả định rằng lập trình viên đủ kỷ luật để không tìm cách vượt qua cơ chế đóng gói dữ liệu. Dù vậy, các ngôn ngữ tự nhận hỗ trợ OO chỉ làm suy yếu tính đóng gói hoàn hảo mà ta từng có trong C.

Kế thừa (Inheritance)?

Nếu các ngôn ngữ OO không cho chúng ta sự đóng gói tốt hơn, chắc chắn chúng cho chúng ta kế thừa.

Cũng có phần như vậy. Kế thừa chỉ đơn giản là khai báo lại một nhóm biến và hàm trong một phạm vi bao quanh. Lập trình viên C[3] đã có thể làm việc này bằng tay từ lâu trước khi có ngôn ngữ OO.

Xét sự bổ sung này cho chương trình `point.h` C gốc của chúng ta:

namedPoint.h

```
struct NamedPoint;

struct NamedPoint* makeNamedPoint(double x, double y, char* name);
void setName(struct NamedPoint* np, char* name);
char* getName(struct NamedPoint* np);
```

namedPoint.c

```
#include "namedPoint.h"
#include <stdlib.h>

struct NamedPoint {
    double x,y;
    char* name;
};

struct NamedPoint* makeNamedPoint(double x, double y, char* name) {
    struct NamedPoint* p = malloc(sizeof(struct NamedPoint));
    p->x = x;
    p->y = y;
    p->name = name;
    return p;
}

void setName(struct NamedPoint* np, char* name) {
    np->name = name;
}

char* getName(struct NamedPoint* np) {
    return np->name;
```

2. Ví dụ: Smalltalk, Python, JavaScript, Lua, và Ruby.

3. Không chỉ các lập trình viên C: Hầu hết các ngôn ngữ thời đó đều có khả năng ngụy trang một cấu trúc dữ liệu thành một cấu trúc khác.

```
}
```

main.c

```
#include "point.h"
#include "namedPoint.h"
#include <stdio.h>

int main(int ac, char** av) {
    struct NamedPoint* origin = makeNamedPoint(0.0, 0.0, "origin");
    struct NamedPoint* upperRight = makeNamedPoint (1.0, 1.0, "upperRight");
    printf("distance=%f\n",
    distance(
    (struct Point*) origin,
    (struct Point*) upperRight));
}
```

Nếu bạn nhìn kỹ vào chương trình chính, bạn sẽ thấy cấu trúc dữ liệu `NamedPoint` hành động như thể nó là một dẫn xuất (derivative) của cấu trúc dữ liệu `Point`. Điều này là vì thứ tự của hai trường đầu tiên trong `NamedPoint` giống với `Point`. Tóm lại, `NamedPoint` có thể ngụy trang thành `Point` vì `NamedPoint` là một siêu tập thuần túy (pure superset) của `Point` và duy trì thứ tự của các thành viên tương ứng với `Point`.

Loại thủ thuật này là một thực hành phổ biến[4] của các lập trình viên trước kỷ nguyên ngôn ngữ OO. Thực tế, thủ thuật đó chính là cách C++ triển khai kế thừa đơn.

Vì vậy chúng ta có thể nói rằng chúng ta đã có một loại kế thừa từ lâu trước khi các ngôn ngữ OO được phát minh. Tuy nhiên phát ngôn đó không hoàn toàn đúng. Chúng ta có một thủ thuật, nhưng nó không tiện lợi như kế thừa thực sự. Hơn nữa, kế thừa đa (multiple inheritance) khó đạt được hơn nhiều bằng thủ thuật như vậy.

Lưu ý thêm rằng trong `main.c`, tôi bị buộc phải ép kiểu (cast) các đối số `NamedPoint` thành `Point`. Trong một ngôn ngữ OO thực sự, việc upcasting như vậy sẽ được ngầm hiểu (implicit).

Nói công bằng, dù ngôn ngữ OO không đem lại điều gì hoàn toàn mới, chúng khiến việc ngụy trang cấu trúc dữ liệu thuận tiện hơn đáng kể.

Tóm lại: Chúng ta không thể cho OO điểm nào cho sự đóng gói, và có lẽ nửa điểm cho kế thừa. Cho đến nay, điểm số không cao lắm.

Nhưng còn một thuộc tính nữa cần xem xét.

Đa hình (Polymorphism)?

Chúng ta có hành vi đa hình trước các ngôn ngữ OO chứ? Tất nhiên là có. Xét chương trình sao chép (copy) C đơn giản này:

---

4. Thực tế nó vẫn là như vậy.

```
#include <stdio.h>

void copy() {
    int c;
    while ((c=getchar()) != EOF)
        putchar(c);
}
```

Hàm `getchar()` đọc từ `STDIN`. Nhưng `STDIN` là thiết bị nào? Hàm `putchar()` ghi ra `STDOUT`. Nhưng thiết bị đó là gì? Hai hàm này có tính đa hình (polymorphic), nghĩa là hành vi của chúng tùy thuộc vào kiểu của `STDIN` và `STDOUT`.

Có thể xem `STDIN` và `STDOUT` như các interface kiểu Java, mỗi interface có cách triển khai cho từng thiết bị. Nhưng chương trình C này không có interface nào. Vậy lời gọi `getchar()` được chuyển đến trình điều khiển thiết bị đọc ký tự bằng cách nào?

Câu trả lời cho câu hỏi đó khá đơn giản. Hệ điều hành UNIX yêu cầu mọi trình điều khiển thiết bị IO cung cấp năm hàm chuẩn:[5] `open`, `close`, `read`, `write`, và `seek`. Chữ ký của những hàm đó phải giống hệt nhau cho mọi trình điều khiển IO.

Cấu trúc dữ liệu `FILE` chứa năm con trỏ hàm. Trong ví dụ của chúng ta, nó có thể trông như sau:

```
struct FILE {
    void (*open)(char* name, int mode);
    void (*close)();
    int (*read)();
    void (*write)(char);
    void (*seek)(long index, int mode);
};
```

Trình điều khiển IO của console định nghĩa các hàm đó rồi nạp địa chỉ của chúng vào một cấu trúc `FILE`, chẳng hạn như sau:

```
#include "file.h"

void open(char* name, int mode) {/*...*/}
void close() {/*...*/};
int read() {int c;/*...*/ return c;}
void write(char c) {/*...*/}
void seek(long index, int mode) {/*...*/}

struct FILE console = {open, close, read, write, seek};
```

Bây giờ nếu `STDIN` được định nghĩa là một `FILE*`, và nếu nó trỏ đến cấu trúc dữ liệu `console`, thì `getchar()` có thể được triển khai như sau:

```
extern struct FILE* STDIN;

int getchar() {
    return STDIN->read();
```

5. Hệ thống UNIX đa dạng; đây chỉ là một ví dụ.

```
}
```

Nói cách khác, `getchar()` đơn giản gọi hàm được trỏ đến bởi con trỏ `read` của cấu trúc dữ liệu `FILE` được trỏ đến bởi `STDIN`.

Chiêu bài đơn giản này là cơ sở cho mọi đa hình trong OO. Trong C++, ví dụ, mọi hàm ảo (virtual function) trong một lớp có một con trỏ trong một bảng gọi là vtable, và mọi lời gọi hàm ảo đều đi qua bảng đó. Hàm tạo (constructor) của các lớp dẫn xuất đơn giản nạp phiên bản của chúng cho những hàm đó vào vtable của đối tượng đang được tạo.

Tóm lại, đa hình là một ứng dụng của con trỏ hàm. Các lập trình viên đã sử dụng con trỏ hàm để đạt được hành vi đa hình kể từ khi kiến trúc Von Neumann được triển khai đầu tiên vào cuối những năm 1940. Nói cách khác, OO không cung cấp gì mới.

À, nhưng điều đó không hoàn toàn đúng. Các ngôn ngữ OO có thể không cho chúng ta đa hình, nhưng chúng đã làm cho nó an toàn hơn nhiều và tiện lợi hơn nhiều.

Vấn đề với việc sử dụng tường minh con trỏ hàm để tạo hành vi đa hình là con trỏ hàm rất nguy hiểm. Việc sử dụng như vậy được điều khiển bởi một tập các quy ước thủ công. Bạn phải nhớ tuân theo quy ước để khởi tạo những con trỏ đó. Bạn phải nhớ tuân theo quy ước để gọi tất cả các hàm của bạn qua những con trỏ đó. Nếu bất kỳ lập trình viên nào không nhớ các quy ước đó, lỗi (bug) kết quả có thể cực kỳ khó để tìm ra và loại bỏ.

Các ngôn ngữ OO loại bỏ những quy ước đó và do vậy loại bỏ các nguy cơ. Với ngôn ngữ OO, việc dùng đa hình trở nên rất dễ dàng. Khả năng này đem lại sức mạnh to lớn mà các lập trình viên C trước kia chỉ có thể mơ ước. Từ đó, ta có thể kết luận rằng OO áp đặt kỷ luật lên việc chuyển giao quyền điều khiển gián tiếp.

Sức mạnh của đa hình

Đa hình có gì hay ho? Để trải nghiệm vẻ đẹp của nó tốt hơn, hãy xem lại ví dụ chương trình sao chép (copy). Chương trình đó sẽ ra sao nếu một thiết bị IO mới được tạo ra? Giả sử chúng ta muốn dùng chương trình sao chép để sao chép dữ liệu từ một thiết bị nhận dạng chữ viết tay đến một thiết bị tổng hợp lời nói: Chúng ta cần thay đổi chương trình sao chép như thế nào để nó hoạt động với những thiết bị mới đó?

Chúng ta **không cần bất kỳ thay đổi nào**! Thực tế, chúng ta thậm chí không cần biên dịch lại chương trình sao chép. Tại sao? Vì mã nguồn của chương trình sao chép **không phụ thuộc** vào mã nguồn của các trình điều khiển IO. Miễn là các trình điều khiển IO đó triển khai năm hàm chuẩn được định nghĩa bởi `FILE`, chương trình sao chép sẽ vui vẻ sử dụng chúng.

Tóm lại, các thiết bị IO đã trở thành plugin (mô-đun mở rộng) của chương trình sao chép.

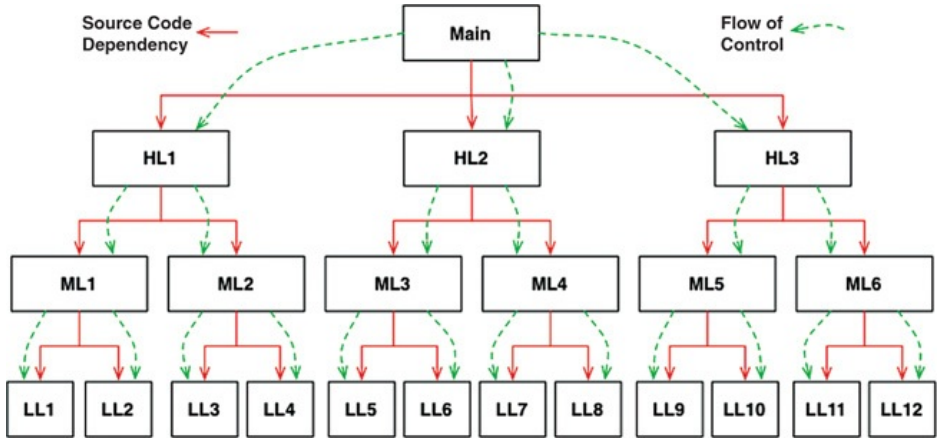
Tại sao hệ điều hành UNIX làm cho các thiết bị IO trở thành plugin? Vì chúng ta đã học, vào cuối những năm 1950, rằng chương trình của chúng ta nên độc lập thiết bị (device independent). Tại sao? Vì chúng ta đã viết nhiều chương trình phụ thuộc thiết bị, chỉ để phát hiện ra rằng chúng ta thực sự muốn những chương trình đó làm cùng một công việc nhưng dùng một thiết bị khác.

Ví dụ, chúng ta thường viết chương trình đọc dữ liệu đầu vào từ bộ thẻ (decks of cards),[6] sau đó đục bộ thẻ mới làm đầu ra. Sau đó, khách hàng của chúng ta ngừng cho chúng ta bộ thẻ và bắt đầu cho chúng ta cuộn băng từ (reels of magnetic tape). Điều này rất bất tiện, vì nó có nghĩa là viết lại phần lớn chương trình gốc. Rất thuận tiện nếu cùng một chương trình hoạt động xen kẽ với thẻ hoặc băng. Kiến trúc plugin được phát minh để hỗ trợ loại độc lập thiết bị IO này, và đã được triển khai trong hầu như mọi hệ điều hành kể từ khi nó được giới thiệu. Dù vậy, hầu hết các lập trình viên không mở rộng ý tưởng đó cho chương trình của chính họ, vì sử dụng con trỏ hàm rất nguy hiểm.

OO cho phép kiến trúc plugin được sử dụng ở bất cứ đâu, cho bất cứ thứ gì.

Đảo ngược phụ thuộc (Dependency Inversion)

Hãy tưởng tượng phần mềm trông như thế nào trước khi một cơ chế đa hình an toàn và tiện lợi có sẵn. Trong cây gọi (calling tree) điển hình, hàm `main` gọi các hàm cấp cao (high-level functions), các hàm cấp cao gọi các hàm cấp trung (mid-level functions), các hàm cấp trung gọi các hàm cấp thấp (low-level functions). Trong cây gọi đó, tuy nhiên, các phụ thuộc mã nguồn (source code dependencies) không thể tránh khỏi phải theo luồng điều khiển (Hình 5.1).



Hình 5.1

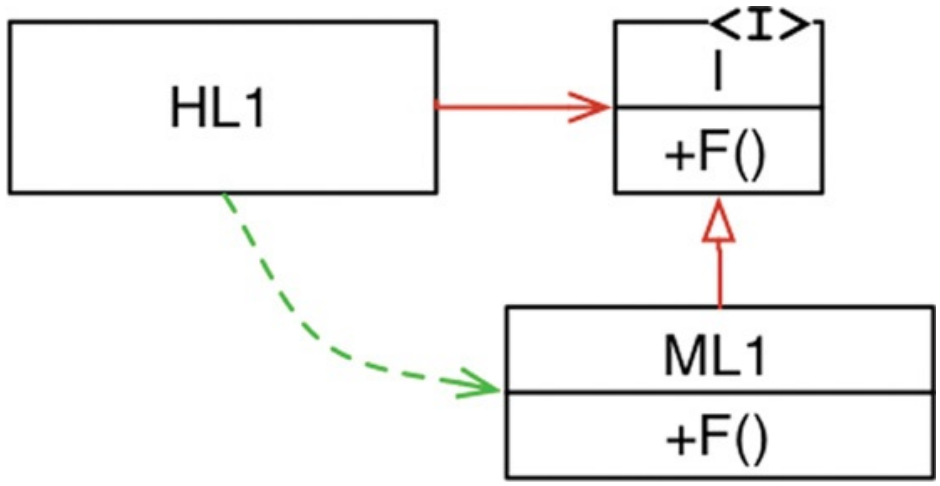
Phụ thuộc mã nguồn so với luồng điều khiển

Để `main` gọi một trong các hàm cấp cao, nó phải nhắc đến tên của mô-đun chứa hàm đó. Trong C, đây là một `#include`. Trong Java, đây là một câu lệnh `import`. Trong C#, đây là một câu lệnh `using`. Thực tế, mọi caller (hàm gọi) đều bị buộc phải nhắc đến tên của mô-đun chứa callee (hàm được gọi).

Yêu cầu này khiến kiến trúc sư phần mềm có rất ít lựa chọn, thậm chí có thể không có

6. Thẻ đục lỗ Hollerith của IBM, rộng 80 cột. Có lẽ nhiều bạn chưa từng thấy loại thẻ này, nhưng chúng rất phổ biến trong các thập niên 1950, 1960, thậm chí cả 1970.

lựa chọn nào. Hành vi của hệ thống quyết định luồng điều khiển, còn luồng điều khiển lại quyết định các phụ thuộc mã nguồn. Tuy nhiên, khi đưa đa hình vào, mọi chuyện có thể khác hẳn (Hình 5.2).



Hình 5.2
Đảo ngược phụ thuộc

Trong Hình 5.2, mô-đun `HL1` gọi hàm `F()` trong mô-đun `ML1`. Việc gọi hàm qua một interface chỉ là thủ thuật ở mã nguồn. Khi chương trình chạy, interface không tồn tại. `HL1` chỉ gọi `F()` trong `ML1`.[7]

Tuy nhiên, hãy lưu ý rằng phụ thuộc mã nguồn (mối quan hệ kế thừa) giữa `ML1` và interface `I` trỏ về hướng **ngược lại** so với luồng điều khiển. Điều này được gọi là **đảo ngược phụ thuộc (dependency inversion)**, và những hệ lụy của nó đối với kiến trúc sư phần mềm là sâu sắc.

Thực tế là các ngôn ngữ OO cung cấp đa hình an toàn và tiện lợi có nghĩa là **bất kỳ** phụ thuộc mã nguồn nào, không quan trọng nó ở đâu, đều có thể bị đảo ngược.

Hãy nhìn lại cây gọi trong Hình 5.1 cùng các phụ thuộc mã nguồn của nó. Có thể đảo chiều bất kỳ phụ thuộc nào bằng cách chèn một interface vào giữa.

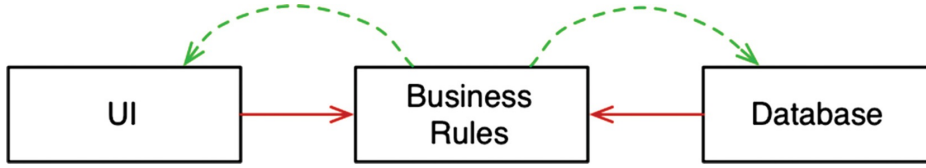
Với cách này, kiến trúc sư làm việc trên hệ thống OO có thể kiểm soát hoàn toàn hướng của mọi phụ thuộc mã nguồn. Họ không cần căn chỉnh các phụ thuộc ấy theo luồng điều khiển. Bất kể mô-đun nào gọi hay được gọi, kiến trúc sư đều có thể định hướng phụ thuộc mã nguồn theo ý mình.

Đó là sức mạnh mà OO đem lại. Ít nhất từ góc nhìn của kiến trúc sư, đó mới là bản chất của OO.

Bạn có thể làm gì với quyền lực đó? Ví dụ, bạn có thể sắp xếp lại các phụ thuộc mã nguồn của hệ thống sao cho cơ sở dữ liệu (database) và giao diện người dùng (UI) phụ thuộc

7. dù là gián tiếp.

vào các quy tắc nghiệp vụ (business rules) (Hình 5.3), thay vì ngược lại.



Hình 5.3

Cơ sở dữ liệu và giao diện người dùng phụ thuộc vào quy tắc nghiệp vụ

Điều này có nghĩa là UI và cơ sở dữ liệu có thể là các plugin cho các quy tắc nghiệp vụ. Nó có nghĩa là mã nguồn của các quy tắc nghiệp vụ **không bao giờ** nhắc đến UI hoặc cơ sở dữ liệu.

Như một hệ quả, các quy tắc nghiệp vụ, UI, và cơ sở dữ liệu có thể được biên dịch thành ba thành phần (component) hoặc đơn vị triển khai (deployment unit) riêng biệt (ví dụ: file jar, DLL, hoặc Gem file) có cùng các phụ thuộc như mã nguồn. Thành phần chứa các quy tắc nghiệp vụ sẽ **không** phụ thuộc vào các thành phần chứa UI và cơ sở dữ liệu.

Lại nữa, các quy tắc nghiệp vụ có thể được triển khai độc lập với UI và cơ sở dữ liệu. Thay đổi đối với UI hoặc cơ sở dữ liệu không cần có bất kỳ tác động nào lên các quy tắc nghiệp vụ. Những thành phần đó có thể được triển khai riêng biệt và độc lập.

Tóm lại, khi mã nguồn trong một thành phần thay đổi, **chỉ** thành phần đó cần được triển khai lại. Đây là **khả năng triển khai độc lập (independent deployability)**.

Nếu các mô-đun trong hệ thống của bạn có thể được triển khai độc lập, thì chúng có thể được phát triển độc lập bởi các đội khác nhau. Đó là **khả năng phát triển độc lập (independent developability)**.

Kết luận

OO là gì? Có nhiều ý kiến và nhiều câu trả lời. Với kiến trúc sư phần mềm, câu trả lời rất rõ: OO là khả năng dùng đa hình để kiểm soát hoàn toàn mọi phụ thuộc mã nguồn trong hệ thống. Nhờ đó, kiến trúc sư có thể tạo ra kiến trúc plugin, trong đó các mô-đun chứa chính sách cấp cao không phụ thuộc vào mô-đun chứa chi tiết cấp thấp. Các chi tiết cấp thấp được tách thành những plugin có thể phát triển và triển khai độc lập với các mô-đun chứa chính sách cấp cao.

6

Lập trình hàm



Ở nhiều phương diện, các khái niệm của lập trình hàm có từ trước cả lập trình máy tính. Mô hình này dựa chủ yếu trên λ-calculus do Alonzo Church phát minh trong thập niên 1930.

Bình phương các số nguyên (SQUARES OF INTEGERS)

Để giải thích lập trình hàm là gì, tốt nhất là xem xét một vài ví dụ. Hãy cùng tìm hiểu một bài toán đơn giản: in ra bình phương của 25 số nguyên đầu tiên.

Trong một ngôn ngữ như Java, chúng ta có thể viết như sau:

```
public class Squint {
    public static void main(String args[]) {
        for (int i=0; i<25; i++)
            System.out.println(i*i);
    }
}
```

Trong một ngôn ngữ như Clojure, một ngôn ngữ phái sinh từ Lisp và mang tính hàm, chúng ta có thể triển khai cùng một chương trình như sau:

```
(println (take 25 (map (fn [x] (* x x)) (range))))
```

Nếu bạn không biết Lisp, đoạn mã trên có thể trông hơi kỳ lạ. Hãy cùng định dạng lại một chút và thêm chú thích:

```
(println ;_____________________ In ra
  (take 25 ;___________________ 25 phần tử đầu
    (map (fn [x] (* x x)) ;___ bình phương
      (range)))) ;____________ của các số nguyên
```

Rõ ràng `println`, `take`, `map`, và `range` đều là các hàm. Trong Lisp, bạn gọi một hàm bằng cách đặt nó trong cặp ngoặc đơn. Ví dụ, `(range)` gọi hàm `range`.

Biểu thức `(fn [x] (* x x))` là một hàm ẩn danh gọi hàm nhân, truyền đối số đầu vào của nó hai lần. Nói cách khác, nó tính bình phương của đầu vào.

Nhìn lại toàn bộ biểu thức, tốt nhất là bắt đầu từ lời gọi hàm nằm sâu nhất bên trong.

- Hàm `range` trả về một danh sách không bao giờ kết thúc các số nguyên bắt đầu từ 0.
- Danh sách này được truyền vào hàm `map`, hàm này gọi hàm ẩn danh tính bình phương trên từng phần tử, tạo ra một danh sách mới không bao giờ kết thúc chứa tất cả các bình phương.
- Danh sách các bình phương được truyền vào hàm `take`, hàm này trả về một danh sách mới chỉ chứa 25 phần tử đầu tiên.
- Hàm `println` in ra đầu vào của nó, là một danh sách 25 bình phương đầu tiên của các số nguyên.

Nếu bạn thấy khái niệm "danh sách không bao giờ kết thúc" đáng sợ, đừng lo. Chỉ có 25 phần tử đầu tiên của những danh sách đó mới thực sự được tạo ra. Đó là vì không phần tử nào của một danh sách không bao giờ kết thúc được đánh giá cho đến khi nó được truy cập.

Nếu tất cả điều trên khiến bạn bối rối, bạn có thể mong chờ một thời gian tuyệt vời khi tìm hiểu về Clojure và lập trình hàm. Mục tiêu của tôi ở đây không phải dạy bạn về các chủ đề này.

Thay vào đó, mục tiêu của tôi là chỉ ra một điểm khác biệt rất đáng chú ý giữa chương trình Clojure và chương trình Java. Chương trình Java sử dụng một biến có thể thay đổi (mutable variable), tức là một biến thay đổi trạng thái trong khi chương trình thực thi. Biến đó là `i`, biến điều khiển vòng lặp. Không có biến có thể thay đổi nào tồn tại trong chương trình Clojure. Trong chương trình Clojure, các biến như `x` được khởi tạo, nhưng chúng không bao giờ bị sửa đổi.

Điều này dẫn chúng ta đến một phát biểu khiến người ta kinh ngạc: **Các biến trong ngôn ngữ hàm không thay đổi.**

Bất biến và kiến trúc (IMMUTABILITY AND ARCHITECTURE)

Tại sao điểm này lại quan trọng như một cân nhắc kiến trúc? Tại sao một kiến trúc sư lại quan tâm đến tính có thể thay đổi của biến? Câu trả lời đơn giản đến mức bất ngờ: **Tất cả các điều kiện đua (race conditions), các tình trạng bế tắc (deadlock conditions), và các vấn đề cập nhật đồng thời (concurrent update problems) đều do các biến có thể thay đổi gây ra.** Bạn không thể có điều kiện đua hay vấn đề cập nhật đồng thời nếu không biến nào từng bị cập nhật. Bạn không thể có bế tắc nếu không có khóa có thể thay đổi (mutable locks).

Nói cách khác, tất cả những vấn đề mà chúng ta đối mặt trong các ứng dụng đồng thời, tất cả những vấn đề trong các ứng dụng yêu cầu nhiều luồng (thread) và nhiều bộ xử lý, đều không thể xảy ra nếu không có biến có thể thay đổi.

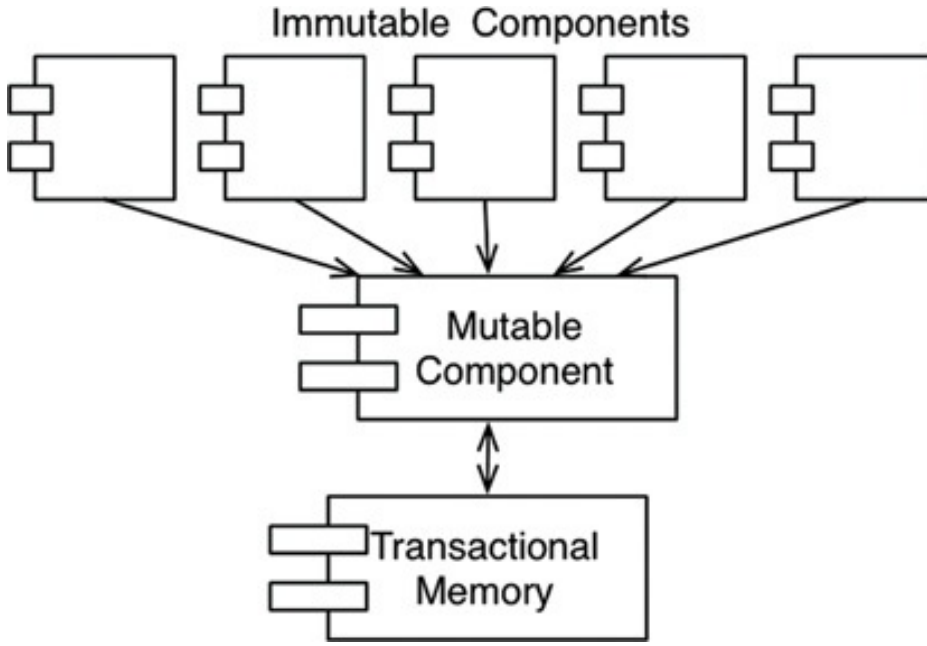
Là một kiến trúc sư, bạn nên rất quan tâm đến các vấn đề đồng thời. Bạn muốn đảm bảo rằng các hệ thống bạn thiết kế sẽ vững chắc khi có nhiều luồng và bộ xử lý. Câu hỏi bạn phải tự hỏi là liệu bất biến có thực tế không.

Câu trả lời cho câu hỏi đó là khẳng định, **nếu bạn có bộ nhớ vô hạn và tốc độ bộ xử lý vô hạn**. Nếu thiếu những tài nguyên vô hạn đó, câu trả lời phức tạp hơn một chút. Có, bất biến có thể thực tế được, **nếu chấp nhận một số sự thỏa hiệp**.

Hãy xem xét một vài sự thỏa hiệp đó.

Tách biệt trạng thái có thể thay đổi (SEGREGATION OF MUTABILITY)

Một trong những sự thỏa hiệp phổ biến nhất liên quan đến bất biến là phân tách ứng dụng, hoặc các dịch vụ trong ứng dụng, thành các thành phần có thể thay đổi và bất biến. Các thành phần bất biến thực hiện nhiệm vụ của chúng theo cách hàm thuần túy, không sử dụng bất kỳ biến có thể thay đổi nào. Các thành phần bất biến giao tiếp với một hoặc nhiều thành phần khác không thuần túy theo hàm, và cho phép trạng thái của các biến bị thay đổi (Hình 6.1).



Hình 6.1
Trạng thái biến đổi và bộ nhớ giao dịch

Vì thay đổi trạng thái làm cho các thành phần đó hứng chịu tất cả các vấn đề của đồng thời, thực hành phổ biến là sử dụng một loại bộ nhớ giao dịch (transactional memory) để bảo vệ các biến có thể thay đổi khỏi các cập nhật đồng thời và điều kiện đua.

Bộ nhớ giao dịch đơn giản chỉ xử lý các biến trong bộ nhớ theo cách tương tự như cơ sở dữ liệu xử lý các bản ghi trên đĩa.[1] Nó bảo vệ những biến đó bằng một sơ đồ dựa trên giao dịch hoặc thử lại (retry).

Một ví dụ đơn giản về cách tiếp cận này là cơ chế `atom` của Clojure:

1. Tôi biết… đĩa là gì cơ?

```
(def counter (atom 0)) ; khởi tạo counter = 0
(swap! counter inc) ; tăng counter an toàn
```

Trong mã này, biến `counter` được định nghĩa là một `atom`. Trong Clojure, `atom` là một loại biến đặc biệt mà giá trị của nó được phép thay đổi dưới những điều kiện rất kỷ luật, được hàm `swap!` thực thi.

Hàm `swap!`, như trong đoạn mã trước, nhận hai đối số: `atom` sẽ bị thay đổi, và một hàm tính giá trị mới để lưu vào `atom`. Trong ví dụ của chúng ta, `atom` counter sẽ được thay đổi thành giá trị do hàm `inc` tính toán, hàm này đơn giản chỉ tăng đối số của nó.

Chiến lược mà `swap!` sử dụng là thuật toán so sánh và hoán đổi (compare and swap) truyền thống. Giá trị của `counter` được đọc và truyền cho `inc`. Khi `inc` trả về, giá trị của `counter` bị khóa và so sánh với giá trị đã truyền cho `inc`. Nếu giá trị giống nhau, giá trị do `inc` trả về được lưu vào `counter` và khóa được giải phóng. Ngược lại, khóa được giải phóng và chiến lược được thử lại từ đầu.

Cơ chế `atom` đủ cho các ứng dụng đơn giản. Thật không may, nó không thể bảo vệ hoàn toàn khỏi các cập nhật đồng thời và bế tắc khi nhiều biến phụ thuộc lẫn nhau xuất hiện. Trong những trường hợp đó, các cơ chế phức tạp hơn có thể được sử dụng.

Điểm mấu chốt là các ứng dụng được cấu trúc tốt sẽ được phân tách thành những thành phần không thay đổi biến và những thành phần làm vậy. Loại phân tách này được hỗ trợ bằng việc sử dụng các kỷ luật phù hợp để bảo vệ những biến bị thay đổi.

Kiến trúc sư sẽ khôn ngoan nếu đẩy càng nhiều xử lý càng tốt vào các thành phần bất biến, và gạt càng nhiều mã càng tốt ra khỏi những thành phần phải cho phép thay đổi.

Nguồn sự kiện (EVENT SOURCING)

Giới hạn về bộ nhớ và sức mạnh xử lý đang nhanh chóng không còn là rào cản. Ngày nay, các bộ xử lý thực thi hàng tỷ lệnh mỗi giây và có hàng tỷ byte RAM là điều phổ biến. Càng nhiều bộ nhớ, máy móc càng nhanh, chúng ta càng ít cần trạng thái có thể thay đổi.

Lấy một ví dụ đơn giản, hãy tưởng tượng một ứng dụng ngân hàng duy trì số dư tài khoản của khách hàng. Nó thay đổi những số dư đó khi các giao dịch nạp và rút được thực thi.

Bây giờ hãy tưởng tượng thay vì lưu trữ số dư tài khoản, chúng ta chỉ lưu trữ các giao dịch. Bất cứ khi nào ai đó muốn biết số dư của một tài khoản, chúng ta đơn giản chỉ cộng tổng tất cả các giao dịch của tài khoản đó, từ đầu thời gian. Sách lược này không đòi hỏi bất kỳ biến có thể thay đổi nào.

Rõ ràng, cách tiếp cận này nghe có vẻ vô lý. Theo thời gian, số lượng giao dịch sẽ tăng không giới hạn, và sức mạnh xử lý cần thiết để tính tổng sẽ trở nên không thể chấp nhận được. Để làm cho sách lược này hoạt động mãi mãi, chúng ta sẽ cần bộ nhớ vô hạn và sức mạnh xử lý vô hạn.

Nhưng có lẽ chúng ta không cần sách lược này hoạt động mãi mãi. Và có lẽ chúng ta có đủ bộ nhớ và đủ sức mạnh xử lý để sách lược này hoạt động trong vòng đời hợp lý của ứng dụng.

Đây chính là ý tưởng đằng sau **nguồn sự kiện (event sourcing)**.[2] Nguồn sự kiện là một chiến lược trong đó chúng ta lưu trữ các giao dịch, chứ không lưu trạng thái. Khi trạng thái được yêu cầu, chúng ta đơn giản áp dụng tất cả các giao dịch từ đầu thời gian.

Tất nhiên, chúng ta có thể lấy đường tắt. Ví dụ, chúng ta có thể tính toán và lưu trạng thái mỗi nửa đêm. Khi đó, khi thông tin trạng thái được yêu cầu, chúng ta chỉ cần tính toán các giao dịch kể từ nửa đêm.

Xét về bộ nhớ dữ liệu cần thiết cho sách lược này: chúng ta sẽ cần rất nhiều. Thực tế, bộ nhớ ngoại tuyến đã tăng trưởng nhanh đến mức chúng ta hiện coi hàng nghìn tỷ byte là nhỏ, nên chúng ta có rất nhiều.

Quan trọng hơn, **không bao giờ có thứ gì bị xóa hoặc cập nhật** từ kho dữ liệu như vậy. Hệ quả là, ứng dụng của chúng ta không phải là CRUD (Create, Read, Update, Delete); chúng chỉ là CR (Create, Read). Ngoài ra, vì không có cập nhật hay xóa xảy ra trong kho dữ liệu, không thể có bất kỳ vấn đề cập nhật đồng thời nào.

Nếu chúng ta có đủ bộ nhớ và đủ sức mạnh bộ xử lý, chúng ta có thể làm cho ứng dụng của mình hoàn toàn bất biến, và do đó, thuần túy theo kiểu hàm.

Nếu điều này vẫn nghe có vẻ vô lý, có thể sẽ giúp ích nếu bạn nhớ rằng **đúng chính cách hoạt động của hệ thống kiểm soát mã nguồn (source code control system) của bạn.**

Kết luận (CONCLUSION)

Tóm lại:

- Lập trình có cấu trúc là kỷ luật được áp đặt lên việc chuyển giao quyền điều khiển trực tiếp.
- Lập trình hướng đối tượng là kỷ luật được áp đặt lên việc chuyển giao quyền điều khiển gián tiếp.
- Lập trình hàm là kỷ luật được áp đặt lên việc gán biến.

Mỗi mô hình lập trình trong số ba này đều đã lấy đi một thứ gì đó từ chúng ta. Mỗi cái đều hạn chế một khía cạnh nào đó của cách chúng ta viết mã. Không cái nào trong số chúng bổ sung quyền năng hay khả năng cho chúng ta.

Điều chúng ta học được trong nửa thế kỷ qua là **điều không nên làm**.

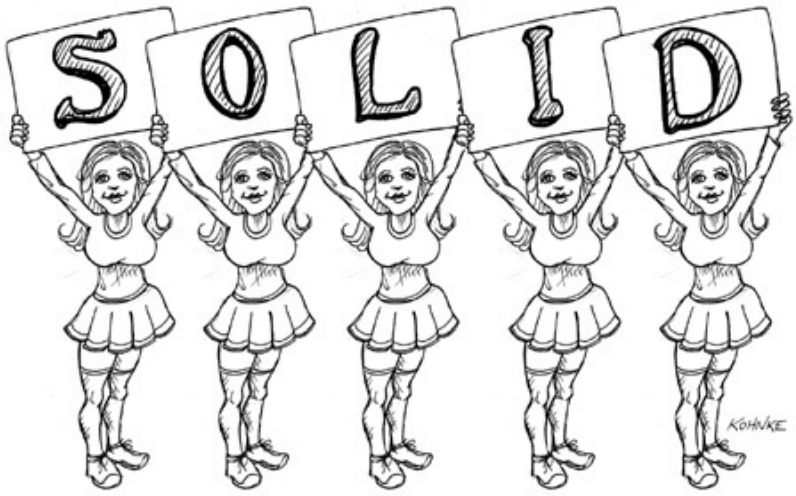
Với sự nhận thức đó, chúng ta phải đối mặt với một thực tế khó chịu: **Phần mềm không phải là một công nghệ tiến bộ nhanh chóng.** Quy tắc của phần mềm ngày nay y hệt như năm 1946, khi Alan Turing viết nên đoạn mã đầu tiên được thực thi trên một máy tính điện tử. Công cụ đã thay đổi, phần cứng đã thay đổi, nhưng bản chất của phần mềm vẫn giữ nguyên.

Phần mềm, nguyên liệu của các chương trình máy tính, bao gồm trình tự (sequence), lựa chọn (selection), lặp (iteration), và gián tiếp (indirection). Không hơn. Không kém.

---

2. Cảm ơn Greg Young đã dạy tôi khái niệm này.

Phần III: Các nguyên tắc thiết kế



Hệ thống phần mềm tốt bắt đầu từ mã nguồn sạch. Về một mặt, nếu những viên gạch không được làm tốt, thì kiến trúc của tòa nhà không còn quan trọng lắm. Về mặt khác, bạn có thể tạo ra một hỗn độn đáng kể ngay cả với những viên gạch làm tốt. Chính lúc này các nguyên tắc SOLID phát huy tác dụng.

Các nguyên tắc SOLID cho chúng ta biết cách sắp xếp các hàm và cấu trúc dữ liệu thành các lớp, và những lớp đó nên được kết nối với nhau như thế nào. Việc dùng từ "lớp" (class) không ngụ ý rằng các nguyên tắc này chỉ áp dụng cho phần mềm hướng đối tượng. Một lớp đơn giản chỉ là một nhóm hàm và dữ liệu được gắn kết chặt chẽ. Mọi hệ thống phần mềm đều có những nhóm như vậy, dù chúng có được gọi là lớp hay không. Các nguyên tắc SOLID áp dụng cho những nhóm đó.

Mục tiêu của các nguyên tắc là tạo ra các cấu trúc phần mềm trung cấp mà:

- Chịu đựng được sự thay đổi,
- Dễ hiểu, và
- Là nền tảng của các thành phần (component) có thể dùng trong nhiều hệ thống phần mềm.

Thuật ngữ "trung cấp" (mid-level) ám chỉ việc các nguyên tắc này được áp dụng bởi các lập trình viên làm việc ở cấp độ mô-đun. Chúng được áp dụng ngay trên cấp độ mã nguồn và giúp định nghĩa những loại cấu trúc phần mềm được sử dụng bên trong các mô-đun và thành phần.

Cũng giống như có thể tạo ra một hỗn độn đáng kể với những viên gạch làm tốt, ta cũng có thể tạo ra một hỗn độn trải rộng toàn hệ thống với những thành phần trung cấp được thiết kế tốt. Vì lý do này, sau khi đã trình bày xong các nguyên tắc SOLID, chúng ta sẽ chuyển sang những đối ứng của chúng trong thế giới thành phần (component), và sau đó là các nguyên tắc của kiến trúc cấp cao.

Lịch sử của các nguyên tắc SOLID đã kéo dài nhiều năm. Tôi bắt đầu tập hợp chúng vào cuối thập niên 1980, khi tranh luận về nguyên tắc thiết kế phần mềm với những người khác trên USENET, một mạng xã hội sơ khai. Theo thời gian, các nguyên tắc thay đổi: một số bị loại bỏ, một số được gộp lại, số khác được bổ sung. Nhóm nguyên tắc cuối cùng ổn định vào đầu những năm 2000, dù tôi trình bày chúng theo một thứ tự khác.

Khoảng năm 2004, Michael Feathers gửi email cho tôi và chỉ ra rằng nếu sắp xếp lại các nguyên tắc, chữ cái đầu sẽ ghép thành từ SOLID. Từ đó, các nguyên tắc SOLID ra đời.

Những chương tiếp theo sẽ mô tả từng nguyên tắc kỹ hơn. Dưới đây là tóm tắt điều hành:

- **SRP: Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle)** Một hệ quả chủ động của định luật Conway: Cấu trúc tốt nhất cho một hệ thống phần mềm chịu ảnh hưởng lớn bởi cấu trúc xã hội của tổ chức sử dụng nó, sao cho mỗi mô-đun phần mềm có một, và chỉ có một, lý do để thay đổi.
- **OCP: Nguyên tắc Đóng-Mở (Open-Closed Principle)** Bertrand Meyer làm cho nguyên tắc này nổi tiếng trong những năm 1980. Ý chính là để hệ thống phần mềm dễ thay đổi, chúng phải được thiết kế để cho phép hành vi của những hệ thống đó được thay đổi bằng cách thêm mã mới, thay vì sửa đổi mã hiện có.
- **LSP: Nguyên tắc Thay thế Liskov (Liskov Substitution Principle)** Định nghĩa nổi tiếng của Barbara Liskov về kiểu con (subtype), từ năm 1988. Tóm lại, nguyên tắc này nói rằng để xây dựng hệ thống phần mềm từ các bộ phận có thể hoán đổi cho nhau, những bộ phận đó phải tuân thủ một hợp đồng cho phép chúng được thay thế lẫn nhau.
- **ISP: Nguyên tắc Tách biệt Giao diện (Interface Segregation Principle)** Nguyên tắc này khuyên các nhà thiết kế phần mềm tránh phụ thuộc vào những thứ mà họ không sử dụng.
- **DIP: Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle)** Mã triển khai chính sách cấp cao không nên phụ thuộc vào mã triển khai chi tiết cấp thấp. Ngược lại, chi tiết nên phụ thuộc vào chính sách.

Các nguyên tắc này đã được mô tả chi tiết trong nhiều ấn bản khác nhau[1] qua nhiều năm. Những chương tiếp theo sẽ tập trung vào những hàm ý kiến trúc của các nguyên tắc thay vì lặp lại những thảo luận chi tiết đó. Nếu bạn chưa quen thuộc với các nguyên tắc này, nội dung tiếp theo không đủ để hiểu chúng một cách chi tiết và bạn nên học chúng trong các tài liệu được trích dẫn ở chân trang.

1. Ví dụ, *Agile Software Development, Principles, Patterns, and Practices*, Robert C. Martin, Prentice Hall, 2002, `http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod`, và `https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)` (hoặc chỉ cần google SOLID).

7

SRP: Nguyên tắc trách nhiệm đơn nhất



Trong số tất cả các nguyên tắc SOLID, Nguyên tắc Trách nhiệm Đơn nhất (SRP) có lẽ là nguyên tắc được hiểu lầm nhiều nhất. Điều này có lẽ vì nó có một cái tên đặc biệt không phù hợp. Quá dễ dàng để các lập trình viên nghe cái tên và giả định rằng nó có nghĩa là mọi mô-đun nên chỉ làm một việc duy nhất.

Đừng nhầm lẫn: có một nguyên tắc nói rằng hàm nên làm một việc, và chỉ một việc. Ta áp dụng nguyên tắc đó khi tái cấu trúc hàm lớn thành các hàm nhỏ hơn, tức ở cấp thấp nhất. Nhưng nguyên tắc ấy không nằm trong SOLID. Đó cũng không phải SRP.

Trước đây, SRP thường được phát biểu như sau:

> *Một mô-đun nên có một, và chỉ một, lý do để thay đổi.*

Hệ thống phần mềm được thay đổi để thỏa mãn người dùng và các bên liên quan; những người dùng và bên liên quan đó chính là "lý do để thay đổi" mà nguyên tắc đề cập đến. Thực tế, chúng ta có thể diễn đạt lại nguyên tắc như sau:

> *Một mô-đun nên chịu trách nhiệm đối với một, và chỉ một, người dùng hoặc bên liên quan.*

Thật không may, "người dùng" và "bên liên quan" không phải là những từ thật phù hợp ở đây. Có thể nhiều người dùng hoặc bên liên quan cùng muốn hệ thống được thay đổi theo một cách. Thực ra, ta đang nói đến một nhóm, gồm một hay nhiều người yêu cầu thay đổi đó. Ta gọi nhóm ấy là một **tác nhân (actor)**.

Như vậy, phiên bản cuối cùng của SRP là:

> *Một mô-đun nên chịu trách nhiệm đối với một, và chỉ một, tác nhân.*

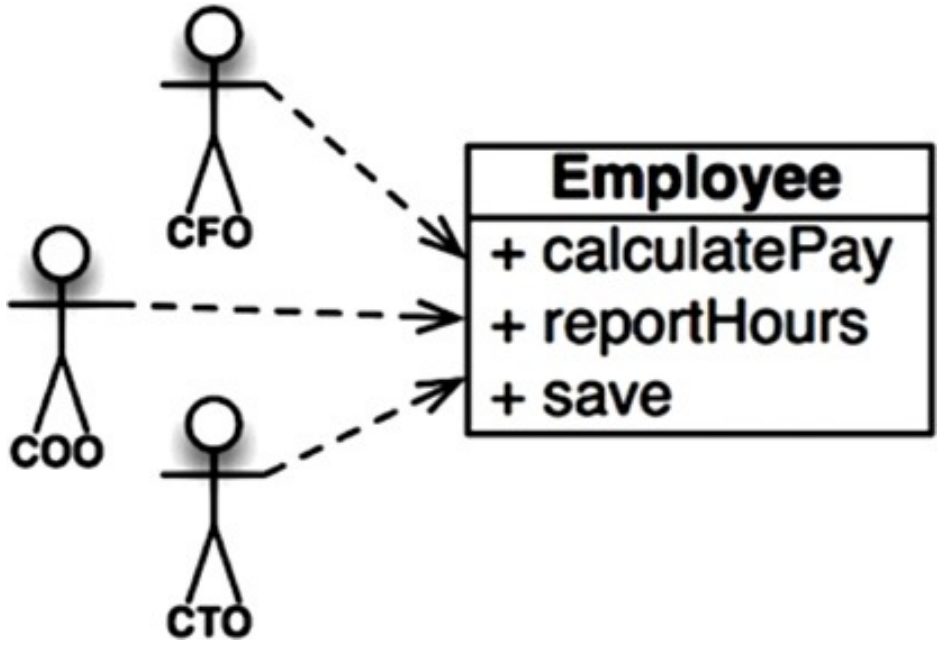
Bây giờ, chúng ta hiểu từ "mô-đun" (module) theo ý gì? Định nghĩa đơn giản nhất chỉ là một tệp nguồn (source file). Hầu như mọi lúc định nghĩa đó đều hoạt động tốt. Một số ngôn ngữ và môi trường phát triển, tuy nhiên, không dùng tệp nguồn để chứa mã của họ. Trong những trường hợp đó, một mô-đun chỉ là một tập hợp các hàm và cấu trúc dữ liệu có sự gắn kết (cohesive).

Từ "gắn kết" hàm ý SRP. Sự gắn kết là lực kết nối mã chịu trách nhiệm cho một tác nhân duy nhất.

Cách tốt nhất để hiểu nguyên tắc này có lẽ là nhìn vào các triệu chứng của việc vi phạm nó.

Triệu chứng 1: Trùng lặp ngẫu nhiên (Accidental Duplication)

Ví dụ yêu thích của tôi là lớp `Employee` (Nhân viên) từ một ứng dụng tính lương. Nó có ba phương thức: `calculatePay()` (tính lương), `reportHours()` (báo cáo giờ làm), và `save()` (lưu) (Hình 7.1).



Hình 7.1
Lớp Employee

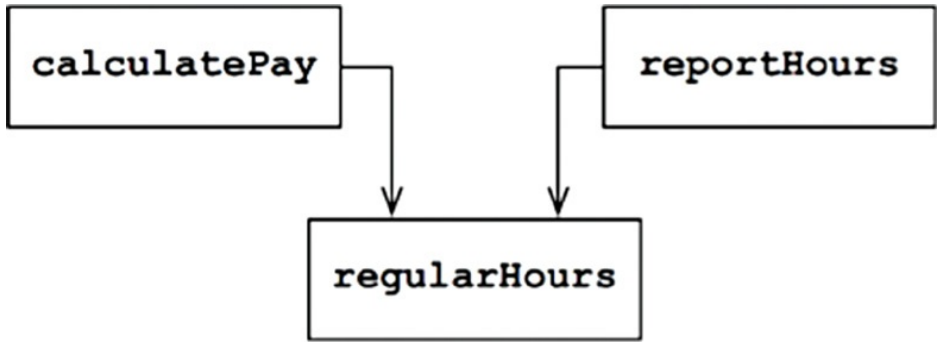
Lớp này vi phạm SRP vì ba phương thức đó chịu trách nhiệm cho ba tác nhân rất khác nhau.

- Phương thức `calculatePay()` được quy định bởi bộ phận kế toán, bộ phận này báo cáo cho CFO (Giám đốc tài chính).
- Phương thức `reportHours()` được quy định và sử dụng bởi bộ phận nhân sự (human resources), bộ phận này báo cáo cho COO (Giám đốc điều hành).
- Phương thức `save()` được quy định bởi các quản trị cơ sở dữ liệu (DBA), những người báo cáo cho CTO (Giám đốc công nghệ).

Bằng cách đặt mã nguồn của ba phương thức này trong cùng một lớp `Employee`, các nhà phát triển đã khiến ba tác nhân phụ thuộc lẫn nhau. Vì thế, hành động của đội ngũ CFO có thể ảnh hưởng đến công việc mà đội ngũ COO dựa vào.

Ví dụ, giả sử hàm `calculatePay()` và hàm `reportHours()` chia sẻ một thuật toán chung để tính giờ làm không tăng ca (non-overtime hours). Giả sử thêm rằng các nhà phát

triển, những người cẩn thận không lặp lại mã, đã đặt thuật toán đó vào một hàm tên là `regularHours()` (Hình 7.2).



Hình 7.2
Thuật toán chung

Bây giờ giả sử đội ngũ CFO quyết định rằng cách tính giờ không tăng ca cần được điều chỉnh. Ngược lại, đội ngũ COO ở bộ phận nhân sự *không* muốn điều chỉnh cụ thể đó vì họ sử dụng giờ không tăng ca cho một mục đích khác.

Một nhà phát triển được giao làm thay đổi, và thấy hàm `regularHours()` tiện lợi được gọi bởi phương thức `calculatePay()`. Thật không may, nhà phát triển đó không nhận ra hàm này *cũng* được gọi bởi hàm `reportHours()`.

Nhà phát triển thực hiện thay đổi yêu cầu và kiểm tra cẩn thận. Đội ngũ CFO xác nhận hàm mới hoạt động như mong muốn, và hệ thống được triển khai.

Đội ngũ COO không biết chuyện đó đang xảy ra. Nhân viên nhân sự vẫn dùng các báo cáo do `reportHours()` tạo ra, nhưng giờ đây chúng chứa số liệu sai. Cuối cùng vấn đề bị phát hiện. COO nổi giận vì dữ liệu sai đã làm ngân sách của ông thiệt hại hàng triệu đô-la.

Chúng ta đều từng thấy chuyện như thế xảy ra. Những vấn đề này xuất hiện vì mã mà các tác nhân khác nhau phụ thuộc vào bị đặt quá gần nhau. SRP yêu cầu tách riêng phần mã mà các tác nhân khác nhau dựa vào.

Triệu chứng 2: Hợp nhất (Merges)

Không khó để thấy rằng các tệp nguồn chứa nhiều phương thức khác nhau thường xuyên phải hợp nhất. Điều này càng dễ xảy ra khi các phương thức phục vụ những tác nhân khác nhau.

Ví dụ, giả sử đội ngũ DBA của CTO quyết định nên có một thay đổi lược đồ (schema) đơn giản cho bảng `Employee` của cơ sở dữ liệu. Giả sử thêm rằng đội ngũ nhân viên nhân sự của COO quyết định họ cần thay đổi định dạng của báo cáo giờ.

Hai nhà phát triển khác nhau, có thể thuộc hai đội khác nhau, check out lớp `Employee` rồi bắt đầu sửa đổi. Thật không may, các thay đổi của họ xung đột và phải hợp nhất.

Hẳn không cần nói thêm rằng các lần hợp nhất đều rủi ro. Công cụ ngày nay khá tốt,

nhưng không công cụ nào xử lý được mọi trường hợp. Cuối cùng vẫn luôn có rủi ro.

Trong ví dụ của chúng ta, lần hợp nhất khiến cả CTO và COO đều phải chịu rủi ro. Không thể loại trừ khả năng CFO cũng bị ảnh hưởng.

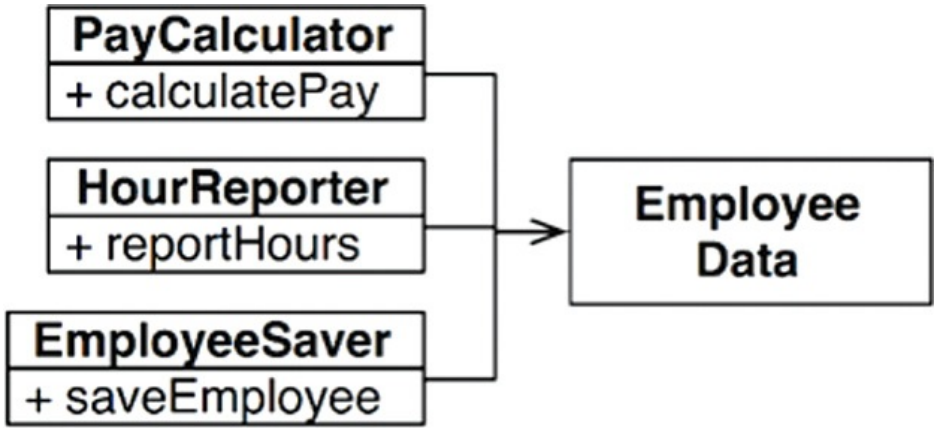
Có nhiều triệu chứng khác mà chúng ta có thể điều tra, nhưng tất cả đều liên quan đến việc nhiều người thay đổi cùng một tệp nguồn vì những lý do khác nhau.

Một lần nữa, cách tránh vấn đề này là tách riêng phần mã hỗ trợ các tác nhân khác nhau.

Giải pháp (Solutions)

Có nhiều giải pháp khác nhau cho vấn đề này. Mỗi giải pháp di chuyển các hàm vào các lớp khác nhau.

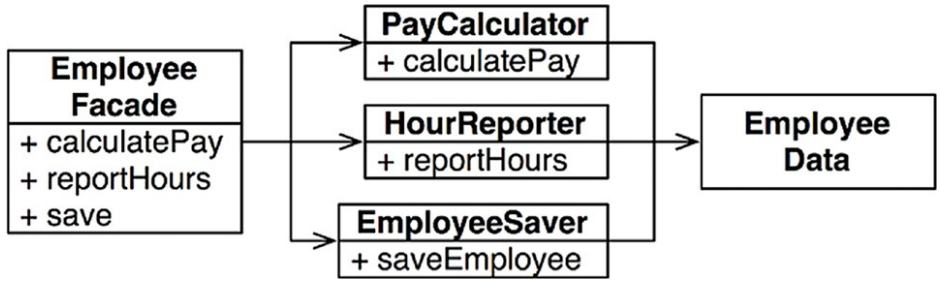
Cách rõ ràng nhất để giải quyết vấn đề có lẽ là tách dữ liệu khỏi các hàm. Ba lớp dùng chung `EmployeeData`, một cấu trúc dữ liệu đơn giản không có phương thức nào (Hình 7.3). Mỗi lớp chỉ chứa mã cần thiết cho chức năng riêng. Ba lớp không được phép biết về nhau. Nhờ vậy, tránh được việc vô tình tạo mã trùng lặp.



Hình 7.3

Ba lớp không biết về nhau

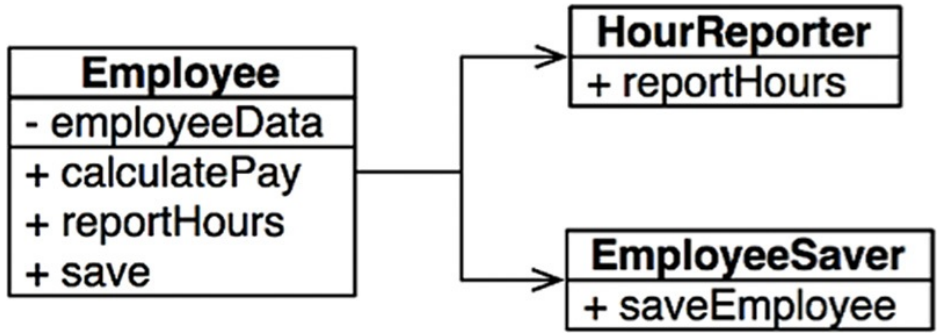
Nhược điểm của giải pháp này là các nhà phát triển phải khởi tạo và quản lý ba lớp. Một cách phổ biến để giải quyết tình thế đó là dùng mẫu Facade (Hình 7.4).



Hình 7.4
Mẫu Facade

`EmployeeFacade` chứa rất ít mã. Nó chịu trách nhiệm khởi tạo các lớp chứa hàm rồi ủy quyền công việc cho chúng.

Một số nhà phát triển thích giữ các quy tắc nghiệp vụ quan trọng nhất gần với dữ liệu. Điều này có thể làm được bằng cách giữ phương thức quan trọng nhất trong lớp `Employee` ban đầu, rồi dùng lớp đó làm Facade cho các chức năng phụ hơn (Hình 7.5).



Hình 7.5
Phương thức quan trọng nhất được giữ trong lớp Employee ban đầu và dùng làm Facade cho các chức năng phụ

Bạn có thể phản đối các giải pháp này với lý do rằng mỗi lớp sẽ chỉ chứa một hàm. Điều này hầu như không đúng. Số lượng hàm cần thiết để tính lương, tạo báo cáo, hoặc lưu dữ liệu trong mỗi trường hợp có thể lớn. Mỗi lớp trong số đó sẽ có nhiều phương thức private (riêng tư).

Mỗi lớp chứa một họ các phương thức như vậy là một phạm vi (scope). Bên ngoài phạm vi đó, không ai biết đến sự tồn tại của các thành viên private trong họ đó.

Kết luận

SRP nói về hàm và lớp, nhưng nó còn xuất hiện dưới hình thức khác ở hai cấp độ nữa. Ở cấp thành phần, nó trở thành Nguyên tắc Đóng chung (Common Closure Principle). Ở cấp kiến trúc, nó trở thành Trục Thay đổi (Axis of Change), yếu tố tạo ra các Ranh giới Kiến trúc (Architectural Boundaries). Các chương sau sẽ bàn về những ý tưởng này.

8

OCP: Nguyên tắc đóng-mở



Nguyên tắc đóng-mở (OCP) được Bertrand Meyer đặt ra năm 1988.[1] Nguyên tắc phát biểu:

Một tạo phẩm phần mềm nên mở cho việc mở rộng nhưng đóng cho việc sửa đổi.

Nói cách khác, hành vi của một tạo phẩm phần mềm nên có thể mở rộng mà không phải

1. Bertrand Meyer. *Object Oriented Software Construction*, Prentice Hall, 1988, tr. 23.

sửa đổi tạo phẩm đó.

Đây, tất nhiên, là lý do cơ bản nhất khiến chúng ta học kiến trúc phần mềm. Rõ ràng, nếu những mở rộng đơn giản của yêu cầu ép buộc phải thay đổi mã lớn trong phần mềm, thì các kiến trúc sư của hệ thống phần mềm đó đã thất bại một cách ngoạn mục. Hầu hết học viên thiết kế phần mềm nhận ra OCP là một nguyên tắc hướng dẫn họ trong thiết kế lớp và mô-đun. Nhưng nguyên tắc này mang ý nghĩa lớn hơn nữa khi xét ở cấp độ các thành phần kiến trúc.

Một thử nghiệm tư duy sẽ làm rõ điều này.

Một thử nghiệm tư duy

Hãy tưởng tượng, chỉ trong giây lát, rằng chúng ta có một hệ thống hiển thị tóm tắt tài chính trên một trang web. Dữ liệu trên trang có thể cuộn, và các số âm được hiển thị bằng màu đỏ.

Bây giờ hãy tưởng tượng các bên liên quan yêu cầu chuyển thông tin cùng loại đó thành một báo cáo để in trên máy in đen trắng. Báo cáo cần được phân trang đúng cách, có tiêu đề trang, chân trang và nhãn cột thích hợp. Các số âm nên được bao quanh bởi dấu ngoặc đơn.

Rõ ràng, một số mã mới phải được viết. Nhưng bao nhiêu mã cũ sẽ phải thay đổi?

Một kiến trúc phần mềm tốt sẽ giảm lượng mã thay đổi xuống mức tối thiểu. Lý tưởng là không thay đổi gì cả.

Làm sao? Bằng cách tách đúng các thứ thay đổi vì lý do khác nhau (Nguyên tắc trách nhiệm đơn nhất), rồi tổ chức các phụ thuộc giữa chúng đúng cách (Nguyên tắc đảo ngược phụ thuộc).

Áp dụng SRP, chúng ta có thể đến được khung nhìn luồng dữ liệu như trong Hình 8.1. Một thủ tục phân tích kiểm tra dữ liệu tài chính và sinh ra dữ liệu có thể báo cáo, dữ liệu đó sau đó được định dạng thích hợp bởi hai tiến trình báo cáo. **Hình 8.1** Áp dụng SRP

Điểm mấu chốt ở đây là việc tạo báo cáo bao gồm hai trách nhiệm riêng biệt: tính toán dữ liệu báo cáo, và trình bày dữ liệu đó dưới dạng thân thiện với web và máy in.

Sau khi đã tách biệt, chúng ta cần tổ chức các phụ thuộc mã nguồn để đảm bảo thay đổi ở một trách nhiệm không gây thay đổi ở trách nhiệm kia. Cũng như vậy, tổ chức mới nên đảm bảo hành vi có thể mở rộng mà không cần sửa đổi.

Chúng ta đạt được điều này bằng cách phân chia các tiến trình thành các lớp, và tách các lớp đó thành các thành phần, như đường kẻ đôi trong sơ đồ Hình 8.2. Trong hình này, thành phần góc trên bên trái là Controller (Bộ điều khiển). Góc trên bên phải là Interactor (Bộ tương tác). Góc dưới bên phải là Database (Cơ sở dữ liệu). Cuối cùng, góc dưới bên trái là bốn thành phần đại diện cho các Presenter (Bộ trình bày) và View (Giao diện). **Hình 8.2** Phân chia tiến trình thành các lớp và tách các lớp thành các thành phần

Các lớp được đánh dấu `<I>` là giao diện; các lớp đánh dấu `<DS>` là cấu trúc dữ liệu. Mũi tên mở là quan hệ sử dụng. Mũi tên kín là quan hệ triển khai hoặc kế thừa.

Điều đầu tiên cần nhận thấy là tất cả các phụ thuộc đều là phụ thuộc mã nguồn. Một mũi tên chỉ từ lớp A đến lớp B nghĩa là mã nguồn của lớp A nhắc đến tên lớp B, nhưng lớp

B không nhắc đến gì về lớp A. Do đó, trong Hình 8.2, `FinancialDataMapper` biết đến `FinancialDataGateway` qua quan hệ triển khai, nhưng `FinancialGateway` không biết bất cứ điều gì về `FinancialDataMapper`.

Điều tiếp theo cần nhận thấy là mỗi đường kẻ đôi chỉ bị cắt theo một chiều. Điều này nghĩa là tất cả các quan hệ giữa các thành phần đều đơn hướng, như đồ thị thành phần trong Hình 8.3. Các mũi tên này chỉ về phía các thành phần chúng ta muốn bảo vệ khỏi sự thay đổi. **Hình 8.3** Các quan hệ giữa các thành phần là đơn hướng

Để tôi nói lại lần nữa: Nếu thành phần A nên được bảo vệ khỏi các thay đổi trong thành phần B, thì thành phần B nên phụ thuộc vào thành phần A.

Chúng ta muốn Interactor được bảo vệ khỏi mọi thay đổi, bất kể chúng đến từ đâu.

Interactor nằm ở vị trí phù hợp nhất với OCP. Thay đổi Database, hoặc Controller, hoặc Presenter, hoặc View sẽ không có tác động nào đến Interactor.

Tại sao Interactor lại được đặc quyền như vậy? Vì nó chứa các quy tắc nghiệp vụ. Interactor chứa các chính sách cấp cao nhất của ứng dụng. Tất cả các thành phần khác lo việc ngoại vi. Interactor lo việc cốt lõi.

Dù Controller là ngoại vi đối với Interactor, nó vẫn là cốt lõi đối với Presenter và View. Và trong khi Presenter có thể là ngoại vi đối với Controller, chúng lại là cốt lõi đối với View.

Chú ý cách này tạo ra một thứ bậc bảo vệ dựa trên khái niệm "cấp độ". Interactor là khái niệm cấp cao nhất, nên được bảo vệ nhất. View nằm trong số các khái niệm cấp thấp nhất, nên được bảo vệ kém nhất. Presenter cấp cao hơn View, nhưng thấp hơn Controller hoặc Interactor.

Đây là cách OCP hoạt động ở cấp độ kiến trúc. Kiến trúc sư tách biệt chức năng dựa trên *cách*, *lý do* và *thời điểm* xảy ra thay đổi, rồi tổ chức các chức năng đã tách biệt thành một thứ bậc thành phần. Các thành phần cấp cao hơn trong thứ bậc đó được bảo vệ khỏi những thay đổi ở các thành phần cấp thấp hơn.

Kiểm soát chiều

Nếu thiết kế lớp ở trên khiến bạn hoảng hốt, hãy nhìn lại. Phần lớn độ phức tạp trong sơ đồ đó nhằm đảm bảo các phụ thuộc giữa các thành phần chỉ đúng chiều.

Ví dụ, giao diện `FinancialDataGateway` nằm giữa `FinancialReportGenerator` và `FinancialDataMapper` để đảo ngược phụ thuộc vốn sẽ hướng từ thành phần Interactor sang thành phần Database. Điều tương tự cũng đúng với giao diện `FinancialReportPresenter` và hai giao diện View.

Che giấu thông tin

Giao diện `FinancialReportRequester` phục vụ mục đích khác. Nó ở đó để bảo vệ `FinancialReportController` khỏi việc biết quá nhiều về nội bộ của Interactor. Nếu giao diện đó không có, Controller sẽ có phụ thuộc bắc cầu đến `FinancialEntities`.

Phụ thuộc bắc cầu vi phạm nguyên tắc chung là các thực thể phần mềm không nên phụ thuộc vào những thứ chúng không sử dụng trực tiếp. Chúng ta sẽ gặp nguyên tắc đó lại khi nói về Nguyên tắc phân tách giao diện và Nguyên tắc tái sử dụng chung.

Vì vậy, dù ưu tiên đầu tiên là bảo vệ Interactor khỏi các thay đổi của Controller, chúng ta cũng muốn bảo vệ Controller khỏi các thay đổi của Interactor bằng cách che giấu nội bộ của Interactor.

Kết luận

OCP là một trong những lực đẩy đằng sau kiến trúc của các hệ thống. Mục tiêu là làm cho hệ thống dễ dàng mở rộng mà không gánh chịu tác động thay đổi lớn. Mục tiêu này đạt được bằng cách phân chia hệ thống thành các thành phần, và sắp xếp các thành phần đó thành một thứ bậc phụ thuộc bảo vệ các thành phần cấp cao khỏi các thay đổi trong các thành phần cấp thấp.

9

LSP: Nguyên tắc thay thế Liskov



Năm 1988, Barbara Liskov đã viết điều sau như một cách để định nghĩa các kiểu con:

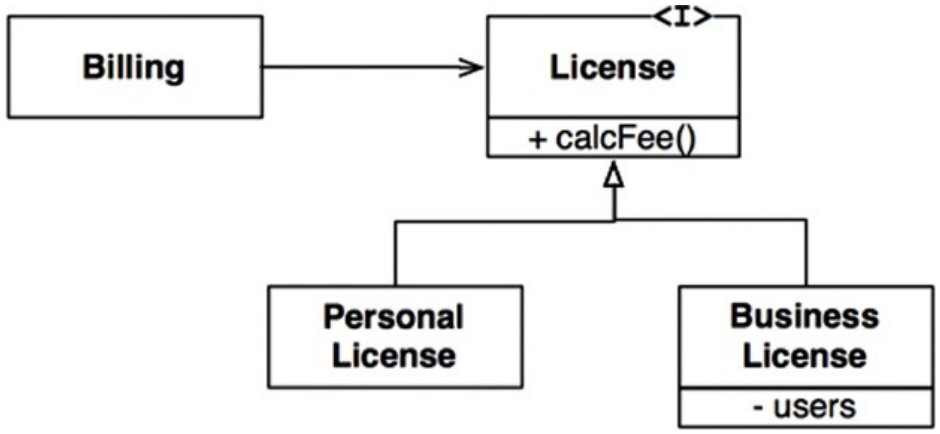
> Điều mong muốn ở đây là một tính chất thay thế như sau: Nếu với mỗi đối tượng o1 kiểu S thì tồn tại một đối tượng o2 kiểu T sao cho với tất cả các chương trình P được định nghĩa dựa trên T, hành vi của P không thay đổi khi o1 được dùng thay cho o2 thì S là một kiểu con của T.[1]

Để hiểu ý tưởng này, được gọi là **Nguyên tắc thay thế Liskov (LSP)**, hãy xem xét một số ví dụ.

1. Barbara Liskov, "Data Abstraction and Hierarchy," SIGPLAN Notices 23, 5 (Tháng 5 năm 1988).

Dẫn dắt việc sử dụng kế thừa

Hãy tưởng tượng rằng chúng ta có một lớp tên là `License`, như minh họa trong Hình 9.1. Lớp này có một phương thức tên `calcFee()`, được gọi bởi ứng dụng Billing. Có hai "kiểu con" của `License`: `PersonalLicense` và `BusinessLicense`. Chúng sử dụng các thuật toán khác nhau để tính phí giấy phép.

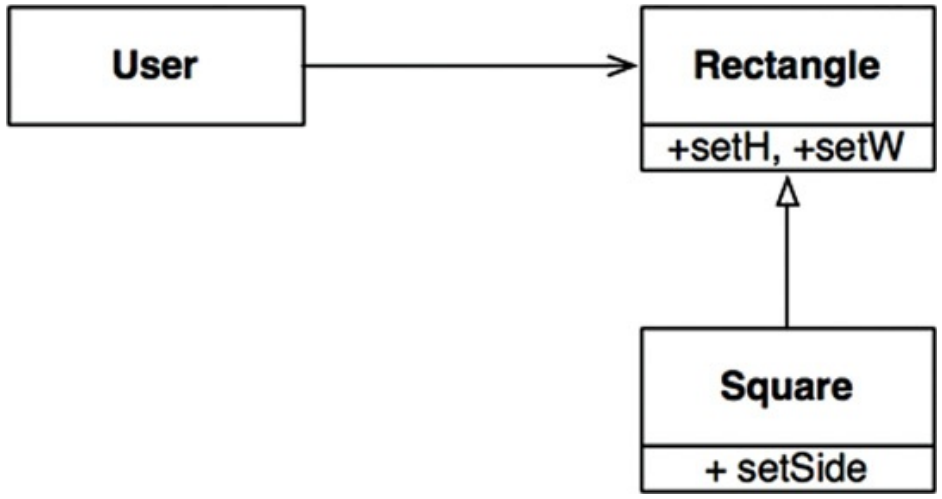


Hình 9.1
License, và các lớp dẫn xuất của nó, tuân thủ LSP

Thiết kế này tuân thủ LSP vì hành vi của ứng dụng Billing **không** phụ thuộc theo bất kỳ cách nào vào hai kiểu con mà nó sử dụng. Cả hai kiểu con đều có thể thay thế cho kiểu `License`.

Vấn đề vuông/hình chữ nhật

Ví dụ điển hình của một vi phạm LSP là bài toán nổi tiếng (hay tồi tệ, tùy góc nhìn) về vuông/hình chữ nhật (Hình 9.2).



Hình 9.2

Bài toán vuông/hình chữ nhật tồi tệ

Trong ví dụ này, `Square` **không** phải là một kiểu con thích hợp của `Rectangle` vì chiều cao và chiều rộng của `Rectangle` có thể thay đổi độc lập; ngược lại, chiều cao và chiều rộng của `Square` phải thay đổi cùng nhau. Vì `User` tin rằng nó đang giao tiếp với một `Rectangle`, nên nó rất dễ bị nhầm lẫn. Đoạn mã sau cho thấy lý do:

```
Rectangle r = …

r.setW(5);

r.setH(2);

assert(r.area() == 10);
```

Nếu mã … tạo ra một `Square`, thì khẳng định (assertion) sẽ thất bại.

Cách duy nhất để phòng thủ trước loại vi phạm LSP này là thêm các cơ chế vào `User` (như một câu lệnh `if`) để phát hiện xem `Rectangle` có thực sự là một `Square` hay không. Vì hành vi của `User` phụ thuộc vào các kiểu mà nó sử dụng, nên những kiểu đó **không** thể thay thế cho nhau.

LSP và kiến trúc

Trong những năm đầu của cuộc cách mạng hướng đối tượng, chúng ta coi LSP là một cách để dẫn dắt việc sử dụng kế thừa, như đã trình bày ở các phần trước. Tuy nhiên, qua nhiều năm, LSP đã biến hình thành một nguyên tắc thiết kế phần mềm rộng hơn, liên quan đến các giao diện (interface) và các triển khai (implementation).

Các giao diện trong vấn đề này có thể mang nhiều hình thức. Chúng ta có thể có một

giao diện kiểu Java, được triển khai bởi nhiều lớp. Hoặc chúng ta có thể có một vài lớp Ruby chia sẻ cùng các chữ ký phương thức. Hoặc chúng ta có thể có một tập hợp các dịch vụ đều phản hồi cùng một giao diện REST.

Trong tất cả các tình huống đó, và nhiều tình huống khác, LSP đều áp dụng được vì có những người dùng (user) phụ thuộc vào các giao diện được định nghĩa rõ ràng, và vào tính khả thi của việc thay thế các triển khai của những giao diện đó.

Cách tốt nhất để hiểu LSP từ góc độ kiến trúc là xem xét điều gì xảy ra với kiến trúc của một hệ thống khi nguyên tắc này bị vi phạm.

Ví dụ vi phạm LSP

Giả sử chúng ta đang xây dựng một bộ tổng hợp (aggregator) cho nhiều dịch vụ điều phối taxi. Khách hàng sử dụng trang web của chúng ta để tìm taxi phù hợp nhất, bất kể công ty taxi nào. Khi khách hàng quyết định, hệ thống của chúng ta điều phối taxi đã chọn bằng cách sử dụng một dịch vụ RESTful.

Bây giờ giả sử URI của dịch vụ điều phối RESTful là một phần của thông tin chứa trong cơ sở dữ liệu tài xế. Khi hệ thống của chúng ta đã chọn một tài xế phù hợp cho khách hàng, nó lấy URI đó từ bản ghi tài xế và dùng nó để điều phối tài xế.

Giả sử tài xế Bob có URI điều phối trông như sau:

```
purplecab.com/driver/Bob
```

Hệ thống của chúng ta sẽ gắn thông tin điều phối vào URI này và gửi nó bằng một yêu cầu PUT, như sau:

```
purplecab.com/driver/Bob
/pickupAddress/24 Maple St.
/pickupTime/153
/destination/ORD
```

Rõ ràng, điều này có nghĩa là **tất cả** các dịch vụ điều phối, của **tất cả** các công ty khác nhau, phải tuân thủ **cùng một** giao diện REST. Chúng phải xử lý các trường `pickupAddress`, `pickupTime` và `destination` theo cách hoàn toàn giống nhau.

Bây giờ giả sử công ty taxi Acme thuê một vài lập trình viên không đọc kỹ spec. Họ viết tắt trường `destination` thành chỉ `dest`. Acme là công ty taxi lớn nhất trong khu vực của chúng ta, và vợ cũ của CEO Acme là vợ mới của CEO chúng ta, và… Bạn hiểu ý rồi. Điều gì sẽ xảy ra với kiến trúc của hệ thống chúng ta?

Chắc chắn, chúng ta sẽ phải thêm một trường hợp đặc biệt (special case). Yêu cầu điều phối cho bất kỳ tài xế Acme nào sẽ phải được xây dựng bằng một bộ quy tắc khác so với tất cả các tài xế còn lại.

Cách đơn giản nhất để đạt được mục tiêu này là thêm một câu lệnh `if` vào mô-đun xây dựng lệnh điều phối:

```
if (driver.getDispatchUri().startsWith("acme.com"))…
```

Không kiến trúc sư giỏi nào lại để kiểu cấu trúc đó tồn tại trong hệ thống. Đưa từ "acme" thẳng vào mã nguồn sẽ mở đường cho đủ loại lỗi khó lường, chưa kể đến các lỗ hổng bảo mật.

Ví dụ, nếu Acme trở nên còn thành công hơn và mua công ty taxi Purple thì sao? Nếu công ty hợp nhất giữ nguyên các thương hiệu riêng biệt và các trang web riêng biệt, nhưng thống nhất tất cả hệ thống của các công ty ban đầu? Chúng ta có phải thêm một câu `if` khác cho "purple" không?

Kiến trúc sư của chúng ta sẽ phải cách ly hệ thống khỏi các lỗi như thế này bằng cách tạo ra một loại mô-đun tạo lệnh điều phối được điều khiển bởi một cơ sở dữ liệu cấu hình (configuration database) được tra cứu theo URI điều phối. Dữ liệu cấu hình có thể trông như sau:

```
URI          Định dạng điều phối
Acme.com     /pickupAddress/%s/pickupTime/%s/dest/%s
*.*          /pickupAddress/%s/pickupTime/%s/destination/%s
```

Và như vậy, kiến trúc sư của chúng ta đã phải thêm một cơ chế đáng kể và phức tạp để đối phó với thực tế rằng các giao diện của các dịch vụ RESTful **không** đều có thể thay thế cho nhau.

Kết luận

LSP có thể, và nên, được mở rộng đến mức độ kiến trúc. Một vi phạm đơn giản về khả năng thay thế, có thể làm ô nhiễm kiến trúc của một hệ thống bằng một lượng lớn các cơ chế thêm vào.

10

ISP: Nguyên tắc phân tách giao diện

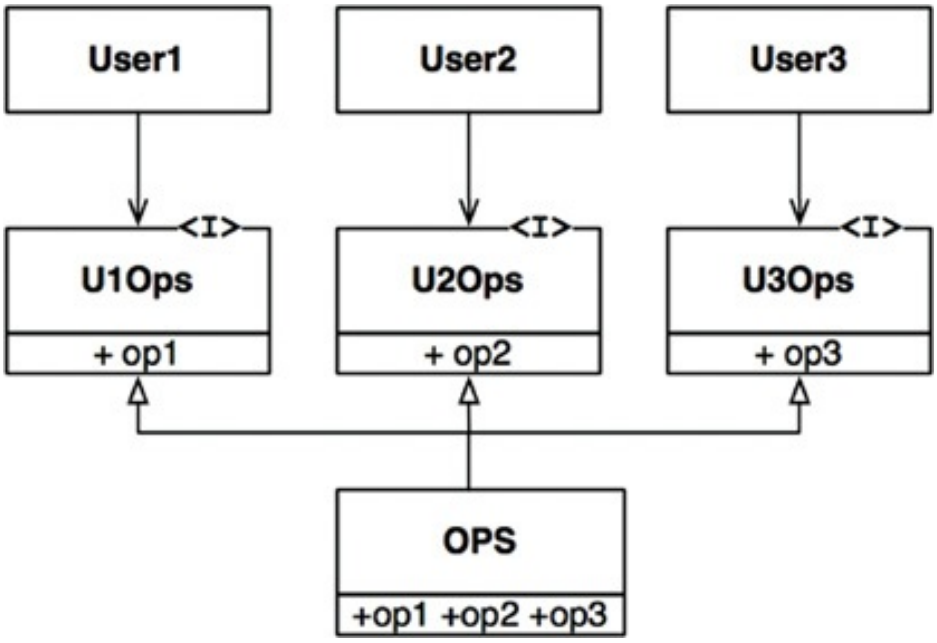


Nguyên tắc phân tách giao diện (ISP) lấy tên từ sơ đồ được hiển thị trong Hình 10.1.
Hình 10.1 Nguyên tắc phân tách giao diện

Trong tình huống được minh họa trong Hình 10.1, có một số người dùng sử dụng các thao tác của lớp OPS. Giả sử User1 chỉ dùng op1, User2 chỉ dùng op2, và User3 chỉ dùng op3.

Bây giờ hãy tưởng tượng OPS là một lớp viết bằng ngôn ngữ kiểu Java. Rõ ràng, trong trường hợp này, mã nguồn của User1 sẽ vô tình phụ thuộc vào op2 và op3, ngay cả khi nó không gọi chúng. Sự phụ thuộc này có nghĩa là một thay đổi đối với mã nguồn của op2 trong OPS sẽ buộc User1 phải biên dịch lại và triển khai lại, mặc dù không có gì mà nó quan tâm thực sự đã thay đổi.

Vấn đề này có thể được giải quyết bằng cách phân tách các thao tác thành các giao diện như được hiển thị trong Hình 10.2.



Hình 10.2
Các thao tác được phân tách

Lại một lần nữa, nếu chúng ta tưởng tượng việc này được triển khai bằng một ngôn ngữ kiểu tĩnh như Java, thì mã nguồn của User1 sẽ phụ thuộc vào U1Ops, và op1, nhưng sẽ *không* phụ thuộc vào OPS. Do đó, một thay đổi đối với OPS mà User1 không quan tâm sẽ không khiến User1 phải biên dịch lại và triển khai lại.

ISP và ngôn ngữ

Rõ ràng, mô tả trước đây phụ thuộc quan trọng vào loại ngôn ngữ. Các ngôn ngữ kiểu tĩnh như Java buộc lập trình viên phải tạo các khai báo mà người dùng phải import, hoặc use, hoặc theo cách nào đó bao gồm. Chính những khai báo được bao gồm này trong mã nguồn tạo ra các phụ thuộc mã nguồn buộc phải biên dịch lại và triển khai lại.

Trong các ngôn ngữ kiểu động như Ruby và Python, những khai báo như vậy không tồn tại trong mã nguồn. Thay vào đó, chúng được suy diễn tại thời điểm chạy. Do đó không có phụ thuộc mã nguồn nào để buộc biên dịch lại và triển khai lại. Đây là lý do chính khiến các ngôn ngữ kiểu động tạo ra các hệ thống linh hoạt hơn và ít bị ràng buộc chặt chẽ hơn so với các ngôn ngữ kiểu tĩnh.

Thực tế này có thể khiến bạn kết luận rằng ISP là một vấn đề ngôn ngữ, chứ không phải vấn đề kiến trúc.

ISP và kiến trúc

Nếu bạn lùi lại một bước và nhìn vào động lực cốt lõi của ISP, bạn có thể thấy một mối quan tâm sâu xa hơn tiềm ẩn bên dưới. Nói chung, việc phụ thuộc vào các mô-đun chứa nhiều thứ hơn những gì bạn cần là có hại. Điều này rõ ràng đúng đối với các phụ thuộc mã nguồn có thể buộc biên dịch lại và triển khai lại không cần thiết; nó cũng đúng ở một cấp độ cao hơn nhiều, cấp độ kiến trúc.

Xét ví dụ sau: một kiến trúc sư đang làm việc trên một hệ thống, S. Anh ta muốn đưa một framework nhất định, F, vào hệ thống. Giả sử các tác giả của F đã ràng buộc nó với một cơ sở dữ liệu cụ thể, D. Vậy S phụ thuộc vào F, F phụ thuộc vào D (Hình 10.3).



Hình 10.3
Một kiến trúc gây vấn đề

Bây giờ giả sử D chứa các tính năng mà F không sử dụng và do đó S không quan tâm. Các thay đổi đối với những tính năng đó bên trong D có thể buộc phải triển khai lại F và do đó triển khai lại S. Tệ hơn, một lỗi của một trong những tính năng bên trong D có thể gây ra lỗi trong F và S.

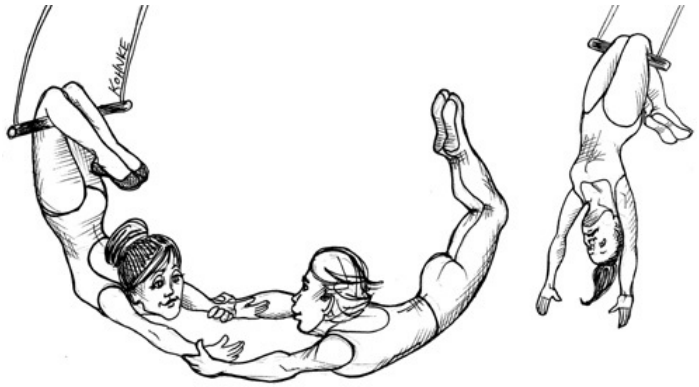
Kết luận

Bài học ở đây là việc phụ thuộc vào thứ mang theo hành lý mà bạn không cần có thể gây ra những rắc rối bạn không ngờ tới.

Chúng ta sẽ khám phá ý tưởng này chi tiết hơn khi thảo luận về Nguyên tắc Tái sử dụng Chung (Common Reuse Principle) trong Chương 13, “Tính gắn kết của thành phần” (Component Cohesion).

11

DIP: Nguyên tắc đảo ngược phụ thuộc



Nguyên tắc đảo ngược phụ thuộc (DIP) khẳng định rằng các hệ thống linh hoạt nhất là những hệ thống có phụ thuộc mã nguồn chỉ tham chiếu đến các trừu tượng, không tham chiếu đến các phần triển khai cụ thể.

Trong một ngôn ngữ kiểu tĩnh, như Java, điều này có nghĩa là các câu lệnh `use`, `import` và `include` nên chỉ tham chiếu đến các mô-đun nguồn chứa các giao diện (interface), các lớp trừu tượng (abstract class), hoặc một kiểu khai báo trừu tượng nào đó. Không có thứ cụ thể (concrete) nào nên bị phụ thuộc vào.

Quy tắc tương tự cũng áp dụng cho các ngôn ngữ kiểu động, như Ruby và Python. Các sự phụ thuộc của mã nguồn không nên tham chiếu đến các mô-đun cụ thể. Tuy nhiên, trong các ngôn ngữ này, việc xác định một mô-đun cụ thể là gì thì hơi khó khăn hơn một chút. Cụ thể, đó là bất kỳ mô-đun nào mà các hàm được gọi được triển khai bên trong.

Rõ ràng, việc coi ý tưởng này như một quy tắc cứng nhắc là không thực tế, bởi vì các hệ thống phần mềm buộc phải phụ thuộc vào nhiều tiện ích cụ thể. Ví dụ, lớp `String` trong

Java là cụ thể, và sẽ không thực tế nếu cố gắng ép buộc nó trở thành trừu tượng. Sự phụ thuộc mã nguồn vào `java.lang.String` cụ thể đó không thể, và cũng không nên, bị tránh né.

Bằng so sánh, lớp `String` rất ổn định. Những thay đổi đối với lớp này rất hiếm và được kiểm soát chặt chẽ. Các lập trình viên và kiến trúc sư không phải lo lắng về những thay đổi thường xuyên và thất thường của `String`.

Với những lý do này, chúng ta có xu hướng bỏ qua nền tảng ổn định của hệ điều hành và các tiện ích nền tảng (platform) khi nói đến DIP. Chúng ta chấp nhận những sự phụ thuộc cụ thể đó vì chúng ta biết rằng chúng ta có thể tin cậy vào việc chúng sẽ không thay đổi.

Đó là những yếu tố cụ thể, thất thường (volatile) của hệ thống chúng ta mà chúng ta muốn tránh việc phụ thuộc vào. Đó là những mô-đun mà chúng ta đang tích cực phát triển, và đang trải qua những thay đổi thường xuyên.

Trừu tượng ổn định

Mọi thay đổi đối với một giao diện trừu tượng đều tương ứng với thay đổi ở các bản triển khai cụ thể của nó. Ngược lại, thay đổi ở các bản triển khai cụ thể không phải lúc nào, thậm chí thường là không, đòi hỏi thay đổi ở các giao diện mà chúng triển khai. Vì thế, giao diện ít thất thường hơn bản triển khai.

Thực tế, những nhà thiết kế và kiến trúc sư phần mềm giỏi luôn cố gắng giảm thiểu sự thất thường của giao diện. Họ tìm cách thêm chức năng vào các bản triển khai mà không thay đổi giao diện. Đây là bài học Cơ bản về Thiết kế Phần mềm 101.

Hệ quả là kiến trúc phần mềm ổn định tránh phụ thuộc vào các phần triển khai cụ thể, dễ thay đổi, và ưu tiên các giao diện trừu tượng ổn định. Sự chuyển dịch tinh tế này dẫn đến một tập hợp những thực hành viết mã rất cụ thể:

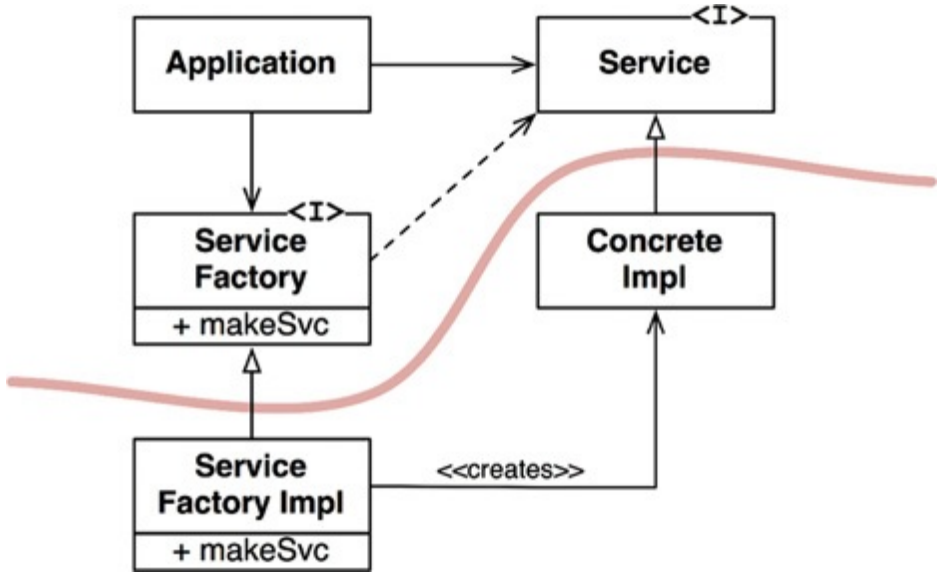
- **Đừng tham chiếu đến các lớp cụ thể và thất thường. Hãy tham chiếu đến các giao diện trừu tượng thay vào đó.** Quy tắc này áp dụng trong mọi ngôn ngữ, dù là kiểu tĩnh hay kiểu động. Nó cũng đặt ra những ràng buộc nghiêm ngặt đối với việc tạo đối tượng (object creation) và nói chung bắt buộc việc sử dụng Abstract Factory (Nhà máy trừu tượng).
- **Đừng kế thừa từ các lớp cụ thể và thất thường.** Đây là một hệ quả của quy tắc trước, nhưng đáng được nhắc riêng. Trong các ngôn ngữ kiểu tĩnh, kế thừa là mối quan hệ mã nguồn mạnh mẽ và cứng nhắc nhất; do đó, nó nên được dùng với sự thận trọng cực đại. Trong các ngôn ngữ kiểu động, kế thừa ít gây vấn đề hơn, nhưng nó vẫn là một sự phụ thuộc, và sự thận trọng luôn là lựa chọn khôn ngoan nhất.
- **Đừng ghi đè (override) các hàm cụ thể.** Hàm cụ thể thường đòi hỏi phụ thuộc mã nguồn. Khi ghi đè những hàm đó, bạn không loại bỏ các phụ thuộc này, mà kế thừa chúng. Để quản lý chúng, hãy làm cho hàm thành trừu tượng (abstract) và tạo ra nhiều bản triển khai.
- **Đừng bao giờ nhắc đến tên của bất cứ thứ gì cụ thể và thất thường.** Điều này thực chất chỉ là cách diễn đạt lại chính nguyên tắc đó.

Nhà máy (Factories)

Để tuân thủ các quy tắc này, việc tạo ra các đối tượng cụ thể và thất thường đòi hỏi sự xử lý đặc biệt. Sự thận trọng này là cần thiết vì, trong hầu như mọi ngôn ngữ, việc tạo một đối tượng đòi hỏi một sự phụ thuộc mã nguồn vào định nghĩa cụ thể của đối tượng đó.

Trong hầu hết các ngôn ngữ hướng đối tượng, như Java, chúng ta sẽ dùng Abstract Factory (Nhà máy trừu tượng) để quản lý sự phụ thuộc không mong muốn này.

Sơ đồ trong **Hình 11.1** cho thấy cấu trúc đó. `Application` sử dụng `ConcreteImpl` thông qua giao diện `Service`. Tuy nhiên, `Application` phải có cách nào đó để tạo ra các thể hiện (instance) của `ConcreteImpl`. Để đạt được điều này mà không tạo ra sự phụ thuộc mã nguồn vào `ConcreteImpl`, `Application` gọi phương thức `makeSvc` của giao diện `ServiceFactory`. Phương thức này được triển khai bởi lớp `ServiceFactoryImpl`, kế thừa từ `ServiceFactory`. Triển khai đó khởi tạo `ConcreteImpl` và trả về nó dưới kiểu `Service`.



Hình 11.1

Sử dụng mẫu Abstract Factory để quản lý sự phụ thuộc

Đường cong trong **Hình 11.1** là một ranh giới kiến trúc (architectural boundary). Nó tách biệt trừu tượng với cụ thể. Tất cả các sự phụ thuộc mã nguồn cắt qua đường cong đó đều chỉ theo một hướng, về phía bên trừu tượng.

Đường cong chia hệ thống thành hai thành phần (component): một trừu tượng và một cụ thể. Thành phần trừu tượng chứa tất cả các quy tắc nghiệp vụ (business rules) cấp cao của ứng dụng. Thành phần cụ thể chứa tất cả các chi tiết triển khai (implementation details) mà những quy tắc nghiệp vụ đó thao tác.

Lưu ý rằng luồng điều khiển (flow of control) cắt qua đường cong đó theo hướng ngược

lại so với các sự phụ thuộc mã nguồn. Các sự phụ thuộc mã nguồn bị đảo ngược so với luồng điều khiển, đó là lý do chúng ta gọi nguyên tắc này là Đảo ngược phụ thuộc (Dependency Inversion).

Thành phần cụ thể (Concrete Components)

Thành phần cụ thể trong **Hình 11.1** chứa một sự phụ thuộc duy nhất, vì vậy nó vi phạm DIP. Đây là điều điển hình. Các vi phạm DIP không thể loại bỏ hoàn toàn, nhưng chúng có thể được gom nhóm vào một số nhỏ các thành phần cụ thể và tách biệt khỏi phần còn lại của hệ thống.

Hầu hết các hệ thống sẽ chứa ít nhất một thành phần cụ thể như vậy, thường được gọi là `main` vì nó chứa hàm `main`[1]. Trong trường hợp được minh họa trong **Hình 11.1**, hàm `main` sẽ khởi tạo `ServiceFactoryImpl` và đặt thể hiện đó vào một biến toàn cục (global variable) kiểu `ServiceFactory`. `Application` sau đó sẽ truy cập nhà máy (factory) thông qua biến toàn cục đó.

Kết luận

Khi chúng ta tiến về phía trước trong cuốn sách này và đề cập đến các nguyên tắc kiến trúc cấp cao hơn, DIP sẽ xuất hiện lại lần nữa và lần nữa. Nó sẽ là nguyên tắc tổ chức dễ thấy nhất trong các sơ đồ kiến trúc của chúng ta. Đường cong trong **Hình 11.1** sẽ trở thành các ranh giới kiến trúc (architectural boundaries) trong các chương sau. Cách mà các sự phụ thuộc cắt qua đường cong đó theo một hướng, và về phía các thực thể trừu tượng hơn, sẽ trở thành một quy tắc mới mà chúng ta sẽ gọi là Quy tắc phụ thuộc (Dependency Rule).

1. Nói cách khác, hàm được gọi bởi hệ điều hành khi ứng dụng lần đầu được khởi động.

Phần IV: Nguyên tắc thành phần

Nếu các nguyên tắc SOLID cho chúng ta biết cách xếp gạch thành tường và phòng, thì các nguyên tắc thành phần cho chúng ta biết cách xếp phòng thành tòa nhà. Hệ thống phần mềm lớn, giống như tòa nhà lớn, được xây dựng từ các thành phần nhỏ hơn.

Ở Phần IV, chúng tôi sẽ thảo luận về thành phần phần mềm là gì, những yếu tố nào nên cấu thành chúng, và cách chúng nên được kết hợp lại với nhau thành hệ thống.

12

Các thành phần



Các thành phần (Components) là đơn vị triển khai (deployment). Chúng là các thực thể nhỏ nhất có thể được triển khai như một phần của hệ thống. Trong Java, chúng là các tệp jar (jar files). Trong Ruby, chúng là các tệp gem (gem files). Trong .Net, chúng là các DLL. Trong các ngôn ngữ biên dịch, chúng là tập hợp các tệp nhị phân (binary files). Trong các ngôn ngữ thông dịch, chúng là tập hợp các tệp nguồn (source files). Trong tất cả các ngôn ngữ, chúng là đơn vị triển khai (granule of deployment).

Các thành phần có thể được liên kết (linked) lại với nhau thành một tệp thực thi đơn lẻ

(single executable). Hoặc chúng có thể được gom lại thành một kho lưu trữ đơn lẻ, chẳng hạn như tệp .war. Hoặc chúng có thể được triển khai độc lập như các plugin được tải động (dynamically loaded plugins) riêng biệt, chẳng hạn như tệp .jar, .dll hoặc .exe. Dù chúng cuối cùng được triển khai như thế nào, các thành phần được thiết kế tốt luôn giữ khả năng triển khai độc lập (independently deployable) và, do đó, phát triển độc lập (independently developable).

Lịch sử ngắn về các thành phần

Trong những năm đầu của phát triển phần mềm, các lập trình viên kiểm soát vị trí và bố cục bộ nhớ của chương trình. Một trong những dòng mã đầu tiên trong chương trình là câu lệnh origin (origin statement), khai báo địa chỉ mà chương trình sẽ được nạp vào.

Xét chương trình PDP-8 đơn giản sau. Nó bao gồm một thủ tục con (subroutine) tên GETSTR, nhập một chuỗi từ bàn phím và lưu vào một bộ đệm (buffer). Nó cũng có một chương trình kiểm thử đơn vị (unit test) nhỏ để chạy thử GETSTR.

```
*200
TLS
START, CLA
TAD BUFR
JMS GETSTR
CLA
TAD BUFR
JMS PUTSTR
JMP START
BUFR, 3000
GETSTR, 0
DCA PTR
NXTCH, KSF
JMP -1
KRB
DCA I PTR
TAD I PTR
AND K177
ISZ PTR
TAD MCR
SZA
JMP NXTCH
K177, 177
MCR, -15
```

Lưu ý lệnh *200 ở đầu chương trình. Nó báo cho trình biên dịch tạo mã sẽ được nạp vào địa chỉ 200_8.

Loại lập trình này là khái niệm xa lạ đối với hầu hết lập trình viên ngày nay. Họ hiếm khi phải nghĩ đến việc chương trình được nạp ở đâu trong bộ nhớ máy tính. Nhưng xưa kia, đây là một trong những quyết định đầu tiên lập trình viên phải làm. Vào những ngày ấy, chương trình **không thể tái định vị (relocatable)**.

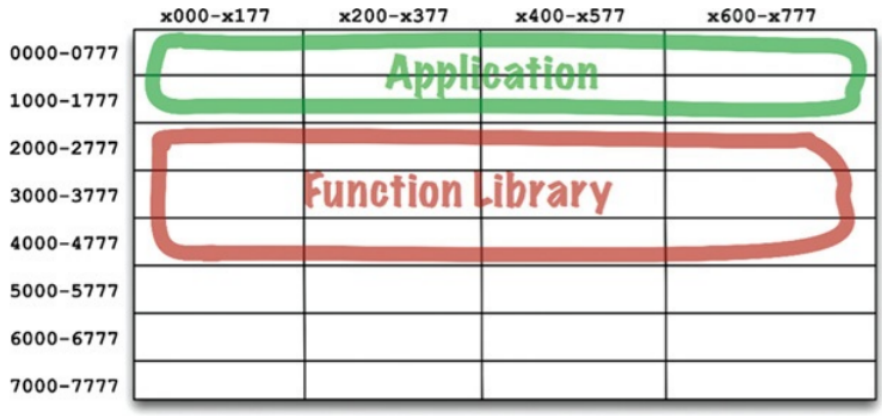
Làm sao bạn truy cập hàm thư viện (library function) trong những ngày xưa ấy? Đoạn mã trước minh họa cách tiếp cận được dùng. Lập trình viên đưa mã nguồn của các hàm thư

viện vào mã ứng dụng của mình, và biên dịch tất cả thành một chương trình đơn lẻ.[1] Thư viện được giữ ở dạng nguồn, không phải nhị phân.

Vấn đề của cách này là, thời đó, thiết bị chậm và bộ nhớ đắt, do đó hạn chế. Trình biên dịch cần thực hiện nhiều lần đọc (passes) qua mã nguồn, nhưng bộ nhớ quá nhỏ để chứa toàn bộ mã nguồn. Do đó, trình biên dịch phải đọc mã nguồn nhiều lần bằng các thiết bị chậm.

Việc này mất nhiều thời gian, và thư viện hàm càng lớn, trình biên dịch càng lâu. Biên dịch một chương trình lớn có thể mất hàng giờ.

Để rút ngắn thời gian biên dịch, lập trình viên tách mã nguồn của thư viện hàm ra khỏi ứng dụng. Chúng biên dịch thư viện hàm riêng biệt và nạp nhị phân vào địa chỉ đã biết, ví dụ 2000_8. Chúng tạo bảng ký hiệu (symbol table) cho thư viện hàm và biên dịch bảng đó cùng mã ứng dụng. Khi muốn chạy ứng dụng, chúng nạp thư viện hàm nhị phân[2] rồi nạp ứng dụng. Bộ nhớ trông như bố cục trong **Hình 12.1**.



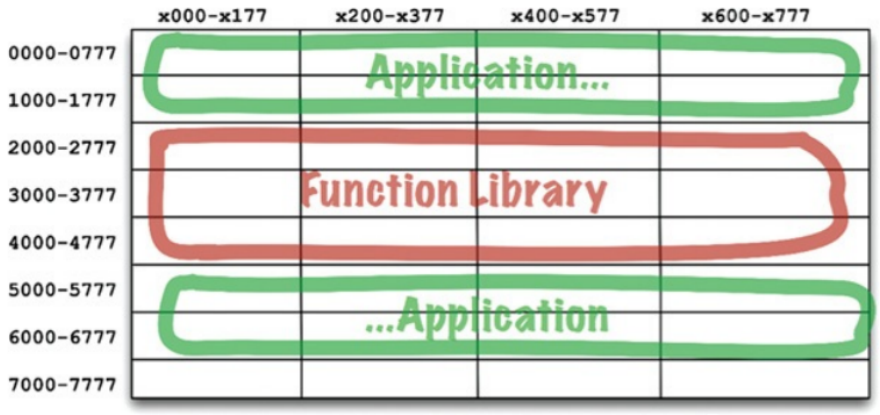
Hình 12.1
Bố cục bộ nhớ sơ khai

Cách này chạy tốt miễn sao ứng dụng vừa trong khoảng địa chỉ 0000_8 đến 1777_8. Nhưng sớm thôi ứng dụng lớn vượt quá không gian dành cho chúng. Lúc đó lập trình viên phải chia ứng dụng thành hai đoạn địa chỉ (address segments), nhảy vòng qua thư viện hàm (**Hình 12.2**).

---

1. Người tuyển dụng đầu tiên của tôi giữ vài chục bộ bài (decks) mã nguồn thư viện thủ tục con trên kệ. Khi bạn viết chương trình mới, bạn chỉ cần lấy một trong những bộ bài đó và gắn nó vào cuối bộ bài của mình.

2. Thực tế, hầu hết những máy cũ dùng bộ nhớ lõi (core memory), không bị xóa khi tắt máy. Chúng ta thường để thư viện hàm nạp sẵn hàng ngày liền.



Hình 12.2
Chia ứng dụng thành hai đoạn địa chỉ

Rõ ràng đây không phải tình trạng bền vững. Khi lập trình viên thêm hàm vào thư viện, nó vượt quá giới hạn, và họ phải cấp thêm không gian (ở ví dụ này, gần 7000_8). Sự phân mảnh (fragmentation) chương trình và thư viện này buộc phải tiếp diễn khi bộ nhớ máy tính tăng lên.

Rõ ràng phải có giải pháp.

Khả năng tái định vị

Giải pháp là **nhị phân có thể tái định vị (relocatable binaries)**. Ý tưởng đằng sau rất đơn giản. Trình biên dịch được thay đổi để xuất mã nhị phân có thể được di chuyển trong bộ nhớ bởi một nạp trình thông minh (smart loader). Nạp trình được báo nơi nạp mã tái định vị (relocatable). Mã tái định vị được gắn cờ (flags) báo cho nạp trình biết phần nào của dữ liệu nạp phải thay đổi để nạp tại địa chỉ đã chọn. Thường chỉ nghĩa là cộng địa chỉ bắt đầu vào bất kỳ địa chỉ tham chiếu bộ nhớ nào trong nhị phân.

Bây giờ lập trình viên có thể bảo nạp trình nạp thư viện hàm ở đâu, và nạp ứng dụng ở đâu. Thực tế, nạp trình chấp nhận nhiều đầu vào nhị phân và chỉ việc nạp chúng vào bộ nhớ lần lượt, tái định vị (relocate) chúng trong khi nạp. Điều này cho phép lập trình viên chỉ nạp những hàm họ cần.

Trình biên dịch cũng được thay đổi để phát ra tên hàm dưới dạng siêu dữ liệu (metadata) trong nhị phân tái định vị. Nếu chương trình gọi hàm thư viện, trình biên dịch phát ra tên đó như một tham chiếu ngoài (external reference). Nếu chương trình định nghĩa hàm thư viện, trình biên dịch phát ra tên đó như một định nghĩa ngoài (external definition). Sau đó nạp trình có thể liên kết (link) các tham chiếu ngoài đến các định nghĩa ngoài khi nó đã xác định được các định nghĩa đó được nạp ở đâu.

Và **trình nạp liên kết (linking loader)** ra đời.

Các trình liên kết

Trình nạp liên kết cho phép lập trình viên chia chương trình thành các đoạn (segments) có thể biên dịch và nạp riêng biệt. Cách này chạy tốt khi các chương trình tương đối nhỏ được liên kết với thư viện nhỏ. Tuy nhiên, cuối những năm 1960 và đầu 1970, lập trình viên có tham vọng lớn hơn, và chương trình của họ lớn hơn rất nhiều.

Cuối cùng, các trình nạp liên kết (linking loaders) quá chậm để chịu đựng. Thư viện hàm lưu trên thiết bị chậm như băng từ (magnetic tape). Ngay cả đĩa, thời đó, cũng khá chậm. Dùng những thiết bị chậm này, trình nạp liên kết phải đọc chục, nếu không phải hàng trăm, thư viện nhị phân để giải quyết tham chiếu ngoài. Khi chương trình càng lớn, và nhiều hàm thư viện tích lũy hơn, một trình nạp liên kết có thể mất hơn một giờ chỉ để nạp chương trình.

Cuối cùng, việc nạp (loading) và liên kết (linking) được tách thành hai giai đoạn. Lập trình viên lấy phần chậm, phần làm việc liên kết, và đưa vào một ứng dụng riêng gọi là **trình liên kết (linker)**. Đầu ra của trình liên kết là một tệp nhị phân tái định vị đã liên kết (linked relocatable) mà một trình nạp tái định vị (relocating loader) có thể nạp rất nhanh. Điều này cho phép lập trình viên chuẩn bị một tệp thực thi (executable) dùng trình liên kết chậm, nhưng sau đó nạp nó nhanh chóng, bất kỳ lúc nào.

Rồi đến những năm 1980. Lập trình viên làm việc với C hoặc ngôn ngữ bậc cao khác. Khi tham vọng tăng, chương trình cũng lớn theo. Chương trình hàng trăm nghìn dòng mã không hiếm.

Các mô-đun nguồn (source modules) được biên dịch từ tệp .c thành tệp .o, rồi đưa vào trình liên kết để tạo tệp thực thi có thể nạp nhanh. Biên dịch từng mô-đun riêng lẻ khá nhanh, nhưng biên dịch tất cả mất chút thời gian. Trình liên kết lại mất thêm thời gian. Chu kỳ hoàn tất (turnaround) lại tăng lên một giờ trở lên trong nhiều trường hợp.

Dường như lập trình viên bị định mệnh phải đua đuổi mình vô tận. Trong suốt những năm 1960, 1970 và 1980, mọi thay đổi làm tăng tốc quy trình đều bị bác bỏ bởi tham vọng của lập trình viên, và kích thước chương trình họ viết. Họ không thể thoát khỏi chu kỳ hoàn tất một giờ. Thời gian nạp vẫn nhanh, nhưng thời gian biên dịch-liên kết (compile-link) là nút cổ chai (bottleneck).

Chúng ta đang trải qua **luật Murphy về kích thước chương trình**:

Chương trình sẽ lớn lên để lấp đầy toàn bộ thời gian biên dịch và liên kết có sẵn.

Nhưng Murphy không phải là đối thủ duy nhất. Moore[3] xuất hiện, và cuối những năm 1980, hai "đấu" nhau. Moore thắng. Đĩa bắt đầu thu nhỏ và nhanh hơn đáng kể. Bộ nhớ máy tính rẻ đến mức dữ liệu trên đĩa phần lớn có thể cache trong RAM. Tần số nhịp máy tăng từ 1 MHz lên 100 MHz.

Đến giữa những năm 1990, thời gian liên kết bắt đầu giảm nhanh hơn tham vọng làm chương trình lớn. Trong nhiều trường hợp, thời gian liên kết giảm đến vài giây. Đối với tác vụ nhỏ, ý tưởng trình nạp liên kết (linking loader) lại trở nên khả thi.

---

3. Định luật Moore: Tốc độ máy tính, bộ nhớ, và mật độ gấp đôi mỗi 18 tháng. Định luật này đúng từ những năm 1950 đến 2000, nhưng sau đó, ít nhất đối với tần số nhịp, dừng hẳn.

Đây là kỷ nguyên của Active-X, thư viện chia sẻ (shared libraries), và khởi đầu của tệp .jar. Máy tính và thiết bị nhanh đến mức ta có thể, một lần nữa, làm liên kết tại thời điểm nạp (load time). Ta có thể liên kết vài tệp .jar, hoặc vài thư viện chia sẻ, trong vài giây, và chạy chương trình kết quả. Và như vậy **kiến trúc plugin thành phần (component plugin architecture)** ra đời.

Ngày nay chúng ta thường xuyên phân phối tệp .jar, DLL, hoặc thư viện chia sẻ như plugin cho ứng dụng hiện có. Nếu bạn muốn tạo mod cho Minecraft, ví dụ, bạn chỉ cần đưa tệp .jar tùy chỉnh vào thư mục nhất định. Nếu bạn muốn gắn Resharper vào Visual Studio, bạn chỉ cần đưa các DLL thích hợp.

Kết luận

Những tệp liên kết động (dynamically linked files) này, có thể gắn kết nhau tại thời điểm chạy (runtime), chính là các thành phần phần mềm (software components) của kiến trúc chúng ta. Đã mất 50 năm, nhưng chúng ta đã đến nơi mà kiến trúc plugin thành phần có thể là lựa chọn mặc định thông thường (casual default), thay vì nỗ lực vĩ đại (herculean effort) như xưa.

13

Tính gắn kết của thành phần



Trong chương này, chúng ta sẽ thảo luận về ba nguyên tắc của tính gắn kết của thành phần (component cohesion):

- REP: Nguyên tắc tương đương giữa tái sử dụng và phát hành (Reuse/Release Equivalence Principle)
- CCP: Nguyên tắc Đóng chung (Common Closure Principle)
- CRP: Nguyên tắc Tái sử dụng chung (Common Reuse Principle)

Nguyên tắc tương đương giữa tái sử dụng và phát hành

Đơn vị của việc tái sử dụng là đơn vị của việc phát hành.

Thập kỷ qua đã chứng kiến sự trỗi dậy của đủ loại công cụ quản lý mô-đun, như Maven, Leiningen và RVM. Các công cụ này tăng lên tầm quan trọng bởi vì, trong khoảng thời gian đó, một số lượng lớn các thành phần có thể tái sử dụng và thư viện thành phần đã được tạo ra. Chúng ta đang sống trong kỷ nguyên của việc tái sử dụng phần mềm, sự hiện thực hóa của một trong những lời hứa lâu đời nhất của mô hình hướng đối tượng.

Nguyên tắc tương đương giữa tái sử dụng và phát hành (REP) là một nguyên tắc có vẻ hiển nhiên, ít nhất là khi nhìn lại từ hiện tại. Những người muốn tái sử dụng các thành phần phần mềm không thể, và sẽ không, làm điều đó trừ khi các thành phần đó được theo dõi qua một quy trình phát hành và được gán số phát hành.

Điều này không đơn giản là vì, nếu không có số phát hành, sẽ không có cách nào để đảm bảo rằng tất cả các thành phần được tái sử dụng đều tương thích với nhau. Thực tế, điều này còn phản ánh việc các nhà phát triển phần mềm cần biết khi nào các bản phát hành mới sẽ ra mắt, và những bản phát hành đó sẽ mang lại những thay đổi gì.

Không hiếm khi các nhà phát triển được thông báo về một bản phát hành mới và quyết định, dựa trên các thay đổi được thực hiện trong bản phát hành đó, tiếp tục sử dụng bản phát hành cũ. Do đó, quy trình phát hành phải tạo ra các thông báo và tài liệu phát hành thích hợp để người dùng có thể đưa ra quyết định có thông tin về việc khi nào và có nên tích hợp bản phát hành mới hay không.

Từ quan điểm thiết kế và kiến trúc phần mềm, nguyên tắc này có nghĩa là các lớp và mô-đun được gom lại thành một thành phần phải thuộc về một nhóm gắn kết. Thành phần không thể đơn thuần bao gồm một hỗn độn ngẫu nhiên các lớp và mô-đun; ngược lại, phải có một chủ đề hoặc mục đích cốt lõi nào đó mà tất cả các mô-đun đó đều chia sẻ.

Tuy nhiên, có một cách nhìn khác về vấn đề này có lẽ không hẳn hiển nhiên. Các lớp và mô-đun được gom nhóm lại vào một thành phần nên có thể được phát hành cùng một lúc. Việc chúng chia sẻ cùng một số phiên bản, cùng một hệ thống theo dõi phát hành, và được bao gồm trong cùng một tài liệu phát hành, nên có ý nghĩa cả với tác giả lẫn với người dùng. Đây là một lời khuyên yếu: Việc nói rằng cái gì đó nên "có ý nghĩa" chỉ là cách vung tay lên không trung và cố gắng nghe có vẻ có thẩm quyền. Lời khuyên yếu vì rất khó giải thích chính xác chất keo dính giữ các lớp và mô-đun lại với nhau thành một thành phần duy nhất. Dù lời khuyên có yếu, bản thân nguyên tắc lại quan trọng, bởi vì các vi phạm rất dễ phát hiện: chúng "không có ý nghĩa". Nếu bạn vi phạm REP, người dùng của bạn sẽ biết, và họ sẽ không ấn tượng với kỹ năng kiến trúc của bạn.

Sự yếu kém của nguyên tắc này được bù đắp nhiều hơn bởi sức mạnh của hai nguyên tắc tiếp theo. Thực tế, CCP và CRP định hình mạnh mẽ nguyên tắc này, nhưng theo nghĩa tiêu cực.

Nguyên tắc Đóng chung

Hãy gom vào thành phần những lớp thay đổi vì cùng lý do và vào cùng lúc. Hãy tách vào các thành phần khác nhau những lớp thay đổi vào lúc khác nhau và vì những lý do khác nhau.

Đây là Nguyên tắc Trách nhiệm đơn nhất (SRP) được diễn đạt lại cho thành phần. Cũng như SRP nói rằng một lớp không nên chứa nhiều lý do để thay đổi, Nguyên tắc Đóng chung (CCP) nói rằng một thành phần không nên có nhiều lý do để thay đổi.

Đối với hầu hết các ứng dụng, khả năng bảo trì quan trọng hơn khả năng tái sử dụng. Nếu mã trong một ứng dụng phải thay đổi, bạn sẽ muốn tất cả các thay đổi đó xảy ra trong một thành phần, thay vì bị phân tán trên nhiều thành phần. Nếu các thay đổi được gói gọn trong một thành phần, thì chúng ta chỉ cần triển khai lại (redeploy) thành phần thay đổi đó. Các thành phần khác không phụ thuộc vào thành phần đã thay đổi không cần được xác thực lại (revalidated) hay triển khai lại.

Nguyên tắc Đóng chung (CCP) thúc đẩy chúng ta gom lại vào một nơi tất cả các lớp có khả năng thay đổi vì cùng lý do. Nếu hai lớp bị ràng buộc chặt chẽ với nhau, về mặt vật lý hay khái niệm, đến mức chúng luôn thay đổi cùng nhau, thì chúng thuộc về cùng một thành phần. Điều này tối thiểu hóa khối lượng công việc liên quan đến việc phát hành, xác thực lại và triển khai lại phần mềm.

Nguyên tắc này liên hệ mật thiết với Nguyên tắc Đóng/Mở (OCP). Thực tế, chính "sự đóng" (closure) theo nghĩa của OCP là điều mà Nguyên tắc Đóng chung giải quyết. OCP nói rằng các lớp nên đóng đối với sửa đổi nhưng mở đối với mở rộng. Vì sự đóng 100% không thể đạt được, sự đóng phải mang tính chiến lược. Chúng ta thiết kế các lớp sao cho chúng đóng đối với những loại thay đổi phổ biến nhất mà chúng ta mong đợi hoặc đã từng trải qua.

Nguyên tắc Đóng chung mở rộng bài học này bằng cách gom lại vào cùng một thành phần những lớp được đóng đối với cùng loại thay đổi. Như vậy, khi một thay đổi yêu cầu xuất hiện, thay đổi đó có cơ hội tốt được giới hạn trong một số lượng thành phần tối thiểu.

Sự tương tự với SRP

Như đã nói trước, CCP là dạng thành phần của SRP. SRP bảo chúng ta tách các phương thức vào các lớp khác nhau, nếu chúng thay đổi vì lý do khác nhau. CCP bảo chúng ta tách các lớp vào các thành phần khác nhau, nếu chúng thay đổi vì lý do khác nhau. Cả hai nguyên tắc đều có thể tóm gọn trong câu slogan sau:

> *Hãy gom lại những thứ thay đổi vào cùng lúc và vì cùng lý do. Hãy tách những thứ thay đổi vào lúc khác nhau hoặc vì những lý do khác nhau.*

Nguyên tắc Tái sử dụng chung

Đừng ép người dùng của một thành phần phải phụ thuộc vào những thứ họ không cần.

Nguyên tắc Tái sử dụng chung (CRP) là thêm một nguyên tắc giúp chúng ta quyết định những lớp và mô-đun nào nên được đặt vào một thành phần. Nó nói rằng các lớp và mô-đun có xu hướng được tái sử dụng cùng nhau nên thuộc về cùng một thành phần.

Các lớp hiếm khi được tái sử dụng đơn lẻ. Thường thấy hơn, các lớp có thể tái sử dụng cộng tác với các lớp khác thuộc về cùng một trừu tượng có thể tái sử dụng. CRP nói rằng các lớp này nên cùng nằm trong một thành phần. Trong một thành phần như vậy, chúng ta sẽ thấy các lớp có rất nhiều sự phụ thuộc lẫn nhau.

Một ví dụ đơn giản có thể là một lớp chứa (container) và các trình lặp (iterator) liên kết của nó. Các lớp này được tái sử dụng cùng nhau vì chúng bị ràng buộc chặt chẽ với nhau. Do đó, chúng nên nằm trong cùng một thành phần.

Nhưng CRP cho chúng ta biết không chỉ về việc lớp nào nên đặt cùng nhau vào một thành phần: Nó còn cho biết lớp nào **không** nên được giữ chung trong một thành phần. Khi một thành phần sử dụng một thành phần khác, một sự phụ thuộc giữa các thành phần được tạo ra. Có lẽ thành phần sử dụng chỉ dùng một lớp trong thành phần được sử dụng, nhưng điều đó vẫn không làm yếu đi sự phụ thuộc. Thành phần sử dụng vẫn phụ thuộc vào thành phần được sử dụng.

Bởi vì sự phụ thuộc đó, mỗi khi thành phần được sử dụng thay đổi, thành phần sử dụng có thể sẽ cần các thay đổi tương ứng. Ngay cả khi không cần thay đổi thành phần sử dụng, nó vẫn có khả năng cao phải được biên dịch lại, xác thực lại và triển khai lại. Điều này đúng ngay cả khi thành phần sử dụng không quan tâm đến thay đổi được thực hiện trong thành phần được sử dụng.

Do đó, khi chúng ta phụ thuộc vào một thành phần, chúng ta muốn đảm bảo rằng chúng ta phụ thuộc vào **mọi** lớp trong thành phần đó. Nói cách khác, chúng ta muốn đảm bảo rằng các lớp chúng ta đưa vào một thành phần là không tách rời được, mà không thể phụ thuộc vào một số mà không phụ thuộc vào những số khác. Nếu không, chúng ta sẽ phải triển khai lại nhiều thành phần hơn cần thiết, và lãng phí công sức đáng kể.

Vì vậy, CRP cho chúng ta biết nhiều hơn về những lớp **không** nên cùng nhau hơn là về những lớp **nên** cùng nhau. CRP nói rằng các lớp không bị ràng buộc chặt chẽ với nhau không nên nằm trong cùng một thành phần.

Mối quan hệ với ISP

CRP là phiên bản tổng quát của ISP. ISP khuyên chúng ta không nên phụ thuộc vào các lớp có phương thức mà chúng ta không dùng. CRP khuyên chúng ta không nên phụ thuộc vào các thành phần có lớp mà chúng ta không dùng.

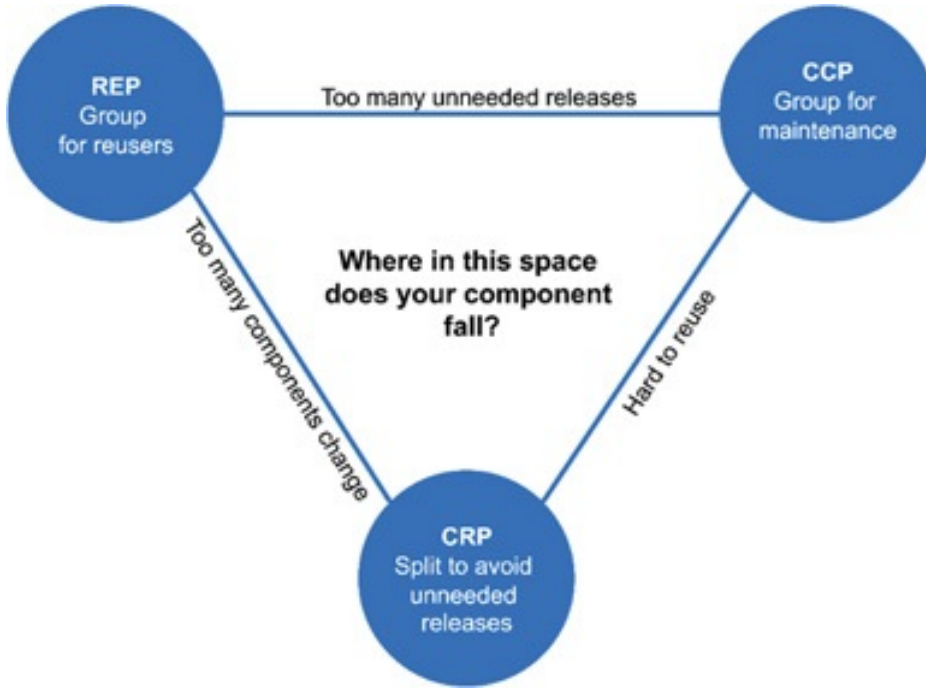
Tất cả lời khuyên này có thể rút gọn thành một câu slogan duy nhất:

Đừng phụ thuộc vào những thứ bạn không cần.

Biểu đồ căng thẳng về tính gắn kết của thành phần

Có thể bạn đã nhận ra rằng ba nguyên tắc gắn kết này có xu hướng chống lại nhau. REP và CCP là các nguyên tắc bao gồm (inclusive): Cả hai đều có xu hướng làm cho thành phần trở nên lớn hơn. CRP là một nguyên tắc loại trừ (exclusive), thúc đẩy thành phần nhỏ hơn. Chính sự căng thẳng giữa các nguyên tắc này là những gì các kiến trúc sư giỏi tìm cách giải quyết.

Hình 13.1 là một biểu đồ căng thẳng (tension diagram)[1] cho thấy ba nguyên tắc gắn kết tương tác với nhau như thế nào. Các cạnh của biểu đồ mô tả chi phí của việc từ bỏ nguyên tắc ở đỉnh đối diện.



Hình 13.1

Biểu đồ căng thẳng các nguyên tắc gắn kết

Một kiến trúc sư chỉ tập trung vào REP và CRP sẽ thấy quá nhiều thành phần bị ảnh hưởng khi thực hiện các thay đổi đơn giản. Ngược lại, một kiến trúc sư tập trung quá mạnh vào CCP và REP sẽ khiến quá nhiều bản phát hành không cần thiết được tạo ra.

Một kiến trúc sư giỏi tìm được một vị trí trong tam giác căng thẳng đó đáp ứng được mối quan tâm hiện tại của nhóm phát triển, nhưng cũng nhận thức rằng những mối quan tâm đó sẽ thay đổi theo thời gian. Ví dụ, sớm trong quá trình phát triển một dự án, CCP

1. Cảm ơn Tim Ottinger vì ý tưởng này.

quan trọng hơn REP nhiều, bởi vì khả năng phát triển (develop-ability) quan trọng hơn tái sử dụng.

Nói chung, các dự án thường bắt đầu ở bên phải của tam giác, nơi hy sinh duy nhất là tái sử dụng. Khi dự án trưởng thành, và các dự án khác bắt đầu lấy từ nó, dự án sẽ trượt sang bên trái. Điều này có nghĩa là cấu trúc thành phần của một dự án có thể thay đổi theo thời gian và mức độ trưởng thành. Nó liên quan nhiều hơn đến cách dự án được phát triển và sử dụng, hơn là đến việc dự án thực sự làm gì.

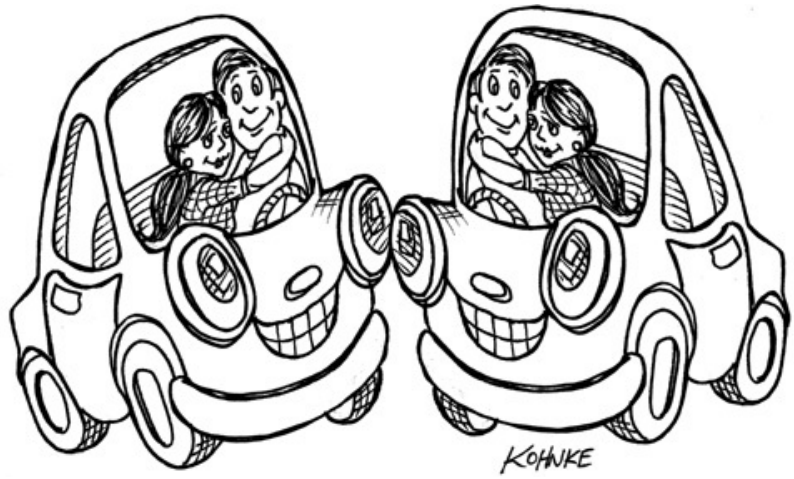
Kết luận

Trong quá khứ, quan điểm của chúng ta về sự gắn kết (cohesion) đơn giản hơn nhiều so với những gì REP, CCP và CRP hàm ý. Chúng ta từng nghĩ rằng sự gắn kết đơn giản là thuộc tính cho thấy một mô-đun thực hiện một, và chỉ một, chức năng. Tuy nhiên, ba nguyên tắc về sự gắn kết của thành phần miêu tả một dạng gắn kết phức tạp hơn nhiều. Khi chọn các lớp để gom nhóm lại thành thành phần, chúng ta phải cân nhắc các lực đối lập liên quan đến khả năng tái sử dụng và khả năng phát triển. Việc cân bằng các lực này với nhu cầu của ứng dụng là không tầm thường. Hơn nữa, sự cân bằng đó gần như luôn động. Tức là, sự phân chia phù hợp hôm nay có thể không còn phù hợp vào năm sau. Hệ quả là, cách tổ chức các thành phần trong hệ thống có thể dao động (jitter)[2] và tiến hóa theo thời gian khi trọng tâm của dự án thay đổi từ khả năng phát triển sang khả năng tái sử dụng.

2. Xem mục “The Kitty Problem” trong Chương 27, “Services: Great and Small”.

14

Sự kết nối giữa các thành phần



Ba nguyên tắc tiếp theo liên quan đến các mối quan hệ giữa các thành phần. Ở đây chúng ta lại gặp phải sự căng thẳng giữa khả năng phát triển và thiết kế logic. Các lực tác động lên kiến trúc của một cấu trúc thành phần mang tính kỹ thuật, chính trị và biến động.

Nguyên tắc phụ thuộc không chu trình (ADP)

Không cho phép chu trình trong biểu đồ phụ thuộc của thành phần.

Bạn có từng làm việc cả ngày, làm cho một số thứ chạy được, rồi về nhà, chỉ để sáng hôm sau nhận thấy những thứ đó không còn hoạt động nữa? Tại sao nó không chạy? Vì có ai đó ở lại muộn hơn bạn và thay đổi thứ mà bạn phụ thuộc vào! Tôi gọi đây là "hội chứng sáng hôm sau".

"Hội chứng sáng hôm sau" xảy ra trong các môi trường phát triển nơi nhiều nhà phát triển cùng sửa đổi các tệp nguồn giống nhau. Trong các dự án nhỏ với chỉ vài nhà phát triển, nó không quá nghiêm trọng. Nhưng khi quy mô dự án và nhóm phát triển tăng lên, những buổi sáng sau có thể trở nên kinh khủng. Không hiếm khi nhiều tuần trôi qua mà nhóm không thể build một phiên bản ổn định của dự án. Thay vào đó, mọi người cứ việc thay đổi mã của mình để làm cho nó hoạt động với những thay đổi vừa được người khác thực hiện.

Trong vài thập kỷ qua, hai giải pháp cho vấn đề này đã được phát triển, cả hai đều đến từ ngành viễn thông. Giải pháp thứ nhất là "build hàng tuần", và giải pháp thứ hai là Nguyên tắc Phụ thuộc Không chu trình (ADP).

Build hàng tuần

Build hàng tuần từng phổ biến trong các dự án trung bình. Nó hoạt động như sau: Tất cả nhà phát triển bỏ qua nhau trong bốn ngày đầu tuần. Tất cả đều làm việc trên các bản sao riêng của mã nguồn, và không lo lắng về việc tích hợp công việc trên cơ sở tập thể. Sau đó, vào thứ Sáu, họ tích hợp tất cả các thay đổi và build hệ thống.

Cách tiếp cận này có ưu điểm tuyệt vời là cho phép các nhà phát triển sống trong một thế giới biệt lập trong bốn ngày trên năm ngày. Nhược điểm, tất nhiên, là chi phí tích hợp lớn phải trả vào thứ Sáu.

Thật không may, khi dự án lớn lên, việc hoàn thành tích hợp vào thứ Sáu trở nên kém khả thi. Gánh nặng tích hợp tăng dần và bắt đầu kéo dài sang thứ Bảy. Sau vài lần làm việc vào thứ Bảy, các nhà phát triển sẽ muốn bắt đầu tích hợp từ thứ Năm, khiến thời điểm tích hợp dần lùi về giữa tuần.

Khi tỷ lệ nhiệm vụ giữa phát triển và tích hợp giảm xuống, hiệu suất của nhóm cũng giảm theo. Cuối cùng, tình huống này trở nên chán nản đến mức các nhà phát triển, hoặc người quản lý dự án, tuyên bố rằng lịch trình nên được thay đổi thành build hai tuần một lần. Điều này đủ dùng một thời gian, nhưng thời gian tích hợp vẫn tiếp tục tăng cùng với quy mô dự án.

Cuối cùng, kịch bản này dẫn đến khủng hoảng. Để duy trì hiệu suất, lịch build phải liên tục được kéo dài, nhưng làm vậy lại tăng rủi ro cho dự án. Việc tích hợp và kiểm thử ngày càng khó khăn, còn nhóm mất đi lợi ích của phản hồi nhanh.

Loại bỏ các chu trình phụ thuộc

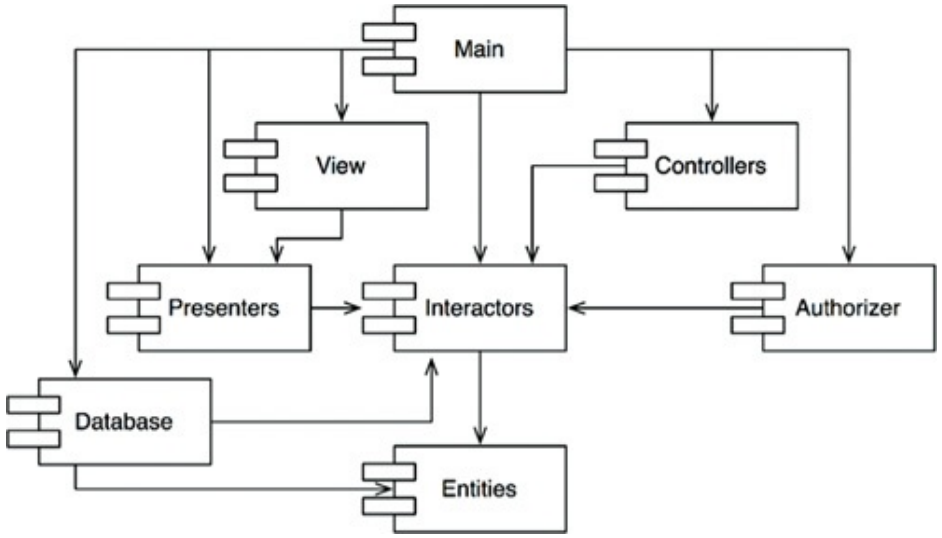
Giải pháp cho vấn đề này là phân chia môi trường phát triển thành các thành phần có thể phát hành. Các thành phần trở thành các đơn vị công việc có thể là trách nhiệm của một nhà phát triển đơn lẻ, hoặc một nhóm nhà phát triển. Khi các nhà phát triển làm cho một thành phần hoạt động, họ phát hành nó để các nhà phát triển khác sử dụng. Họ gán cho nó một số phát hành và di chuyển nó vào một thư mục để các nhóm khác dùng. Sau đó họ tiếp tục sửa đổi thành phần của mình trong khu vực riêng tư. Mọi người khác sử dụng phiên bản đã phát hành.

Khi các phát hành mới của một thành phần có sẵn, các nhóm khác có thể quyết định xem họ có nên áp dụng ngay phát hành mới hay không. Nếu họ quyết định không, họ chỉ cần tiếp tục dùng phiên bản cũ. Khi họ quyết định đã sẵn sàng, họ bắt đầu dùng phát hành mới.

Nhờ đó, không nhóm nào phải chờ nhóm khác. Thay đổi ở một thành phần không nhất thiết tác động ngay đến các nhóm khác. Mỗi nhóm tự quyết định khi nào cập nhật thành phần của mình để dùng bản phát hành mới từ các thành phần khác. Hơn nữa, việc tích hợp diễn ra từng bước nhỏ. Không có một thời điểm duy nhất mà tất cả nhà phát triển phải cùng tập trung để tích hợp mọi thứ họ đang làm.

Đây là một quy trình rất đơn giản và hợp lý, và nó được sử dụng rộng rãi. Tuy nhiên, để làm cho nó thành công, bạn phải quản lý cấu trúc phụ thuộc của các thành phần. **Không được có chu trình**. Nếu có chu trình trong cấu trúc phụ thuộc, thì "hội chứng sáng hôm sau" không thể tránh khỏi.

Xét biểu đồ thành phần trong Hình 14.1. Nó cho thấy một cấu trúc điển hình của các thành phần được lắp ráp thành một ứng dụng. Chức năng của ứng dụng này không quan trọng cho mục đích của ví dụ này. Điều quan trọng là cấu trúc phụ thuộc của các thành phần. Chú ý rằng cấu trúc này là một đồ thị có hướng. Các thành phần là các nút, và các mối quan hệ phụ thuộc là các cạnh có hướng.



Hình 14.1
Biểu đồ thành phần điển hình

Chú ý thêm một điều: Dù bạn bắt đầu từ thành phần nào, đều không thể theo dõi các mối quan hệ phụ thuộc và quay trở lại thành phần đó. Cấu trúc này không có chu trình. Nó là một đồ thị có hướng không chu trình (DAG).

Bây giờ xét điều xảy ra khi nhóm chịu trách nhiệm về Presenters tạo một phát hành mới của thành phần của họ. Dễ dàng tìm ra ai bị ảnh hưởng bởi phát hành này; bạn chỉ cần theo dõi các mũi tên phụ thuộc ngược lại. Do đó View và Main đều sẽ bị ảnh hưởng. Các nhà phát triển hiện đang làm việc trên các thành phần đó sẽ phải quyết định khi nào nên tích hợp công việc của họ với phát hành mới của Presenters.

Chú ý thêm rằng khi Main được phát hành, nó hoàn toàn không có tác động đến bất kỳ thành phần nào khác trong hệ thống. Chúng không biết về Main, và không quan tâm khi Main thay đổi. Điều này tốt. Nó có nghĩa là tác động của việc phát hành Main tương đối nhỏ.

Khi các nhà phát triển làm việc trên thành phần Presenters muốn chạy kiểm thử cho thành phần đó, họ chỉ cần build phiên bản Presenters của họ với các phiên bản của Interactors và Entities mà họ đang dùng. Không thành phần nào khác trong hệ thống cần được liên quan. Điều này tốt. Nó có nghĩa là các nhà phát triển làm việc trên Presenters có ít việc phải làm để thiết lập kiểm thử, và họ có ít biến số phải cân nhắc.

Khi đến lúc phát hành toàn bộ hệ thống, quy trình diễn ra từ dưới lên trên. Trước tiên thành phần Entities được biên dịch, kiểm thử, và phát hành. Sau đó làm tương tự cho Database và Interactors. Các thành phần này được theo sau bởi Presenters, View, Controllers, và sau đó là Authorizer. Main là cuối cùng. Quy trình này rất rõ ràng và dễ xử lý. Chúng ta biết cách build hệ thống vì chúng ta hiểu các phụ thuộc giữa các phần của nó.

Tác động của chu trình trong biểu đồ phụ thuộc thành phần

Giả sử một yêu cầu mới ép buộc chúng ta phải thay đổi một trong các lớp trong Entities để nó sử dụng một lớp trong Authorizer. Ví dụ, hãy nói rằng lớp User trong Entities sử dụng lớp Permissions trong Authorizer. Điều này tạo ra một chu trình phụ thuộc, như được minh họa trong Hình 14.2.

Chu trình này lập tức gây ra một số vấn đề. Chẳng hạn, các nhà phát triển làm việc trên thành phần Database biết rằng để phát hành, thành phần này phải tương thích với Entities. Nhưng khi đã có chu trình, Database còn phải tương thích với Authorizer, trong khi Authorizer lại phụ thuộc vào Interactors. Vì vậy, Database trở nên khó phát hành hơn nhiều. Trên thực tế, Entities, Authorizer và Interactors đã hợp thành một thành phần lớn. Mọi nhà phát triển làm việc trên bất kỳ thành phần nào trong số đó đều có thể gặp "hội chứng sáng hôm sau", vì họ buộc phải dùng đúng cùng một bản phát hành của các thành phần kia.

Nhưng đó mới chỉ là một phần vấn đề. Khi muốn kiểm thử thành phần Entities, thật thất vọng khi thấy chúng ta phải build và tích hợp nó với Authorizer và Interactors. Mức độ kết nối này giữa các thành phần thật đáng lo ngại, nếu không muốn nói là khó chịu.

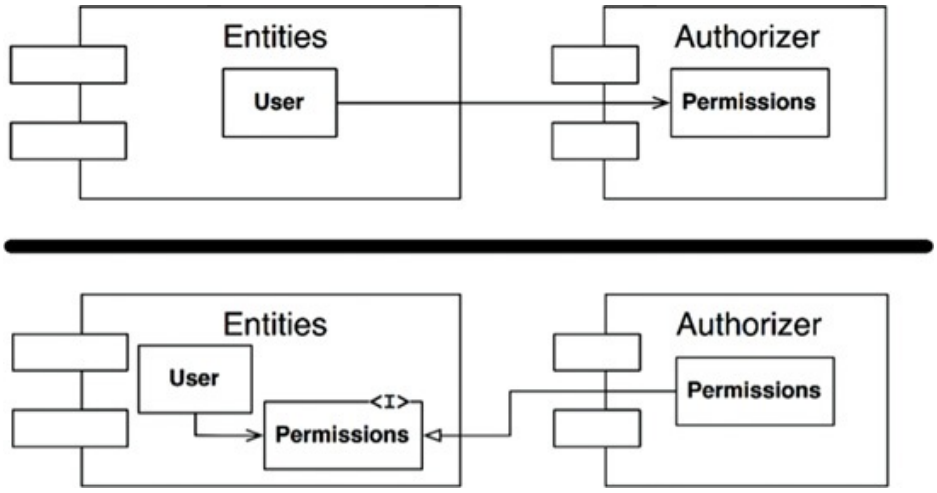
Bạn có thể đã thắc mắc tại sao phải đưa vào nhiều thư viện khác nhau đến vậy, và nhiều mã của người khác đến vậy, chỉ để chạy một bài kiểm thử đơn vị đơn giản của một lớp của mình. Nếu bạn điều tra một chút, bạn có thể sẽ phát hiện ra rằng có chu trình trong biểu đồ phụ thuộc. Các chu trình như vậy làm cho việc tách biệt các thành phần trở nên rất khó khăn. Kiểm thử đơn vị và phát hành trở nên rất khó khăn và dễ sai sót. Ngoài ra, các vấn đề về build tăng lên theo cấp số nhân với số lượng mô-đun.

Hơn nữa, khi có chu trình trong biểu đồ phụ thuộc, có thể rất khó xác định thứ tự mà bạn phải build các thành phần. Thực tế, có lẽ không có thứ tự đúng nào. Điều này có thể dẫn đến những vấn đề rất tồi tệ trong các ngôn ngữ như Java, nơi các khai báo được đọc từ các tệp nhị phân đã biên dịch.

Phá vỡ chu trình

Luôn có thể phá vỡ một chu trình của các thành phần và khôi phục biểu đồ phụ thuộc trở thành DAG. Có hai cơ chế chính để làm như vậy:

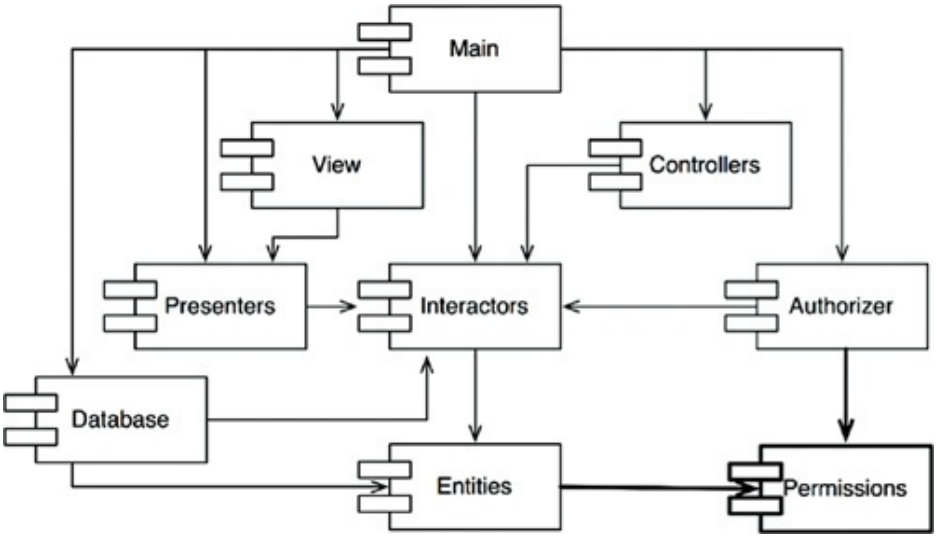
1. **Áp dụng Nguyên tắc Đảo ngược Phụ thuộc (DIP).** Trong trường hợp của Hình 14.3, chúng ta có thể tạo một giao diện có các phương thức mà User cần. Sau đó chúng ta đặt giao diện đó vào Entities và kế thừa nó vào Authorizer. Điều này đảo ngược phụ thuộc giữa Entities và Authorizer, từ đó phá vỡ chu trình.



Hình 14.3

Đảo ngược phụ thuộc giữa Entities và Authorizer

1. **Tạo một thành phần mới mà cả Entities và Authorizer đều phụ thuộc vào.** Di chuyển các lớp mà chúng cùng phụ thuộc vào thành phần mới đó (Hình 14.4).



Hình 14.4

Thành phần mới mà cả Entities và Authorizer đều phụ thuộc vào

Sự dao động (The Jitters)

Giải pháp thứ hai ngụ ý rằng cấu trúc thành phần có tính biến động khi các yêu cầu thay đổi. Thực tế, khi ứng dụng lớn lên, cấu trúc phụ thuộc của thành phần sẽ dao động và phát triển. Do đó, cấu trúc phụ thuộc phải luôn được giám sát để tìm chu trình. Khi chu trình xuất hiện, chúng phải được phá vỡ bằng cách nào đó. Đôi khi điều này sẽ có nghĩa là tạo ra các thành phần mới, làm cho cấu trúc phụ thuộc phát triển.

Thiết kế từ trên xuống (Top-Down Design)

Các vấn đề chúng ta đã thảo luận dẫn đến một kết luận không thể tránh khỏi: **Cấu trúc thành phần không thể được thiết kế từ trên xuống**. Nó không phải là một trong những thứ được thiết kế đầu tiên của hệ thống, mà là thứ tiến hóa khi hệ thống lớn lên và thay đổi.

Một số người đọc có thể thấy điểm này đi ngược trực giác. Chúng ta đã quen mong đợi rằng các phân chia lớn, như các thành phần, cũng sẽ là các phân chia chức năng ở cấp cao.

Khi chúng ta nhìn thấy một cấu trúc lớn như biểu đồ phụ thuộc thành phần, chúng ta tin rằng các thành phần nên đại diện cho các chức năng của hệ thống. Tuy nhiên, dường như đây không phải là một thuộc tính của các biểu đồ phụ thuộc thành phần.

Thực tế, các biểu đồ phụ thuộc thành phần có rất ít liên quan đến việc mô tả chức năng của ứng dụng. Thay vào đó, chúng là một bản đồ cho khả năng build và khả năng bảo trì của ứng dụng. Đây là lý do tại sao chúng không được thiết kế ở đầu dự án. Không có phần mềm nào để build hoặc bảo trì, nên không có nhu cầu cho một bản đồ build và bảo trì. Nhưng khi càng nhiều mô-đun tích lũy trong các giai đoạn đầu của triển khai và thiết kế, có một nhu cầu ngày càng lớn để quản lý các phụ thuộc sao cho dự án có thể được phát triển mà không gặp "hội chứng sáng hôm sau". Hơn nữa, chúng ta muốn giữ các thay đổi càng cục bộ càng tốt, nên chúng ta bắt đầu chú ý đến SRP và CCP và đặt cùng nhau các lớp có khả năng thay đổi cùng lúc.

Một trong những mối quan tâm áp đảo với cấu trúc phụ thuộc này là sự cô lập sự biến động. Chúng ta không muốn các thành phần thay đổi thường xuyên và vì những lý do ngẫu nhiên tác động đến các thành phần mà nếu không thì nên ổn định. Ví dụ, chúng ta không muốn các thay đổi về mặt thẩm mỹ của GUI tác động đến các quy tắc kinh doanh của chúng ta. Chúng ta không muốn việc thêm hoặc sửa đổi báo cáo tác động đến các chính sách ở cấp cao nhất. Do đó, biểu đồ phụ thuộc thành phần được tạo ra và định hình bởi các kiến trúc sư để bảo vệ các thành phần ổn định, giá trị cao khỏi các thành phần biến động.

Khi ứng dụng tiếp tục phát triển, chúng ta bắt đầu quan tâm đến việc tạo ra các phần tử có thể tái sử dụng. Tại thời điểm này, CRP bắt đầu ảnh hưởng đến cấu thành của các thành phần. Cuối cùng, khi chu trình xuất hiện, ADP được áp dụng và biểu đồ phụ thuộc thành phần dao động và phát triển.

Nếu chúng ta cố gắng thiết kế cấu trúc phụ thuộc thành phần trước khi thiết kế bất kỳ lớp nào, chúng ta có thể sẽ thất bại khá tệ. Chúng ta sẽ không biết nhiều về sự đóng chung, chúng ta sẽ không biết về bất kỳ phần tử có thể tái sử dụng nào, và chúng ta gần như chắc chắn sẽ tạo ra các thành phần sản sinh ra chu trình phụ thuộc. Do đó, cấu trúc phụ thuộc thành phần lớn lên và tiến hóa cùng với thiết kế logic của hệ thống.

Nguyên tắc phụ thuộc ổn định (SDP)

Phụ thuộc theo hướng ổn định.

Thiết kế không thể hoàn toàn tĩnh. Một số sự biến động là cần thiết nếu thiết kế phải được bảo trì. Bằng cách tuân theo Nguyên tắc Đóng chung (CCP), chúng ta tạo ra các thành phần nhạy cảm với một số loại thay đổi nhưng miễn dịch với những loại khác. Một số thành phần trong số này được thiết kế để biến động. Chúng ta mong đợi chúng thay đổi.

Bất kỳ thành phần nào mà chúng ta mong đợi sẽ biến động **không nên** được phụ thuộc vào bởi một thành phần khó thay đổi. Nếu không, thành phần biến động cũng sẽ trở nên khó thay đổi.

Đây là sự nghịch lý của phần mềm: một mô-đun mà bạn đã thiết kế để dễ thay đổi có thể bị biến thành khó thay đổi bởi ai đó chỉ cần treo một sự phụ thuộc lên nó. Không một dòng mã nguồn nào trong mô-đun của bạn cần thay đổi, nhưng mô-đun của bạn đột ngột trở nên khó thay đổi hơn. Bằng cách tuân theo Nguyên tắc Phụ thuộc Ổn định (SDP), chúng ta đảm bảo rằng các mô-đun được dự định dễ thay đổi không bị phụ thuộc vào bởi các mô-đun khó thay đổi hơn.

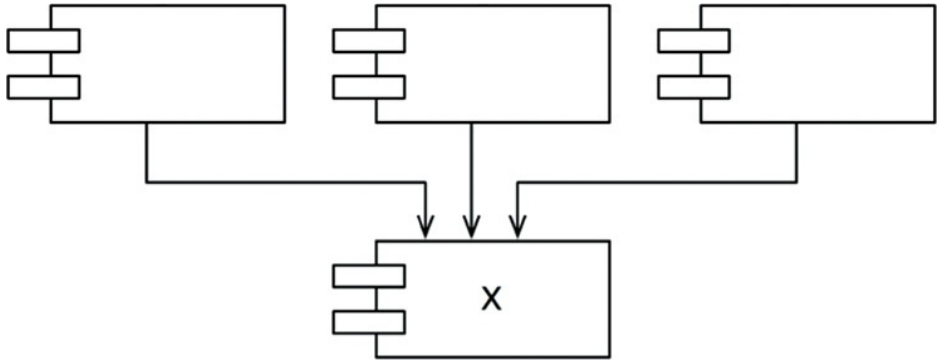
Ổn định (Stability)

"Ổn định" ở đây có nghĩa là gì? Đặt một đồng xu đứng dọc. Nó có ổn định ở tư thế đó không? Bạn có thể nói "không". Tuy nhiên, trừ khi bị xao động, nó sẽ giữ nguyên tư thế đó trong một thời gian rất dài. Do đó, sự ổn định không liên quan trực tiếp đến tần suất thay đổi. Đồng xu không thay đổi, nhưng khó mà coi nó là ổn định.

Từ điển Webster nói rằng một thứ là ổn định nếu nó "không dễ bị di chuyển". Sự ổn định liên quan đến lượng công sức cần thiết để thực hiện một thay đổi. Một mặt, đồng xu đứng dọc không ổn định vì nó cần rất ít công sức để đẩy ngã nó. Mặt khác, một chiếc bàn rất ổn định vì nó cần một lượng công sức đáng kể để lật nó ngửa.

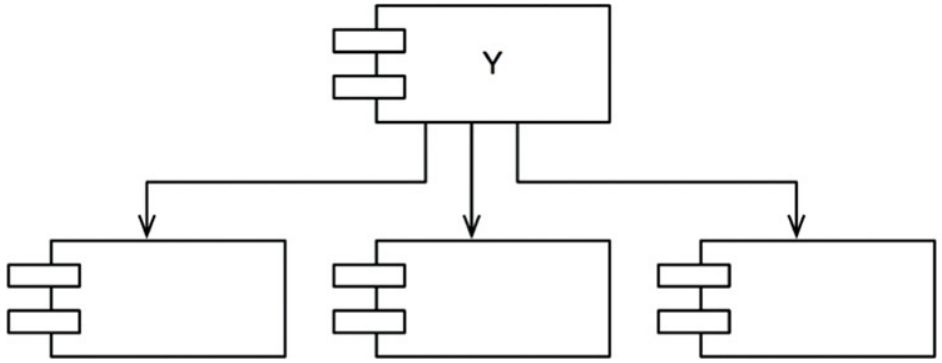
Điều này liên quan gì đến phần mềm? Nhiều yếu tố có thể khiến một thành phần phần mềm khó thay đổi, chẳng hạn kích thước, độ phức tạp và mức độ rõ ràng. Ở đây, chúng ta sẽ bỏ qua những yếu tố đó và tập trung vào điểm khác biệt sau: **Một cách chắc chắn để khiến thành phần phần mềm khó thay đổi là làm cho nhiều thành phần khác phụ thuộc vào nó.** Thành phần có nhiều phụ thuộc đến rất ổn định, vì cần nhiều công sức phối hợp với mọi thành phần phụ thuộc trước khi thay đổi được thực hiện.

Biểu đồ trong Hình 14.5 cho thấy X, một thành phần ổn định. Ba thành phần phụ thuộc vào X, nên nó có ba lý do tốt để không thay đổi. Chúng ta nói rằng X chịu trách nhiệm đối với ba thành phần đó. Ngược lại, X không phụ thuộc vào bất cứ thứ gì, nên nó không có ảnh hưởng bên ngoài để làm nó thay đổi. Chúng ta nói nó độc lập.



Hình 14.5
X: một thành phần ổn định

Hình 14.6 cho thấy Y, một thành phần rất không ổn định. Không thành phần nào khác phụ thuộc vào Y, nên chúng ta nói rằng nó không chịu trách nhiệm. Y cũng có ba thành phần mà nó phụ thuộc vào, nên thay đổi có thể đến từ ba nguồn bên ngoài. Chúng ta nói rằng Y phụ thuộc.



Hình 14.6
Y: một thành phần rất không ổn định

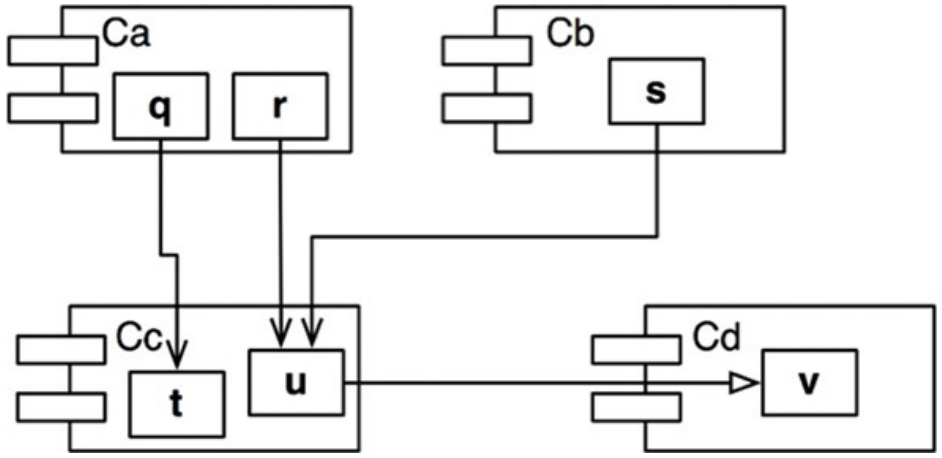
Các chỉ số Ổn định

Làm thế nào để đo lường sự ổn định của một thành phần? Một cách là đếm số phụ thuộc đi vào và đi ra khỏi thành phần đó. Các số đếm này sẽ cho phép chúng ta tính toán sự ổn định vị trí của thành phần.

- **Fan-in (Độ vào):** Phụ thuộc đến. Chỉ số này xác định số lượng lớp bên ngoài thành phần này mà phụ thuộc vào các lớp bên trong thành phần.
- **Fan-out (Độ ra):** Phụ thuộc đi. Chỉ số này xác định số lượng lớp bên trong thành phần này mà phụ thuộc vào các lớp bên ngoài thành phần.

- **I: Độ không ổn định (Instability):** I = Fan-out / (Fan-in + Fan-out). Chỉ số này có phạm vi [0, 1]. I = 0 chỉ ra một thành phần ổn định tối đa. I = 1 chỉ ra một thành phần không ổn định tối đa.

Các chỉ số Fan-in và Fan-out[1] được tính bằng cách đếm số lượng các lớp bên ngoài thành phần đang xét mà có sự phụ thuộc với các lớp bên trong thành phần đang xét. Xét ví dụ trong Hình 14.7.



Hình 14.7
Ví dụ của chúng ta

Giả sử chúng ta muốn tính sự ổn định của thành phần Cc. Chúng ta thấy có ba lớp bên ngoài Cc phụ thuộc vào các lớp trong Cc. Do đó, Fan-in = 3. Hơn nữa, có một lớp bên ngoài Cc mà các lớp trong Cc phụ thuộc vào. Do đó, Fan-out = 1 và I = 1/4.

Trong C++, các phụ thuộc này thường được biểu diễn bằng các câu lệnh #include. Thực tế, chỉ số I dễ tính nhất khi bạn tổ chức mã nguồn sao cho mỗi tệp nguồn chứa một lớp. Trong Java, chỉ số I có thể được tính bằng cách đếm các câu lệnh import và tên đủ điều kiện.

Khi chỉ số I bằng 1, điều đó có nghĩa là không thành phần nào khác phụ thuộc vào thành phần này (Fan-in = 0), và thành phần này phụ thuộc vào các thành phần khác (Fan-out > 0). Tình huống này là mức không ổn định nhất mà một thành phần có thể đạt được; nó là không chịu trách nhiệm và phụ thuộc. Sự thiếu hụt thành phần phụ thuộc của nó không cho thành phần lý do nào để không thay đổi, và các thành phần mà nó phụ thuộc vào có thể cho nó nhiều lý do để thay đổi.

Ngược lại, khi chỉ số I bằng 0, điều đó có nghĩa là thành phần được các thành phần khác phụ thuộc vào (Fan-in > 0), nhưng bản thân nó không phụ thuộc vào bất kỳ thành phần nào khác (Fan-out = 0). Thành phần như vậy là chịu trách nhiệm và độc lập. Nó ổn định

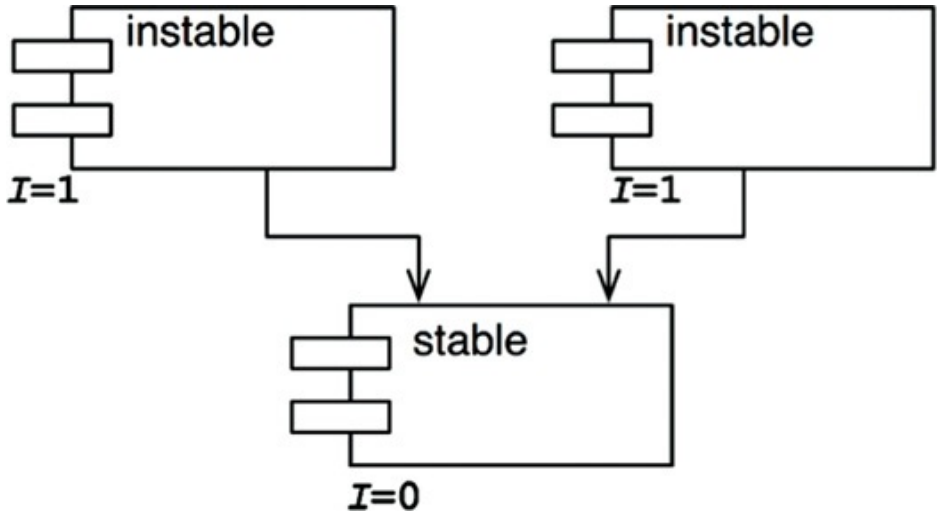
1. Trong các xuất bản trước, tôi dùng tên Efferent và Afferent couplings (Ce và Ca) cho Fan-out và Fan-in, tương ứng. Đó chỉ là sự kiêu ngạo của tôi: tôi thích ẩn dụ về hệ thần kinh trung ương.

nhất có thể. Các thành phần phụ thuộc của nó làm cho việc thay đổi thành phần trở nên khó khăn, và nó không có phụ thuộc nào có thể ép nó phải thay đổi.

SDP nói rằng chỉ số I của một thành phần nên lớn hơn các chỉ số I của các thành phần mà nó phụ thuộc vào. Đó là, các chỉ số I nên giảm theo hướng phụ thuộc.

Không phải thành phần nào cũng nên ổn định

Nếu tất cả các thành phần trong một hệ thống đều ổn định tối đa, hệ thống sẽ không thể thay đổi. Đây không phải là tình huống mong muốn. Thực tế, chúng ta muốn thiết kế cấu trúc thành phần của mình sao cho một số thành phần không ổn định và một số ổn định. Biểu đồ trong Hình 14.8 cho thấy một cấu hình lý tưởng cho một hệ thống với ba thành phần.

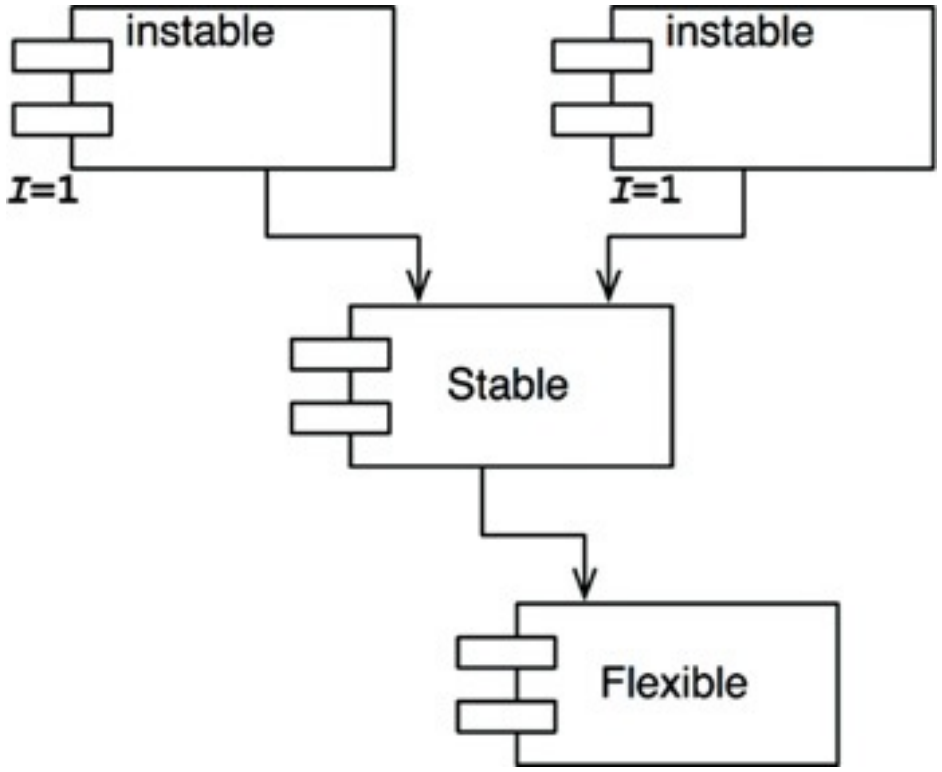


Hình 14.8

Cấu hình lý tưởng cho hệ thống ba thành phần

Các thành phần có thể thay đổi nằm ở trên và phụ thuộc vào thành phần ổn định ở dưới. Việc đặt các thành phần không ổn định ở trên cùng của biểu đồ là một quy ước hữu ích vì bất kỳ mũi tên nào chỉ lên trên đều vi phạm SDP (và, như chúng ta sẽ thấy sau, cả ADP).

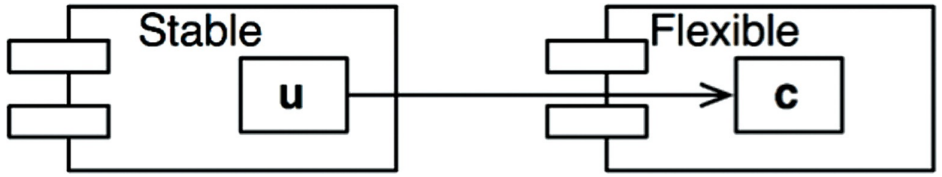
Biểu đồ trong Hình 14.9 cho thấy SDP có thể bị vi phạm như thế nào.



Hình 14.9
Vi phạm SDP

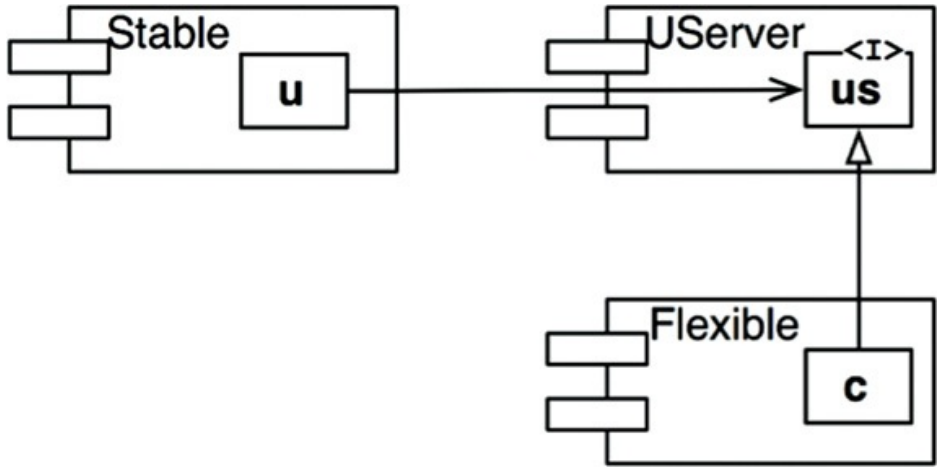
Flexible là một thành phần mà chúng ta đã thiết kế để dễ thay đổi. Chúng ta muốn Flexible không ổn định. Tuy nhiên, một nhà phát triển nào đó, đang làm việc trong thành phần tên là Stable, đã treo một sự phụ thuộc lên Flexible. Điều này vi phạm SDP vì chỉ số I cho Stable nhỏ hơn nhiều so với chỉ số I cho Flexible. Hệ quả là Flexible sẽ không còn dễ thay đổi nữa. Một thay đổi đối với Flexible sẽ ép buộc chúng ta phải đối phó với Stable và tất cả các thành phần phụ thuộc của nó.

Để khắc phục vấn đề này, chúng ta phải làm cách nào đó phá vỡ sự phụ thuộc của Stable vào Flexible. Tại sao sự phụ thuộc này tồn tại? Giả sử có một lớp C bên trong Flexible mà một lớp U bên trong Stable cần sử dụng (Hình 14.10).



Hình 14.10
U trong Stable sử dụng C trong Flexible

Chúng ta có thể khắc phục bằng cách sử dụng DIP. Chúng ta tạo một lớp giao diện tên là US và đặt nó trong một thành phần tên là UServer. Chúng ta đảm bảo rằng giao diện này khai báo tất cả các phương thức mà U cần dùng. Sau đó chúng ta làm cho C triển khai giao diện này như được minh họa trong Hình 14.11. Điều này phá vỡ sự phụ thuộc của Stable vào Flexible, và ép buộc cả hai thành phần phụ thuộc vào UServer. UServer rất ổn định (I = 0), và Flexible giữ được sự không ổn định cần thiết của nó (I = 1). Tất cả các phụ thuộc bây giờ đều chảy theo hướng giảm I.



Hình 14.11
C triển khai lớp giao diện US

Các thành phần trừu tượng (Abstract Components)

Bạn có thể thấy lạ khi chúng ta tạo một thành phần, trong ví dụ này là UService, chỉ chứa một giao diện. Thành phần như vậy không có mã thực thi! Tuy nhiên, đây là chiến thuật phổ biến và cần thiết khi sử dụng ngôn ngữ định kiểu tĩnh như Java và C#. Những thành phần trừu tượng này rất ổn định, nên là đích phụ thuộc lý tưởng cho các thành phần kém ổn định hơn.

Trong các ngôn ngữ định kiểu động như Ruby và Python, những thành phần trừu tượng này hoàn toàn không tồn tại, cũng như các phụ thuộc vốn sẽ hướng đến chúng. Cấu trúc phụ thuộc trong các ngôn ngữ này đơn giản hơn nhiều vì đảo ngược phụ thuộc không cần khai báo hay kế thừa giao diện.

Nguyên tắc trừu tượng ổn định (SAP)

Một thành phần nên trừu tượng đến mức nó ổn định.

Chúng ta đặt chính sách cấp cao ở đâu?

Một số phần mềm trong hệ thống không nên thay đổi thường xuyên. Chúng đại diện cho các quyết định kiến trúc và chính sách cấp cao. Chúng ta không muốn những quyết định kinh doanh và kiến trúc này biến động. Vì vậy, mã nguồn đóng gói các chính sách cấp cao nên nằm trong các thành phần ổn định (I = 0). Các thành phần không ổn định (I = 1) chỉ nên chứa phần mềm có tính biến động, tức phần mềm cần được thay đổi nhanh chóng và dễ dàng.

Tuy nhiên, nếu các chính sách cấp cao được đặt vào các thành phần ổn định, thì mã nguồn đại diện cho các chính sách đó sẽ khó thay đổi. Điều này có thể làm cho kiến trúc tổng thể trở nên cứng nhắc. Làm thế nào một thành phần ổn định tối đa (I = 0) có thể đủ linh hoạt để chịu đựng được thay đổi? Câu trả lời nằm trong OCP. Nguyên tắc này nói rằng có thể và mong muốn tạo ra các lớp đủ linh hoạt để được mở rộng mà không yêu cầu sửa đổi. Loại lớp nào tuân theo nguyên tắc này? Lớp trừu tượng.

Giới thiệu Nguyên tắc Trừu tượng Ổn định

Nguyên tắc Trừu tượng Ổn định (SAP) thiết lập một mối quan hệ giữa sự ổn định và mức độ trừu tượng. Một mặt, nó nói rằng một thành phần ổn định cũng nên là trừu tượng để sự ổn định của nó không ngăn cản nó được mở rộng. Mặt khác, nó nói rằng một thành phần không ổn định nên là cụ thể vì sự không ổn định của nó cho phép mã cụ thể bên trong nó dễ dàng được thay đổi.

Do đó, nếu một thành phần sẽ ổn định, nó nên bao gồm các giao diện và các lớp trừu tượng để có thể được mở rộng. Các thành phần ổn định có thể mở rộng là linh hoạt và không khiến kiến trúc bị ràng buộc quá chặt.

SDP và SAP kết hợp lại tương đương với DIP cho các thành phần. Điều này đúng vì SDP nói rằng các phụ thuộc nên chạy theo hướng ổn định, và SAP nói rằng ổn định ngụ ý trừu tượng. Do đó, các phụ thuộc chạy theo hướng trừu tượng.

Tuy nhiên, DIP là nguyên tắc áp dụng cho các lớp, và lớp không có vùng xám: hoặc trừu tượng, hoặc không. SDP kết hợp với SAP áp dụng cho các thành phần, nên một thành phần có thể vừa trừu tượng một phần, vừa ổn định một phần.

Đo lường mức độ trừu tượng

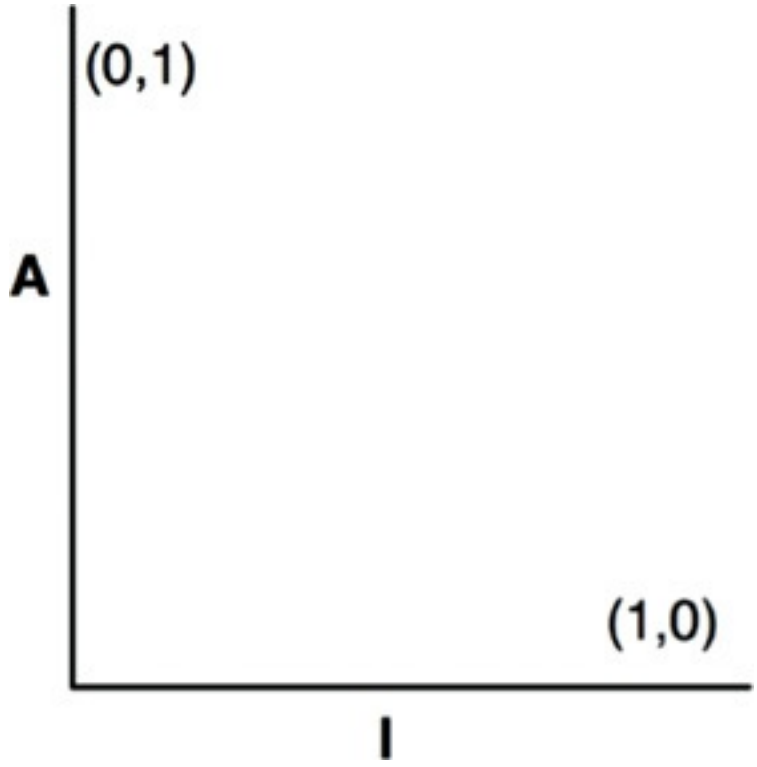
Chỉ số A là một thước đo của mức độ trừu tượng của một thành phần. Giá trị của nó đơn giản là tỷ lệ giữa số giao diện và lớp trừu tượng trong một thành phần với tổng số lớp trong thành phần đó.

- **Nc:** Số lượng lớp trong thành phần.
- **Na:** Số lượng lớp trừu tượng và giao diện trong thành phần.
- **A: Mức độ trừu tượng (Abstractness).** A = Na ÷ Nc.

Chỉ số A dao động từ 0 đến 1. Giá trị 0 ngụ ý rằng thành phần không có lớp trừu tượng nào. Giá trị 1 ngụ ý rằng thành phần chỉ chứa các lớp trừu tượng.

Dãy chính (Main Sequence)

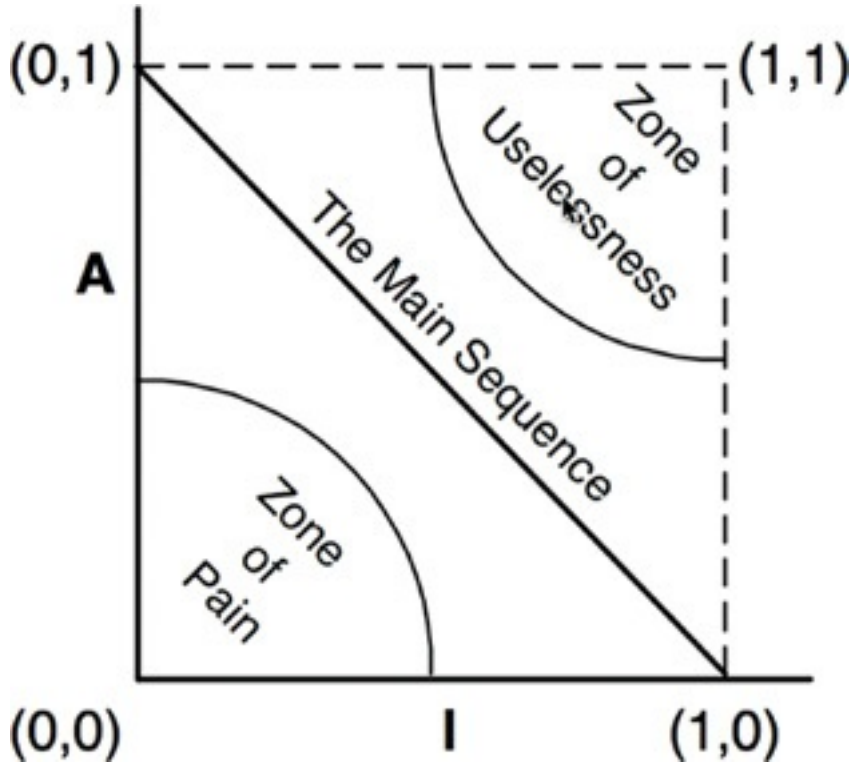
Chúng ta hiện đang ở vị trí xác định mối quan hệ giữa sự ổn định (I) và mức độ trừu tượng (A). Để làm được điều này, chúng ta tạo một biểu đồ với A trên trục tung và I trên trục hoành (Hình 14.12). Nếu chúng ta vẽ hai loại thành phần "tốt" lên biểu đồ này, chúng ta sẽ tìm thấy các thành phần ổn định tối đa và trừu tượng tối đa ở góc trên bên trái tại (0, 1). Các thành phần không ổn định tối đa và cụ thể nằm ở góc dưới bên phải tại (1, 0).



Hình 14.12
Biểu đồ I/A

Không phải thành phần nào cũng rơi vào một trong hai vị trí này, vì các thành phần thường có các độ trừu tượng và ổn định. Ví dụ, rất phổ biến khi một lớp trừu tượng kế thừa từ một lớp trừu tượng khác. Lớp dẫn xuất là một trừu tượng có một phụ thuộc. Do đó, dù nó trừu tượng tối đa, nó sẽ không ổn định tối đa. Phụ thuộc của nó sẽ giảm sự ổn định của nó.

Vì không thể yêu cầu mọi thành phần phải nằm tại (0, 1) hoặc (1, 0), ta cần xác định một tập hợp các điểm hợp lý trên biểu đồ A/I. Có thể tìm ra tập hợp đó bằng cách xác định những vùng mà thành phần **không** nên nằm, tức các vùng loại trừ (Hình 14.13).



Hình 14.13
Các vùng loại trừ

Vùng đau khổ (Zone of Pain)

Xét một thành phần trong khu vực (0, 0). Đây là một thành phần ổn định cao và cụ thể. Thành phần như vậy không mong muốn vì nó cứng nhắc. Nó không thể được mở rộng vì nó không trừu tượng, và nó rất khó thay đổi do sự ổn định của nó. Do đó chúng ta thường không mong đợi thấy các thành phần thiết kế tốt ngồi gần (0, 0). Khu vực quanh (0, 0) là

một vùng loại trừ được gọi là Vùng đau khổ.

Một số thực thể phần mềm thực sự rơi vào Vùng đau khổ. Một ví dụ là lược đồ cơ sở dữ liệu. Các lược đồ cơ sở dữ liệu nổi tiếng là biến động, cực kỳ cụ thể, và bị phụ thuộc nhiều. Đây là một lý do tại sao giao diện giữa ứng dụng hướng đối tượng và cơ sở dữ liệu rất khó quản lý, và tại sao cập nhật lược đồ thường đau đớn.

Một ví dụ khác về phần mềm ngồi gần khu vực (0, 0) là thư viện tiện ích cụ thể. Dù thư viện như vậy có chỉ số I = 1, nó thực tế có thể không biến động. Xét thành phần String, ví dụ. Dù tất cả các lớp trong nó đều cụ thể, nó được dùng phổ biến đến mức thay đổi nó sẽ tạo ra hỗn loạn. Do đó String là không biến động.

Các thành phần không biến động là vô hại trong vùng (0, 0) vì chúng không có khả năng bị thay đổi. Với lý do đó, chỉ có các thành phần phần mềm biến động mới gây vấn đề trong Vùng đau khổ. Càng biến động một thành phần trong Vùng đau khổ, nó càng "đau khổ". Thực tế, chúng ta có thể coi sự biến động là trục thứ ba của biểu đồ. Với hiểu biết này, Hình 14.13 chỉ cho thấy mặt phẳng đau khổ nhất, nơi độ biến động = 1.

Vùng vô dụng (Zone of Uselessness)

Xét một thành phần gần (1, 1). Vị trí này không mong muốn vì nó trừu tượng tối đa, nhưng không có thành phần phụ thuộc. Các thành phần như vậy là vô dụng. Do đó khu vực này được gọi là Vùng vô dụng.

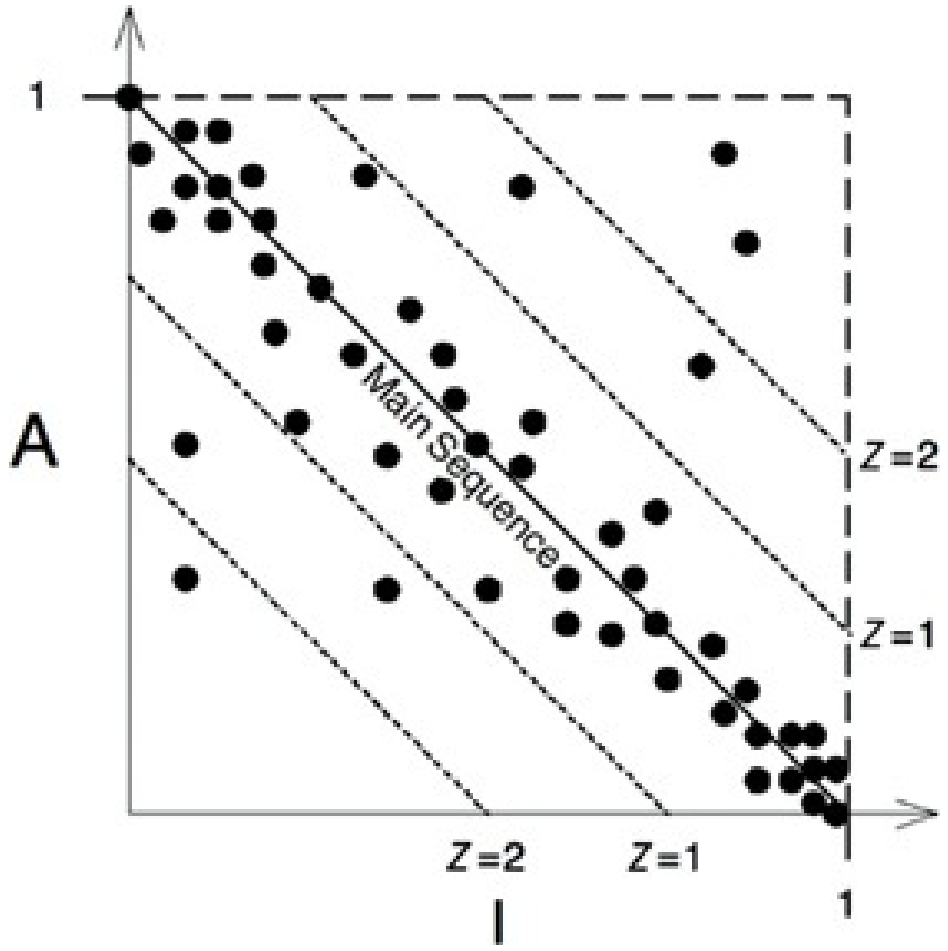
Các thực thể phần mềm trú ngụ trong khu vực này là một loại phế liệu. Chúng thường là các lớp trừu tượng thừa không ai bao giờ triển khai. Chúng ta tìm thấy chúng trong các hệ thống từ thời gian qua, ngồi trong cơ sở mã, không được dùng.

Một thành phần có vị trí sâu bên trong Vùng vô dụng phải chứa một phần đáng kể các thực thể như vậy. Rõ ràng, sự hiện diện của các thực thể vô dụng như vậy là không mong muốn.

Tránh các vùng loại trừ

Rõ ràng là các thành phần biến động nhất của chúng ta nên được giữ càng xa càng tốt khỏi cả hai vùng loại trừ. Tập hợp các điểm xa nhất từ mỗi vùng là đường nối (1, 0) và (0, 1). Tôi gọi đường này là Dãy chính[2].

2. Tác giả xin phép người đọc cho phép mượn một thuật ngữ quan trọng như vậy từ thiên văn học.



Hình 14.14
Biểu đồ phân tán của các thành phần

Một thành phần ngồi trên Dãy chính không "quá trừu tượng" cho sự ổn định của nó, cũng không "quá không ổn định" cho mức độ trừu tượng của nó. Nó không vô dụng cũng không đặc biệt đau khổ. Nó bị phụ thuộc vào ở mức độ mà nó trừu tượng, và nó phụ thuộc vào người khác ở mức độ mà nó cụ thể.

Vị trí mong muốn nhất cho một thành phần là ở một trong hai đầu mút của Dãy chính. Các kiến trúc sư giỏi nỗ lực đặt phần lớn thành phần của họ ở các đầu mút đó. Tuy nhiên, theo kinh nghiệm của tôi, một phần nhỏ các thành phần trong một hệ thống lớn không hoàn toàn trừu tượng cũng không hoàn toàn ổn định. Các thành phần đó có đặc điểm tốt nhất nếu chúng ở trên, hoặc gần, Dãy chính.

Khoảng cách từ Dãy chính

Điều này dẫn chúng ta đến chỉ số cuối cùng. Nếu các thành phần mong muốn nằm trên, hoặc gần, Dãy chính, thì chúng ta có thể tạo một chỉ số đo lường một thành phần xa đến đâu từ lý tưởng này.

- **D: Khoảng cách (Distance).** $D = |A + I - 1|^3$. Phạm vi của chỉ số là [0, 1]. Giá trị 0 cho biết thành phần nằm đúng trên Dãy chính. Giá trị 1 cho biết nó nằm xa Dãy chính nhất có thể.

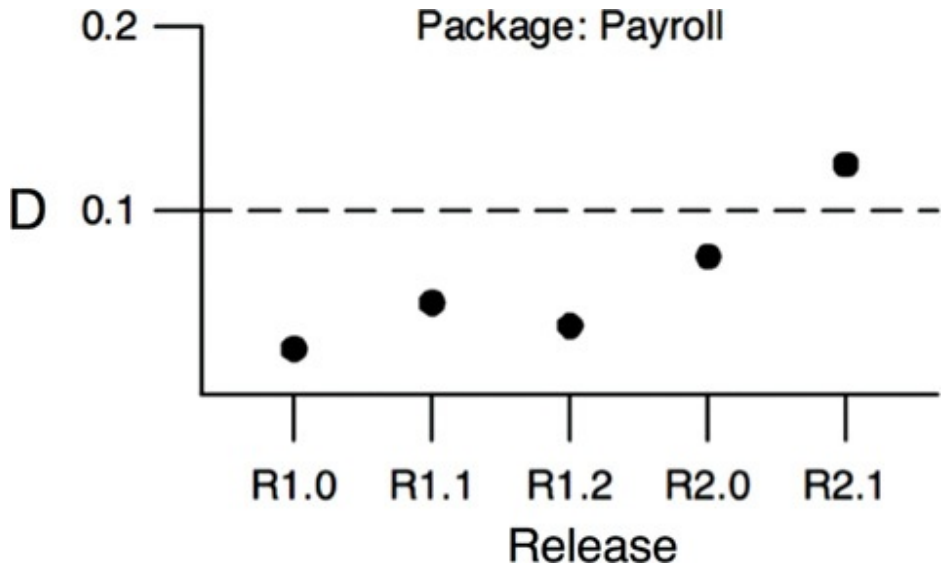
Cho chỉ số này, một thiết kế có thể được phân tích cho sự tuân thủ tổng thể đối với Dãy chính. Chỉ số D cho mỗi thành phần có thể được tính. Bất kỳ thành phần nào có giá trị D không gần bằng không có thể được xem xét lại và cấu trúc lại. Phân tích thống kê của một thiết kế cũng là khả thi. Chúng ta có thể tính trung bình và phương sai của tất cả các chỉ số D cho các thành phần trong một thiết kế. Chúng ta mong đợi một thiết kế tuân thủ có trung bình và phương sai gần bằng không. Phương sai có thể được dùng để thiết lập "giới hạn kiểm soát" để nhận diện các thành phần "ngoài lệ" so với tất cả các thành phần khác.

Trong biểu đồ phân tán trong Hình 14.14, chúng ta thấy phần lớn các thành phần nằm dọc theo Dãy chính, nhưng một số trong số chúng xa hơn một độ lệch chuẩn (Z = 1) từ trung bình. Những thành phần bất thường này đáng giá xem xét kỹ hơn. Với một lý do nào đó, chúng hoặc rất trừu tượng với ít thành phần phụ thuộc, hoặc rất cụ thể với nhiều thành phần phụ thuộc.

Một cách khác để dùng các chỉ số là vẽ chỉ số D của mỗi thành phần theo thời gian. Biểu đồ trong Hình 14.15 là bản mô phỏng của một biểu đồ như vậy. Bạn có thể thấy một số phụ thuộc kỳ lạ đã từ từ len lỏi vào thành phần Payroll trong vài phát hành gần đây. Biểu đồ cho thấy ngưỡng kiểm soát tại D = 0.1. Điểm R2.1 đã vượt quá giới hạn kiểm soát này, vì vậy sẽ đáng giá để tìm hiểu tại sao thành phần này lại xa Dãy chính đến vậy.

---

3. Trong các xuất bản trước, tôi gọi chỉ số này là D′. Tôi không thấy lý do nào để tiếp tục thực hành đó.



Hình 14.15

Biểu đồ D của một thành phần đơn lẻ theo thời gian

Kết luận

Các chỉ số quản lý phụ thuộc được mô tả trong chương này đo lường sự tuân thủ của một thiết kế đối với một mẫu phụ thuộc và trừu tượng mà tôi nghĩ là một mẫu "tốt". Kinh nghiệm đã chỉ ra rằng một số phụ thuộc là tốt và những cái khác xấu. Mẫu này phản ánh kinh nghiệm đó. Tuy nhiên, một chỉ số không phải là thần thánh; nó chỉ là phép đo theo một tiêu chuẩn tùy ý. Các chỉ số này, ở mức tốt nhất, cũng chỉ không hoàn hảo, nhưng tôi hy vọng bạn thấy chúng hữu ích.

Phần V: Kiến trúc

15

Kiến trúc là gì?



Từ "kiến trúc" gợi lên những hình ảnh về quyền lực và sự bí ẩn. Nó khiến ta nghĩ đến những quyết định quan trọng và sự am hiểu kỹ thuật thâm sâu. Kiến trúc phần mềm nằm ở đỉnh cao của thành tựu kỹ thuật. Khi nghĩ đến một kiến trúc sư phần mềm, ta hình dung một người có quyền lực và được kính trọng. Lập trình viên trẻ nào không từng mơ ước một ngày nào đó trở thành kiến trúc sư phần mềm?

Nhưng kiến trúc phần mềm là gì? Kiến trúc sư phần mềm làm gì, và khi nào họ làm

nó?

Thứ nhất, một kiến trúc sư phần mềm là một lập trình viên; và tiếp tục là một lập trình viên. Đừng bao giờ tin vào lời nói dối cho rằng kiến trúc sư phần mềm lui lại khỏi mã nguồn để tập trung vào các vấn đề cấp độ cao hơn. Họ không làm vậy! Kiến trúc sư phần mềm là những lập trình viên xuất sắc nhất, và họ tiếp tục nhận các nhiệm vụ lập trình, đồng thời dẫn dắt phần còn lại của nhóm về phía một thiết kế giúp tối đa hóa năng suất. Kiến trúc sư phần mềm có thể không viết nhiều mã như các lập trình viên khác, nhưng họ tiếp tục tham gia vào các công việc lập trình. Họ làm như vậy vì không thể thực hiện tốt công việc của mình nếu không trải qua những vấn đề mà họ tạo ra cho các lập trình viên khác.

Kiến trúc của một hệ thống phần mềm là hình dáng được trao cho hệ thống đó bởi những người xây dựng nó. Hình dáng đó thể hiện trong việc chia hệ thống thành các thành phần, sự sắp xếp của các thành phần đó, và cách thức mà các thành phần đó giao tiếp với nhau.

Mục đích của hình dáng đó là để tạo điều kiện cho việc phát triển, triển khai, vận hành và bảo trì hệ thống phần mềm bên trong nó.

> *Chiến lược đằng sau sự tạo điều kiện đó là để giữ càng nhiều lựa chọn mở càng tốt, trong thời gian càng dài càng tốt.*

Có lẽ câu nói này khiến bạn ngạc nhiên. Có lẽ bạn nghĩ mục tiêu của kiến trúc phần mềm là làm cho hệ thống hoạt động đúng đắn. Chắc chắn chúng ta muốn hệ thống hoạt động đúng, và chắc chắn kiến trúc của hệ thống phải hỗ trợ điều đó như một trong những ưu tiên cao nhất.

Tuy nhiên, kiến trúc của một hệ thống hầu như không có ảnh hưởng gì đến việc hệ thống đó có hoạt động hay không. Có rất nhiều hệ thống bên ngoài kia, với kiến trúc tồi tệ, vẫn hoạt động tốt đẹp. Khó khăn của chúng không nằm ở việc vận hành; mà nằm ở việc triển khai, bảo trì và phát triển liên tục.

Đây không có nghĩa là kiến trúc không đóng vai trò trong việc hỗ trợ hành vi đúng đắn của hệ thống. Nó chắc chắn đóng vai trò, và vai trò đó là quan trọng. Nhưng vai trò đó là bị động và hình thức, không phải chủ động hay thiết yếu. Có rất ít, nếu không có, tùy chọn hành vi nào mà kiến trúc của một hệ thống có thể để mở.

Mục đích chính của kiến trúc là hỗ trợ vòng đời của hệ thống. Kiến trúc tốt làm cho hệ thống dễ hiểu, dễ phát triển, dễ bảo trì và dễ triển khai. Mục tiêu tối thượng là giảm thiểu chi phí vòng đời của hệ thống và tối đa hóa năng suất của lập trình viên.

Phát triển

Một hệ thống phần mềm khó phát triển khó lòng có một vòng đời lâu dài và khỏe mạnh. Vì vậy, kiến trúc của một hệ thống nên làm cho hệ thống đó dễ dàng phát triển, cho các nhóm phát triển nó.

Các cấu trúc nhóm khác nhau ngụ ý các quyết định kiến trúc khác nhau. Mặt khác, một nhóm nhỏ năm lập trình viên có thể làm việc cùng nhau rất hiệu quả để phát triển một hệ thống nguyên khối (monolithic) nếu không có các thành phần hay giao diện được định nghĩa rõ ràng. Thực tế, nhóm như vậy có thể thấy các quy phạm của một kiến trúc trở thành một trở ngại trong những ngày đầu của quá trình phát triển. Đây có lẽ là lý do tại sao nhiều hệ thống lại thiếu kiến trúc tốt: Chúng bắt đầu không có kiến trúc, vì nhóm nhỏ và không muốn gánh nặng của một siêu cấu trúc.

Mặt khác, một hệ thống được phát triển bởi năm nhóm khác nhau, mỗi nhóm bảy lập trình viên, không thể tiến triển trừ khi hệ thống được chia thành các thành phần được định nghĩa rõ ràng với các giao diện ổn định và tin cậy. Nếu không xem xét yếu tố nào khác, kiến trúc của hệ thống đó có thể sẽ tiến hóa thành năm thành phần, một cho mỗi nhóm.

Kiến trúc một-thành-phần-một-nhóm (component-per-team) như vậy khó có thể là kiến trúc tốt nhất cho việc triển khai, vận hành và bảo trì. Dù vậy, nếu các nhóm chỉ bị thúc đẩy bởi lịch trình phát triển, hệ thống rất dễ hình thành theo mô hình này.

Triển khai

Để có hiệu quả, một hệ thống phần mềm phải có thể triển khai được. Chi phí triển khai càng cao, hệ thống càng ít hữu ích. Mục tiêu của một kiến trúc phần mềm, do đó, nên là tạo ra một hệ thống có thể dễ dàng triển khai bằng một thao tác duy nhất.

Thật không may, chiến lược triển khai hiếm khi được xem xét trong giai đoạn phát triển ban đầu. Điều này dẫn đến những kiến trúc có thể làm cho hệ thống dễ phát triển, nhưng lại khiến nó rất khó triển khai.

Ví dụ, trong giai đoạn phát triển sớm của một hệ thống, các lập trình viên có thể quyết định sử dụng "kiến trúc vi dịch vụ" (micro-service architecture). Họ có thể thấy cách tiếp cận này làm cho hệ thống rất dễ phát triển vì các ranh giới thành phần rất chặt chẽ và các giao diện tương đối ổn định. Tuy nhiên, khi đến lúc triển khai hệ thống, họ có thể phát hiện số lượng vi dịch vụ đã trở nên đáng sợ; việc cấu hình các kết nối giữa chúng, và thời điểm khởi động của chúng, cũng có thể trở thành một nguồn lỗi khổng lồ.

Nếu các kiến trúc sư đã xem xét các vấn đề triển khai sớm, họ có thể đã quyết định dùng ít dịch vụ hơn, một sự kết hợp giữa dịch vụ và các thành phần trong tiến trình (in-process), và một cách quản lý liên kết tích hợp hơn.

Vận hành

Tác động của kiến trúc đến vận hành hệ thống thường ít rõ rệt hơn tác động của nó đến phát triển, triển khai và bảo trì. Hầu hết khó khăn trong vận hành đều có thể giải quyết bằng cách bổ sung phần cứng mà không cần thay đổi đáng kể kiến trúc phần mềm.

Chúng ta đã thấy điều này xảy ra lặp đi lặp lại. Những hệ thống phần mềm có kiến trúc kém hiệu quả thường có thể được làm cho hoạt động hiệu quả chỉ bằng cách thêm bộ nhớ và thêm máy chủ. Thực tế là phần cứng rẻ và con người đắt có nghĩa là những kiến trúc cản trở vận hành không tốn kém bằng những kiến trúc cản trở phát triển, triển khai và bảo trì.

Điều này không có nghĩa là một kiến trúc được tinh chỉnh tốt cho việc vận hành hệ thống là không đáng mong muốn. Ngược lại, điều đó rất đáng mong muốn. Chỉ là phép tính chi phí nghiêng về phía phát triển, triển khai và bảo trì.

Dù vậy, kiến trúc còn có một vai trò khác đối với việc vận hành hệ thống: một kiến trúc phần mềm tốt truyền đạt các nhu cầu vận hành của hệ thống.

Có thể nói chính xác hơn rằng kiến trúc khiến cách hệ thống vận hành hiện rõ với lập trình viên. Kiến trúc cần làm rõ yêu cầu vận hành. Nó nên nâng các ca sử dụng, tính năng và hành vi bắt buộc lên thành những yếu tố hạng nhất, những mốc dễ nhận biết đối với lập trình viên. Nhờ vậy, hệ thống dễ hiểu hơn, hỗ trợ đáng kể cho phát triển và bảo trì.

Bảo trì

Trong mọi khía cạnh của một hệ thống phần mềm, bảo trì là phần tốn kém nhất. Chuỗi tính năng mới không dứt cùng hàng loạt lỗi và bản sửa chữa tiêu tốn một lượng lớn nguồn lực con người.

Chi phí bảo trì chủ yếu đến từ việc lần mò trong mã nguồn và rủi ro. Lần mò là thời gian đào sâu vào phần mềm hiện có để tìm nơi thích hợp nhất và cách tốt nhất để bổ sung tính năng hoặc sửa lỗi. Trong khi thực hiện những thay đổi ấy, nguy cơ vô tình tạo thêm lỗi luôn hiện hữu, làm tăng chi phí rủi ro.

Một kiến trúc được suy xét cẩn thận giúp giảm đáng kể các chi phí này. Việc chia hệ thống thành các thành phần và cô lập chúng bằng những giao diện ổn định giúp làm rõ lối đi cho các tính năng tương lai, đồng thời giảm mạnh nguy cơ vô tình làm hỏng hệ thống.

Giữ các lựa chọn mở

Như chúng ta đã mô tả ở một chương trước, phần mềm có hai loại giá trị: giá trị của hành vi và giá trị của cấu trúc. Loại thứ hai là lớn hơn vì chính giá trị này làm cho phần mềm "mềm" (soft).

Phần mềm ra đời vì chúng ta cần một cách nhanh chóng và dễ dàng để thay đổi hành vi của máy móc. Nhưng tính linh hoạt ấy phụ thuộc phần lớn vào hình dáng của hệ thống, cách sắp xếp các thành phần và cách chúng kết nối với nhau.

Cách bạn giữ phần mềm mềm là để càng nhiều lựa chọn mở càng tốt, trong thời gian càng dài càng tốt. Những lựa chọn chúng ta cần để mở là gì? Chúng là những chi tiết không quan trọng.

Tất cả các hệ thống phần mềm có thể được phân tách thành hai yếu tố chính: chính sách (policy) và chi tiết (details). Yếu tố chính sách thể hiện tất cả các quy tắc và thủ tục nghiệp vụ. Chính sách là nơi giá trị thực sự của hệ thống nằm.

Chi tiết là những thứ cần thiết để con người, các hệ thống khác và lập trình viên giao tiếp với chính sách, nhưng hoàn toàn không ảnh hưởng đến hành vi của chính sách. Chúng bao gồm thiết bị IO, cơ sở dữ liệu, hệ thống web, máy chủ, framework, giao thức truyền thông và những thứ tương tự.

Mục tiêu của kiến trúc sư là định hình hệ thống sao cho chính sách nổi bật lên như yếu tố cốt lõi, còn các chi tiết không ảnh hưởng đến chính sách. Nhờ vậy, các quyết định về chi tiết có thể được trì hoãn.

Ví dụ:

- Không cần chọn một hệ thống cơ sở dữ liệu trong những ngày đầu của phát triển, vì chính sách cấp cao không nên quan tâm loại cơ sở dữ liệu nào sẽ được sử dụng. Thực tế, nếu kiến trúc sư cẩn thận, chính sách cấp cao sẽ không quan tâm cơ sở dữ liệu là quan hệ (relational), phân tán (distributed), phân cấp (hierarchical), hay chỉ là các tệp phẳng (flat files) đơn thuần.
- Không cần chọn máy chủ web ngay từ đầu, vì chính sách cấp cao không nên biết mình được cung cấp qua web. Nếu chính sách cấp cao không nhận biết HTML, AJAX, JSP, JSF hay bất kỳ từ viết tắt nào trong mớ thuật ngữ phát triển web, thì bạn có thể trì hoãn việc chọn hệ thống web đến giai đoạn sau của dự án. Thậm chí, bạn chưa cần quyết định hệ thống có được cung cấp qua web hay không.
- Không cần áp dụng REST sớm trong phát triển, vì chính sách cấp cao nên trung lập về giao diện với thế giới bên ngoài. Cũng không cần áp dụng một khung vi dịch vụ (micro-services framework), hay một khung SOA. Lại nữa, chính sách cấp cao không nên quan tâm đến những thứ này.
- Không cần áp dụng một khung tiêm phụ thuộc (dependency injection framework) sớm trong phát triển, vì chính sách cấp cao không nên quan tâm phụ thuộc được giải quyết như thế nào.

Tôi nghĩ bạn đã hiểu ý. Nếu bạn có thể phát triển chính sách cấp cao mà không cam kết vào các chi tiết bao quanh nó, bạn có thể trì hoãn và trì hoãn thêm các quyết định về những chi tiết đó trong một thời gian dài. Và càng đợi lâu để đưa ra những quyết định đó, bạn càng

có nhiều thông tin hơn để đưa ra chúng đúng đắn.

Điều này cũng cho phép bạn thử nhiều phương án khác nhau. Nếu chính sách cấp cao đã hoạt động và không phụ thuộc vào cơ sở dữ liệu, bạn có thể kết nối nó với vài cơ sở dữ liệu để kiểm tra độ phù hợp và hiệu năng. Điều tương tự cũng đúng với các hệ thống web, framework web, thậm chí cả bản thân web.

Càng để các lựa chọn mở lâu, bạn càng có thể chạy nhiều thí nghiệm hơn, thử nhiều thứ hơn, và có nhiều thông tin hơn khi bạn đạt đến thời điểm những quyết định đó không thể trì hoãn thêm được.

Nếu người khác đã đưa ra các quyết định đó thì sao? Nếu công ty bạn đã cam kết dùng một cơ sở dữ liệu, máy chủ web hay framework cụ thể thì sao? Một kiến trúc sư giỏi vẫn làm việc như thể các quyết định ấy chưa được chốt, và định hình hệ thống sao cho chúng có thể tiếp tục được trì hoãn hoặc thay đổi càng lâu càng tốt.

Một kiến trúc sư tốt tối đa hóa số lượng quyết định chưa *được đưa ra.*

Độc lập thiết bị

Để minh họa lối tư duy này, hãy quay về những năm 1960, khi máy tính còn non trẻ và phần lớn lập trình viên là các nhà toán học hoặc kỹ sư thuộc những ngành khác. Ít nhất một phần ba trong số họ là phụ nữ.

Những ngày ấy chúng ta phạm nhiều sai lầm. Chúng ta không biết đó là sai lầm vào lúc đó, tất nhiên. Làm sao có thể biết?

Một trong những sai lầm đó là gắn mã của chúng ta trực tiếp vào các thiết bị IO. Nếu chúng ta cần in một cái gì đó trên máy in, chúng ta viết mã sử dụng các lệnh IO để điều khiển máy in. Mã của chúng ta phụ thuộc vào thiết bị (device dependent).

Ví dụ, khi tôi viết chương trình PDP-8 in trên máy in điện báo (teleprinter), tôi dùng một tập lệnh máy trông như sau:

```
PRTCHR, 0
TSF
JMP .-1
TLS
JMP I PRTCHR
```

`PRTCHR` là một chương trình con in một ký tự trên máy in điện báo. Số không ở đầu được dùng làm nơi lưu trữ địa chỉ trả về. (Đừng hỏi.) Lệnh `TSF` bỏ qua lệnh tiếp theo nếu máy in điện báo sẵn sàng in một ký tự. Nếu máy in bận, `TSF` chỉ việc rơi xuống lệnh `JMP .-1`, nhảy ngược lại chính lệnh `TSF`. Nếu máy in sẵn sàng, `TSF` sẽ nhảy qua lệnh `TLS`, gửi ký tự trong thanh ghi A đến máy in. Sau đó lệnh `JMP I PRTCHR` trả về cho người gọi.

Ban đầu, chiến lược này hoạt động tốt. Khi cần đọc thẻ, chúng ta dùng mã giao tiếp trực tiếp với máy đọc thẻ. Khi cần đục lỗ thẻ, chúng ta viết mã điều khiển trực tiếp máy đục lỗ. Các chương trình hoạt động hoàn hảo. Làm sao biết đó là một sai lầm?

Nhưng các lô thẻ đục lỗ lớn rất khó quản lý. Chúng có thể bị mất, hư hỏng, cong vênh, xáo trộn hoặc rơi vãi. Từng thẻ có thể bị thất lạc hoặc thẻ thừa có thể bị chèn vào. Vì vậy,

tính toàn vẹn dữ liệu trở thành vấn đề đáng kể.

Băng từ là giải pháp. Ta có thể chuyển ảnh của các tấm thẻ sang băng. Nếu làm rơi một cuộn băng từ, các bản ghi không bị xáo trộn. Không thể vô tình làm mất một bản ghi hay chèn bản ghi trống chỉ vì cầm cuộn băng. Băng an toàn hơn nhiều. Nó cũng đọc và ghi nhanh hơn, đồng thời dễ sao lưu.

Thật không may, toàn bộ phần mềm của chúng ta đều được viết để điều khiển máy đọc thẻ và máy đục lỗ thẻ. Muốn chuyển sang dùng băng từ, chúng ta phải viết lại các chương trình. Đó là một công việc lớn.

Đến cuối những năm 1960, chúng ta đã rút ra bài học và phát minh ra khái niệm *độc lập thiết bị* (device independence). Hệ điều hành thời đó trừu tượng hóa thiết bị IO thành các hàm phần mềm xử lý những bản ghi đơn vị (unit records) có dạng như thẻ. Chương trình gọi các dịch vụ của hệ điều hành để xử lý những bản ghi trừu tượng này. Người vận hành có thể yêu cầu hệ điều hành kết nối các dịch vụ ấy với máy đọc thẻ, băng từ hoặc bất kỳ thiết bị xử lý bản ghi đơn vị nào khác.

Bây giờ cùng một chương trình có thể đọc và ghi thẻ, hoặc đọc và ghi băng, mà không cần bất kỳ thay đổi nào. Nguyên tắc Đóng-Mở (Open-Closed Principle) đã ra đời (dù chưa được đặt tên).

Thư quảng cáo hàng loạt (Junk Mail)

Bạn biết loại này:

> *Kính chào anh Martin, Chúc mừng! Chúng tôi đã chọn ANH trong tất cả mọi người sống trên đường Witchwood để tham gia vào ưu đãi tuyệt vời một lần duy nhất…*

Khách hàng gửi cho chúng ta những cuộn thư mẫu khổng lồ chứa tất cả các từ trừ tên và địa chỉ, và bất kỳ yếu tố nào khác họ muốn chúng ta in. Chúng ta viết chương trình trích xuất tên, địa chỉ, và các yếu tố khác từ băng từ, và in chính xác những yếu tố đó lên đúng vị trí trên các mẫu.

Những cuộn thư mẫu này nặng 500 pound và chứa hàng ngàn lá thư. Khách hàng gửi cho chúng ta hàng trăm cuộn như vậy. Chúng ta in từng cuộn một.

Ban đầu, chúng ta có một IBM 360 làm việc in trên máy in dòng (line printer) duy nhất của nó. Chúng ta có thể in vài ngàn lá thư mỗi ca. Thật không may, điều này chiếm dụng một máy rất đắt trong thời gian rất dài. Những ngày ấy, IBM 360 thuê với giá hàng chục nghìn đô la mỗi tháng.

Vì vậy, chúng ta bảo hệ điều hành dùng băng từ thay vì máy in dòng. Chương trình của chúng ta không quan tâm, vì chúng đã được viết để dùng các trừu tượng IO của hệ điều hành.

IBM 360 có thể đẩy ra một cuộn băng đầy trong 10 phút, đủ để in vài cuộn thư mẫu.

Giá trị của độc lập thiết bị là rất lớn! Chúng ta có thể viết chương trình mà không cần biết hay quan tâm thiết bị nào sẽ được sử dụng. Ta có thể kiểm thử chương trình trên máy tính nối với máy in dòng cục bộ. Sau đó, ta có thể yêu cầu hệ điều hành "in" ra băng từ để

tạo hàng trăm nghìn bức thư quảng cáo cá nhân hóa.

Chương trình của chúng ta có một hình dáng. Hình dáng đó tách biệt chính sách khỏi chi tiết. Chính sách là việc định dạng các bản ghi tên và địa chỉ. Chi tiết là thiết bị. Chúng ta đã trì hoãn quyết định về việc sẽ dùng thiết bị nào.

Địa chỉ vật lý (Physical Addressing)

Đầu những năm 1970, tôi làm việc trên một hệ thống kế toán lớn cho một công đoàn tài xế xe tải địa phương. Chúng tôi có một ổ đĩa 25 MB để lưu bản ghi về Đại lý (Agents), Nhà tuyển dụng (Employers) và Thành viên (Members). Các bản ghi có kích thước khác nhau, nên chúng tôi định dạng vài trụ đầu tiên của đĩa sao cho mỗi sector vừa với một bản ghi Đại lý. Vài trụ tiếp theo được định dạng để chứa các sector vừa với bản ghi Nhà tuyển dụng. Những trụ cuối cùng dành cho các bản ghi Thành viên.

Chúng ta viết phần mềm biết chính xác cấu trúc vật lý của đĩa. Mã biết đĩa có 200 trụ (cylinders) và 10 đầu đọc (heads), mỗi trụ có vài chục sector trên mỗi đầu. Nó biết trụ nào chứa dữ liệu Đại lý, Nhà tuyển dụng và Thành viên. Tất cả thông tin đó được đóng cứng (hard-wired) vào mã.

Chúng ta duy trì một chỉ mục (index) trên đĩa để tra cứu từng Đại lý, Nhà tuyển dụng và Thành viên. Chỉ mục nằm trên một nhóm trụ khác được định dạng đặc biệt. Chỉ mục Đại lý chứa các bản ghi lưu ID của đại lý cùng số trụ, số đầu đọc và số sector của bản ghi đó. Nhà tuyển dụng và Thành viên có chỉ mục tương tự. Các bản ghi Thành viên còn được nối thành danh sách liên kết đôi (doubly linked list) trên đĩa. Mỗi bản ghi lưu số trụ, số đầu đọc và số sector của bản ghi Thành viên kế tiếp và trước đó.

Nếu cần nâng cấp lên ổ đĩa mới có nhiều đầu đọc hơn, nhiều trụ hơn hoặc nhiều sector hơn trên mỗi trụ thì sao? Chúng ta phải viết chương trình đặc biệt để đọc dữ liệu từ ổ cũ rồi ghi sang ổ mới, điều chỉnh mọi số trụ, đầu đọc và sector. Chúng ta cũng phải sửa tất cả các giá trị đóng cứng trong mã, mà chúng lại nằm rải rác khắp nơi. Mọi quy tắc nghiệp vụ đều biết sơ đồ trụ, đầu đọc và sector chi tiết.

Một ngày nọ, một lập trình viên giàu kinh nghiệm hơn gia nhập nhóm. Khi thấy những gì chúng ta đã làm, mặt anh tái mét. Anh nhìn chúng ta kinh hoàng như thể chúng ta là người ngoài hành tinh. Rồi anh nhẹ nhàng khuyên nên đổi sơ đồ địa chỉ sang địa chỉ tương đối (relative addresses).

Người đồng nghiệp giàu kinh nghiệm hơn gợi ý xem đĩa như một mảng tuyến tính khổng lồ gồm các sector, mỗi sector được đánh địa chỉ bằng một số nguyên liên tiếp. Khi đó, ta có thể viết một chương trình chuyển đổi nhỏ, biết cấu trúc vật lý của đĩa và chuyển đổi địa chỉ tương đối sang số trụ, đầu đọc và sector ngay khi cần (on the fly).

May mắn cho chúng ta, chúng ta đã nghe theo lời khuyên. Chúng ta thay đổi chính sách cấp cao của hệ thống để trung lập về cấu trúc vật lý của đĩa. Điều đó cho phép chúng ta tách rời quyết định về cấu trúc ổ đĩa khỏi ứng dụng.

Kết luận

Hai câu chuyện nhỏ trong chương này minh họa nguyên tắc mà các kiến trúc sư áp dụng ở quy mô lớn. Kiến trúc sư giỏi tách biệt chi tiết khỏi chính sách, rồi tách chính sách khỏi chi tiết đến mức chính sách không biết và không phụ thuộc vào bất kỳ chi tiết nào. Họ thiết kế chính sách sao cho các quyết định về chi tiết có thể được trì hoãn, và giữ nguyên lựa chọn ấy càng lâu càng tốt.

16

Tính độc lập



Như đã nêu trước đó, một kiến trúc tốt phải hỗ trợ:

- Các trường hợp sử dụng (use cases) và vận hành của hệ thống.
- Bảo trì hệ thống.
- Phát triển hệ thống.
- Triển khai (deployment) hệ thống.

Trường hợp sử dụng (Use Cases)

Mục đầu tiên, trường hợp sử dụng, có nghĩa là kiến trúc của hệ thống phải hỗ trợ ý định của hệ thống. Nếu hệ thống là một ứng dụng giỏ hàng mua sắm, thì kiến trúc phải hỗ trợ các trường hợp sử dụng của giỏ hàng mua sắm. Thực tế, đây là mối quan tâm đầu tiên của kiến trúc sư, và ưu tiên đầu tiên của kiến trúc. Kiến trúc phải hỗ trợ các trường hợp sử dụng.

Tuy nhiên, như chúng ta đã thảo luận trước đó, kiến trúc không có nhiều ảnh hưởng lên hành vi của hệ thống. Có rất ít các lựa chọn hành vi mà kiến trúc có thể để mở. Nhưng ảnh hưởng không phải là mọi thứ. Điều quan trọng nhất mà một kiến trúc tốt có thể làm để hỗ trợ hành vi là làm rõ và trình bày hành vi đó để ý định của hệ thống nhìn thấy được ở cấp độ kiến trúc.

Một ứng dụng giỏ hàng mua sắm với kiến trúc tốt sẽ trông như một ứng dụng giỏ hàng mua sắm. Các trường hợp sử dụng của hệ thống đó sẽ nhìn thấy rõ ràng trong cấu trúc của hệ thống. Các nhà phát triển sẽ không phải săn lùng các hành vi, bởi vì những hành vi đó sẽ là các yếu tố hạng nhất (first-class elements) nhìn thấy được ở cấp cao nhất của hệ thống. Các yếu tố đó sẽ là các lớp hoặc hàm hoặc mô-đun có vị trí nổi bật trong kiến trúc, và chúng sẽ có tên mô tả rõ ràng chức năng của chúng.

Chương 21, “Kiến trúc Tự Hào” (Screaming Architecture), sẽ làm rõ điểm này hơn.

Vận hành (Operation)

Kiến trúc đóng vai trò quan trọng hơn nhiều, và ít mang tính trang trí hơn, trong việc hỗ trợ vận hành của hệ thống. Nếu hệ thống phải xử lý 100.000 khách hàng mỗi giây, kiến trúc phải hỗ trợ loại thông lượng (throughput) và thời gian phản hồi đó cho mỗi trường hợp sử dụng yêu cầu nó. Nếu hệ thống phải truy vấn các khối dữ liệu lớn (big data cubes) trong mili-giây, thì kiến trúc phải được cấu trúc để cho phép loại vận hành đó.

Đối với một số hệ thống, điều này có nghĩa là sắp xếp các yếu tố xử lý của hệ thống thành một mảng các dịch vụ nhỏ (little services) có thể chạy song song trên nhiều máy chủ khác nhau. Đối với các hệ thống khác, nó sẽ nghĩa là một loạt luồng nhẹ (lightweight threads) chia sẻ không gian địa chỉ của một tiến trình (process) đơn lẻ trong một bộ xử lý đơn. Những hệ thống khác sẽ chỉ cần một vài tiến trình chạy trong các không gian địa chỉ biệt lập. Và một số hệ thống thậm chí có thể tồn tại dưới dạng các chương trình nguyên khối (monolithic) đơn giản chạy trong một tiến trình đơn.

Nghe có vẻ lạ lùng, nhưng quyết định này chính là một trong những lựa chọn mà một kiến trúc tốt để mở. Một hệ thống được viết dưới dạng nguyên khối (monolith), và phụ thuộc vào cấu trúc nguyên khối đó, không thể dễ dàng nâng cấp lên nhiều tiến trình, nhiều luồng, hoặc micro-services nếu nhu cầu nảy sinh. Ngược lại, một kiến trúc duy trì sự biệt lập thích hợp của các thành phần (components) của nó, và không giả định cách thức giao tiếp giữa các thành phần đó, sẽ dễ dàng hơn nhiều trong việc chuyển đổi qua lại xuyên qua phổ các luồng, tiến trình, và dịch vụ khi nhu cầu vận hành của hệ thống thay đổi theo thời gian.

Phát triển (Development)

Kiến trúc đóng vai trò đáng kể trong việc hỗ trợ môi trường phát triển. Đây là nơi định luật Conway (Conway's law) phát huy tác dụng. Định luật Conway nói:

> *Mọi tổ chức thiết kế một hệ thống sẽ tạo ra một thiết kế có cấu trúc sao chép cấu trúc truyền thông của tổ chức đó.*

Một hệ thống phải được phát triển bởi một tổ chức có nhiều nhóm (teams) và nhiều mối quan tâm (concerns) phải có một kiến trúc tạo điều kiện cho các hành động độc lập của các nhóm đó, để các nhóm không can thiệp lẫn nhau trong quá trình phát triển. Điều này đạt được bằng cách phân vùng (partitioning) hệ thống đúng cách thành các thành phần biệt lập tốt, có thể phát triển độc lập. Các thành phần đó sau đó có thể được phân bổ cho các nhóm có thể làm việc độc lập với nhau.

Triển khai (Deployment)

Kiến trúc cũng đóng vai trò lớn trong việc xác định mức độ dễ dàng triển khai hệ thống. Mục tiêu là "triển khai tức thì" (immediate deployment). Một kiến trúc tốt không dựa vào hàng chục tập tin kịch bản cấu hình (configuration scripts) nhỏ và các chỉnh sửa tệp thuộc tính (property file tweaks). Nó không yêu cầu tạo thủ công các thư mục hoặc tệp phải được sắp xếp đúng cách. Một kiến trúc tốt giúp hệ thống có thể triển khai ngay lập tức sau khi build (xây dựng).

Lại một lần nữa, điều này đạt được thông qua việc phân vùng và biệt lập thích hợp các thành phần của hệ thống, bao gồm cả các thành phần chủ (master components) gắn kết toàn bộ hệ thống lại với nhau và đảm bảo mỗi thành phần được khởi động, tích hợp, và giám sát (supervised) đúng cách.

Giữ các lựa chọn mở (Leaving Options Open)

Tóm lại, các mục tiêu chúng ta phải đạt được là không rõ ràng và thiếu ổn định (indistinct and inconstant). Chào mừng đến với thế giới thực.

Nhưng chưa phải là đường cùng: Một số nguyên tắc kiến trúc thực hiện với chi phí tương đối thấp và có thể giúp cân bằng những mối quan tâm đó, ngay cả khi bạn không có bức tranh rõ ràng về các mục tiêu bạn phải đánh trúng. Những nguyên tắc đó giúp chúng ta phân vùng hệ thống thành các thành phần biệt lập tốt, cho phép chúng ta giữ càng nhiều lựa chọn mở càng tốt, trong càng lâu càng tốt.

Một kiến trúc tốt làm cho hệ thống dễ dàng thay đổi, theo tất cả các cách mà nó phải thay đổi, bằng cách giữ các lựa chọn mở.

Tách biệt các lớp (Decoupling Layers)

…là một hệ thống xử lý đơn hàng. Vì vậy, kiến trúc sư có thể sử dụng Nguyên tắc Trách nhiệm Đơn lẻ (Single Responsibility Principle - SRP) và Nguyên tắc Đóng chung (Common Closure Principle - CCP) để tách biệt những thứ thay đổi vì những lý do khác nhau, và gom nhóm những thứ thay đổi vì cùng một lý do, trong bối cảnh ý định của hệ thống.

Điều gì thay đổi vì những lý do khác nhau? Có một số điều hiển nhiên. Giao diện người dùng (UI) thay đổi vì những lý do không liên quan gì đến các quy tắc nghiệp vụ (business rules). Các trường hợp sử dụng có cả hai yếu tố. Rõ ràng, một kiến trúc sư tốt sẽ muốn tách phần UI của một trường hợp sử dụng ra khỏi phần quy tắc nghiệp vụ sao cho chúng có thể thay đổi độc lập lẫn nhau, đồng thời vẫn giữ các trường hợp sử dụng nhìn thấy rõ ràng và rõ ràng.

…giá trị tính toán lãi suất trên một tài khoản và việc đếm tồn kho là các quy tắc nghiệp vụ gắn liền chặt chẽ hơn với miền (domain). Hai loại quy tắc khác nhau này sẽ thay đổi ở tốc độ khác nhau, và vì những lý do khác nhau, vì vậy chúng nên được tách biệt để có thể thay đổi độc lập.

Cơ sở dữ liệu (database), ngôn ngữ truy vấn, và thậm chí schema là các chi tiết kỹ thuật không có liên quan gì đến các quy tắc nghiệp vụ hay UI. Chúng sẽ thay đổi ở các tỷ lệ, và vì những lý do, độc lập với các khía cạnh khác của hệ thống. Do đó, kiến trúc nên tách chúng ra khỏi phần còn lại của hệ thống để chúng có thể thay đổi độc lập.

Do đó, chúng ta thấy hệ thống bị chia thành các lớp ngang (horizontal layers) tách biệt: UI, quy tắc nghiệp vụ đặc thù ứng dụng (application-specific business rules), quy tắc nghiệp vụ độc lập ứng dụng (application-independent business rules), và cơ sở dữ liệu, chỉ để nhắc đến một vài.

Tách biệt các trường hợp sử dụng (Decoupling Use Cases)

Điều gì khác thay đổi vì những lý do khác nhau? Các trường hợp sử dụng bản thân! Trường hợp sử dụng để thêm một đơn hàng vào hệ thống nhập đơn hàng gần như chắc chắn sẽ thay đổi ở tỷ lệ khác nhau, và vì những lý do khác nhau, so với trường hợp sử dụng xóa một đơn hàng khỏi hệ thống. Các trường hợp sử dụng là một cách rất tự nhiên để chia hệ thống.

Cùng lúc đó, các trường hợp sử dụng là các lát cắt dọc hẹp (narrow vertical slices) cắt xuyên qua các lớp ngang của hệ thống. Mỗi trường hợp sử dụng sử dụng một chút UI, một chút quy tắc nghiệp vụ đặc thù ứng dụng, một chút quy tắc nghiệp vụ độc lập ứng dụng, và một chút chức năng cơ sở dữ liệu. Do đó, khi chúng ta chia hệ thống thành các lớp ngang, chúng ta cũng chia hệ thống thành các trường hợp sử dụng mỏng dọc cắt xuyên qua các lớp đó.

Để đạt được sự tách biệt này, chúng ta tách UI của trường hợp sử dụng thêm đơn hàng (add-order) ra khỏi UI của trường hợp sử dụng xóa đơn hàng (delete-order). Chúng ta làm tương tự với các quy tắc nghiệp vụ, và với cơ sở dữ liệu. Chúng ta giữ các trường hợp sử dụng tách biệt dọc theo chiều cao dọc của hệ thống.

Bạn có thể thấy khuôn mẫu ở đây. Nếu bạn tách biệt các yếu tố của hệ thống thay đổi vì những lý do khác nhau, thì bạn có thể tiếp tục thêm các trường hợp sử dụng mới mà không

can thiệp vào các trường hợp cũ. Nếu bạn cũng gom nhóm UI và cơ sở dữ liệu để hỗ trợ các trường hợp sử dụng đó, sao cho mỗi trường hợp sử dụng sử dụng một khía cạnh khác nhau của UI và cơ sở dữ liệu, thì việc thêm các trường hợp sử dụng mới sẽ khó làm ảnh hưởng đến các trường hợp cũ.

Chế độ tách biệt (Decoupling Mode)

Bây giờ hãy nghĩ về ý nghĩa của tất cả sự tách biệt đó đối với mục thứ hai: vận hành. Nếu các khía cạnh khác nhau của các trường hợp sử dụng được tách biệt, thì những khía cạnh phải chạy ở thông lượng cao có thể đã được tách biệt khỏi những khía cạnh phải chạy ở thông lượng thấp. Nếu UI và cơ sở dữ liệu đã được tách biệt khỏi các quy tắc nghiệp vụ, thì chúng có thể chạy trên các máy chủ khác nhau. Những khía cạnh yêu cầu băng thông cao hơn có thể được nhân bản (replicated) trên nhiều máy chủ.

Tóm lại, sự tách biệt mà chúng ta làm vì các trường hợp sử dụng cũng giúp ích cho vận hành. Tuy nhiên, để tận hưởng lợi ích vận hành đó, sự tách biệt phải có chế độ (mode) thích hợp. Để chạy trên các máy chủ riêng biệt, các thành phần đã tách biệt không thể phụ thuộc vào việc cùng nằm trong không gian địa chỉ của một bộ xử lý. Chúng phải là các dịch vụ độc lập (independent services), giao tiếp qua một mạng nào đó.

Nhiều kiến trúc sư gọi các thành phần như vậy là “dịch vụ” (services) hoặc “micro-services”, tùy thuộc vào một khái niệm mơ hồ về số dòng mã. Thực tế, một kiến trúc dựa trên dịch vụ thường được gọi là kiến trúc hướng dịch vụ (service-oriented architecture - SOA).

Nếu thuật ngữ đó làm bật lên chuông cảnh báo trong đầu bạn, đừng lo. Tôi không sẽ nói rằng SOA là kiến trúc tốt nhất có thể, hay rằng micro-services là làn sóng của tương lai. Điểm được nói ở đây là đôi khi chúng ta phải tách biệt các thành phần của mình hoàn toàn đến cấp độ dịch vụ.

Hãy nhớ, một kiến trúc tốt để các lựa chọn mở. Chế độ tách biệt là một trong những lựa chọn đó.

Trước khi khám phá chủ đề đó sâu hơn, hãy nhìn sang hai mục còn lại.

Khả năng phát triển độc lập (Independent Develop-ability)

Mục thứ ba là phát triển. Rõ ràng khi các thành phần được tách biệt mạnh (strongly decoupled), sự can thiệp giữa các nhóm được giảm thiểu. Nếu các quy tắc nghiệp vụ không biết về UI, thì một nhóm tập trung vào UI không thể ảnh hưởng nhiều đến một nhóm tập trung vào các quy tắc nghiệp vụ. Nếu các trường hợp sử dụng bản thân được tách biệt lẫn nhau, thì một nhóm tập trung vào trường hợp sử dụng thêm đơn hàng (addOrder) không có khả năng can thiệp vào một nhóm tập trung vào trường hợp sử dụng xóa đơn hàng (deleteOrder).

Miễn là các lớp và trường hợp sử dụng được tách biệt, kiến trúc của hệ thống sẽ hỗ trợ tổ chức của các nhóm, bất kể chúng được tổ chức như các nhóm tính năng (feature teams), nhóm thành phần (component teams), nhóm lớp (layer teams), hoặc một biến thể nào khác.

Khả năng triển khai độc lập (Independent Deployability)

Sự tách biệt của các trường hợp sử dụng và các lớp cũng cung cấp một mức độ linh hoạt cao trong triển khai. Thực tế, nếu sự tách biệt được làm tốt, thì nên có thể hot-swap (thay đổi nóng) các lớp và trường hợp sử dụng trong các hệ thống đang chạy. Việc thêm một trường hợp sử dụng mới có thể đơn giản như việc thêm một vài tệp jar mới hoặc dịch vụ vào hệ thống trong khi để phần còn lại nguyên vẹn.

Trùng lặp (Duplication)

Các kiến trúc sư thường rơi vào một cái bẫy, cái bẫy dựa trên nỗi sợ hãi của họ về sự trùng lặp (duplication).

Sự trùng lặp nói chung là một điều xấu trong phần mềm. Chúng ta không thích mã trùng lặp. Khi mã thực sự bị trùng lặp (true duplication), chúng ta bị ràng buộc bởi đạo đức nghề nghiệp phải giảm thiểu và loại bỏ nó.

Nhưng có nhiều loại trùng lặp khác nhau. Có trùng lặp thật (true duplication), nơi mọi thay đổi đối với một thể hiện (instance) đòi hỏi cùng một thay đổi đối với mọi bản sao của thể hiện đó. Sau đó có trùng lặp giả hoặc trùng lặp ngẫu nhiên (false or accidental duplication). Nếu hai phần mã có vẻ trùng lặp phát triển theo các con đường khác nhau, nếu chúng thay đổi ở các tỷ lệ khác nhau, và vì những lý do khác nhau, thì chúng không phải là bản sao thật. Quay lại chúng sau vài năm, và bạn sẽ thấy chúng rất khác nhau.

Hãy tưởng tượng hai trường hợp sử dụng có cấu trúc màn hình rất giống nhau. Các kiến trúc sư sẽ có xu hướng mạnh muốn chia sẻ mã cho cấu trúc đó. Nhưng họ nên làm như vậy không? Đó có phải là trùng lặp thật? Hay là ngẫu nhiên?

Có khả năng lớn là ngẫu nhiên. Theo thời gian, khả năng là hai màn hình đó sẽ phân hóa (diverge) và cuối cùng trông rất khác nhau. Vì lý do đó, phải cẩn thận tránh việc hợp nhất (unifying) chúng. Nếu không, việc tách biệt chúng sau này sẽ là một thách thức.

Khi bạn tách dọc các trường hợp sử dụng ra khỏi nhau, bạn sẽ gặp vấn đề này, và cám dỗ của bạn sẽ là gắn kết (coupling) các trường hợp sử dụng vì chúng có cấu trúc màn hình giống nhau, hoặc thuật toán giống nhau, hoặc các truy vấn và/hoặc schema cơ sở dữ liệu giống nhau. Hãy cẩn thận. Hãy kháng cự cám dỗ loại bỏ trùng lặp theo phản xạ gân cốt (knee-jerk). Hãy chắc chắn rằng sự trùng lặp là thật.

Theo cùng token đó, khi bạn tách các lớp theo chiều ngang, bạn có thể nhận thấy cấu trúc dữ liệu của một bản ghi cơ sở dữ liệu cụ thể rất giống với cấu trúc dữ liệu của một khung nhìn màn hình (screen view) cụ thể. Bạn có thể bị cám dỗ đơn giản truyền bản ghi cơ sở dữ liệu lên cho UI, thay vì tạo một view model trông giống y hệt và sao chép các phần tử qua. Hãy cẩn thận: Sự trùng lặp này gần như chắc chắn là ngẫu nhiên. Việc tạo view model riêng biệt không tốn nhiều công sức, và nó sẽ giúp bạn giữ các lớp tách biệt đúng cách.

Các chế độ tách biệt (lại) (Decoupling Modes (Again))

Quay lại các chế độ. Có nhiều cách để tách biệt các lớp và trường hợp sử dụng. Chúng có thể được tách biệt ở cấp độ mã nguồn (source code level), ở cấp độ mã nhị phân/triển khai (binary code/deployment level), và ở cấp độ đơn vị thực thi/dịch vụ (execution unit/service level).

- **Cấp mã nguồn (Source level).** Chúng ta có thể kiểm soát các phụ thuộc giữa các mô-đun mã nguồn sao cho thay đổi đối với một mô-đun không buộc thay đổi hoặc biên dịch lại các mô-đun khác (ví dụ: Ruby Gems). Trong chế độ tách biệt này, tất cả các thành phần thực thi trong cùng một không gian địa chỉ, và giao tiếp với nhau bằng các lời gọi hàm đơn giản (function calls). Có một tệp thực thi đơn (single executable) được tải vào bộ nhớ máy tính. Mọi người thường gọi đây là cấu trúc nguyên khối (monolithic structure).
- **Cấp triển khai (Deployment level).** Chúng ta có thể kiểm soát các phụ thuộc giữa các đơn vị triển khai (deployable units) như tệp jar, DLL, hoặc thư viện chia sẻ (shared libraries), sao cho thay đổi mã nguồn trong một mô-đun không buộc các mô-đun khác phải được build lại và triển khai lại. Nhiều thành phần vẫn có thể nằm trong cùng một không gian địa chỉ, và giao tiếp qua lời gọi hàm. Các thành phần khác có thể nằm trong các tiến trình khác trên cùng một bộ xử lý, và giao tiếp qua truyền thông liên tiến trình (interprocess communications), socket, hoặc bộ nhớ chia sẻ. Điều quan trọng ở đây là các thành phần tách biệt được phân vùng thành các đơn vị triển khai độc lập như tệp jar, Gem files, hoặc DLL.
- **Cấp dịch vụ (Service level).** Chúng ta có thể giảm các phụ thuộc xuống cấp độ cấu trúc dữ liệu (data structures), và giao tiếp hoàn toàn qua các gói tin mạng (network packets) sao cho mọi đơn vị thực thi hoàn toàn độc lập với thay đổi mã nguồn và nhị phân của các đơn vị khác (ví dụ: services hoặc micro-services).

Chế độ nào là tốt nhất để sử dụng?

Câu trả lời là khó biết chế độ nào tốt nhất trong giai đoạn đầu của dự án. Thực tế, khi dự án trưởng thành, chế độ tối ưu có thể thay đổi.

Ví dụ, không khó tưởng tượng một hệ thống chạy thoải mái trên một máy chủ ngay bây giờ có thể phát triển đến mức một số thành phần của nó nên chạy trên các máy chủ riêng biệt. Trong khi hệ thống chạy trên một máy chủ đơn, tách biệt ở cấp mã nguồn có thể đã đủ. Tuy nhiên, sau này, nó có thể đòi hỏi tách biệt thành các đơn vị triển khai, hoặc thậm chí các dịch vụ.

Một giải pháp (có vẻ phổ biến lúc này) là đơn giản tách biệt ở cấp dịch vụ theo mặc định. Một vấn đề với cách tiếp cận này là nó tốn kém và khuyến khích tách biệt thô (coarse-grained decoupling). Dù micro-services có "micro" đến đâu, tách biệt đó không có khả năng đủ mịn (fine-grained).

Một vấn đề khác với tách biệt cấp dịch vụ là nó tốn kém, cả về thời gian phát triển và tài nguyên hệ thống. Xử lý ranh giới dịch vụ ở nơi không cần thiết là lãng phí công sức, bộ nhớ, và chu kỳ CPU. Và, đúng, tôi biết hai cái sau rẻ, nhưng cái đầu thì không.

Sở thích của tôi là đẩy sự tách biệt đến điểm mà một dịch vụ có thể được hình thành,

nếu nó trở nên cần thiết; nhưng sau đó để các thành phần ở trong cùng không gian địa chỉ càng lâu càng tốt. Điều này để lựa chọn dịch vụ mở.

Với cách tiếp cận này, ban đầu các thành phần được tách biệt ở cấp mã nguồn. Điều đó có thể đủ tốt cho suốt vòng đời dự án. Nếu, tuy nhiên, các vấn đề triển khai hoặc phát triển nảy sinh, đẩy một số tách biệt lên cấp triển khai có thể đủ, ít nhất là một thời gian.

Khi các vấn đề phát triển, triển khai, và vận hành tăng lên, tôi cẩn trọng chọn những đơn vị triển khai nào để biến thành dịch vụ, và dần dần chuyển dịch hệ thống theo hướng đó.

Theo thời gian, nhu cầu vận hành của hệ thống có thể giảm. Điều từng đòi hỏi tách biệt ở cấp dịch vụ bây giờ có thể chỉ yêu cầu tách biệt cấp triển khai hoặc thậm chí cấp mã nguồn.

Một kiến trúc tốt sẽ cho phép một hệ thống ra đời như một nguyên khối (monolith), được triển khai trong một tệp đơn, nhưng sau đó phát triển thành một tập hợp các đơn vị triển khai độc lập, và luôn luôn đến các dịch vụ và/hoặc micro-services độc lập. Sau này, khi mọi thứ thay đổi, nó nên cho phép đảo ngược tiến trình đó và trượt ngược về nguyên khối.

Một kiến trúc tốt bảo vệ phần lớn mã nguồn khỏi những thay đổi đó. Nó để chế độ tách biệt mở như một lựa chọn sao cho các triển khai lớn có thể dùng một chế độ, trong khi các triển khai nhỏ có thể dùng chế độ khác.

Kết luận (Conclusion)

Đúng, đây là chuyện rắc rối. Và tôi không nói rằng việc thay đổi chế độ tách biệt nên là một tùy chọn cấu hình hời hợt (trivial configuration option) (dù đôi khi điều đó thích hợp). Tôi đang nói rằng chế độ tách biệt của một hệ thống là một trong những thứ có khả năng thay đổi theo thời gian, và một kiến trúc sư tốt dự đoán và tạo điều kiện thích hợp cho những thay đổi đó.

17

Ranh giới: Vạch đường phân cách



Hai câu chuyện buồn

Đây là câu chuyện buồn về công ty P, lời cảnh báo cho việc ra quyết định quá sớm. Trong những năm 1980, những người sáng lập P viết một ứng dụng desktop nguyên khối đơn giản. Sản phẩm thành công vang dội và trong thập niên 1990 phát triển thành một ứng dụng desktop GUI phổ biến.

Nhưng rồi, vào cuối những năm 1990, web nổi lên như một thế lực. Bỗng nhiên, ai cũng cần giải pháp web, và P cũng không ngoại lệ. Khách hàng của P đòi hỏi một phiên bản sản phẩm trên web. Để đáp ứng nhu cầu này, công ty tuyển một nhóm lập trình viên Java tài năng ở độ tuổi đôi mươi, rồi bắt đầu dự án chuyển sản phẩm sang web.

Những anh chàng Java có giấc mơ về các trang trại máy chủ, nên họ áp dụng một "kiến trúc" ba tầng phức tạp[1] mà họ có thể phân phối qua các trang trại đó. Sẽ có máy chủ cho GUI, máy chủ cho middleware, và máy chủ cho cơ sở dữ liệu. Tất nhiên.

Các lập trình viên quyết định từ rất sớm rằng mỗi đối tượng miền sẽ có ba bản thể hiện (instances): một ở tầng GUI, một ở tầng middleware và một ở tầng cơ sở dữ liệu. Vì các bản thể hiện đó chạy trên những máy khác nhau, họ xây dựng một cơ chế giao tiếp phức tạp giữa các bộ xử lý và các tầng. Các lời gọi phương thức giữa các tầng được chuyển thành đối tượng, tuần tự hóa (serialized) rồi tổ chức truyền (marshaled) qua mạng.

Bây giờ hãy tưởng tượng việc triển khai một tính năng đơn giản như thêm một trường mới vào một bản ghi hiện có tốn bao nhiêu công sức. Trường đó phải được thêm vào các lớp ở cả ba tầng, và vào một vài thông điệp liên tầng. Vì dữ liệu di chuyển theo cả hai hướng, bốn giao thức thông điệp cần được thiết kế. Mỗi giao thức có một phía gửi và một phía nhận, do đó cần tám trình xử lý giao thức. Ba file thực thi phải được build, mỗi file có ba đối tượng nghiệp vụ cập nhật, bốn thông điệp mới, và tám trình xử lý mới.

Và hãy nghĩ xem những file thực thi đó phải làm gì để triển khai đơn giản nhất các tính năng. Hãy nghĩ đến tất cả các lần khởi tạo đối tượng, tất cả các lần tuần tự hóa, tất cả các lần tổ chức truyền và giải mã (de-marshaling), tất cả việc xây dựng và phân tích thông điệp, tất cả giao tiếp socket, trình quản lý timeout, kịch bản thử lại (retry scenarios), và tất cả những công việc phụ trợ khác mà bạn phải làm chỉ để hoàn thành một việc đơn giản.

Tất nhiên, trong quá trình phát triển, các lập trình viên không có trang trại máy chủ. Thực tế, họ chỉ chạy ba file thực thi đó trong ba tiến trình khác nhau trên một máy đơn lẻ. Họ phát triển theo cách này trong vài năm. Nhưng họ tin rằng kiến trúc của mình là đúng. Vì vậy, dù họ đang chạy trên một máy đơn lẻ, họ vẫn tiếp tục tất cả các lần khởi tạo đối tượng, tất cả các lần tuần tự hóa, tất cả các lần tổ chức truyền và giải mã, tất cả việc xây dựng và phân tích thông điệp, tất cả giao tiếp socket, và tất cả những thứ rườm rà khác trong một máy đơn lẻ.

Điều trớ trêu là công ty P chưa bao giờ bán một hệ thống cần trang trại máy chủ. Mọi hệ thống họ triển khai đều chạy trên một máy chủ duy nhất. Dù vậy, cả ba chương trình thực thi trong hệ thống ấy vẫn thực hiện mọi thao tác khởi tạo đối tượng, tuần tự hóa, tổ chức truyền và giải mã, tạo và phân tích thông điệp, giao tiếp qua socket cùng vô số công

1. Từ "architecture" xuất hiện trong ngoặc kép ở đây vì ba tầng (three-tier) không phải là kiến trúc; đó là một topology (cấu trúc liên kết). Đó chính là loại quyết định mà một kiến trúc tốt nỗ lực trì hoãn.

việc phụ trợ khác, chỉ để chờ một trang trại máy chủ chưa từng tồn tại và sẽ chẳng bao giờ xuất hiện.

Bi kịch là các kiến trúc sư, bằng cách ra một quyết định quá sớm, đã làm tăng nỗ lực phát triển lên gấp nhiều lần.

Câu chuyện của P không phải là trường hợp độc lập. Tôi đã thấy nó nhiều lần ở nhiều nơi. Thực ra, P là sự chồng chéo (superposition) của tất cả những nơi đó.

Nhưng có số phận tồi tệ hơn P.

Hãy xét W, một doanh nghiệp địa phương quản lý đội xe. Gần đây, họ tuyển một "kiến trúc sư" để kiểm soát nỗ lực phát triển phần mềm vốn đang rối ren. Và anh ta đúng là người thích kiểm soát. Anh ta nhanh chóng kết luận rằng doanh nghiệp nhỏ này cần một kiến trúc hướng dịch vụ (SoA) toàn diện ở cấp doanh nghiệp. Anh ta xây dựng mô hình miền khổng lồ cho mọi "đối tượng" trong doanh nghiệp, thiết kế một bộ dịch vụ quản lý chúng, rồi đẩy các lập trình viên vào một cơn ác mộng. Chẳng hạn, muốn thêm tên, địa chỉ và số điện thoại của một người liên hệ vào bản ghi bán hàng, bạn phải truy vấn `ServiceRegistry` để lấy ID dịch vụ `ContactService`. Sau đó, bạn gửi thông điệp `CreateContact` đến `ContactService`. Thông điệp này có hàng chục trường, mỗi trường phải chứa dữ liệu hợp lệ, nhưng lập trình viên không có những dữ liệu đó, vì họ chỉ biết tên, địa chỉ và số điện thoại. Sau khi tự bịa dữ liệu, lập trình viên phải chèn ID của người liên hệ mới vào bản ghi bán hàng rồi gửi thông điệp `UpdateContact` đến `SaleRecordService`. Tất nhiên, để kiểm thử bất kỳ chức năng nào, bạn phải lần lượt khởi động mọi dịch vụ cần thiết, rồi khởi động message bus và máy chủ BPEL. Sau đó còn phải chờ thông điệp truyền từ dịch vụ này sang dịch vụ khác và nằm trong hết hàng đợi này đến hàng đợi kia.

Và rồi nếu bạn muốn thêm một tính năng mới, bạn có thể tưởng tượng được sự kết dính giữa tất cả những dịch vụ đó, và khối lượng WSDL khổng lồ cần thay đổi, và tất cả các lần triển khai lại mà những thay đổi đó bắt buộc …

Địa ngục bắt đầu trông như một nơi dễ chịu so với cái này.

Không có gì sai về bản chất của một hệ thống phần mềm được tổ chức quanh các dịch vụ. Sai lầm ở W là triển khai và áp đặt quá sớm một bộ công cụ hứa hẹn SoA, tức là quá sớm dựng lên cả một hệ thống dịch vụ đồ sộ cho các đối tượng miền. Cái giá phải trả chỉ là giờ công, rất nhiều giờ công, bị ném vào hố đen SoA.

Tôi có thể tiếp tục liệt kê thất bại kiến trúc này mãi. Nhưng hãy nói về một thành công kiến trúc nào.

FitNesse

Con trai tôi, Micah, và tôi bắt đầu làm việc trên FitNesse vào năm 2001. Ý tưởng là tạo một wiki đơn giản bao bọc công cụ FIT của Ward Cunningham để viết các kiểm thử chấp nhận.

Đây là những ngày trước khi Maven "giải quyết" vấn đề file jar. Tôi kiên quyết rằng bất cứ thứ gì chúng tôi tạo ra không nên yêu cầu người dùng tải xuống hơn một file jar. Tôi gọi quy tắc này là "Tải về và Chạy" (Download and Go). Quy tắc này chi phối nhiều quyết định của chúng tôi.

Một trong những quyết định đầu tiên là viết máy chủ web của chính chúng tôi, chuyên biệt cho nhu cầu của FitNesse. Điều này nghe có vẻ ngớ ngẩn. Ngay cả năm 2001 đã có nhiều máy chủ web mã nguồn mở chúng tôi có thể dùng. Tuy nhiên, viết máy chủ của riêng mình kết quả là một quyết định thực sự tốt vì một máy chủ web cơ bản là một phần mềm rất đơn giản để viết và nó cho phép chúng tôi hoãn quyết định về framework web đến muộn hơn nhiều[2].

Một quyết định sớm khác là tránh nghĩ về cơ sở dữ liệu. Chúng tôi vẫn nghĩ đến MySQL, nhưng chúng tôi cố ý trì hoãn quyết định đó bằng cách dùng một thiết kế khiến quyết định trở nên vô nghĩa. Thiết kế đó đơn giản là đặt một giao diện (interface) giữa tất cả các truy cập dữ liệu và kho dữ liệu (repository) chính nó.

Chúng tôi đặt các phương thức truy cập dữ liệu vào một giao diện tên là `WikiPage`. Những phương thức đó cung cấp tất cả chức năng chúng tôi cần để tìm, tải, và lưu các trang. Tất nhiên, chúng tôi không triển khai các phương thức đó ngay; chúng tôi chỉ để các phương thức đó ở trạng thái stub trong khi làm việc trên các tính năng không liên quan đến tải và lưu dữ liệu.

Thực tế, trong ba tháng chúng tôi chỉ làm việc dịch văn bản wiki sang HTML. Điều này không yêu cầu bất kỳ loại lưu trữ dữ liệu nào, nên chúng tôi tạo một lớp tên là `MockWikiPage` chỉ để các phương thức truy cập dữ liệu ở trạng thái stub.

Cuối cùng, những stub đó trở nên không đủ cho các tính năng chúng tôi muốn viết. Chúng tôi cần truy cập dữ liệu thật, không phải stub. Vì vậy chúng tôi tạo một lớp dẫn xuất mới của `WikiPage` tên là `InMemoryPage`. Lớp dẫn xuất này triển khai phương thức truy cập dữ liệu để quản lý một bảng băm (hash table) các trang wiki, được giữ trong RAM.

Điều này cho phép chúng tôi viết tính năng theo tính năng suốt một năm trời. Thực tế, chúng tôi làm chạy được toàn bộ phiên bản đầu tiên của chương trình FitNesse theo cách này. Chúng tôi có thể tạo trang, liên kết đến trang khác, làm tất cả định dạng wiki phức tạp, thậm chí chạy kiểm thử với FIT. Điều chúng tôi không làm được là lưu bất kỳ công việc nào.

Khi đến lúc triển khai tính năng lưu trữ (persistence), chúng tôi lại nghĩ đến MySQL, nhưng quyết định rằng không cần thiết trong ngắn hạn, vì viết bảng băm ra file phẳng (flat file) sẽ rất dễ dàng. Vì vậy chúng tôi triển khai `FileSystemWikiPage`, chỉ chuyển chức năng ra file phẳng, và tiếp tục phát triển thêm tính năng. Ba tháng sau, chúng tôi kết luận giải pháp file phẳng đủ tốt; quyết định bỏ qua ý định MySQL hoàn toàn. Chúng tôi hoãn quyết định đó đến mức nó biến mất và không bao giờ nhìn lại.

---

2. Nhiều năm sau chúng tôi mới nhúng framework Velocity vào FitNesse.

Câu chuyện sẽ kết thúc ở đây nếu không có một khách hàng quyết định anh ta cần đưa wiki vào MySQL cho mục đích riêng. Chúng tôi chỉ cho anh ta kiến trúc `WikiPage` đã cho phép chúng tôi trì hoãn quyết định. Anh ta quay lại một ngày sau với toàn bộ hệ thống chạy trên MySQL. Anh ta chỉ viết một lớp dẫn xuất `MySqlWikiPage` và làm cho nó chạy.

Chúng tôi từng gói tùy chọn đó cùng FitNesse, nhưng không ai khác dùng nó, nên cuối cùng chúng tôi bỏ nó. Ngay cả khách hàng viết lớp dẫn xuất cuối cùng cũng bỏ nó.

Sớm trong quá trình phát triển FitNesse, chúng tôi vạch một đường ranh giới giữa quy tắc nghiệp vụ và cơ sở dữ liệu. Đường đó ngăn cản quy tắc nghiệp vụ biết bất cứ điều gì về cơ sở dữ liệu, trừ các phương thức truy cập dữ liệu đơn giản. Quyết định đó cho phép chúng tôi trì hoãn lựa chọn và triển khai cơ sở dữ liệu được hơn một năm. Nó cho phép chúng tôi thử tùy chọn hệ thống file, và cho phép chúng tôi đổi hướng khi thấy giải pháp tốt hơn. Tuy nhiên nó không cản trở, hay thậm chí làm chậm, việc di chuyển theo hướng ban đầu (MySQL) khi ai đó muốn.

Việc không có cơ sở dữ liệu chạy trong 18 tháng phát triển có nghĩa là, trong 18 tháng, chúng tôi không có vấn đề schema, vấn đề truy vấn, vấn đề máy chủ cơ sở dữ liệu, vấn đề mật khẩu, vấn đề thời gian kết nối, và tất cả những vấn đề xấu xí khác xuất hiện khi bạn khởi động một cơ sở dữ liệu. Nó cũng có nghĩa là tất cả kiểm thử của chúng tôi chạy nhanh, vì không có cơ sở dữ liệu làm chậm chúng.

Tóm lại, vạch các đường ranh giới giúp chúng tôi trì hoãn và hoãn các quyết định, và cuối cùng nó giúp chúng tôi tiết kiệm một lượng lớn thời gian và đau đầu. Và đó chính là những gì một kiến trúc tốt nên làm.

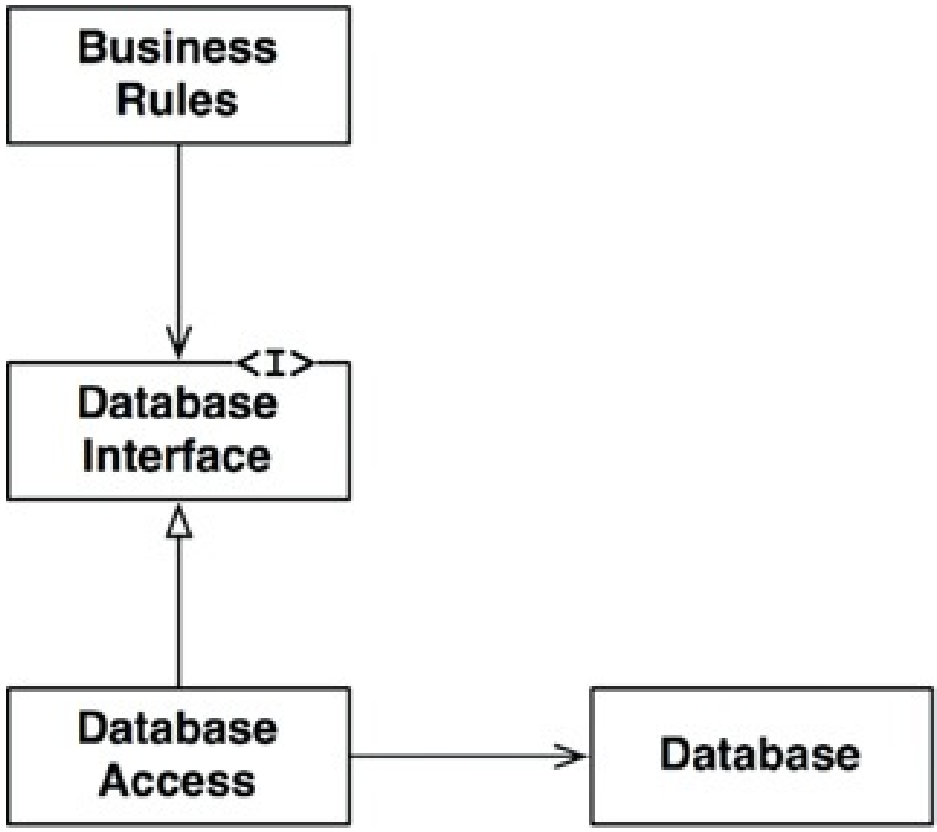
Vạch những đường nào, và vạch khi nào?

Bạn vạch đường giữa những thứ quan trọng và những thứ không quan trọng. GUI không quan trọng đối với quy tắc nghiệp vụ, nên có một đường ranh giới giữa chúng. Cơ sở dữ liệu không quan trọng đối với GUI, nên có một đường ranh giới giữa chúng. Cơ sở dữ liệu không quan trọng đối với quy tắc nghiệp vụ, nên có một đường ranh giới giữa chúng.

Một số bạn có thể đã bác bỏ một hoặc nhiều phát biểu trên, đặc biệt là phần về quy tắc nghiệp vụ không quan tâm đến cơ sở dữ liệu. Nhiều trong chúng ta đã được dạy tin rằng cơ sở dữ liệu gắn liền không tách rời với quy tắc nghiệp vụ. Một số trong chúng ta thậm chí đã bị thuyết phục rằng cơ sở dữ liệu là bản thể hiện của quy tắc nghiệp vụ.

Nhưng, như chúng ta sẽ thấy ở một chương khác, ý tưởng này sai lầm. Cơ sở dữ liệu là một công cụ mà quy tắc nghiệp vụ có thể dùng gián tiếp. Quy tắc nghiệp vụ không cần biết về schema, ngôn ngữ truy vấn, hoặc bất kỳ chi tiết nào khác về cơ sở dữ liệu. Tất cả những gì quy tắc nghiệp vụ cần biết là có một tập hợp hàm có thể dùng để tải hoặc lưu dữ liệu. Điều này cho phép chúng ta đặt cơ sở dữ liệu sau một giao diện.

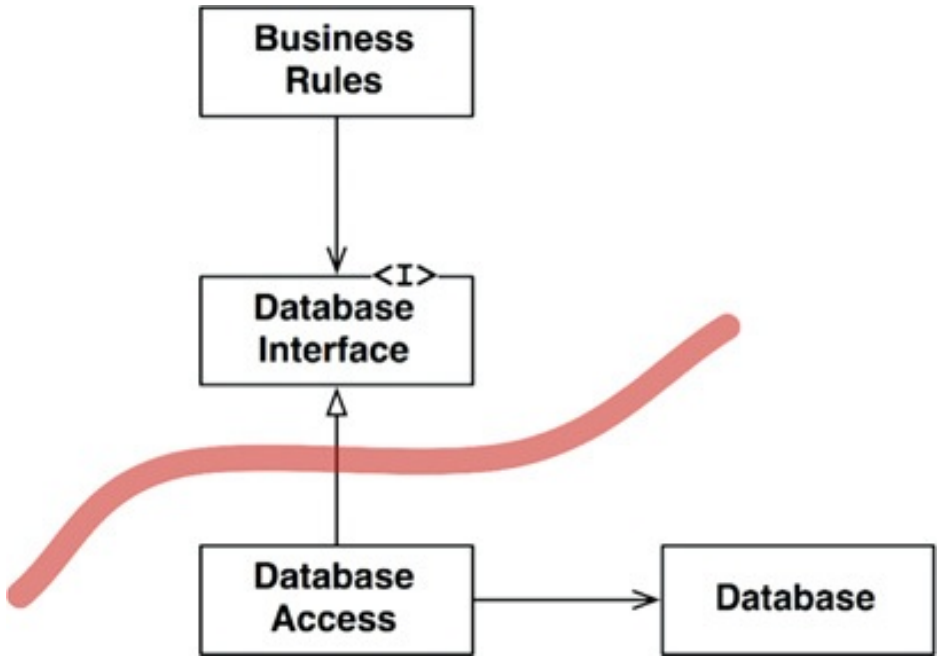
Bạn có thể thấy rõ điều này trong Hình 17.1. `BusinessRules` (Quy tắc nghiệp vụ) dùng `DatabaseInterface` (Giao diện cơ sở dữ liệu) để tải và lưu dữ liệu. `Database Access` (Truy cập cơ sở dữ liệu) triển khai giao diện đó và điều khiển hoạt động của `Database` (Cơ sở dữ liệu) thực tế.



Hình 17.1
Cơ sở dữ liệu sau một giao diện

Các lớp và giao diện trong sơ đồ này mang tính biểu tượng. Trong một ứng dụng thực, sẽ có nhiều lớp quy tắc nghiệp vụ, nhiều lớp giao diện cơ sở dữ liệu, và nhiều triển khai truy cập cơ sở dữ liệu. Tất cả chúng, tuy nhiên, sẽ tuân theo đại thể cùng một mẫu.

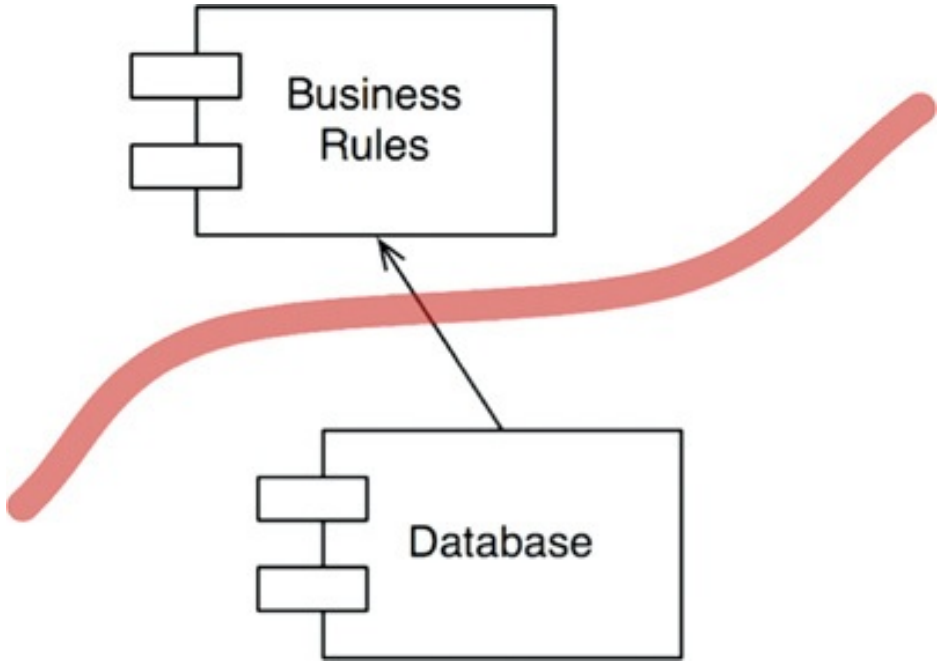
Đường ranh giới nằm ở đâu? Nó đi qua quan hệ kế thừa (inheritance), ngay bên dưới `DatabaseInterface` (Hình 17.2).



Hình 17.2
Đường ranh giới

Hãy chú ý hai mũi tên hướng ra khỏi `DatabaseAccess`. Điều đó cho thấy các lớp còn lại không biết `DatabaseAccess` tồn tại.

Bây giờ hãy lùi lại một bước. Ta sẽ xem thành phần chứa các quy tắc nghiệp vụ và thành phần chứa cơ sở dữ liệu cùng mọi lớp truy cập của nó (Hình 17.3).



Hình 17.3

Thành phần quy tắc nghiệp vụ và thành phần cơ sở dữ liệu

Chú ý hướng của mũi tên. `Database` (Cơ sở dữ liệu) biết về `BusinessRules` (Quy tắc nghiệp vụ). `BusinessRules` không biết về `Database`. Điều này ngụ ý các lớp `Database Interface` sống trong thành phần `BusinessRules`, trong khi các lớp `DatabaseAccess` sống trong thành phần `Database`.

Hướng của đường này quan trọng. Nó cho thấy `Database` không quan trọng đối với `BusinessRules`, nhưng `Database` không thể tồn tại nếu không có `BusinessRules`.

Nếu điều đó nghe lạ, hãy nhớ điểm này: Thành phần `Database` chứa mã dịch các lời gọi do `BusinessRules` thực hiện sang ngôn ngữ truy vấn của cơ sở dữ liệu. Chính mã dịch đó biết về `BusinessRules`.

Đã vạch đường ranh giới giữa hai thành phần này, và đã thiết lập hướng mũi tên trỏ về `BusinessRules`, chúng ta giờ thấy `BusinessRules` có thể dùng bất kỳ loại cơ sở dữ liệu nào. Thành phần `Database` có thể được thay thế bằng nhiều triển khai khác nhau, `BusinessRules` không quan tâm.

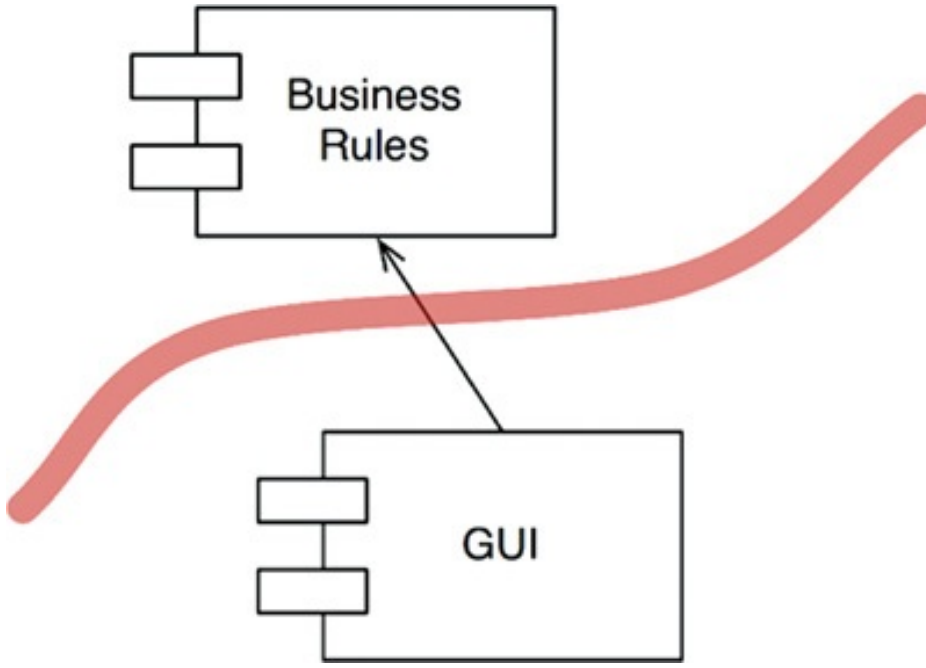
Cơ sở dữ liệu có thể triển khai bằng Oracle, hoặc MySQL, hoặc Couch, hoặc Datomic, thậm chí file phẳng. Quy tắc nghiệp vụ hoàn toàn không quan tâm. Và điều đó có nghĩa là quyết định về cơ sở dữ liệu có thể trì hoãn, và bạn có thể tập trung vào việc viết và kiểm thử quy tắc nghiệp vụ trước khi phải ra quyết định về cơ sở dữ liệu.

Còn đầu vào và đầu ra thì sao?

Lập trình viên và khách hàng thường nhầm lẫn về bản chất của hệ thống. Họ nhìn thấy GUI và cho rằng đó chính là hệ thống. Họ định nghĩa hệ thống qua GUI, nên tin rằng GUI phải xuất hiện ngay lập tức. Họ bỏ qua một nguyên tắc cực kỳ quan trọng: **IO không liên quan** (The IO is irrelevant).

Điều này ban đầu có thể khó hiểu. Ta thường mô tả hành vi hệ thống qua hành vi IO. Hãy xét trò chơi điện tử. Trải nghiệm của bạn chịu ảnh hưởng bởi giao diện: màn hình, chuột, nút bấm và âm thanh. Bạn có thể quên rằng phía sau giao diện là một mô hình, gồm những cấu trúc dữ liệu và hàm tinh vi điều khiển trò chơi. Quan trọng hơn, mô hình ấy không cần giao diện. Nó vẫn thực hiện công việc của mình, duy trì trạng thái của mọi sự kiện trong trò chơi, ngay cả khi trò chơi không hề được hiển thị trên màn hình. Giao diện không liên quan đến mô hình, tức là các quy tắc nghiệp vụ.

Và một lần nữa, chúng ta thấy các thành phần `GUI` và `BusinessRules` được tách biệt bởi một đường ranh giới (Hình 17.4). Thành phần ít liên quan hơn phụ thuộc vào thành phần quan trọng hơn. Các mũi tên cho thấy thành phần nào biết về thành phần kia, và do đó thành phần nào quan tâm đến thành phần kia. `GUI` quan tâm đến `BusinessRules`.



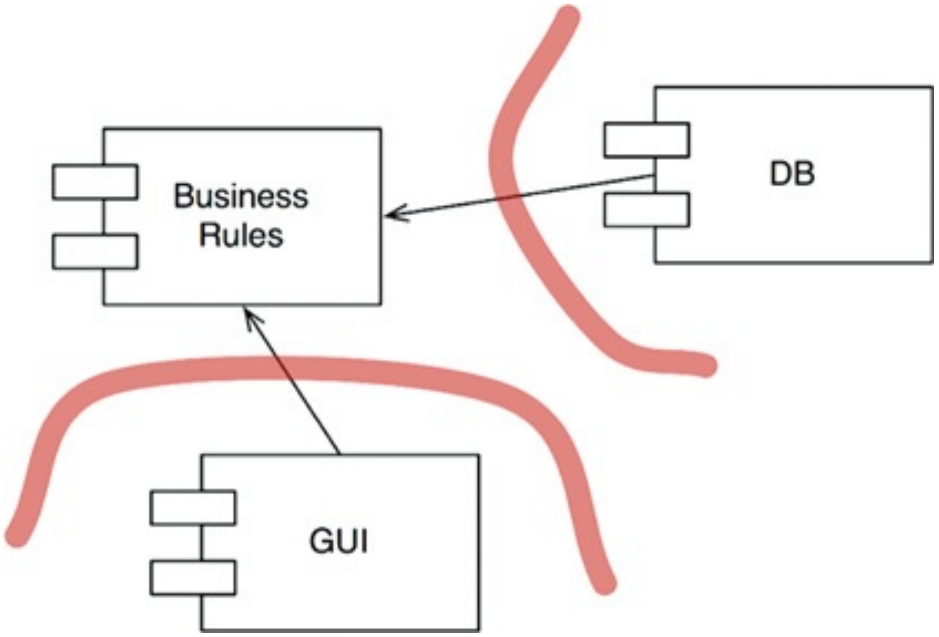
Hình 17.4
Ranh giới giữa thành phần GUI và thành phần BusinessRules

Đã vạch ranh giới này và mũi tên này, chúng ta giờ thấy `GUI` có thể được thay thế bằng bất kỳ loại giao diện nào khác, và `BusinessRules` sẽ không quan tâm.

Kiến trúc plugin

Kết hợp lại, hai quyết định này về cơ sở dữ liệu và GUI tạo ra một kiểu mẫu để thêm các thành phần khác. Mẫu đó chính là mẫu được các hệ thống cho phép plugin của bên thứ ba sử dụng.

Thực tế, lịch sử phát triển phần mềm là câu chuyện về việc tạo ra các plugin để xây dựng kiến trúc hệ thống có thể mở rộng và bảo trì. Các quy tắc nghiệp vụ cốt lõi được giữ tách biệt, độc lập với những thành phần tùy chọn hoặc có thể được triển khai theo nhiều cách khác nhau (Hình 17.5).



Hình 17.5

Cắm vào quy tắc nghiệp vụ

Vì giao diện người dùng trong thiết kế này được coi là một plugin, chúng ta đã làm cho việc cắm nhiều loại giao diện người dùng khác nhau trở nên khả thi. Chúng có thể dựa trên web, dựa trên client/server, dựa trên SOA, dựa trên Console, hoặc dựa trên bất kỳ công nghệ giao diện người dùng nào khác.

Tương tự đối với cơ sở dữ liệu. Vì chúng ta chọn coi nó như một plugin, chúng ta có thể thay thế nó bằng bất kỳ cơ sở dữ liệu SQL nào, hoặc cơ sở dữ liệu NoSQL, hoặc cơ sở dữ liệu dựa trên hệ thống file, hoặc bất kỳ công nghệ cơ sở dữ liệu nào chúng ta có thể cho là cần thiết trong tương lai.

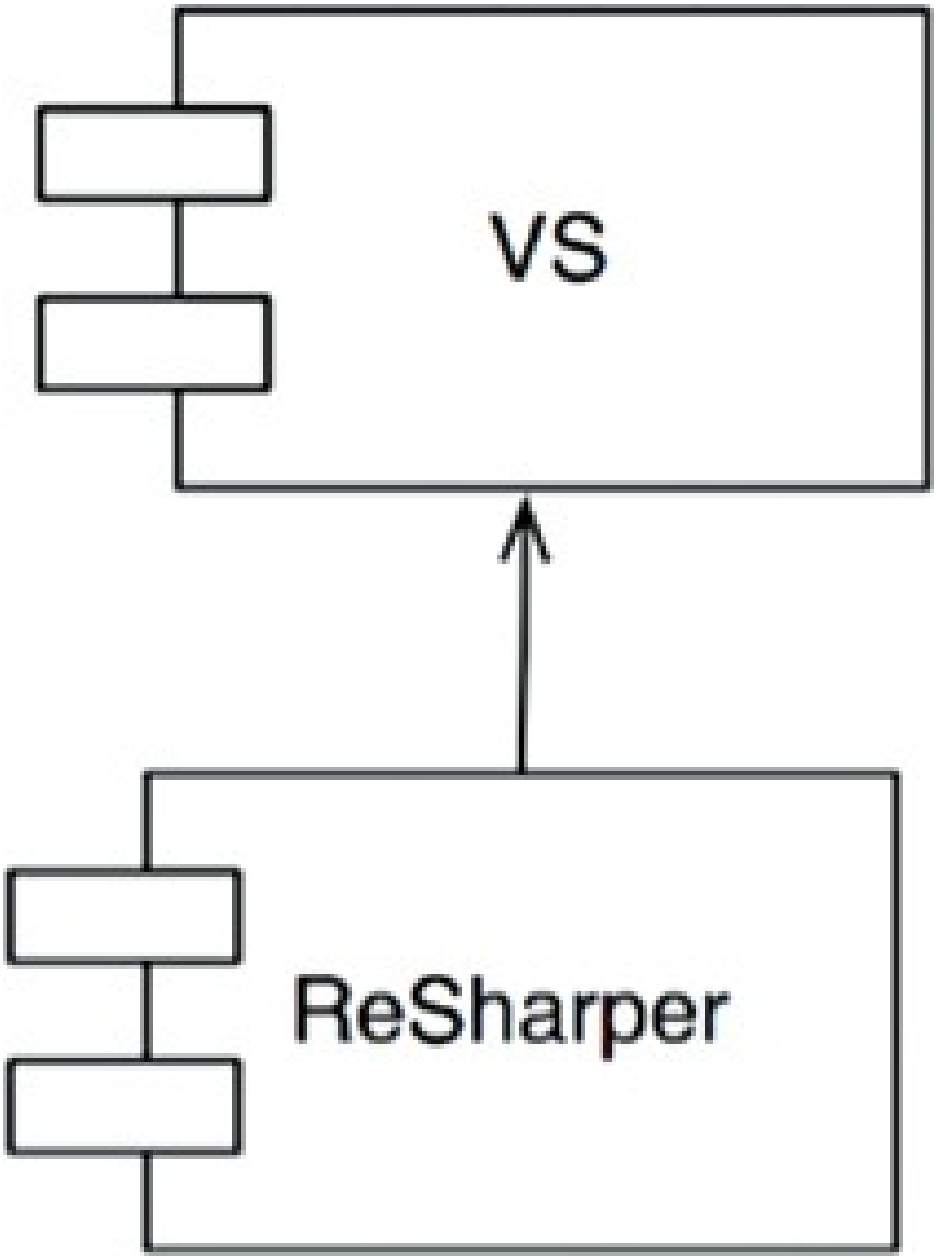
Những sự thay thế này có thể không tầm thường. Nếu triển khai ban đầu của hệ thống là dựa trên web, thì viết plugin cho giao diện client-server có thể thách thức. Có khả năng

một số giao tiếp giữa quy tắc nghiệp vụ và GUI mới phải được viết lại. Tuy nhiên, bằng cách bắt đầu với giả định một cấu trúc plugin, chúng ta đã ít nhất làm cho một thay đổi như vậy trở nên thực tế.

Lý lẽ về plugin

Xét mối quan hệ giữa ReSharper và Visual Studio. Hai thành phần này được tạo ra bởi các nhóm phát triển hoàn toàn khác nhau trong các công ty hoàn toàn khác nhau. Thực tế, JetBrains, nhà làm ReSharper, đặt tại Nga. Microsoft, tất nhiên, đặt tại Redmond, Washington. Khó tưởng tượng hai nhóm phát triển tách biệt hơn thế.

Nhóm nào có thể gây hại cho nhóm kia? Nhóm nào được bảo vệ khỏi nhóm kia? Cấu trúc phụ thuộc cho thấy rõ điều đó (Hình 17.6). Mã nguồn của ReSharper phụ thuộc vào mã nguồn của Visual Studio. Vì thế, nhóm ReSharper không thể làm ảnh hưởng đến nhóm Visual Studio. Nhưng nếu muốn, nhóm Visual Studio có thể khiến ReSharper hoàn toàn không hoạt động.



Hình 17.6

ReSharper phụ thuộc vào Visual Studio

Đó là một mối quan hệ bất đối xứng sâu sắc, và đó là mối quan hệ chúng ta mong muốn trong hệ thống của mình. Chúng ta muốn một số mô-đun ít bị ảnh hưởng bởi các mô-đun khác. Ví dụ, chúng ta không muốn quy tắc nghiệp vụ bị phá vỡ khi ai đó thay đổi định dạng trang web, hoặc thay đổi schema cơ sở dữ liệu. Chúng ta không muốn thay đổi ở một phần của hệ thống gây hỏng các phần không liên quan khác. Chúng ta không muốn hệ thống thể hiện sự mong manh (fragility) kiểu đó.

Việc sắp xếp hệ thống thành kiến trúc plugin tạo ra các tường lửa (firewalls) mà thay đổi không thể lan truyền qua. Nếu GUI cắm vào quy tắc nghiệp vụ, thì thay đổi trong GUI không thể ảnh hưởng đến những quy tắc nghiệp vụ đó.

Ranh giới được vạch ở nơi có trục thay đổi (axis of change). Các thành phần ở một bên ranh giới thay đổi với tốc độ khác nhau, và vì lý do khác nhau, so với các thành phần ở bên kia ranh giới.

GUI thay đổi ở các thời điểm và tốc độ khác nhau so với quy tắc nghiệp vụ, nên có ranh giới giữa chúng. Quy tắc nghiệp vụ thay đổi ở các thời điểm và vì những lý do khác nhau so với framework tiêm phụ thuộc (dependency injection framework), nên có ranh giới giữa chúng.

Đây đơn giản là Nguyên tắc Trách nhiệm Đơn nhất (SRP) trở lại. SRP cho chúng ta biết vạch ranh giới ở đâu.

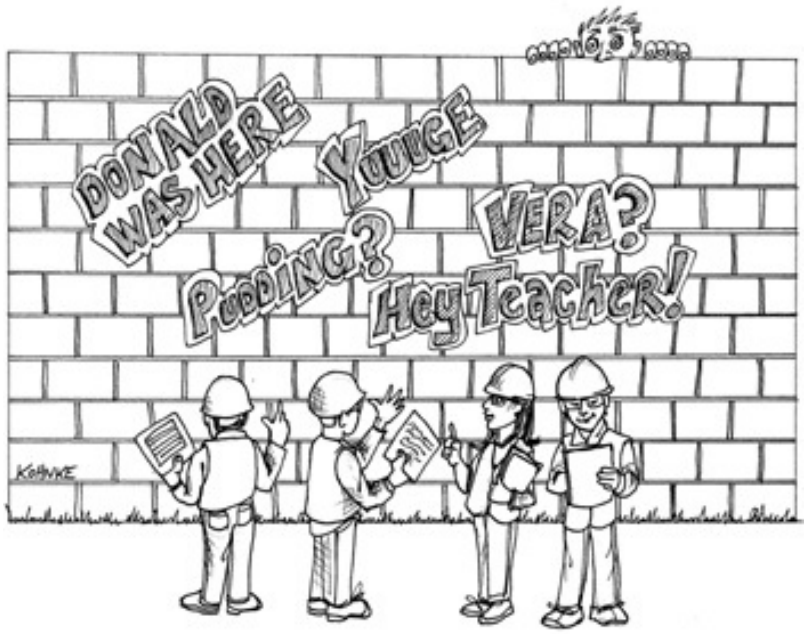
Kết luận

Để vạch đường ranh giới trong một kiến trúc phần mềm, bạn trước tiên phân chia hệ thống thành các thành phần. Một số thành phần là quy tắc nghiệp vụ cốt lõi; những thành phần khác là plugin chứa các chức năng cần thiết nhưng không liên quan trực tiếp đến nghiệp vụ cốt lõi. Sau đó bạn sắp xếp mã trong các thành phần đó sao cho các mũi tên giữa chúng chỉ theo một hướng, về phía nghiệp vụ cốt lõi.

Bạn nên nhận ra đây là một ứng dụng của Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle) và Nguyên tắc Trừu tượng Ổn định (Stable Abstractions Principle). Các mũi tên phụ thuộc được sắp xếp để trỏ từ chi tiết mức thấp về phía trừu tượng mức cao.

18

Giải phẫu ranh giới



Kiến trúc của một hệ thống được xác định bởi tập hợp các thành phần phần mềm và những ranh giới ngăn cách chúng. Các ranh giới ấy có nhiều dạng khác nhau. Trong chương này, chúng ta sẽ xem xét một số dạng phổ biến nhất.

Vượt qua ranh giới

Khi chương trình đang chạy, vượt qua ranh giới chỉ đơn giản là một hàm ở phía này gọi một hàm ở phía kia và truyền theo một ít dữ liệu. Bí quyết để tạo ranh giới phù hợp là quản lý các phụ thuộc mã nguồn.

Vì sao phải quan tâm đến mã nguồn? Vì khi một mô-đun mã nguồn thay đổi, các mô-đun khác có thể cũng phải thay đổi, biên dịch lại rồi triển khai lại. Ranh giới có nhiệm vụ quản lý và ngăn chặn sự lan truyền của những thay đổi đó.

Khối nguyên khối đáng sợ

Ranh giới kiến trúc đơn giản và phổ biến nhất không có biểu hiện vật lý cụ thể. Đó chỉ là sự phân tách có kỷ luật giữa các hàm và dữ liệu trong cùng một bộ xử lý và cùng một không gian địa chỉ. Trong chương trước, tôi gọi đây là chế độ tách rời ở cấp mã nguồn.

Xét từ góc độ triển khai, cấu trúc này chỉ gồm một tệp thực thi duy nhất, thường được gọi là khối nguyên khối. Tệp đó có thể là dự án C hoặc C++ liên kết tĩnh, tập hợp các tệp lớp Java đóng gói trong một tệp jar thực thi, các tệp nhị phân .NET gộp vào một tệp .EXE, v.v.

Việc các ranh giới không hiện ra trong quá trình triển khai khối nguyên khối không có nghĩa là chúng không tồn tại hoặc không có ý nghĩa. Ngay cả khi được liên kết tĩnh thành một tệp thực thi duy nhất, khả năng phát triển độc lập các thành phần khác nhau rồi điều phối chúng để lắp ráp cuối cùng vẫn vô cùng giá trị.

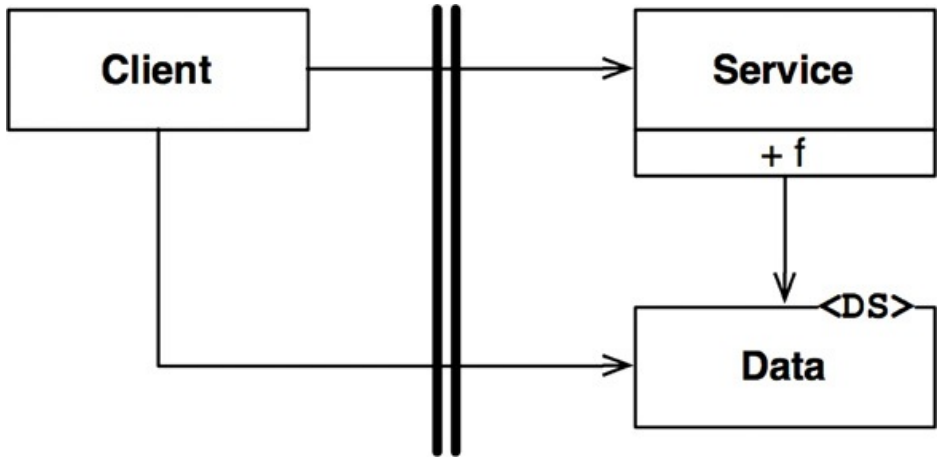
Những kiến trúc như vậy hầu như luôn dựa vào đa hình động[1] để quản lý các phụ thuộc nội bộ. Đây là một trong những lý do khiến phát triển hướng đối tượng trở thành mô hình quan trọng trong vài thập kỷ gần đây. Nếu không có hướng đối tượng, hoặc một hình thức đa hình tương đương, kiến trúc sư phải dùng đến cách nguy hiểm là sử dụng con trỏ hàm để đạt được sự tách rời cần thiết. Hầu hết kiến trúc sư cho rằng dùng con trỏ hàm tràn lan quá rủi ro, nên họ buộc phải từ bỏ việc phân chia hệ thống thành các thành phần.

Cách vượt ranh giới đơn giản nhất là một hàm ở phía khách hàng cấp thấp gọi dịch vụ cấp cao hơn. Cả phụ thuộc lúc chạy lẫn phụ thuộc lúc biên dịch đều cùng hướng về thành phần cấp cao hơn.

Trong Hình 18.1, luồng điều khiển đi qua ranh giới từ trái sang phải. `Client` gọi hàm `f()` của `Service` và truyền theo một thể hiện của `Data`. Ký hiệu `<DS>` chỉ một cấu trúc dữ liệu. `Data` có thể được truyền như đối số hàm hoặc bằng một phương thức phức tạp hơn. Lưu ý rằng định nghĩa của `Data` nằm ở phía được gọi của ranh giới.

---

1. Đa hình tĩnh (chẳng hạn generics hoặc templates) đôi khi có thể là cách quản lý phụ thuộc khả thi trong hệ thống nguyên khối, nhất là với các ngôn ngữ như C++. Tuy nhiên, sự tách rời nhờ generics không thể tránh yêu cầu biên dịch lại và triển khai lại như đa hình động.

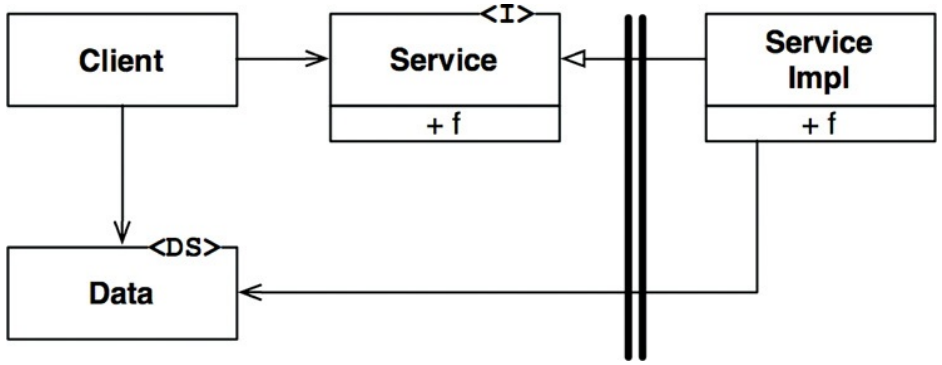


Hình 18.1

Luồng điều khiển đi qua ranh giới từ cấp thấp lên cấp cao

Khi một khách hàng cấp cao cần gọi dịch vụ cấp thấp, đa hình động được dùng để đảo ngược phụ thuộc, ngược với hướng luồng điều khiển. Phụ thuộc lúc chạy đi ngược hướng với phụ thuộc lúc biên dịch.

Trong Hình 18.2, luồng điều khiển vẫn đi qua ranh giới từ trái sang phải. `Client` cấp cao gọi hàm `f()` của `ServiceImpl` cấp thấp thông qua giao diện `Service`. Tuy nhiên, hãy lưu ý rằng mọi phụ thuộc đều đi qua ranh giới từ phải sang trái, hướng về thành phần cấp cao hơn. Cũng cần lưu ý định nghĩa của cấu trúc dữ liệu nằm ở phía gọi.



Hình 18.2

Vượt ranh giới ngược với luồng điều khiển

Ngay cả trong một tệp thực thi nguyên khối được liên kết tĩnh, việc phân chia có kỷ luật như vậy cũng có thể hỗ trợ đáng kể cho quá trình phát triển, kiểm thử và triển khai dự án. Các nhóm có thể làm việc độc lập trên thành phần của mình mà không cản trở nhau. Thành phần cấp cao vẫn độc lập với các chi tiết cấp thấp.

Giao tiếp giữa các thành phần trong một khối nguyên khối rất nhanh và ít tốn kém. Thông thường, chúng chỉ là các lời gọi hàm. Vì vậy, giao tiếp qua các ranh giới tách rời ở cấp mã nguồn có thể diễn ra rất thường xuyên.

Vì triển khai khối nguyên khối thường đòi hỏi biên dịch và liên kết tĩnh, các thành phần trong những hệ thống này thường được phân phối dưới dạng mã nguồn.

Các thành phần triển khai

Biểu hiện vật lý đơn giản nhất của một ranh giới kiến trúc là thư viện liên kết động, chẳng hạn DLL của .NET, tệp jar của Java, Gem của Ruby hoặc thư viện dùng chung của UNIX. Việc triển khai không cần biên dịch. Thay vào đó, các thành phần được phân phối dưới dạng nhị phân hoặc một dạng tương đương có thể triển khai. Đây là chế độ tách rời ở cấp triển khai. Việc triển khai chỉ đơn giản là tập hợp các đơn vị có thể triển khai vào một dạng thuận tiện, chẳng hạn tệp WAR hoặc một thư mục.

Ngoại trừ điểm đó, các thành phần ở cấp triển khai giống với khối nguyên khối. Các hàm thường vẫn chạy trong cùng một bộ xử lý và không gian địa chỉ. Cách phân tách thành phần và quản lý phụ thuộc cũng như nhau.[2]

Tương tự khối nguyên khối, giao tiếp qua ranh giới giữa các thành phần triển khai chỉ là lời gọi hàm nên rất ít tốn kém. Có thể phát sinh chi phí một lần cho việc liên kết động hoặc nạp lúc chạy, nhưng giao tiếp qua những ranh giới này vẫn có thể diễn ra rất thường xuyên.

Luồng

Cả khối nguyên khối lẫn các thành phần triển khai đều có thể sử dụng luồng. Luồng không phải là ranh giới kiến trúc hay đơn vị triển khai; chúng là cách tổ chức lịch trình và thứ tự thực thi. Luồng có thể nằm hoàn toàn trong một thành phần hoặc trải rộng qua nhiều thành phần.

Tiến trình cục bộ

Một ranh giới kiến trúc vật lý mạnh hơn nhiều là tiến trình cục bộ. Tiến trình cục bộ thường được tạo từ dòng lệnh hoặc một lệnh gọi hệ thống tương đương. Các tiến trình cục bộ chạy trên cùng một bộ xử lý, hoặc trong cùng một nhóm bộ xử lý đa lõi, nhưng ở các không gian địa chỉ riêng biệt. Cơ chế bảo vệ bộ nhớ thường ngăn các tiến trình chia sẻ bộ nhớ, dù các phân vùng bộ nhớ dùng chung vẫn thường được sử dụng.

Thông thường, các tiến trình cục bộ giao tiếp với nhau qua socket hoặc một phương tiện giao tiếp khác của hệ điều hành, chẳng hạn mailbox hay hàng đợi thông điệp.

Mỗi tiến trình cục bộ có thể là một khối nguyên khối liên kết tĩnh, hoặc được cấu thành

2. Trong trường hợp này, đa hình tĩnh không phải là một lựa chọn.

từ các thành phần triển khai liên kết động. Trong trường hợp thứ nhất, một số tiến trình nguyên khối có thể chứa cùng các thành phần đã được biên dịch và liên kết vào chúng. Trong trường hợp thứ hai, chúng có thể dùng chung các thành phần triển khai liên kết động.

Hãy xem tiến trình cục bộ như một siêu thành phần: nó gồm các thành phần cấp thấp quản lý phụ thuộc của mình bằng đa hình động.

Chiến lược tách biệt giữa các tiến trình cục bộ cũng giống như với khối nguyên khối và các thành phần nhị phân. Phụ thuộc mã nguồn qua ranh giới vẫn cùng hướng, luôn hướng về thành phần cấp cao hơn.

Với tiến trình cục bộ, điều đó có nghĩa mã nguồn của tiến trình cấp cao hơn không được chứa tên, địa chỉ vật lý hoặc khóa tra cứu trong registry của tiến trình cấp thấp hơn. Hãy nhớ rằng mục tiêu kiến trúc là biến các tiến trình cấp thấp thành plugin của tiến trình cấp cao.

Giao tiếp qua ranh giới giữa các tiến trình cục bộ đòi hỏi lời gọi hệ điều hành, tổ chức truyền và giải mã dữ liệu, cùng các lần chuyển ngữ cảnh giữa tiến trình. Những thao tác này tốn kém ở mức vừa phải. Vì vậy, cần hạn chế việc giao tiếp quá thường xuyên.

Dịch vụ

Ranh giới mạnh nhất là dịch vụ. Dịch vụ là một tiến trình, thường được khởi chạy từ dòng lệnh hoặc thông qua một lệnh gọi hệ thống tương đương. Dịch vụ không phụ thuộc vào vị trí vật lý của nó. Hai dịch vụ giao tiếp với nhau có thể chạy trên cùng một bộ xử lý hoặc nhóm bộ xử lý đa lõi, nhưng cũng có thể không. Các dịch vụ giả định rằng mọi giao tiếp đều diễn ra qua mạng.

Giao tiếp qua ranh giới dịch vụ chậm hơn rất nhiều so với lời gọi hàm. Thời gian phản hồi có thể từ vài chục mili giây đến vài giây. Cần tránh giao tiếp quá thường xuyên khi có thể. Giao tiếp ở cấp này phải tính đến độ trễ cao.

Ngoài ra, các quy tắc áp dụng cho dịch vụ cũng giống như với tiến trình cục bộ. Dịch vụ cấp thấp nên “cắm vào” dịch vụ cấp cao. Mã nguồn của dịch vụ cấp cao không được chứa bất kỳ hiểu biết cụ thể nào về mặt vật lý, chẳng hạn URI, của dịch vụ cấp thấp.

Kết luận

Hầu hết hệ thống, ngoại trừ các khối nguyên khối, sử dụng nhiều hơn một chiến lược ranh giới. Hệ thống dùng ranh giới dịch vụ cũng có thể có một số ranh giới tiến trình cục bộ. Thực tế, một dịch vụ thường chỉ là mặt tiền cho một tập hợp các tiến trình cục bộ tương tác với nhau. Dịch vụ hoặc tiến trình cục bộ gần như chắc chắn là một khối nguyên khối gồm các thành phần mã nguồn, hoặc một tập hợp các thành phần triển khai liên kết động.

Điều này có nghĩa ranh giới trong một hệ thống thường là sự pha trộn giữa các ranh giới cục bộ cho phép giao tiếp thường xuyên và các ranh giới chú trọng hơn đến độ trễ.

19

Chính sách và cấp độ

Hệ thống phần mềm là những tuyên bố về chính sách. Thực ra, cốt lõi của chương trình máy tính chỉ có vậy. Một chương trình máy tính mô tả chi tiết chính sách dùng để chuyển đổi đầu vào thành đầu ra.

Trong hầu hết các hệ thống không tầm thường, chính sách có thể được chia thành nhiều chính sách nhỏ hơn. Một số mô tả cách tính các quy tắc nghiệp vụ cụ thể. Số khác mô tả cách định dạng báo cáo. Những chính sách khác nữa quy định cách xác thực dữ liệu đầu vào.

Một phần nghệ thuật xây dựng kiến trúc phần mềm là tách các chính sách đó một cách cẩn thận, rồi nhóm lại theo cách chúng thay đổi. Những chính sách thay đổi vì cùng lý do và cùng thời điểm thuộc cùng cấp độ, nên được đặt chung trong một thành phần. Những chính sách thay đổi vì lý do khác nhau hoặc vào thời điểm khác nhau thuộc các cấp độ khác nhau, nên cần được tách thành các thành phần riêng.

Nghệ thuật kiến trúc thường bao gồm việc tổ chức lại các thành phần thành một đồ thị có hướng không chu trình. Các nút của đồ thị là những thành phần chứa các chính sách cùng cấp độ. Các cạnh có hướng biểu thị quan hệ phụ thuộc giữa những thành phần đó, nối các thành phần ở những cấp độ khác nhau.

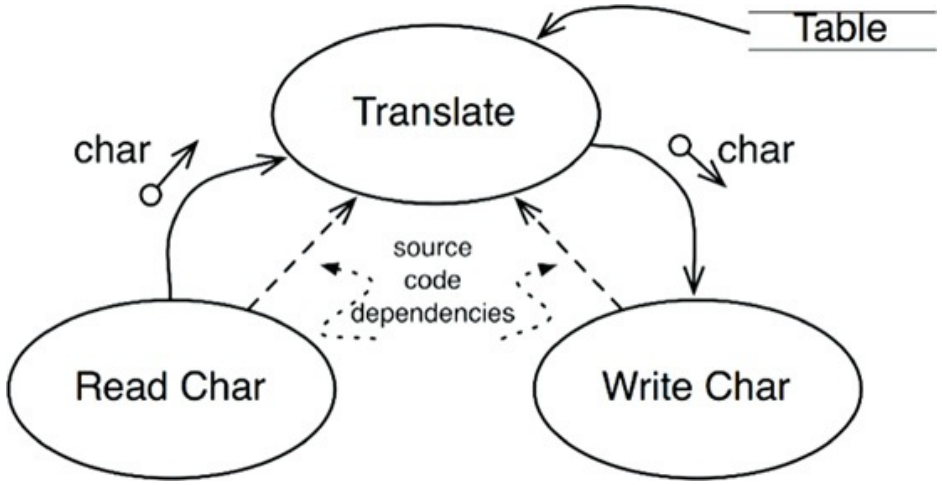
Đó là các phụ thuộc mã nguồn, tức phụ thuộc lúc biên dịch. Trong Java, chúng là các câu lệnh `import`. Trong C#, chúng là các câu lệnh `using`. Trong Ruby, chúng là các câu lệnh `require`. Đây là những phụ thuộc cần thiết để trình biên dịch hoạt động.

Trong một kiến trúc tốt, hướng của các phụ thuộc dựa trên cấp độ của những thành phần mà chúng nối với nhau. Trong mọi trường hợp, thành phần cấp thấp được thiết kế để phụ thuộc vào thành phần cấp cao.

Cấp độ

Định nghĩa chặt chẽ của "cấp độ" là "khoảng cách đến đầu vào và đầu ra". Một chính sách càng cách xa cả đầu vào lẫn đầu ra của hệ thống thì cấp độ của nó càng cao. Các chính sách quản lý đầu vào và đầu ra là những chính sách cấp thấp nhất trong hệ thống.

Sơ đồ luồng dữ liệu ở Hình 19.1 mô tả một chương trình mã hóa đơn giản: chương trình đọc ký tự từ thiết bị đầu vào, dùng một bảng để chuyển đổi ký tự, rồi ghi ký tự đã chuyển đổi ra thiết bị đầu ra. Luồng dữ liệu được biểu diễn bằng các mũi tên liền cong. Các phụ thuộc mã nguồn được thiết kế đúng được biểu diễn bằng những đường nét đứt thẳng.



Hình 19.1
Một chương trình mã hóa đơn giản

Thành phần `Translate` là thành phần cấp cao nhất trong hệ thống này vì nó nằm xa đầu vào và đầu ra nhất.[1]

Lưu ý rằng luồng dữ liệu và phụ thuộc mã nguồn không phải lúc nào cũng cùng hướng. Đây cũng là một phần của nghệ thuật kiến trúc phần mềm. Chúng ta muốn tách phụ thuộc mã nguồn khỏi luồng dữ liệu và gắn chúng với cấp độ.

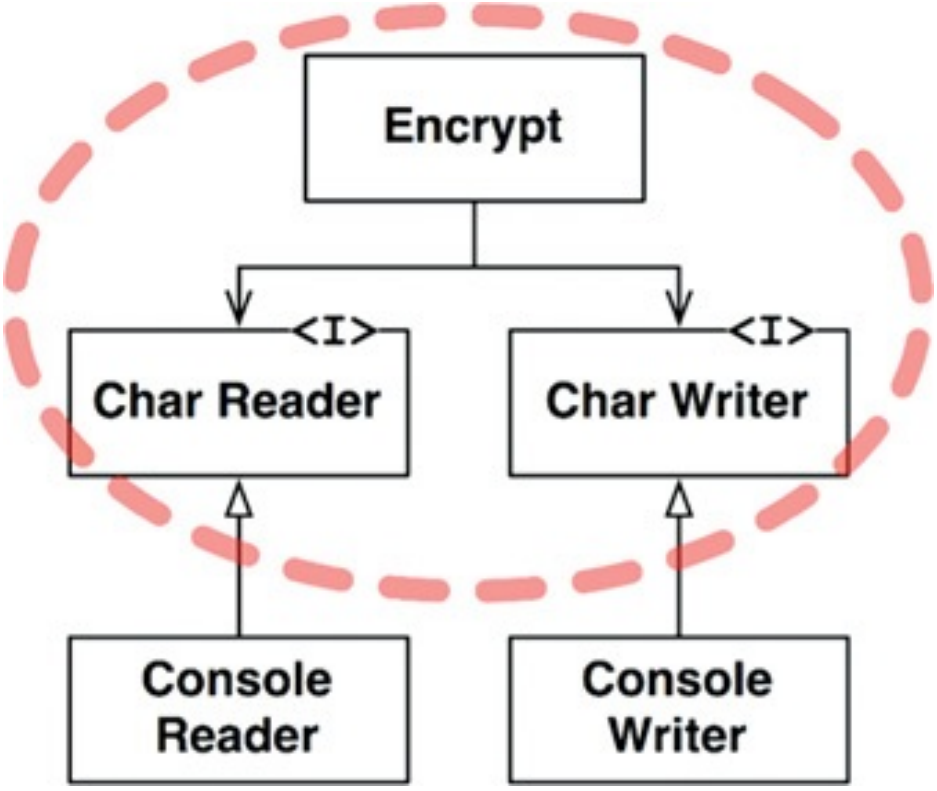
Sẽ rất dễ tạo ra một kiến trúc sai nếu viết chương trình mã hóa như sau:

```
function encrypt() {
    while (true)
        writeChar(translate(readChar()));
}
```

Đây là kiến trúc sai vì hàm cấp cao `encrypt` phụ thuộc vào các hàm cấp thấp `readChar` và `writeChar`.

1. Meilir Page-Jones gọi thành phần này là "Central Transform" trong cuốn *The Practical Guide to Structured Systems Design*, ấn bản thứ hai (Yourdon Press, 1988).

Một kiến trúc tốt hơn cho hệ thống này được thể hiện trong sơ đồ lớp ở Hình 19.2. Hãy chú ý đường viền nét đứt bao quanh lớp `Encrypt` và các giao diện `CharWriter` và `CharReader`. Mọi phụ thuộc đi qua đường viền đều hướng vào trong. Đây là đơn vị cấp cao nhất của hệ thống.



Hình 19.2
Sơ đồ lớp thể hiện kiến trúc tốt hơn cho hệ thống

Ở đây, `ConsoleReader` và `ConsoleWriter` được thể hiện dưới dạng các lớp. Chúng ở cấp thấp vì nằm gần đầu vào và đầu ra.

Cấu trúc này tách chính sách mã hóa cấp cao khỏi các chính sách đầu vào/đầu ra cấp thấp. Nhờ đó, chính sách mã hóa có thể được dùng trong nhiều bối cảnh khác nhau. Khi các chính sách đầu vào và đầu ra thay đổi, chính sách mã hóa ít có khả năng bị ảnh hưởng.

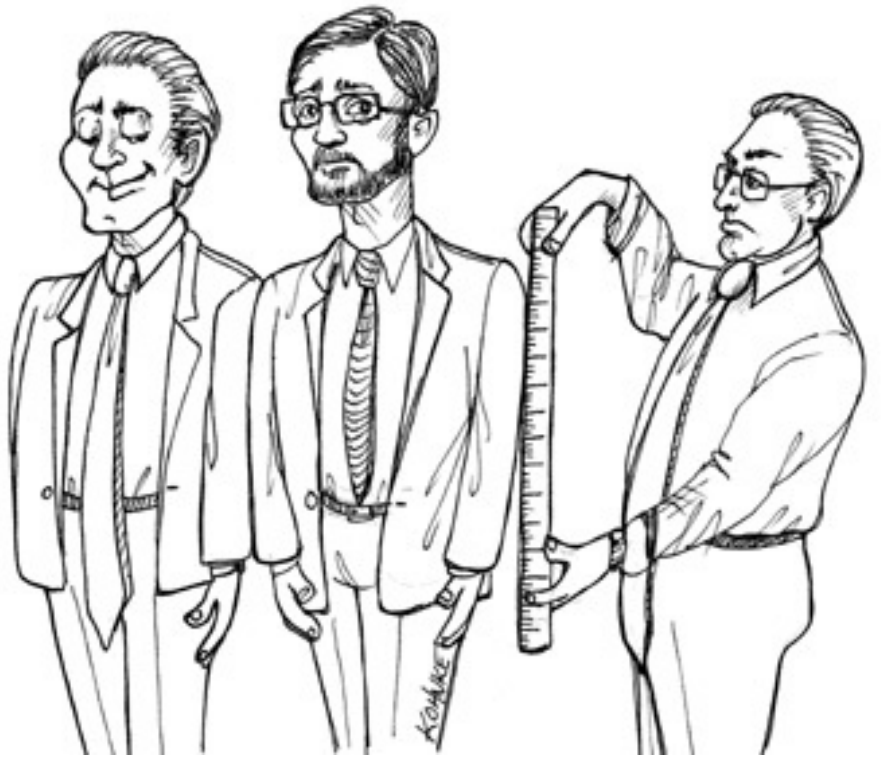
Hãy nhớ rằng các chính sách được nhóm thành thành phần dựa trên cách chúng thay đổi. SRP và CCP nhóm chung những chính sách thay đổi vì cùng lý do và cùng thời điểm. Các chính sách cấp cao, tức những chính sách xa đầu vào và đầu ra nhất, thường thay đổi ít thường xuyên hơn và vì những lý do quan trọng hơn so với chính sách cấp thấp. Các chính sách cấp thấp, tức những chính sách gần đầu vào và đầu ra nhất, thường thay đổi thường

xuyên và cấp bách hơn, nhưng vì những lý do ít quan trọng hơn.

Chẳng hạn, ngay cả trong ví dụ đơn giản về chương trình mã hóa, thiết bị IO có khả năng thay đổi cao hơn nhiều so với thuật toán mã hóa. Nếu thuật toán mã hóa thay đổi, nguyên nhân thường sẽ quan trọng hơn việc thay đổi một thiết bị IO.

Giữ các chính sách này tách biệt, với mọi phụ thuộc mã nguồn hướng về các chính sách cấp cao hơn, sẽ giảm ảnh hưởng của thay đổi. Những thay đổi nhỏ nhưng cấp bách ở các cấp thấp nhất của hệ thống ít hoặc không ảnh hưởng đến các cấp cao hơn, quan trọng hơn.

Một cách khác để nhìn nhận vấn đề này là xem các thành phần cấp thấp như plugin của thành phần cấp cao. Sơ đồ thành phần ở Hình 19.3 thể hiện cách bố trí đó. Thành phần `Encryption` không biết gì về thành phần `IODevices`; thành phần `IODevices` phụ thuộc vào `Encryption`.



Hình 19.3

Các thành phần cấp thấp nên cắm vào thành phần cấp cao

Kết luận

Cho đến đây, phần thảo luận về chính sách đã kết hợp nhiều nguyên tắc: Nguyên tắc Trách nhiệm Đơn nhất, Nguyên tắc Đóng-Mở, Nguyên tắc Đóng chung, Nguyên tắc Đảo ngược Phụ thuộc, Nguyên tắc Phụ thuộc Ổn định và Nguyên tắc Trừu tượng Ổn định. Hãy nhìn lại và thử xác định mỗi nguyên tắc được dùng ở đâu và vì sao.

20

Quy tắc nghiệp vụ



Nếu muốn chia ứng dụng thành quy tắc nghiệp vụ và plugin, trước hết chúng ta cần hiểu rõ quy tắc nghiệp vụ là gì. Hóa ra có nhiều loại quy tắc khác nhau.

Nói chính xác, quy tắc nghiệp vụ là những quy tắc hoặc thủ tục tạo ra hay tiết kiệm tiền cho doanh nghiệp. Nói chặt chẽ hơn, các quy tắc ấy vẫn tạo ra hoặc tiết kiệm tiền cho doanh nghiệp bất kể chúng có được thực thi bằng máy tính hay không. Ngay cả khi được thực hiện thủ công, chúng vẫn tạo ra hoặc tiết kiệm tiền.

Việc ngân hàng tính lãi suất N% cho một khoản vay là quy tắc nghiệp vụ giúp ngân hàng kiếm tiền. Lãi suất do chương trình máy tính tính hay do một nhân viên dùng bàn tính tính đều không quan trọng.

Chúng ta gọi những quy tắc này là **Quy tắc Nghiệp vụ Cốt lõi**, vì chúng thiết yếu đối với chính doanh nghiệp và vẫn tồn tại ngay cả khi không có hệ thống tự động hóa chúng.

Quy tắc nghiệp vụ cốt lõi thường cần dữ liệu để xử lý. Chẳng hạn, khoản vay cần số dư nợ, lãi suất và lịch thanh toán.

Chúng ta gọi dữ liệu này là **Dữ liệu Nghiệp vụ Cốt lõi**. Đây là dữ liệu vẫn tồn tại ngay cả khi hệ thống không được tự động hóa.

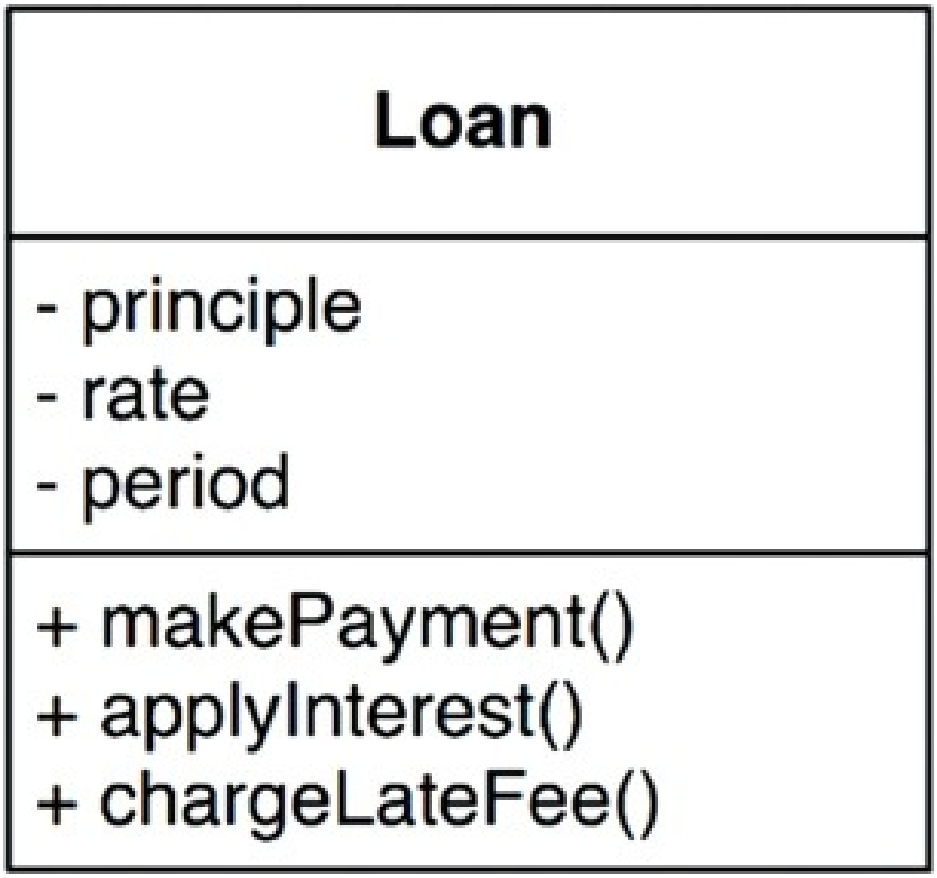
Quy tắc cốt lõi và dữ liệu cốt lõi gắn bó không thể tách rời, nên thích hợp để kết hợp thành một đối tượng. Chúng ta gọi loại đối tượng này là một **Entity**.[1]

1. Đây là tên gọi Ivar Jacobson dùng cho khái niệm này (I. Jacobson và cộng sự, *Object Oriented Software Engineering*, Addison-Wesley, 1992).

Entity

Entity là một đối tượng trong hệ thống máy tính, chứa một tập nhỏ các quy tắc nghiệp vụ cốt lõi hoạt động trên dữ liệu nghiệp vụ cốt lõi. Đối tượng Entity tự chứa dữ liệu nghiệp vụ cốt lõi hoặc có thể truy cập dữ liệu đó dễ dàng. Giao diện của Entity gồm các hàm triển khai những quy tắc nghiệp vụ cốt lõi vận hành trên dữ liệu ấy.

Ví dụ, Hình 20.1 cho thấy Entity `Loan` có thể được biểu diễn như một lớp UML. Lớp này có ba mục dữ liệu nghiệp vụ cốt lõi và cung cấp ba quy tắc nghiệp vụ cốt lõi qua giao diện của nó.



Hình 20.1
Entity khoản vay dưới dạng lớp UML

Khi tạo loại lớp này, chúng ta gom phần mềm triển khai một khái niệm thiết yếu đối với doanh nghiệp lại với nhau, tách nó khỏi mọi mối quan tâm khác trong hệ thống tự động đang xây dựng. Lớp này đứng độc lập, đại diện cho doanh nghiệp. Nó không bị vướng bận

bởi cơ sở dữ liệu, giao diện người dùng hay framework của bên thứ ba. Nó có thể phục vụ doanh nghiệp trong bất kỳ hệ thống nào, bất kể hệ thống được trình bày ra sao, lưu dữ liệu ở đâu hay bố trí máy tính thế nào. Entity chỉ chứa nghiệp vụ, không có gì khác.

Một số bạn có thể băn khoăn vì tôi gọi nó là một lớp. Đừng lo. Không nhất thiết phải dùng ngôn ngữ hướng đối tượng để tạo Entity. Chỉ cần gắn dữ liệu nghiệp vụ cốt lõi với các quy tắc nghiệp vụ cốt lõi trong một mô-đun phần mềm riêng biệt.

Ca sử dụng

Không phải quy tắc nghiệp vụ nào cũng thuần túy như trong Entity. Một số quy tắc tạo ra hoặc tiết kiệm tiền cho doanh nghiệp bằng cách xác định và giới hạn cách hệ thống tự động vận hành. Chúng không được dùng trong môi trường thủ công vì chỉ có ý nghĩa khi nằm trong một hệ thống tự động.

Chẳng hạn, hãy tưởng tượng một ứng dụng được nhân viên ngân hàng dùng để tạo khoản vay mới. Ngân hàng có thể quyết định rằng nhân viên cho vay không được đưa ra ước tính khoản thanh toán cho đến khi đã thu thập và xác thực thông tin liên hệ, đồng thời xác nhận điểm tín dụng của khách hàng tiềm năng từ 500 trở lên. Vì vậy, ngân hàng có thể quy định hệ thống không được chuyển sang màn hình ước tính khoản thanh toán cho đến khi thông tin liên hệ đã được điền và xác minh, đồng thời điểm tín dụng được xác nhận là cao hơn ngưỡng quy định.

Đây là một ca sử dụng.[2] Ca sử dụng mô tả cách một hệ thống tự động được sử dụng. Nó quy định dữ liệu người dùng cần cung cấp, dữ liệu hệ thống trả lại và các bước xử lý để tạo ra kết quả đó. Ca sử dụng mô tả những quy tắc nghiệp vụ riêng của ứng dụng, khác với các quy tắc nghiệp vụ cốt lõi nằm trong Entity.

Hình 20.2 là một ví dụ về ca sử dụng. Lưu ý dòng cuối có nhắc đến `Customer`. Đây là tham chiếu đến Entity `Customer`, nơi chứa các quy tắc nghiệp vụ cốt lõi chi phối mối quan hệ giữa ngân hàng và khách hàng.

---

2. Như trên.



Hình 20.2
Ví dụ về ca sử dụng

Ca sử dụng chứa các quy tắc xác định cách thức và thời điểm gọi các quy tắc nghiệp vụ cốt lõi trong Entity. Ca sử dụng điều khiển nhịp phối hợp của các Entity.

Cũng cần lưu ý rằng ca sử dụng không mô tả giao diện người dùng, ngoài việc nêu không chính thức dữ liệu đi vào từ giao diện và dữ liệu được gửi ngược ra qua giao diện đó. Chỉ dựa vào ca sử dụng, không thể biết ứng dụng được cung cấp qua web, qua ứng dụng máy tính cài đặt cục bộ, qua console hay dưới dạng dịch vụ thuần túy.

Điều này rất quan trọng. Ca sử dụng không mô tả hệ thống xuất hiện như thế nào với người dùng. Thay vào đó, nó mô tả các quy tắc nghiệp vụ riêng của ứng dụng, chi phối tương tác giữa người dùng và Entity. Cách dữ liệu đi vào và ra khỏi hệ thống không liên quan đến ca sử dụng.

Ca sử dụng là một đối tượng. Nó có một hoặc nhiều hàm triển khai các quy tắc nghiệp vụ riêng của ứng dụng. Nó cũng chứa các phần tử dữ liệu gồm dữ liệu đầu vào, dữ liệu đầu ra và tham chiếu đến những Entity thích hợp mà nó tương tác.

Entity không biết về các ca sử dụng điều khiển chúng. Đây là một ví dụ khác cho thấy hướng phụ thuộc tuân theo Nguyên tắc Đảo ngược Phụ thuộc. Các khái niệm cấp cao như Entity không biết gì về khái niệm cấp thấp hơn như ca sử dụng. Ngược lại, ca sử dụng cấp thấp phụ thuộc vào Entity cấp cao.

Vì sao Entity ở cấp cao còn ca sử dụng ở cấp thấp? Vì ca sử dụng dành riêng cho một

ứng dụng, nên gần với đầu vào và đầu ra của hệ thống đó hơn. Entity là khái quát hóa có thể được dùng trong nhiều ứng dụng, nên nằm xa đầu vào và đầu ra hơn. Ca sử dụng phụ thuộc vào Entity; Entity không phụ thuộc vào ca sử dụng.

Mô hình yêu cầu và phản hồi

Ca sử dụng nhận dữ liệu đầu vào và tạo dữ liệu đầu ra. Tuy nhiên, một đối tượng ca sử dụng được thiết kế tốt không nên biết dữ liệu được truyền đến người dùng hay thành phần khác bằng cách nào. Chúng ta chắc chắn không muốn mã trong lớp ca sử dụng biết về HTML hay SQL!

Lớp ca sử dụng nhận các cấu trúc dữ liệu yêu cầu đơn giản làm đầu vào và trả về các cấu trúc dữ liệu phản hồi đơn giản làm đầu ra. Những cấu trúc dữ liệu này không phụ thuộc vào bất kỳ thứ gì. Chúng không kế thừa từ các giao diện chuẩn của framework như `HttpRequest` và `HttpResponse`. Chúng không biết gì về web và cũng không mang theo những đặc điểm của giao diện người dùng đang được sử dụng.

Việc không có phụ thuộc này rất quan trọng. Nếu mô hình yêu cầu và phản hồi không độc lập, các ca sử dụng phụ thuộc vào chúng sẽ bị ràng buộc gián tiếp với mọi phụ thuộc mà mô hình mang theo.

Bạn có thể muốn đặt tham chiếu đến các đối tượng Entity trong những cấu trúc dữ liệu này, vì Entity và mô hình yêu cầu/phản hồi có nhiều dữ liệu chung. Đừng làm vậy! Mục đích của hai loại đối tượng này rất khác nhau. Theo thời gian, chúng sẽ thay đổi vì những lý do rất khác nhau. Gắn chúng với nhau dưới bất kỳ hình thức nào đều vi phạm Nguyên tắc Đóng chung và Nguyên tắc Trách nhiệm Đơn nhất. Kết quả là mã nguồn phải chuyển tiếp nhiều dữ liệu thừa và chứa thêm nhiều câu lệnh điều kiện.

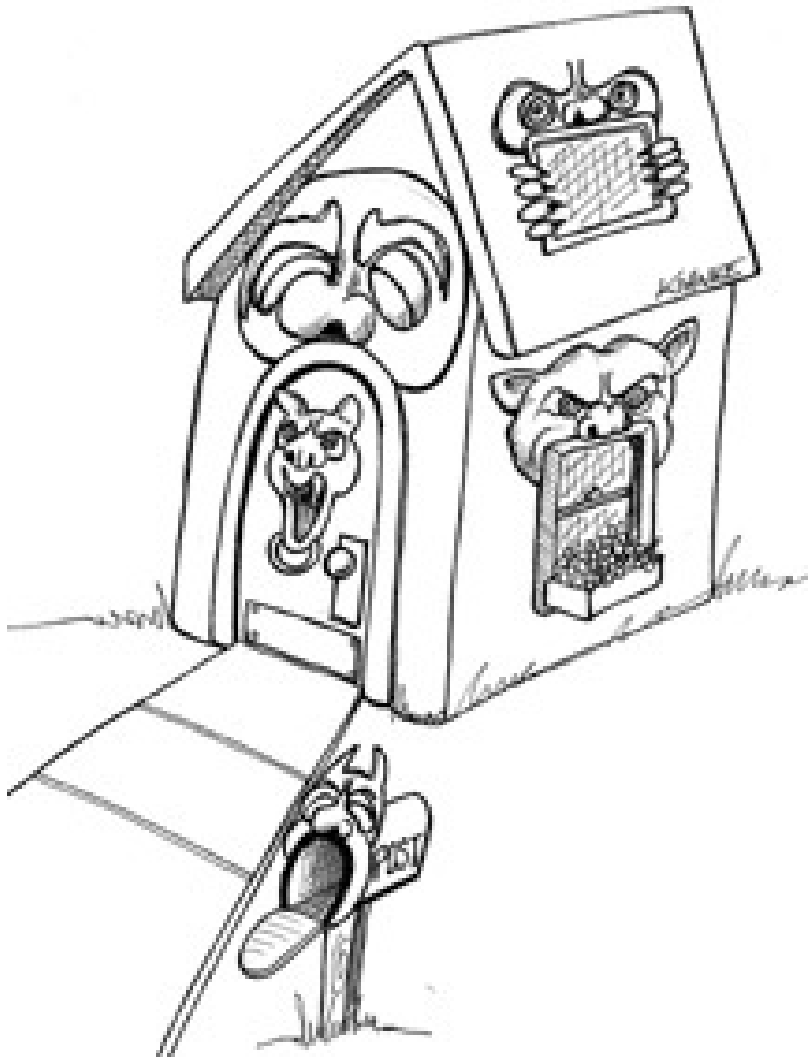
Kết luận

Quy tắc nghiệp vụ là lý do một hệ thống phần mềm tồn tại. Chúng là chức năng cốt lõi, chứa mã tạo ra hoặc tiết kiệm tiền. Chúng là những báu vật quý giá nhất của doanh nghiệp.

Quy tắc nghiệp vụ cần được giữ nguyên vẹn, không bị vấy bẩn bởi những mối quan tâm thứ yếu như giao diện người dùng hay cơ sở dữ liệu được sử dụng. Lý tưởng nhất, mã thể hiện quy tắc nghiệp vụ phải là trái tim của hệ thống, còn các mối quan tâm ít quan trọng hơn được cắm vào nó. Quy tắc nghiệp vụ nên là phần mã độc lập và có thể tái sử dụng nhiều nhất trong hệ thống.

21

Kiến trúc lên tiếng



Hãy tưởng tượng bạn đang xem bản vẽ một tòa nhà. Tài liệu do kiến trúc sư chuẩn bị này trình bày kế hoạch xây dựng. Những bản vẽ đó cho bạn biết điều gì?

Nếu đó là bản vẽ một ngôi nhà dành cho một gia đình, bạn sẽ thấy lối vào chính, tiền sảnh dẫn đến phòng khách và có thể cả phòng ăn. Nhà bếp có lẽ ở gần đó, cách phòng ăn không xa. Bên cạnh nhà bếp có thể là một góc ăn uống nhỏ, và phòng sinh hoạt chung có lẽ cũng ở gần. Nhìn vào bản vẽ, bạn sẽ không thể nhầm đó là một ngôi nhà dành cho một

gia đình. Kiến trúc của nó sẽ cất tiếng: "NHÀ".

Còn nếu bạn đang xem bản vẽ thư viện thì sao? Có lẽ bạn sẽ thấy lối vào lớn, khu vực dành cho nhân viên làm thủ tục mượn và trả sách, các khu đọc sách, những phòng họp nhỏ, cùng nhiều dãy không gian đặt giá sách chứa toàn bộ sách trong thư viện. Kiến trúc ấy sẽ cất tiếng: "THƯ VIỆN".

Vậy kiến trúc ứng dụng của bạn thể hiện điều gì? Khi nhìn vào cấu trúc thư mục cấp cao nhất và các tệp mã nguồn trong gói cấp cao nhất, bạn có nhận ra "Hệ thống Chăm sóc Sức khỏe", "Hệ thống Kế toán" hay "Hệ thống Quản lý Hàng tồn kho" không? Hay bạn chỉ thấy "Rails", "Spring/Hibernate" hoặc "ASP"?

Chủ đề của kiến trúc

Hãy đọc lại công trình có ảnh hưởng lớn của Ivar Jacobson về kiến trúc phần mềm, *Object Oriented Software Engineering*. Hãy chú ý phụ đề: *A Use Case Driven Approach*. Trong sách này, Jacobson cho rằng kiến trúc phần mềm là cấu trúc hỗ trợ các ca sử dụng của hệ thống. Cũng như bản vẽ ngôi nhà hay thư viện thể hiện rõ các ca sử dụng của công trình, kiến trúc ứng dụng phần mềm cũng nên thể hiện rõ các ca sử dụng của ứng dụng.

Kiến trúc không phải, và cũng không nên là, framework. Framework không nên quyết định cấu trúc kiến trúc. Framework là công cụ để sử dụng, không phải khuôn mẫu mà kiến trúc phải tuân theo. Nếu kiến trúc của bạn dựa trên framework, thì nó không thể đồng thời dựa trên các ca sử dụng.

Mục đích của kiến trúc

Kiến trúc tốt lấy các ca sử dụng làm trung tâm, để kiến trúc sư có thể mô tả an toàn cấu trúc hỗ trợ chúng mà không phải cam kết với framework, công cụ hay môi trường cụ thể. Hãy nghĩ lại về bản vẽ ngôi nhà. Mối quan tâm đầu tiên của kiến trúc sư là bảo đảm ngôi nhà có thể sử dụng được, chứ không phải bảo đảm nó được xây bằng gạch. Thực tế, kiến trúc sư còn cố gắng để chủ nhà có thể quyết định vật liệu ngoại thất, như gạch, đá hay gỗ tuyết tùng, vào giai đoạn sau, khi bản vẽ đã bảo đảm đáp ứng các ca sử dụng.

Kiến trúc phần mềm tốt cho phép trì hoãn các quyết định về framework, cơ sở dữ liệu, máy chủ web và những vấn đề hay công cụ thuộc môi trường khác. Framework là những lựa chọn nên được để mở. Kiến trúc tốt giúp bạn không cần quyết định dùng Rails, Spring, Hibernate, Tomcat hay MySQL cho đến giai đoạn muộn hơn của dự án. Kiến trúc tốt cũng giúp bạn dễ dàng đổi ý về những lựa chọn ấy. Nó nhấn mạnh các ca sử dụng và tách chúng khỏi những mối quan tâm ngoại vi.

Thế còn web thì sao?

Web có phải là kiến trúc không? Việc hệ thống được cung cấp qua web có quyết định kiến trúc hệ thống không? Tất nhiên là không. Web là một cơ chế phân phối, một thiết bị IO, và kiến trúc ứng dụng nên xem nó như vậy. Việc ứng dụng được cung cấp qua web chỉ là chi tiết, không nên chi phối cấu trúc hệ thống. Thực ra, quyết định cung cấp ứng dụng qua web là quyết định nên được trì hoãn. Kiến trúc hệ thống nên càng ít biết về cách phân phối càng tốt. Bạn nên có thể cung cấp hệ thống dưới dạng ứng dụng console, ứng dụng web, ứng dụng máy tính cài đặt cục bộ hoặc dịch vụ web, mà không cần thay đổi hay làm phức tạp quá mức kiến trúc nền tảng.

Framework là công cụ, không phải lối sống

Framework có thể rất mạnh mẽ và hữu ích. Tác giả framework thường tin tưởng sâu sắc vào framework của mình. Những ví dụ họ viết về cách sử dụng framework thường được trình bày từ góc nhìn của một người hết lòng tin theo. Các tác giả khác viết về framework cũng thường là môn đồ của niềm tin ấy. Họ chỉ cho bạn cách dùng framework. Họ thường giả định rằng framework phải bao trùm mọi thứ và làm tất cả.

Đó không phải là cách tiếp cận bạn nên chọn.

Hãy xem xét từng framework với thái độ tỉnh táo và hoài nghi. Có thể nó hữu ích, nhưng cái giá phải trả là gì? Hãy tự hỏi nên dùng nó ra sao và làm thế nào để tự bảo vệ khỏi nó. Hãy nghĩ cách giữ cho kiến trúc tiếp tục đặt các ca sử dụng làm trọng tâm. Xây dựng chiến lược ngăn framework chiếm quyền kiểm soát kiến trúc.

Kiến trúc có thể kiểm thử

Nếu kiến trúc hệ thống tập trung vào các ca sử dụng và bạn giữ framework ở khoảng cách an toàn, bạn sẽ có thể kiểm thử đơn vị tất cả ca sử dụng mà không cần cài đặt bất kỳ framework nào. Bạn không cần máy chủ web đang chạy để thực hiện kiểm thử. Bạn không cần kết nối cơ sở dữ liệu. Các đối tượng Entity nên là những đối tượng thông thường, không phụ thuộc vào framework, cơ sở dữ liệu hay những thứ phức tạp khác. Các đối tượng ca sử dụng nên điều phối các Entity. Cuối cùng, tất cả những thành phần đó phải có thể được kiểm thử tại chỗ, không vướng các phức tạp do framework gây ra.

Kết luận

Kiến trúc của bạn nên cho người đọc biết về hệ thống, không phải về framework bạn đã dùng. Nếu bạn đang xây dựng hệ thống chăm sóc sức khỏe, thì khi lập trình viên mới xem kho mã nguồn, ấn tượng đầu tiên của họ nên là: "Ồ, đây là hệ thống chăm sóc sức khỏe". Họ phải có thể tìm hiểu mọi ca sử dụng của hệ thống mà vẫn chưa cần biết hệ thống được phân phối bằng cách nào. Có thể họ sẽ hỏi:

> *"Chúng tôi thấy vài thứ trông giống như model, nhưng view và controller ở đâu?"*

Và bạn nên trả lời:

> *"Ồ, đó là những chi tiết hiện chưa cần quan tâm. Chúng ta sẽ quyết định sau."*

22

Kiến trúc sạch



Trong vài thập kỷ qua, chúng ta đã thấy rất nhiều ý tưởng về kiến trúc hệ thống. Trong số đó có:

- Kiến trúc Lục giác, còn gọi là Cổng và Bộ chuyển đổi (Ports and Adapters), do Alistair Cockburn phát triển và được Steve Freeman cùng Nat Pryce áp dụng trong cuốn sách tuyệt vời *Growing Object Oriented Software with Tests*.
- DCI của James Coplien và Trygve Reenskaug.
- BCE, do Ivar Jacobson giới thiệu trong cuốn *Object Oriented Software Engineering: A Use-Case Driven Approach*.

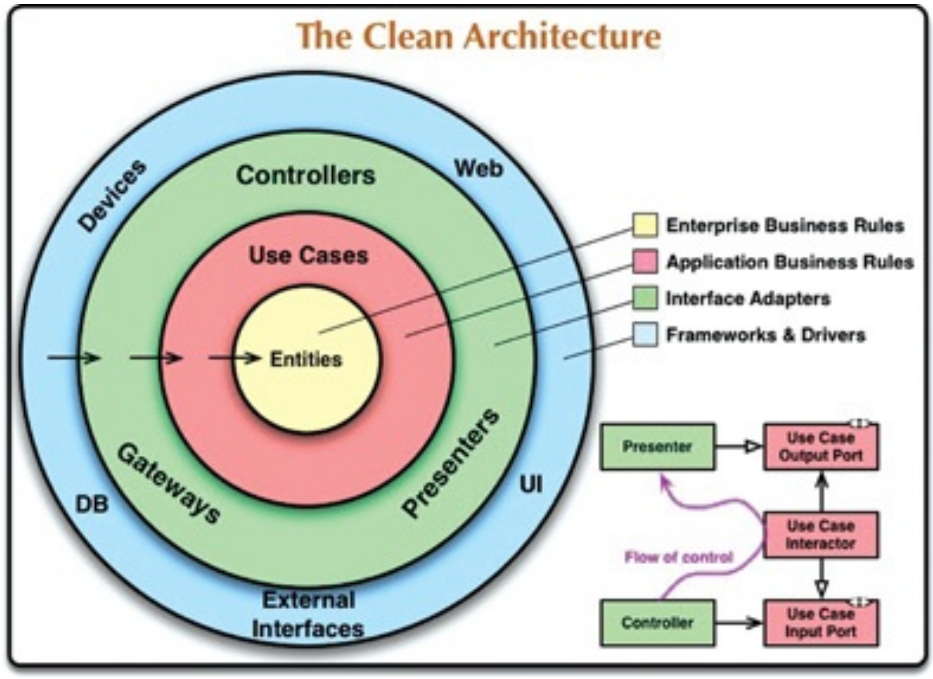
Dù các kiến trúc này khác nhau đôi chút về chi tiết, chúng rất giống nhau. Tất cả đều hướng

đến cùng một mục tiêu: phân tách các mối quan tâm. Chúng đều đạt được sự phân tách đó bằng cách chia phần mềm thành các lớp. Mỗi kiến trúc có ít nhất một lớp dành cho quy tắc nghiệp vụ và một lớp khác dành cho giao diện người dùng và hệ thống.

Mỗi kiến trúc này tạo ra các hệ thống có những đặc điểm sau:

- **Độc lập với framework.** Kiến trúc không phụ thuộc vào sự tồn tại của một thư viện phần mềm nhiều tính năng. Nhờ vậy, bạn có thể dùng framework như công cụ, thay vì phải nhồi hệ thống vào những giới hạn chật hẹp của framework.
- **Có thể kiểm thử.** Có thể kiểm thử các quy tắc nghiệp vụ mà không cần giao diện người dùng, cơ sở dữ liệu, máy chủ web hay bất kỳ yếu tố bên ngoài nào khác.
- **Độc lập với giao diện người dùng.** Có thể dễ dàng thay đổi giao diện mà không làm thay đổi phần còn lại của hệ thống. Chẳng hạn, có thể thay giao diện web bằng giao diện console mà không thay đổi quy tắc nghiệp vụ.
- **Độc lập với cơ sở dữ liệu.** Bạn có thể thay Oracle hoặc SQL Server bằng Mongo, BigTable, CouchDB hay một hệ thống khác. Quy tắc nghiệp vụ không bị ràng buộc với cơ sở dữ liệu.
- **Độc lập với mọi tác nhân bên ngoài.** Thực tế, quy tắc nghiệp vụ hoàn toàn không biết gì về các giao diện với thế giới bên ngoài.

Sơ đồ ở Hình 22.1 là nỗ lực kết hợp tất cả những kiến trúc này thành một ý tưởng có thể áp dụng.



Hình 22.1
Kiến trúc sạch

Quy tắc phụ thuộc

Các vòng tròn đồng tâm trong Hình 22.1 biểu thị những khu vực phần mềm khác nhau. Nhìn chung, càng đi vào trong, cấp độ phần mềm càng cao. Các vòng tròn bên ngoài là cơ chế. Các vòng tròn bên trong là chính sách.

Quy tắc chi phối giúp kiến trúc này hoạt động là Quy tắc Phụ thuộc:

> *Phụ thuộc mã nguồn chỉ được hướng vào trong, về phía các chính sách cấp cao hơn.*

Không có thành phần nào ở vòng tròn bên trong được biết bất kỳ điều gì về thành phần ở vòng tròn bên ngoài. Cụ thể, mã trong vòng tròn bên trong không được nhắc đến tên của bất kỳ thứ gì được khai báo ở vòng tròn bên ngoài. Điều đó bao gồm hàm, lớp, biến hay bất kỳ thực thể phần mềm nào có tên.

Tương tự, không nên dùng trong vòng tròn bên trong các định dạng dữ liệu được khai báo ở vòng tròn bên ngoài, nhất là khi những định dạng đó do một framework ở vòng ngoài tạo ra. Chúng ta không muốn bất kỳ thứ gì ở vòng ngoài ảnh hưởng đến các vòng tròn bên trong.

Entity

Entity đóng gói các quy tắc nghiệp vụ cốt lõi trên phạm vi toàn doanh nghiệp. Entity có thể là một đối tượng với các phương thức, hoặc một tập hợp cấu trúc dữ liệu và hàm. Điều quan trọng là Entity có thể được dùng trong nhiều ứng dụng khác nhau của doanh nghiệp.

Nếu bạn không làm việc trong một doanh nghiệp lớn mà chỉ viết một ứng dụng đơn lẻ, thì các Entity này là đối tượng nghiệp vụ của ứng dụng. Chúng đóng gói những quy tắc tổng quát và cấp cao nhất. Chúng ít có khả năng thay đổi nhất khi có thay đổi bên ngoài. Chẳng hạn, bạn không kỳ vọng các đối tượng này bị ảnh hưởng khi thay đổi điều hướng trang hoặc bảo mật. Không thay đổi vận hành nào của một ứng dụng cụ thể nên ảnh hưởng đến lớp Entity.

Ca sử dụng

Phần mềm ở lớp ca sử dụng chứa các quy tắc nghiệp vụ riêng của ứng dụng. Nó đóng gói và triển khai tất cả ca sử dụng của hệ thống. Các ca sử dụng điều phối luồng dữ liệu đi đến và đi khỏi Entity, đồng thời chỉ đạo Entity dùng các quy tắc nghiệp vụ cốt lõi để đạt mục tiêu của ca sử dụng.

Chúng ta không kỳ vọng thay đổi ở lớp này sẽ ảnh hưởng đến Entity. Cũng không kỳ vọng lớp này bị ảnh hưởng bởi những thay đổi bên ngoài, chẳng hạn cơ sở dữ liệu, giao diện người dùng hay các framework phổ biến. Lớp ca sử dụng được cách ly khỏi những mối quan tâm đó.

Tuy nhiên, chúng ta kỳ vọng thay đổi trong cách ứng dụng vận hành sẽ ảnh hưởng đến các ca sử dụng và do đó ảnh hưởng đến phần mềm ở lớp này. Nếu chi tiết của một ca sử dụng thay đổi, chắc chắn một phần mã trong lớp này cũng phải thay đổi.

Bộ chuyển đổi giao diện

Phần mềm ở lớp bộ chuyển đổi giao diện là tập hợp các bộ chuyển đổi, giúp chuyển dữ liệu từ định dạng thuận tiện nhất cho ca sử dụng và Entity sang định dạng thuận tiện nhất cho tác nhân bên ngoài, chẳng hạn cơ sở dữ liệu hoặc web. Chẳng hạn, lớp này chứa toàn bộ kiến trúc MVC của giao diện người dùng đồ họa. Presenter, view và controller đều thuộc lớp bộ chuyển đổi giao diện. Model thường chỉ là các cấu trúc dữ liệu được truyền từ controller đến ca sử dụng, rồi từ ca sử dụng trở lại presenter và view.

Tương tự, ở lớp này dữ liệu được chuyển từ dạng thuận tiện nhất cho Entity và ca sử dụng sang dạng thuận tiện nhất cho framework lưu trữ đang dùng, tức cơ sở dữ liệu. Không mã nào ở phía trong vòng tròn này được biết bất kỳ điều gì về cơ sở dữ liệu. Nếu cơ sở dữ liệu là SQL, thì toàn bộ câu lệnh SQL phải nằm trong lớp này, đặc biệt là những phần liên quan đến cơ sở dữ liệu.

Lớp này cũng chứa mọi bộ chuyển đổi cần thiết để chuyển dữ liệu từ dạng bên ngoài, chẳng hạn dịch vụ bên ngoài, sang dạng nội bộ mà ca sử dụng và Entity sử dụng.

Framework và trình điều khiển

Lớp ngoài cùng của mô hình trong Hình 22.1 thường gồm framework và công cụ như cơ sở dữ liệu và framework web. Thông thường, bạn không viết nhiều mã trong lớp này, ngoài mã kết nối để giao tiếp với vòng tròn kế tiếp ở phía trong.

Lớp framework và trình điều khiển chứa mọi chi tiết. Web là chi tiết. Cơ sở dữ liệu là chi tiết. Chúng ta giữ những thứ này ở bên ngoài để chúng ít gây hại nhất.

Chỉ có bốn vòng tròn?

Các vòng tròn trong Hình 22.1 chỉ mang tính sơ đồ. Bạn có thể cần nhiều hơn bốn vòng tròn. Không có quy tắc nào bắt buộc lúc nào cũng phải có đúng bốn vòng tròn. Tuy nhiên, Quy tắc Phụ thuộc luôn được áp dụng. Phụ thuộc mã nguồn luôn hướng vào trong. Càng đi vào trong, mức độ trừu tượng và chính sách càng tăng. Vòng tròn ngoài cùng gồm các chi tiết cụ thể cấp thấp. Khi đi vào trong, phần mềm trở nên trừu tượng hơn và đóng gói các chính sách cấp cao hơn. Vòng tròn trong cùng là cấp cao và tổng quát nhất.

Vượt qua ranh giới

Ở góc dưới bên phải của sơ đồ Hình 22.1 có ví dụ về cách vượt qua ranh giới giữa các vòng tròn. Sơ đồ cho thấy controller và presenter giao tiếp với ca sử dụng ở lớp kế tiếp. Hãy chú ý luồng điều khiển: nó bắt đầu từ controller, đi qua ca sử dụng rồi kết thúc bằng việc thực thi trong presenter. Cũng hãy chú ý các phụ thuộc mã nguồn, tất cả đều hướng vào trong, về phía các ca sử dụng.

Thông thường, chúng ta giải quyết mâu thuẫn có vẻ như tồn tại này bằng Nguyên tắc Đảo ngược Phụ thuộc. Chẳng hạn, trong Java, chúng ta sắp xếp các giao diện và quan hệ kế thừa sao cho tại những điểm thích hợp trên ranh giới, phụ thuộc mã nguồn đi ngược với luồng điều khiển.

Ví dụ, giả sử ca sử dụng cần gọi presenter. Không thể gọi trực tiếp vì như vậy sẽ vi phạm Quy tắc Phụ thuộc: vòng tròn bên trong không được nhắc đến tên nào thuộc vòng tròn bên ngoài. Vì thế, ca sử dụng gọi một giao diện ở vòng tròn trong, được ghi là “cổng đầu ra của ca sử dụng” trong Hình 22.1, còn presenter ở vòng tròn ngoài triển khai giao diện đó.

Kỹ thuật tương tự được dùng để vượt qua mọi ranh giới trong các kiến trúc này. Chúng ta tận dụng đa hình động để tạo các phụ thuộc mã nguồn đi ngược với luồng điều khiển, nhờ đó tuân theo Quy tắc Phụ thuộc bất kể luồng điều khiển đi theo hướng nào.

Dữ liệu nào đi qua ranh giới?

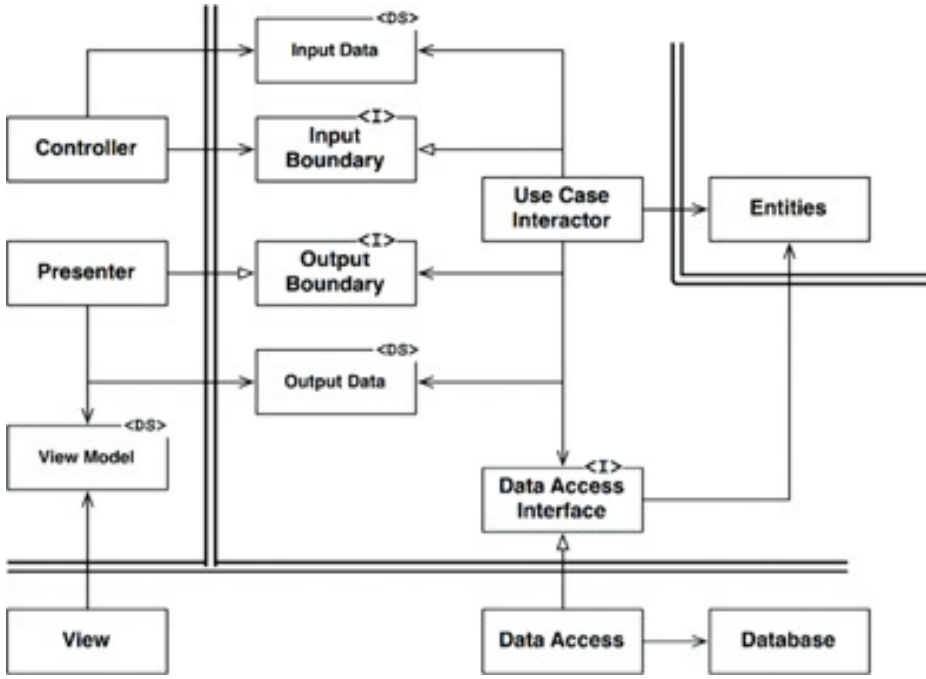
Thông thường, dữ liệu đi qua ranh giới là các cấu trúc dữ liệu đơn giản. Bạn có thể dùng struct cơ bản hoặc đối tượng truyền dữ liệu đơn giản. Dữ liệu cũng có thể chỉ là các đối số trong lời gọi hàm. Hoặc bạn có thể gói nó trong một hashmap hay tạo thành một đối tượng. Điều quan trọng là truyền qua ranh giới các cấu trúc dữ liệu đơn giản, độc lập. Chúng ta không được lách quy tắc bằng cách truyền đối tượng Entity hoặc hàng dữ liệu từ cơ sở dữ liệu. Các cấu trúc dữ liệu ấy không được có bất kỳ phụ thuộc nào vi phạm Quy tắc Phụ thuộc.

Chẳng hạn, nhiều framework cơ sở dữ liệu trả về một định dạng dữ liệu tiện dụng sau khi truy vấn. Ta có thể gọi đó là "cấu trúc hàng". Không nên truyền cấu trúc hàng ấy qua ranh giới vào phía trong. Làm vậy sẽ vi phạm Quy tắc Phụ thuộc vì buộc vòng tròn bên trong phải biết về thứ thuộc vòng tròn bên ngoài.

Vì vậy, khi truyền dữ liệu qua ranh giới, dữ liệu luôn ở dạng thuận tiện nhất cho vòng tròn bên trong.

Kịch bản điển hình

Sơ đồ ở Hình 22.2 cho thấy một kịch bản điển hình cho hệ thống Java chạy trên web và dùng cơ sở dữ liệu. Máy chủ web thu thập dữ liệu đầu vào từ người dùng rồi chuyển cho `Controller` ở góc trên bên trái. `Controller` đóng gói dữ liệu thành một đối tượng Java thông thường rồi truyền đối tượng này qua `InputBoundary` đến `UseCaseInteractor`. `UseCaseInteractor` diễn giải dữ liệu và dùng nó để điều phối các Entity. Nó cũng dùng `DataAccessInterface` để nạp từ cơ sở dữ liệu vào bộ nhớ dữ liệu mà các Entity cần. Khi hoàn tất, `UseCaseInteractor` thu thập dữ liệu từ các Entity và tạo `OutputData` dưới dạng một đối tượng Java thông thường khác. Sau đó, `OutputData` được truyền qua giao diện `OutputBoundary` đến `Presenter`.



Hình 22.2

Kịch bản điển hình cho hệ thống Java trên web sử dụng cơ sở dữ liệu

Nhiệm vụ của `Presenter` là đóng gói lại `OutputData` thành dạng có thể hiển thị, gọi là `ViewModel`, cũng là một đối tượng Java thông thường. `ViewModel` chủ yếu chứa chuỗi và cờ mà `View` dùng để hiển thị dữ liệu. Trong khi `OutputData` có thể chứa các đối tượng `Date`, `Presenter` sẽ điền vào `ViewModel` các chuỗi tương ứng đã được định dạng phù hợp cho người dùng. Điều tương tự cũng áp dụng cho đối tượng `Currency` và mọi dữ liệu nghiệp vụ khác. Tên nút bấm và mục menu được đặt trong `ViewModel`, cùng với các cờ cho `View` biết những nút bấm và mục menu nào cần chuyển sang màu xám.

Nhờ vậy, `View` hầu như không phải làm gì ngoài việc chuyển dữ liệu từ `ViewModel` vào trang HTML.

Hãy chú ý hướng của các phụ thuộc. Tất cả đều vượt qua ranh giới và hướng vào trong, đúng theo Quy tắc Phụ thuộc.

Kết luận

Tuân theo những quy tắc đơn giản này không khó, và điều đó sẽ giúp bạn tránh được nhiều phiền toái về sau. Bằng cách chia phần mềm thành các lớp và tuân theo Quy tắc Phụ thuộc, bạn sẽ tạo ra một hệ thống vốn có thể kiểm thử, cùng mọi lợi ích đi kèm. Khi một thành phần bên ngoài nào đó trở nên lỗi thời, chẳng hạn cơ sở dữ liệu hoặc framework web, bạn

có thể thay thế nó mà không gặp nhiều phiền phức.

23

Presenter và đối tượng khiêm tốn



Trong Chương 22, chúng ta đã giới thiệu khái niệm presenter. Presenter là một dạng của mẫu Đối tượng Khiêm tốn, giúp nhận diện và bảo vệ các ranh giới kiến trúc. Thực tế, Kiến trúc Sạch ở chương trước có rất nhiều cách triển khai mẫu Đối tượng Khiêm tốn.

Mẫu Đối tượng Khiêm tốn

Mẫu Đối tượng Khiêm tốn[1] là một mẫu thiết kế ban đầu được xác định như cách giúp người viết kiểm thử đơn vị tách hành vi khó kiểm thử khỏi hành vi dễ kiểm thử. Ý tưởng rất đơn giản: chia hành vi thành hai mô-đun hoặc lớp. Một mô-đun là đối tượng khiêm tốn, chứa những hành vi khó kiểm thử đã được rút gọn đến mức tối thiểu. Mô-đun còn lại chứa các hành vi có thể kiểm thử đã được tách khỏi đối tượng khiêm tốn.

Chẳng hạn, giao diện đồ họa khó kiểm thử đơn vị vì rất khó viết kiểm thử có thể quan sát màn hình và xác nhận các thành phần thích hợp được hiển thị. Tuy nhiên, phần lớn hành vi của giao diện đồ họa thực ra dễ kiểm thử. Với mẫu Đối tượng Khiêm tốn, chúng ta có thể tách hai loại hành vi này vào hai lớp khác nhau, gọi là Presenter và View.

Presenter và View

View là đối tượng khiêm tốn, khó kiểm thử. Mã trong đối tượng này được giữ đơn giản hết mức có thể. Nó đưa dữ liệu lên giao diện đồ họa nhưng không xử lý dữ liệu đó.

Presenter là đối tượng có thể kiểm thử. Nhiệm vụ của nó là nhận dữ liệu từ ứng dụng và định dạng để View chỉ cần đưa dữ liệu lên màn hình. Chẳng hạn, nếu ứng dụng muốn hiển thị ngày tháng trong một trường, nó sẽ chuyển cho Presenter một đối tượng `Date`. Presenter định dạng dữ liệu thành chuỗi thích hợp rồi đặt chuỗi vào một cấu trúc dữ liệu đơn giản gọi là ViewModel, nơi View có thể lấy dữ liệu.

Nếu ứng dụng muốn hiển thị số tiền, nó có thể chuyển cho Presenter một đối tượng `Currency`. Presenter định dạng đối tượng đó với số chữ số thập phân và ký hiệu tiền tệ thích hợp, rồi tạo chuỗi để đặt vào ViewModel. Nếu giá trị tiền tệ cần hiển thị màu đỏ khi âm, một cờ boolean đơn giản trong ViewModel sẽ được đặt cho phù hợp.

Mỗi nút trên màn hình đều có tên. Tên đó là một chuỗi trong ViewModel do Presenter đặt vào. Nếu các nút cần chuyển sang màu xám, Presenter đặt một cờ boolean thích hợp trong ViewModel. Tên từng mục menu cũng là chuỗi trong ViewModel do Presenter nạp vào. Presenter nạp tên của mọi nút radio, hộp kiểm và trường văn bản vào các chuỗi và giá trị boolean phù hợp trong ViewModel. Các bảng số liệu cần hiển thị cũng được Presenter chuyển thành những bảng chuỗi đã định dạng phù hợp trong ViewModel.

Mọi thứ xuất hiện trên màn hình và nằm trong phạm vi kiểm soát của ứng dụng đều được biểu diễn trong ViewModel dưới dạng chuỗi, giá trị boolean hoặc enum. View không còn việc gì khác ngoài nạp dữ liệu từ ViewModel lên màn hình. Vì vậy, View là đối tượng khiêm tốn.

---

1. *xUnit Patterns*, Meszaros, Addison-Wesley, 2007, tr. 695.

Kiểm thử và kiến trúc

Từ lâu người ta đã biết khả năng kiểm thử là đặc tính của kiến trúc tốt. Mẫu Đối tượng Khiêm tốn là một ví dụ điển hình, vì việc tách hành vi thành phần có thể kiểm thử và phần không thể kiểm thử thường xác định một ranh giới kiến trúc. Ranh giới Presenter/View là một trong những ranh giới đó, nhưng còn nhiều ranh giới khác.

Cổng cơ sở dữ liệu

Giữa các interactor của ca sử dụng và cơ sở dữ liệu là các cổng cơ sở dữ liệu.[2] Đây là những giao diện đa hình, chứa phương thức cho mọi thao tác tạo, đọc, cập nhật hoặc xóa mà ứng dụng có thể thực hiện trên cơ sở dữ liệu. Chẳng hạn, nếu ứng dụng cần biết họ của tất cả người dùng đã đăng nhập hôm qua, giao diện `UserGateway` sẽ có phương thức tên `getLastNamesOfUsersWhoLoggedInAfter`, nhận một đối tượng `Date` làm đối số và trả về danh sách họ.

Hãy nhớ rằng chúng ta không cho phép SQL xuất hiện trong lớp ca sử dụng. Thay vào đó, ta dùng giao diện cổng với các phương thức thích hợp. Những cổng này được triển khai bằng các lớp trong lớp cơ sở dữ liệu. Phần triển khai là đối tượng khiêm tốn. Nó chỉ dùng SQL, hoặc giao diện cơ sở dữ liệu tương ứng, để truy cập dữ liệu mà mỗi phương thức cần. Ngược lại, interactor không phải đối tượng khiêm tốn vì nó đóng gói các quy tắc nghiệp vụ riêng của ứng dụng. Dù không khiêm tốn, interactor vẫn có thể kiểm thử vì có thể thay cổng bằng stub hoặc đối tượng kiểm thử thích hợp.

Bộ ánh xạ dữ liệu

Quay lại chủ đề cơ sở dữ liệu, bạn nghĩ các ORM như Hibernate thuộc lớp nào?

Trước hết, hãy làm rõ một điều: không có thứ gọi là bộ ánh xạ quan hệ đối tượng (ORM). Lý do rất đơn giản: đối tượng không phải là cấu trúc dữ liệu, ít nhất là từ góc nhìn của người dùng đối tượng. Người dùng không thể nhìn thấy dữ liệu bên trong đối tượng vì dữ liệu đó đều là private. Họ chỉ thấy các phương thức public. Vì vậy, từ góc nhìn của người dùng, đối tượng chỉ là tập hợp các thao tác.

Ngược lại, cấu trúc dữ liệu là tập hợp các biến dữ liệu public, không hàm ý hành vi nào. Gọi ORM là "bộ ánh xạ dữ liệu" sẽ chính xác hơn, vì chúng nạp dữ liệu từ các bảng trong cơ sở dữ liệu quan hệ vào cấu trúc dữ liệu.

Những hệ thống ORM như vậy nên nằm ở đâu? Dĩ nhiên là trong lớp cơ sở dữ liệu. Thực tế, ORM tạo thành một dạng ranh giới Đối tượng Khiêm tốn khác giữa giao diện cổng và cơ sở dữ liệu.

---

2. *Patterns of Enterprise Application Architecture*, Martin Fowler và cộng sự, Addison-Wesley, 2003, tr. 466.

Bộ lắng nghe dịch vụ

Còn dịch vụ thì sao? Nếu ứng dụng cần giao tiếp với dịch vụ khác, hoặc nếu ứng dụng cung cấp một tập hợp dịch vụ, liệu mẫu Đối tượng Khiêm tốn có tạo ra ranh giới dịch vụ không?

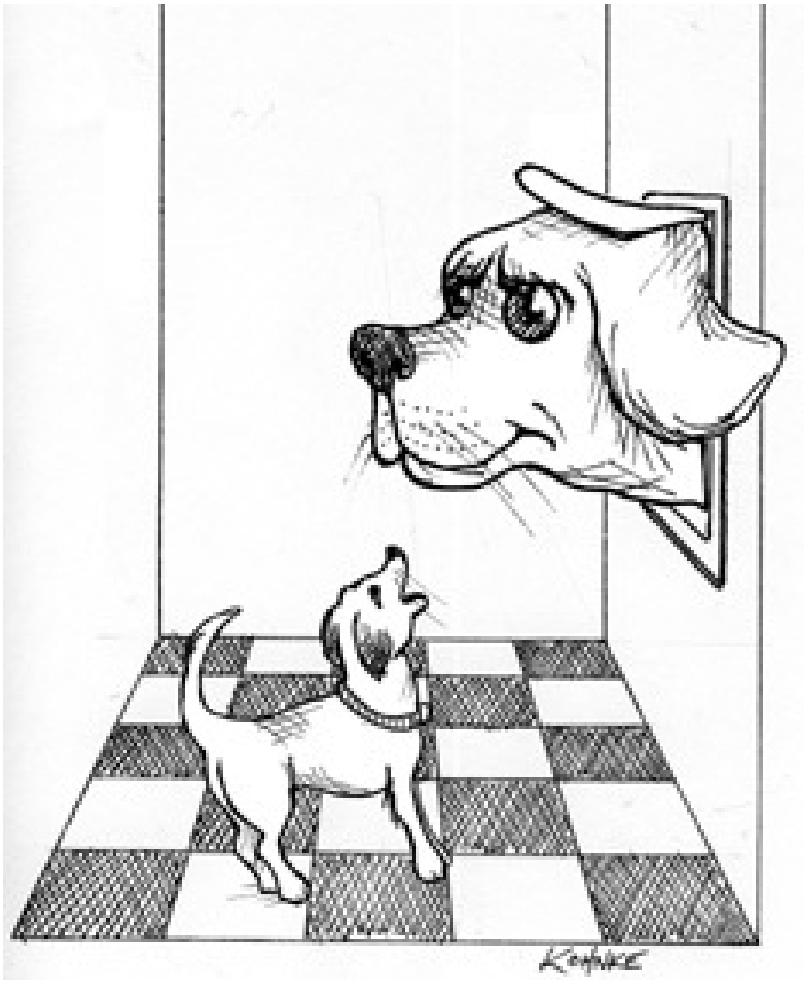
Tất nhiên là có! Ứng dụng sẽ nạp dữ liệu vào các cấu trúc đơn giản rồi truyền chúng qua ranh giới đến các mô-đun có nhiệm vụ định dạng dữ liệu và gửi đến dịch vụ bên ngoài. Ở phía đầu vào, bộ lắng nghe dịch vụ nhận dữ liệu từ giao diện dịch vụ và định dạng thành cấu trúc dữ liệu đơn giản để ứng dụng sử dụng. Sau đó, cấu trúc dữ liệu được truyền qua ranh giới dịch vụ.

Kết luận

Tại mỗi ranh giới kiến trúc, có thể sẽ có mẫu Đối tượng Khiêm tốn ở gần đó. Giao tiếp qua ranh giới hầu như luôn dùng một dạng cấu trúc dữ liệu đơn giản, và ranh giới thường phân tách phần khó kiểm thử khỏi phần dễ kiểm thử. Áp dụng mẫu này tại các ranh giới kiến trúc giúp tăng đáng kể khả năng kiểm thử của toàn hệ thống.

24

Ranh giới một phần



Các ranh giới kiến trúc hoàn chỉnh rất tốn kém. Chúng đòi hỏi các giao diện ranh giới đa hình hai chiều, cấu trúc dữ liệu đầu vào và đầu ra, cùng toàn bộ cơ chế quản lý phụ thuộc cần thiết để cách ly hai phía thành các thành phần có thể biên dịch và triển khai độc lập. Việc đó đòi hỏi nhiều công sức, và cũng tốn công duy trì.

Trong nhiều tình huống, kiến trúc sư giỏi có thể kết luận rằng chi phí cho một ranh giới như vậy quá cao, nhưng vẫn muốn chừa sẵn chỗ cho nó nếu sau này cần đến.

Nhiều người trong cộng đồng Agile thường không tán thành kiểu thiết kế dự đoán này vì cho rằng nó vi phạm YAGNI: “You Aren’t Going to Need It” (Bạn sẽ không cần đến nó).

Tuy nhiên, đôi khi kiến trúc sư nhìn vào vấn đề và nghĩ: "Đúng vậy, nhưng có thể tôi sẽ cần". Khi đó, họ có thể triển khai một ranh giới một phần.

Bỏ qua bước cuối

Một cách tạo ranh giới một phần là thực hiện mọi công việc cần thiết để tạo các thành phần có thể biên dịch và triển khai độc lập, rồi giữ tất cả chúng trong cùng một thành phần. Các giao diện hai chiều đã có, cấu trúc dữ liệu đầu vào và đầu ra đã có, mọi thứ đều được chuẩn bị xong, nhưng chúng ta biên dịch và triển khai tất cả như một thành phần duy nhất.

Rõ ràng, ranh giới một phần kiểu này vẫn cần lượng mã và công việc thiết kế chuẩn bị tương đương một ranh giới hoàn chỉnh. Tuy nhiên, không cần quản lý nhiều thành phần. Không phải theo dõi số phiên bản hay gánh việc quản lý bản phát hành. Khác biệt đó không nên bị xem nhẹ.

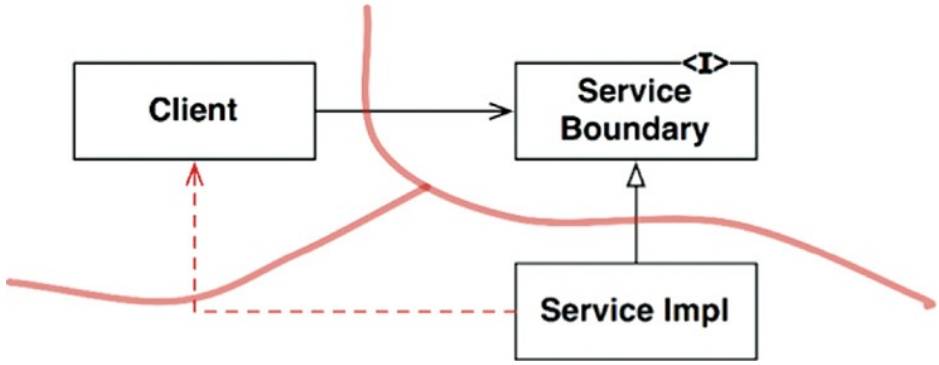
Đây là chiến lược ban đầu của FitNesse. Thành phần máy chủ web của FitNesse được thiết kế để có thể tách khỏi phần wiki và kiểm thử. Ý tưởng là có thể dùng thành phần web đó để tạo các ứng dụng chạy trên web khác. Đồng thời, chúng tôi không muốn người dùng phải tải xuống hai thành phần. Hãy nhớ một trong những mục tiêu thiết kế của chúng tôi là "download and go". Chúng tôi muốn người dùng tải xuống một tệp jar duy nhất và chạy nó mà không phải tìm thêm tệp jar khác hay tự xử lý tính tương thích giữa các phiên bản.

Câu chuyện FitNesse cũng cho thấy một trong những rủi ro của cách tiếp cận này. Theo thời gian, khi thấy rõ sẽ không bao giờ cần một thành phần web riêng, ranh giới giữa thành phần web và thành phần wiki bắt đầu suy yếu. Các phụ thuộc dần vượt qua ranh giới theo hướng sai. Ngày nay, việc tách chúng ra lần nữa sẽ khá vất vả.

Ranh giới một chiều

Ranh giới kiến trúc hoàn chỉnh dùng các giao diện hai chiều để duy trì sự cách ly ở cả hai hướng. Duy trì sự tách biệt theo cả hai hướng tốn kém, cả lúc thiết lập ban đầu lẫn trong quá trình bảo trì.

Một cấu trúc đơn giản hơn, dùng để chừa chỗ cho việc mở rộng thành ranh giới hoàn chỉnh sau này, được minh họa ở Hình 24.1. Cấu trúc này là mẫu Strategy truyền thống. Các client sử dụng giao diện `ServiceBoundary`, còn các lớp `ServiceImpl` triển khai giao diện đó.

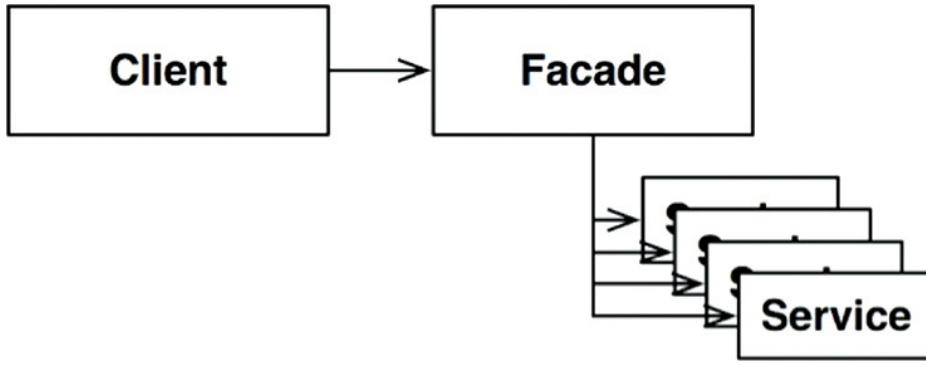


Hình 24.1
Mẫu Strategy

Rõ ràng cấu trúc này chuẩn bị sẵn nền tảng cho một ranh giới kiến trúc trong tương lai. Cơ chế đảo ngược phụ thuộc cần thiết đã có để cố gắng cách ly `Client` khỏi `ServiceImpl`. Cũng rõ ràng rằng sự tách biệt này có thể suy giảm rất nhanh, như mũi tên chấm khó coi trong sơ đồ cho thấy. Khi không có giao diện hai chiều, không có gì ngăn được kiểu kênh phụ thuộc ngược này ngoài sự cẩn trọng và kỷ luật của các nhà phát triển và kiến trúc sư.

Mẫu Facade

Một ranh giới còn đơn giản hơn là mẫu Facade, được minh họa trong Hình 24.2. Trong trường hợp này, ngay cả việc đảo ngược phụ thuộc cũng bị bỏ qua. Ranh giới chỉ được xác định bởi lớp `Facade`, lớp này liệt kê tất cả dịch vụ dưới dạng các phương thức và chuyển tiếp lời gọi dịch vụ đến những lớp mà client không nên truy cập.



Hình 24.2
Mẫu Facade

Tuy nhiên, hãy lưu ý rằng `Client` có phụ thuộc bắc cầu đến tất cả các lớp dịch vụ đó.

Trong ngôn ngữ tĩnh, thay đổi mã nguồn của một lớp `Service` sẽ buộc `Client` phải biên dịch lại. Bạn cũng có thể hình dung việc tạo các kênh phụ thuộc ngược dễ dàng đến mức nào trong cấu trúc này.

Kết luận

Chúng ta đã thấy ba cách đơn giản để triển khai một phần ranh giới kiến trúc. Tất nhiên còn nhiều cách khác. Ba chiến lược này chỉ nhằm làm ví dụ.

Mỗi cách tiếp cận đều có chi phí và lợi ích riêng. Trong một số bối cảnh nhất định, mỗi cách đều phù hợp để làm chỗ giữ chỗ cho một ranh giới hoàn chỉnh trong tương lai. Nếu ranh giới hoàn chỉnh không bao giờ được tạo ra, từng cách cũng có thể suy giảm.

Một trong những nhiệm vụ của kiến trúc sư là quyết định nơi nào có thể cần đến ranh giới kiến trúc trong tương lai, và ranh giới đó nên được triển khai hoàn chỉnh hay chỉ một phần.

25

Các lớp và ranh giới



Ta dễ hình dung hệ thống gồm ba thành phần: giao diện người dùng, quy tắc nghiệp vụ và cơ sở dữ liệu. Với một số hệ thống đơn giản, như vậy là đủ. Nhưng với hầu hết hệ thống, số thành phần nhiều hơn.

Chẳng hạn, hãy nghĩ đến một trò chơi máy tính đơn giản. Ta dễ hình dung ba thành phần. Giao diện người dùng xử lý mọi thông điệp từ người chơi gửi đến luật chơi. Luật

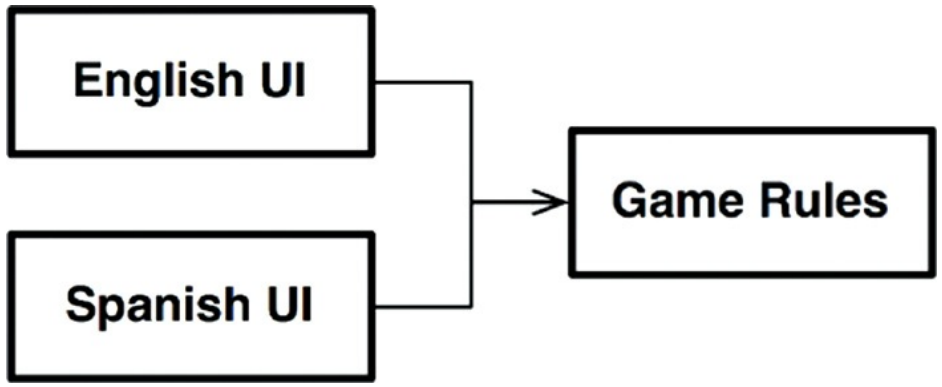
chơi lưu trạng thái trò chơi trong một cấu trúc dữ liệu bền vững nào đó. Nhưng liệu chỉ có vậy thôi không?

Săn Wumpus

Hãy cụ thể hóa ví dụ này. Giả sử trò chơi là cuộc phiêu lưu *Hunt the Wumpus* nổi tiếng từ năm 1972. Trò chơi dạng văn bản này dùng các lệnh rất đơn giản như `GO EAST` và `SHOOT WEST`. Người chơi nhập lệnh, máy tính phản hồi bằng những gì người chơi nhìn thấy, ngửi thấy, nghe thấy và trải nghiệm. Người chơi săn Wumpus trong một hệ thống hang động, đồng thời phải tránh bẫy, hố sâu và những mối nguy khác đang rình rập. Nếu quan tâm, bạn có thể dễ dàng tìm thấy luật chơi trên web.

Giả sử ta giữ giao diện dạng văn bản nhưng tách nó khỏi luật chơi, để có thể dùng các ngôn ngữ khác nhau ở những thị trường khác nhau. Luật chơi sẽ giao tiếp với thành phần giao diện qua một API độc lập với ngôn ngữ, còn giao diện sẽ chuyển API đó sang ngôn ngữ thích hợp cho người dùng.

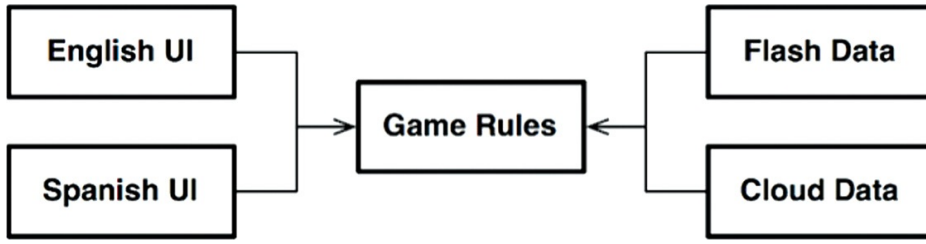
Nếu quản lý phụ thuộc mã nguồn đúng cách như Hình 25.1, bất kỳ số lượng thành phần giao diện nào cũng có thể dùng chung luật chơi. Luật chơi không biết và cũng không quan tâm người dùng đang sử dụng ngôn ngữ nào.



Hình 25.1
Nhiều thành phần giao diện có thể dùng chung luật chơi

Giả sử thêm rằng trạng thái trò chơi được lưu trong một kho lưu trữ bền vững, có thể là bộ nhớ flash, đám mây hoặc chỉ là RAM. Dù trong trường hợp nào, ta cũng không muốn luật chơi biết các chi tiết đó. Vì vậy, một lần nữa, ta tạo API để luật chơi giao tiếp với thành phần lưu trữ dữ liệu.

Ta không muốn luật chơi biết gì về các loại lưu trữ dữ liệu khác nhau. Do đó, phụ thuộc phải được sắp xếp đúng theo Quy tắc Phụ thuộc, như Hình 25.2 cho thấy.



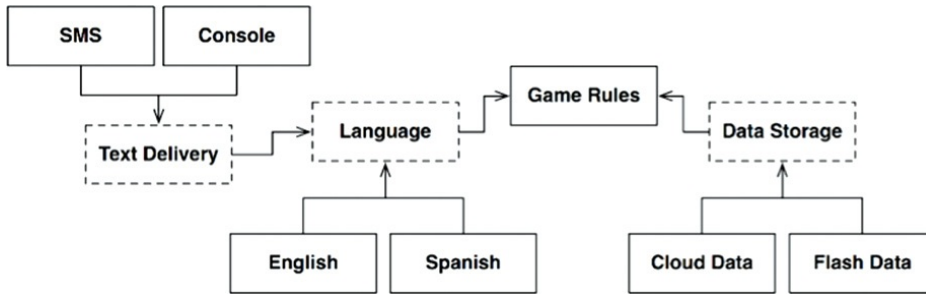
Hình 25.2
Tuân theo Quy tắc Phụ thuộc

Kiến trúc sạch?

Rõ ràng ta có thể dễ dàng áp dụng cách tiếp cận Kiến trúc Sạch trong bối cảnh này,[1] với đầy đủ ca sử dụng, ranh giới, Entity và cấu trúc dữ liệu tương ứng. Nhưng liệu ta đã tìm ra tất cả ranh giới kiến trúc quan trọng chưa?

Chẳng hạn, ngôn ngữ không phải trục thay đổi duy nhất của giao diện người dùng. Ta cũng có thể muốn thay đổi cơ chế giao tiếp nội dung. Ví dụ, ta có thể dùng cửa sổ shell thông thường, tin nhắn văn bản hoặc ứng dụng chat. Có rất nhiều khả năng.

Điều đó có nghĩa là trục thay đổi này có thể xác định một ranh giới kiến trúc. Có lẽ ta nên tạo một API đi qua ranh giới đó và tách biệt ngôn ngữ khỏi cơ chế giao tiếp. Ý tưởng này được minh họa ở Hình 25.3.



Hình 25.3
Sơ đồ đã chỉnh sửa

Sơ đồ ở Hình 25.3 đã hơi phức tạp, nhưng không có gì bất ngờ. Các đường viền nét đứt biểu thị những thành phần trừu tượng xác định API do các thành phần phía trên hoặc phía dưới triển khai. Chẳng hạn, API `Language` được triển khai bởi `English` và `Spanish`.

`GameRules` giao tiếp với `Language` qua một API do `GameRules` xác định và

1. Cũng cần thấy rõ rằng ta sẽ không áp dụng Kiến trúc Sạch cho một trò chơi đơn giản như thế này. Toàn bộ chương trình có lẽ chỉ cần viết trong 200 dòng mã hoặc ít hơn. Ở đây, ta dùng một chương trình đơn giản làm đại diện cho hệ thống lớn hơn với các ranh giới kiến trúc quan trọng.

`Language` triển khai. `Language` giao tiếp với `TextDelivery` qua API do `Language` xác định nhưng `TextDelivery` triển khai. API do thành phần sử dụng quyết định và sở hữu, không phải thành phần triển khai.

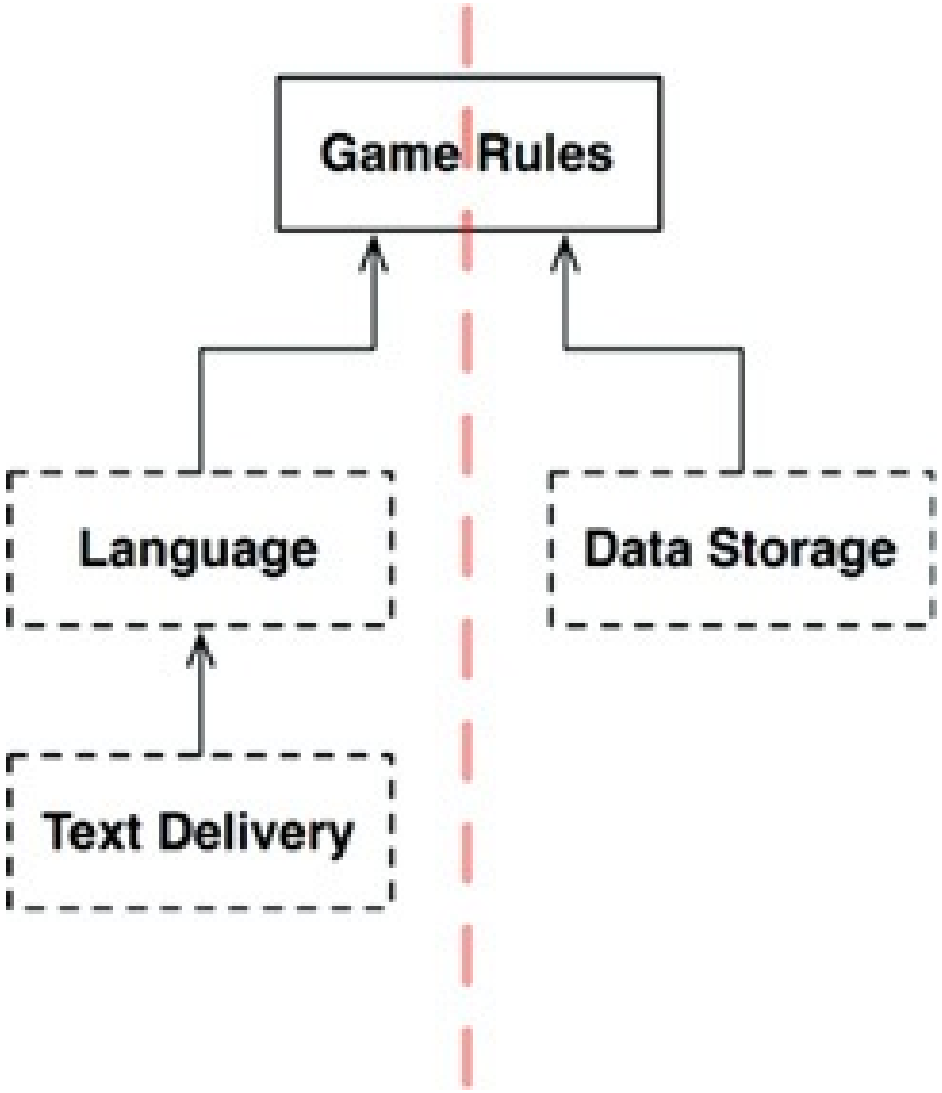
Nếu nhìn vào bên trong `GameRules`, ta sẽ thấy các giao diện ranh giới đa hình được mã bên trong `GameRules` sử dụng và mã bên trong thành phần `Language` triển khai. Ta cũng sẽ thấy các giao diện ranh giới đa hình được `Language` sử dụng và mã bên trong `GameRules` triển khai.

Bên trong `Language` cũng có cấu trúc tương tự: các giao diện ranh giới đa hình được mã trong `TextDelivery` triển khai, cùng các giao diện ranh giới đa hình do `TextDelivery` sử dụng và `Language` triển khai.

Trong mỗi trường hợp, API do các giao diện ranh giới xác định thuộc quyền sở hữu của thành phần ở phía gọi.

Các biến thể như `English`, `SMS` và `CloudData` được cung cấp qua các giao diện đa hình do thành phần API trừu tượng xác định, rồi được các thành phần cụ thể phục vụ chúng triển khai. Chẳng hạn, ta kỳ vọng các giao diện đa hình được xác định trong `Language` sẽ được `English` và `Spanish` triển khai.

Ta có thể đơn giản hóa sơ đồ bằng cách bỏ mọi biến thể và chỉ tập trung vào các thành phần API. Hình 25.4 cho thấy sơ đồ này.



Hình 25.4
Sơ đồ đã đơn giản hóa

Hãy lưu ý rằng Hình 25.4 được bố trí sao cho mọi mũi tên đều hướng lên trên. Như vậy, `GameRules` nằm ở trên cùng. Cách bố trí này hợp lý vì `GameRules` chứa các chính sách cấp cao nhất.

Xét hướng luồng thông tin. Mọi đầu vào từ người dùng đi vào `TextDelivery` ở góc dưới bên trái. Thông tin đi lên qua thành phần `Language`, được chuyển thành lệnh gửi đến `GameRules`. `GameRules` xử lý đầu vào của người dùng rồi gửi dữ liệu thích hợp xuống

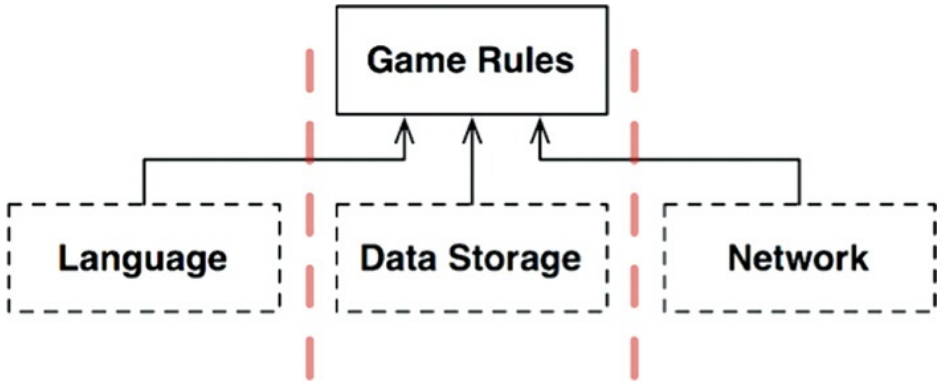
`DataStorage` ở góc dưới bên phải.

Sau đó, `GameRules` gửi kết quả xuống `Language`. Thành phần này chuyển API trở lại ngôn ngữ phù hợp rồi chuyển ngôn ngữ đó đến người dùng qua `TextDelivery`.

Cấu trúc này chia luồng dữ liệu thành hai nhánh.[2] Nhánh bên trái đảm nhiệm giao tiếp với người dùng, còn nhánh bên phải xử lý việc lưu trữ dữ liệu. Hai nhánh gặp nhau ở phía trên,[3] tại `GameRules`, thành phần cuối cùng xử lý dữ liệu đi qua cả hai nhánh.

Giao cắt các luồng

Có phải lúc nào cũng chỉ có hai luồng dữ liệu như trong ví dụ này không? Không hề. Hãy tưởng tượng ta muốn chơi *Hunt the Wumpus* trên mạng với nhiều người cùng lúc. Khi đó, ta cần thêm một thành phần mạng như trong Hình 25.5. Cấu trúc này chia luồng dữ liệu thành ba nhánh, tất cả đều do `GameRules` điều khiển.



Hình 25.5
Thêm thành phần mạng

Khi hệ thống phức tạp hơn, cấu trúc thành phần có thể tách thành nhiều nhánh như vậy.

Tách các luồng

Có thể bạn đang nghĩ rằng mọi luồng cuối cùng đều gặp nhau ở trên cùng trong một thành phần duy nhất. Giá mà mọi chuyện đơn giản như vậy! Thực tế phức tạp hơn nhiều.

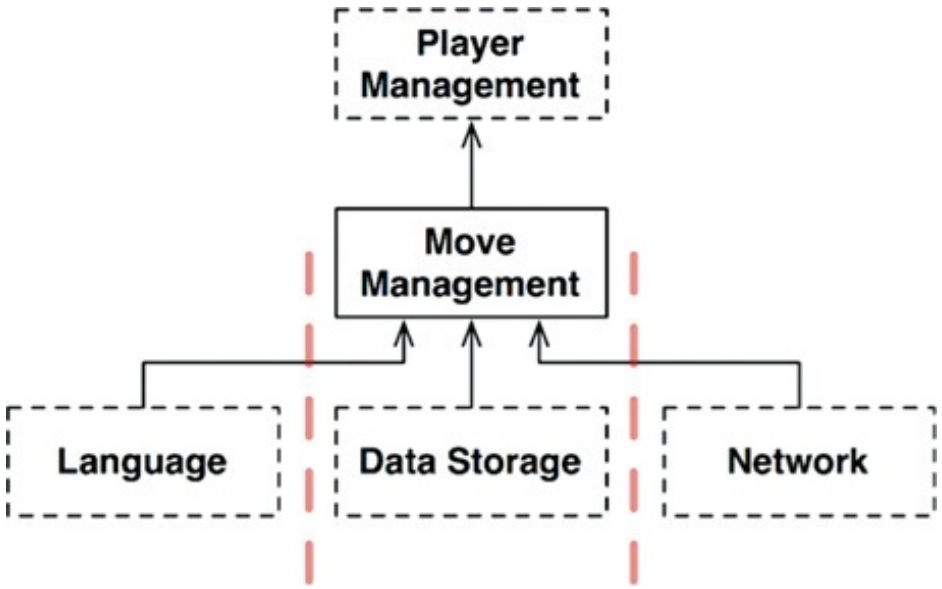
Hãy xét thành phần `GameRules` của *Hunt the Wumpus*. Một phần luật chơi xử lý cơ chế bản đồ. Phần này biết các hang động nối với nhau như thế nào và vật thể nào nằm trong

2. Nếu thấy khó hiểu hướng của các mũi tên, hãy nhớ rằng chúng chỉ hướng phụ thuộc mã nguồn, không phải hướng luồng dữ liệu.

3. Ngày trước, ta có thể gọi thành phần trên cùng là Central Transform. Xem *Practical Guide to Structured Systems Design*, ấn bản thứ hai, Meilir Page-Jones, 1988.

từng hang. Nó biết cách di chuyển người chơi từ hang này sang hang khác và xác định những sự kiện người chơi phải xử lý.

Nhưng còn một tập hợp chính sách khác ở cấp cao hơn nữa, biết tình trạng sức khỏe của người chơi cùng chi phí hoặc lợi ích của từng sự kiện. Các chính sách này có thể khiến sức khỏe người chơi giảm dần hoặc tăng lên khi tìm được thức ăn. Chính sách cơ chế cấp thấp sẽ thông báo sự kiện cho chính sách cấp cao, chẳng hạn `FoundFood` hoặc `FellInPit`. Chính sách cấp cao hơn sau đó quản lý trạng thái người chơi, như Hình 25.6 cho thấy. Cuối cùng, chính sách đó quyết định người chơi thắng hay thua.

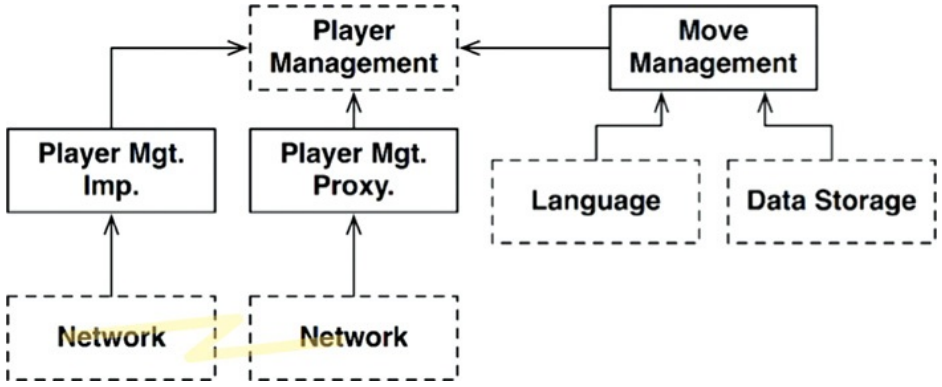


Hình 25.6
Chính sách cấp cao hơn quản lý người chơi

Đây có phải là một ranh giới kiến trúc không? Ta có cần API tách `MoveManagement` khỏi `PlayerManagement` không? Hãy làm ví dụ thú vị hơn một chút bằng cách thêm vi dịch vụ.

Giả sử ta có phiên bản *Hunt the Wumpus* nhiều người chơi quy mô lớn. `MoveManagement` chạy cục bộ trên máy tính của người chơi, còn `PlayerManagement` chạy trên máy chủ. `PlayerManagement` cung cấp API vi dịch vụ cho mọi thành phần `MoveManagement` được kết nối.

Hình 25.7 mô tả kịch bản này theo cách rút gọn. Các thành phần mạng phức tạp hơn những gì sơ đồ thể hiện, nhưng có lẽ bạn vẫn hình dung được. Trong trường hợp này, giữa `MoveManagement` và `PlayerManagement` có một ranh giới kiến trúc hoàn chỉnh.



Hình 25.7
Thêm API vi dịch vụ

Kết luận

Điều này có ý nghĩa gì? Vì sao tôi lấy một chương trình đơn giản đến mức vô lý, có thể viết trong 200 dòng Kornshell, rồi mở rộng nó với tất cả những ranh giới kiến trúc phức tạp này?

Ví dụ này nhằm cho thấy ranh giới kiến trúc tồn tại ở khắp nơi. Với tư cách kiến trúc sư, chúng ta phải cẩn thận nhận ra khi nào cần đến chúng. Ta cũng phải hiểu rằng khi được triển khai đầy đủ, những ranh giới ấy rất tốn kém. Đồng thời, nếu bỏ qua ranh giới, chi phí bổ sung chúng về sau có thể rất cao, ngay cả khi đã có bộ kiểm thử toàn diện và quy trình tái cấu trúc có kỷ luật.

Vậy chúng ta, những kiến trúc sư, phải làm gì? Câu trả lời không dễ chịu. Một mặt, nhiều người thông minh đã khuyên chúng ta không nên dự đoán trước nhu cầu trừu tượng hóa. Đó là triết lý YAGNI: “You Aren’t Going to Need It” (Bạn sẽ không cần đến nó). Lời khuyên này có phần đúng, vì thiết kế quá mức thường tệ hơn thiết kế chưa đủ. Mặt khác, khi phát hiện mình thật sự cần một ranh giới kiến trúc nhưng chưa có, chi phí và rủi ro để bổ sung có thể rất cao.

Vậy đấy. Hỡi kiến trúc sư phần mềm, bạn phải nhìn thấy tương lai. Bạn phải dự đoán một cách sáng suốt. Bạn phải cân nhắc chi phí và xác định vị trí các ranh giới kiến trúc, đồng thời quyết định ranh giới nào cần triển khai đầy đủ, ranh giới nào chỉ triển khai một phần và ranh giới nào nên bỏ qua.

Nhưng đây không phải quyết định chỉ đưa ra một lần. Bạn không thể ngay đầu dự án quyết định ranh giới nào sẽ triển khai, ranh giới nào sẽ bỏ qua rồi thôi. Thay vào đó, hãy quan sát. Chú ý khi hệ thống phát triển. Ghi nhận nơi có thể cần ranh giới, rồi cẩn thận theo dõi dấu hiệu ma sát đầu tiên do thiếu những ranh giới ấy.

Khi đó, hãy cân nhắc chi phí triển khai ranh giới so với chi phí bỏ qua nó, rồi thường xuyên xem xét lại quyết định. Mục tiêu là triển khai ranh giới đúng tại điểm mà chi phí triển khai bắt đầu thấp hơn chi phí bỏ qua.

Cần một con mắt luôn cảnh giác.

26

Thành phần Main



Trong mọi hệ thống đều có ít nhất một thành phần tạo ra, điều phối và giám sát các thành phần khác. Tôi gọi thành phần này là `Main`.

Chi tiết tối hậu

Thành phần `Main` là chi tiết tối hậu, chính sách cấp thấp nhất. Đây là điểm vào ban đầu của hệ thống. Không có gì ngoài hệ điều hành phụ thuộc vào nó. Nhiệm vụ của nó là tạo mọi Factory, Strategy và tiện ích toàn cục khác, rồi chuyển quyền điều khiển cho những phần trừu tượng cấp cao của hệ thống.

Chính trong thành phần `Main`, framework tiêm phụ thuộc nên đưa các phụ thuộc vào hệ thống. Sau khi được tiêm vào `Main`, các phụ thuộc nên được phân phối theo cách thông thường, không dùng framework.

Hãy xem `Main` là thành phần bẩn nhất trong tất cả các thành phần bẩn.

Dưới đây là thành phần `Main` trong một phiên bản gần đây của *Hunt the Wumpus*. Hãy chú ý cách nó nạp tất cả các chuỗi mà ta không muốn phần mã chính phải biết.

```
public class Main implements HtwMessageReceiver {
private static HuntTheWumpus game;
private static int hitPoints = 10;
private static final List<String> caverns = new ArrayList<>();
private static final String[] environments = new String[]{
"bright",
"humid",
"dry",
"creepy",
"ugly",
"foggy",
"hot",
"cold",
"drafty",
"dreadful"
};

private static final String[] shapes = new String[] {
"round",
"square",
"oval",
"irregular",
"long",
"craggy",
"rough",
"tall",
"narrow"
};

private static final String[] cavernTypes = new String[] {
"cavern",
"room",
"chamber",
"catacomb",
"crevasse",
"cell",
"tunnel",
"passageway",
"hall",
"expanse"
};
```

```
private static final String[] adornments = new String[] {
"smelling of sulfur",
"with engravings on the walls",
"with a bumpy floor",
"",
"littered with garbage",
"spattered with guano",
"with piles of Wumpus droppings",
"with bones scattered around",
"with a corpse on the floor",
"that seems to vibrate",
"that feels stuffy",
"that fills you with dread"
};
```

Tiếp theo là hàm chính. Hãy chú ý cách nó dùng `HtwFactory` để tạo trò chơi. Nó truyền vào tên lớp `htw.game.HuntTheWumpusFacade`, vì lớp đó còn bẩn hơn cả `Main`. Nhờ vậy, thay đổi trong lớp đó không buộc `Main` phải biên dịch lại hay triển khai lại.

```
public static void main(String[] args) throws IOException {
game = HtwFactory.makeGame("htw.game.HuntTheWumpusFacade",
new Main());
createMap();
BufferedReader br =
new BufferedReader(new InputStreamReader(System.in));
game.makeRestCommand().execute();
while (true) {
System.out.println(game.getPlayerCavern());
System.out.println("Health: " + hitPoints + " arrows: " +
game.getQuiver());
HuntTheWumpus.Command c = game.makeRestCommand();
System.out.println(">");
String command = br.readLine();
if (command.equalsIgnoreCase("e"))
c = game.makeMoveCommand(EAST);
else if (command.equalsIgnoreCase("w"))
c = game.makeMoveCommand(WEST);
else if (command.equalsIgnoreCase("n"))
c = game.makeMoveCommand(NORTH);
else if (command.equalsIgnoreCase("s"))
c = game.makeMoveCommand(SOUTH);
else if (command.equalsIgnoreCase("r"))
c = game.makeRestCommand();
else if (command.equalsIgnoreCase("sw"))
c = game.makeShootCommand(WEST);
else if (command.equalsIgnoreCase("se"))
c = game.makeShootCommand(EAST);
else if (command.equalsIgnoreCase("sn"))
c = game.makeShootCommand(NORTH);
else if (command.equalsIgnoreCase("ss"))
c = game.makeShootCommand(SOUTH);
else if (command.equalsIgnoreCase("q"))
return;

c.execute();
```

```
    }
    }
```

Cũng cần lưu ý rằng `main` tạo luồng đầu vào và chứa vòng lặp chính của trò chơi. Nó diễn giải các lệnh đầu vào đơn giản nhưng giao toàn bộ phần xử lý cho các thành phần cấp cao hơn.

Cuối cùng, hãy chú ý rằng `main` tạo bản đồ.

```
    private static void createMap() {
    int nCaverns = (int) (Math.random() * 30.0 + 10.0);
    while (nCaverns-- > 0)
    caverns.add(makeName());

    for (String cavern : caverns) {
    maybeConnectCavern(cavern, NORTH);
    maybeConnectCavern(cavern, SOUTH);
    maybeConnectCavern(cavern, EAST);
    maybeConnectCavern(cavern, WEST);
    }

    String playerCavern = anyCavern();
    game.setPlayerCavern(playerCavern);
    game.setWumpusCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));

    game.addPitCavern(anyOther(playerCavern));
    game.addPitCavern(anyOther(playerCavern));
    game.addPitCavern(anyOther(playerCavern));

    game.setQuiver(5);
    }
    // much code removed…
    }
```

Điểm chính là `Main` là mô-đun bẩn, cấp thấp, nằm ở vòng tròn ngoài cùng của Kiến trúc Sạch. Nó nạp mọi thứ cần thiết cho hệ thống cấp cao, rồi chuyển quyền điều khiển cho hệ thống đó.

Kết luận

Hãy xem `Main` như một plugin của ứng dụng, plugin thiết lập các điều kiện và cấu hình ban đầu, thu thập tài nguyên bên ngoài, rồi chuyển quyền điều khiển cho chính sách cấp cao của ứng dụng. Vì là plugin, ứng dụng có thể có nhiều thành phần `Main`, mỗi thành phần dành cho một cấu hình.

Chẳng hạn, bạn có thể có một plugin `Main` cho môi trường phát triển, một plugin khác cho kiểm thử và một plugin nữa cho môi trường production. Bạn cũng có thể có một plugin `Main` cho từng quốc gia triển khai, từng khu vực pháp lý hoặc từng khách hàng.

Khi xem `Main` như thành phần plugin nằm sau một ranh giới kiến trúc, vấn đề cấu hình sẽ dễ giải quyết hơn nhiều.

27

Dịch vụ: lớn và nhỏ



"Kiến trúc" hướng dịch vụ và "kiến trúc" vi dịch vụ gần đây trở nên rất phổ biến. Những lý do khiến chúng được ưa chuộng gồm:

- Dịch vụ dường như tách rời chặt chẽ khỏi nhau. Như chúng ta sẽ thấy, điều này chỉ đúng một phần.
- Dịch vụ dường như hỗ trợ phát triển và triển khai độc lập. Một lần nữa, điều này chỉ đúng một phần.

Kiến trúc dịch vụ?

Trước hết, hãy xem xét quan niệm cho rằng việc dùng dịch vụ, tự bản chất, đã tạo thành một kiến trúc. Điều đó hoàn toàn sai. Kiến trúc của một hệ thống được xác định bởi các ranh giới tách chính sách cấp cao khỏi chi tiết cấp thấp và tuân theo Quy tắc Phụ thuộc. Các dịch vụ chỉ đơn giản tách hành vi ứng dụng thì chẳng khác nào những lời gọi hàm tốn kém, và không nhất thiết có ý nghĩa về mặt kiến trúc.

Điều này không có nghĩa mọi dịch vụ đều phải có ý nghĩa kiến trúc. Việc tạo dịch vụ để tách chức năng giữa các tiến trình và nền tảng thường đem lại lợi ích đáng kể, dù chúng có tuân theo Quy tắc Phụ thuộc hay không. Chỉ có điều bản thân dịch vụ không định nghĩa kiến trúc.

Có thể so sánh với cách tổ chức hàm. Kiến trúc của hệ thống nguyên khối hoặc hệ thống dựa trên thành phần được xác định bởi một số lời gọi hàm vượt qua ranh giới kiến trúc và tuân theo Quy tắc Phụ thuộc. Tuy nhiên, nhiều hàm khác trong những hệ thống đó chỉ tách hành vi này khỏi hành vi khác và không có ý nghĩa kiến trúc.

Dịch vụ cũng vậy. Xét cho cùng, dịch vụ chỉ là lời gọi hàm vượt qua ranh giới tiến trình hoặc nền tảng. Một số dịch vụ có ý nghĩa kiến trúc, số khác thì không. Trong chương này, chúng ta quan tâm đến nhóm thứ nhất.

Lợi ích của dịch vụ?

Dấu hỏi trong tiêu đề cho biết phần này sẽ thách thức quan điểm phổ biến hiện nay về kiến trúc dịch vụ. Hãy xem xét từng lợi ích một.

Ngụy biện về tính tách rời

Một lợi ích lớn được cho là của việc chia hệ thống thành các dịch vụ là chúng tách rời chặt chẽ khỏi nhau. Xét cho cùng, mỗi dịch vụ chạy trong một tiến trình riêng, thậm chí trên một bộ xử lý khác. Vì vậy, dịch vụ không thể truy cập biến của nhau. Hơn nữa, giao diện của mỗi dịch vụ phải được định nghĩa rõ ràng.

Điều này có phần đúng, nhưng không đúng nhiều. Đúng là các dịch vụ tách rời ở cấp độ biến riêng lẻ. Tuy nhiên, chúng vẫn có thể bị ràng buộc với nhau thông qua tài nguyên dùng chung trong bộ xử lý hoặc trên mạng. Quan trọng hơn, chúng bị ràng buộc chặt chẽ bởi dữ liệu dùng chung.

Chẳng hạn, nếu thêm một trường mới vào bản ghi dữ liệu được truyền giữa các dịch vụ, thì mọi dịch vụ sử dụng trường đó đều phải thay đổi. Các dịch vụ cũng phải thống nhất chặt chẽ về cách diễn giải dữ liệu trong trường mới. Vì vậy, chúng phụ thuộc chặt chẽ vào bản ghi dữ liệu và gián tiếp phụ thuộc lẫn nhau.

Giao diện được định nghĩa rõ ràng ư? Điều đó chắc chắn đúng, nhưng hàm cũng vậy. Giao diện dịch vụ không chính thức hơn, không nghiêm ngặt hơn và cũng không được định nghĩa tốt hơn giao diện hàm. Rõ ràng lợi ích này phần nào chỉ là ảo tưởng.

Ngụy biện về phát triển và triển khai độc lập

Một lợi ích khác được cho là của dịch vụ là có thể giao quyền sở hữu và vận hành chúng cho một nhóm chuyên trách. Nhóm đó chịu trách nhiệm viết, bảo trì và vận hành dịch vụ theo chiến lược dev-ops. Người ta cho rằng khả năng phát triển và triển khai độc lập này có thể mở rộng theo quy mô. Họ tin rằng có thể xây dựng hệ thống doanh nghiệp lớn từ hàng chục, hàng trăm, thậm chí hàng nghìn dịch vụ có thể được phát triển và triển khai độc lập. Việc phát triển, bảo trì và vận hành hệ thống có thể được chia cho số lượng nhóm độc lập tương ứng.

Niềm tin này có phần đúng, nhưng chỉ một phần. Trước hết, lịch sử cho thấy có thể xây dựng hệ thống doanh nghiệp lớn bằng kiến trúc nguyên khối và kiến trúc dựa trên thành phần, cũng như kiến trúc dựa trên dịch vụ. Vì vậy, dịch vụ không phải lựa chọn duy nhất để xây dựng hệ thống có khả năng mở rộng.

Thứ hai, ngụy biện về tính tách rời cho thấy không phải lúc nào cũng có thể phát triển, triển khai và vận hành dịch vụ một cách độc lập. Khi các dịch vụ phụ thuộc nhau qua dữ liệu hoặc hành vi, việc phát triển, triển khai và vận hành phải được phối hợp.

Vấn đề mèo con

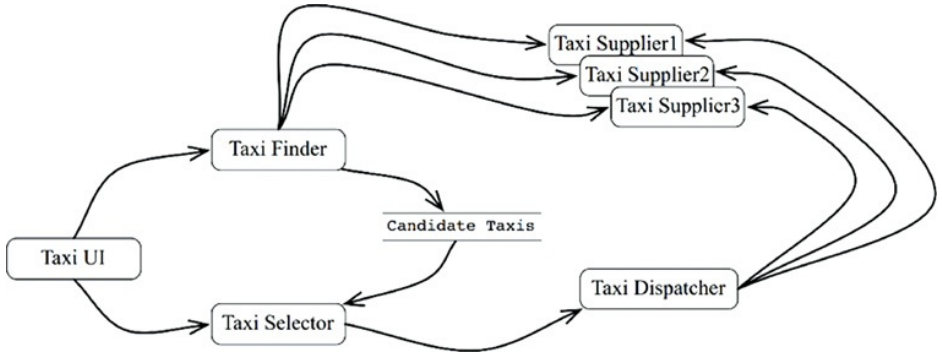
Để minh họa cho hai ngụy biện này, hãy xem lại hệ thống tổng hợp dịch vụ taxi. Hãy nhớ rằng hệ thống biết nhiều nhà cung cấp taxi trong một thành phố và cho phép khách hàng đặt xe. Giả sử khách hàng chọn taxi dựa trên nhiều tiêu chí, chẳng hạn thời gian đón, chi phí, mức độ sang trọng và kinh nghiệm của tài xế.

Chúng ta muốn hệ thống có thể mở rộng, nên chọn xây dựng nó từ nhiều vi dịch vụ nhỏ. Chúng ta chia đội ngũ phát triển thành nhiều nhóm nhỏ, mỗi nhóm chịu trách nhiệm phát triển, bảo trì và vận hành một số lượng nhỏ dịch vụ tương ứng.[1]

Hình 27.1 cho thấy cách các kiến trúc sư hư cấu sắp xếp dịch vụ để triển khai ứng dụng. Dịch vụ `TaxiUI` giao tiếp với khách hàng, những người dùng thiết bị di động để đặt taxi. Dịch vụ `TaxiFinder` kiểm tra danh sách xe của các nhà cung cấp `TaxiSupplier` khác nhau và xác định những taxi có thể phù hợp với người dùng. Nó đưa các xe này vào một bản ghi dữ liệu tạm thời gắn với người dùng đó. Dịch vụ `TaxiSelector` nhận các tiêu chí như chi phí, thời gian, mức độ sang trọng và những tiêu chí khác, rồi chọn taxi phù hợp trong số các ứng viên. Sau đó nó chuyển taxi đã chọn cho dịch vụ `TaxiDispatcher`, dịch vụ này sẽ đặt xe phù hợp.

---

1. Vì vậy, số lượng vi dịch vụ sẽ xấp xỉ số lượng lập trình viên.



Hình 27.1

Các dịch vụ được sắp xếp để triển khai hệ thống tổng hợp taxi

Giả sử hệ thống này đã hoạt động hơn một năm. Đội ngũ phát triển của chúng ta vẫn vui vẻ bổ sung tính năng mới, đồng thời bảo trì và vận hành mọi dịch vụ.

Một ngày đẹp trời, bộ phận tiếp thị họp với nhóm phát triển. Họ thông báo kế hoạch cung cấp dịch vụ giao mèo con trong thành phố. Người dùng có thể đặt giao mèo con đến nhà hoặc nơi làm việc.

Công ty sẽ mở một số điểm tập kết mèo con khắp thành phố. Khi có đơn đặt mèo con, một taxi gần đó sẽ được chọn để đón mèo tại một trong các điểm tập kết rồi giao đến địa chỉ phù hợp.

Một nhà cung cấp taxi đã đồng ý tham gia chương trình. Các nhà cung cấp khác có thể sẽ tham gia theo, còn một số có thể từ chối.

Dĩ nhiên, một số tài xế có thể dị ứng với mèo nên không bao giờ được chọn cho dịch vụ này. Một số khách hàng cũng chắc chắn bị dị ứng. Vì vậy, không được chọn xe từng giao mèo con trong ba ngày gần nhất cho khách hàng khai báo bị dị ứng.

Hãy nhìn sơ đồ dịch vụ. Có bao nhiêu dịch vụ phải thay đổi để triển khai tính năng này? Tất cả. Rõ ràng, việc phát triển và triển khai tính năng mèo con phải được phối hợp rất chặt chẽ.

Nói cách khác, các dịch vụ phụ thuộc lẫn nhau, không thể được phát triển, triển khai và bảo trì độc lập.

Đây là vấn đề của các mối quan tâm cắt ngang. Hệ thống phần mềm nào cũng phải đối mặt với vấn đề này, dù có hướng dịch vụ hay không. Các phân rã chức năng như trong sơ đồ dịch vụ ở Hình 27.1 rất dễ bị ảnh hưởng bởi những tính năng mới cắt ngang mọi hành vi chức năng.

Đối tượng giải cứu tình hình

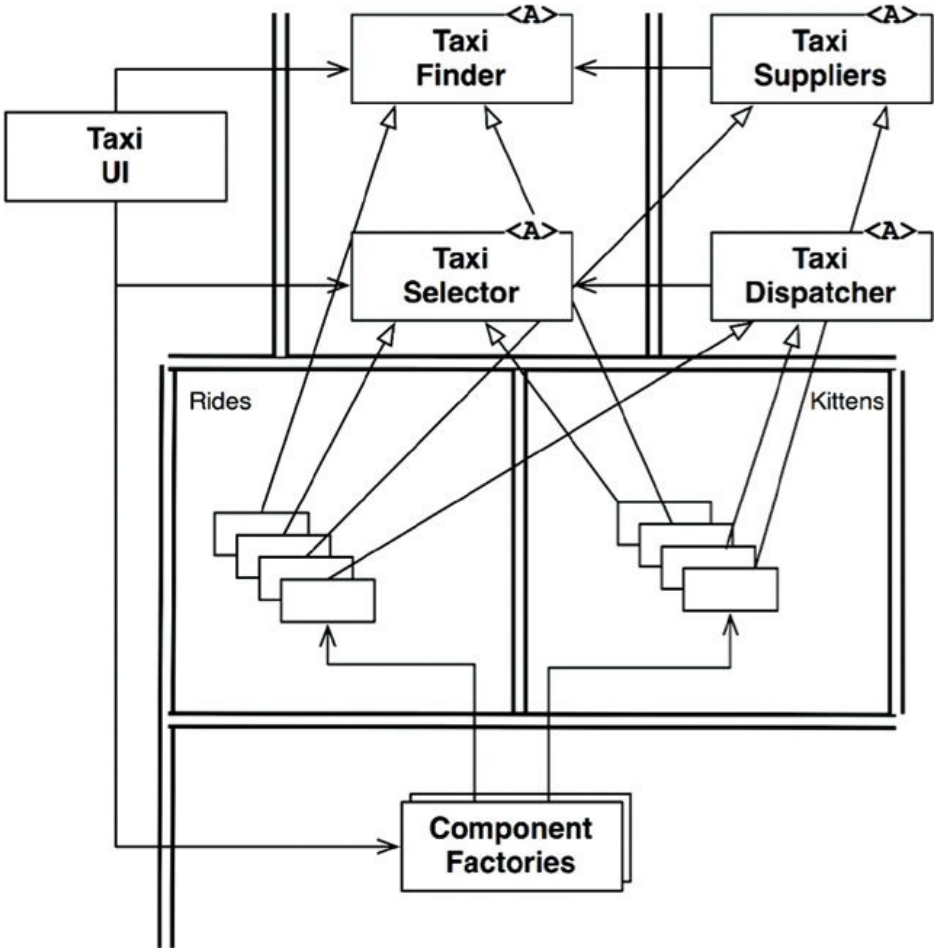
Làm thế nào để giải quyết vấn đề này trong kiến trúc dựa trên thành phần? Việc cân nhắc cẩn thận các nguyên tắc thiết kế SOLID sẽ gợi ý tạo một tập hợp lớp có thể được mở rộng đa hình để xử lý tính năng mới.

Hình 27.2 cho thấy chiến lược đó. Các lớp trong sơ đồ gần tương ứng với những dịch vụ ở Hình 27.1, nhưng hãy chú ý các ranh giới. Cũng cần lưu ý rằng các phụ thuộc tuân theo Quy tắc Phụ thuộc. Phần lớn logic của dịch vụ ban đầu vẫn nằm trong các lớp cơ sở của mô hình đối tượng. Tuy nhiên, phần logic dành riêng cho chuyến đi được tách vào thành phần `Rides`. Tính năng mới dành cho mèo con được đặt vào thành phần `Kittens`. Hai thành phần này ghi đè các lớp cơ sở trừu tượng trong những thành phần ban đầu bằng mẫu như Template Method hoặc Strategy.

Cũng cần lưu ý rằng hai thành phần mới, `Rides` và `Kittens`, tuân theo Quy tắc Phụ thuộc. Các lớp triển khai tính năng được tạo bởi các factory do giao diện người dùng kiểm soát.

Rõ ràng, khi triển khai tính năng mèo con theo cách này, `TaxiUI` phải thay đổi. Nhưng không cần thay đổi gì khác. Chỉ cần thêm một tệp jar, Gem hoặc DLL mới vào hệ thống rồi nạp động lúc chạy.

Như vậy, tính năng mèo con được tách rời và có thể được phát triển, triển khai độc lập.



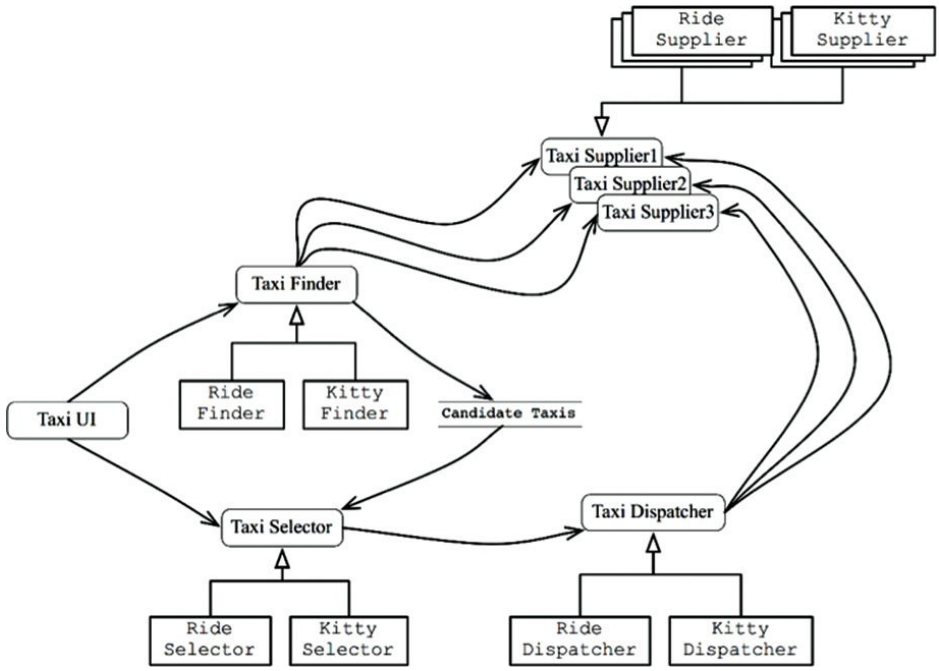
Hình 27.2
Dùng cách tiếp cận hướng đối tượng để xử lý các mối quan tâm cắt ngang

Dịch vụ dựa trên thành phần

Câu hỏi hiển nhiên là: có thể làm điều đó với dịch vụ không? Tất nhiên là có. Dịch vụ không nhất thiết phải là các khối nguyên khối nhỏ. Thay vào đó, có thể thiết kế dịch vụ theo các nguyên tắc SOLID và tạo cấu trúc thành phần để thêm thành phần mới mà không phải thay đổi các thành phần hiện có trong dịch vụ.

Hãy xem một dịch vụ Java như tập hợp các lớp trừu tượng trong một hoặc nhiều tệp jar. Mỗi tính năng mới hoặc phần mở rộng của tính năng có thể là một tệp jar khác, chứa các lớp mở rộng những lớp trừu tượng trong các tệp jar ban đầu. Khi đó, triển khai tính năng mới không còn đòi hỏi triển khai lại dịch vụ. Chỉ cần thêm tệp jar mới vào đường dẫn nạp của dịch vụ. Nói cách khác, việc thêm tính năng mới tuân theo Nguyên tắc Đóng-Mở.

Sơ đồ dịch vụ ở Hình 27.3 cho thấy cấu trúc này. Các dịch vụ vẫn tồn tại như trước, nhưng mỗi dịch vụ có thiết kế thành phần nội bộ riêng, cho phép thêm tính năng mới bằng các lớp dẫn xuất mới. Những lớp dẫn xuất này nằm trong các thành phần riêng.



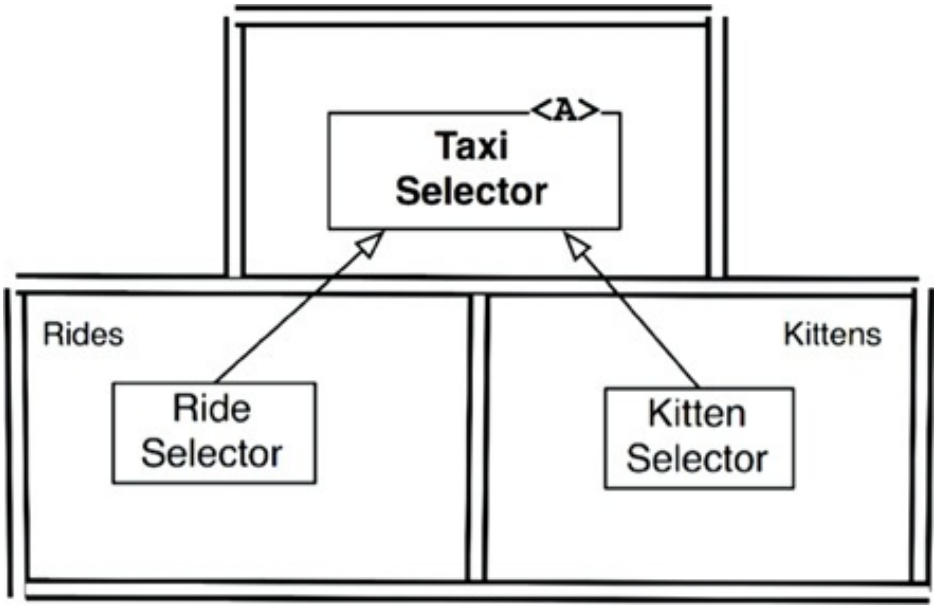
Hình 27.3

Mỗi dịch vụ có thiết kế thành phần nội bộ, cho phép thêm tính năng mới bằng các lớp dẫn xuất

Mối quan tâm cắt ngang

Điều chúng ta rút ra là ranh giới kiến trúc không nằm giữa các dịch vụ. Thay vào đó, chúng đi xuyên qua các dịch vụ và chia chúng thành các thành phần.

Để xử lý các mối quan tâm cắt ngang mà mọi hệ thống quan trọng đều gặp phải, dịch vụ phải được thiết kế với kiến trúc thành phần nội bộ tuân theo Quy tắc Phụ thuộc, như Hình 27.4 cho thấy. Dịch vụ không xác định ranh giới kiến trúc của hệ thống. Thay vào đó, các thành phần bên trong dịch vụ mới xác định chúng.



Hình 27.4

Dịch vụ cần có kiến trúc thành phần nội bộ tuân theo Quy tắc Phụ thuộc

Kết luận

Dịch vụ hữu ích cho khả năng mở rộng và khả năng phát triển của hệ thống, nhưng bản thân chúng không phải là yếu tố có ý nghĩa kiến trúc. Kiến trúc hệ thống được xác định bởi các ranh giới được vạch bên trong hệ thống và các phụ thuộc đi qua những ranh giới đó. Kiến trúc không được xác định bởi cơ chế vật lý mà các thành phần dùng để giao tiếp và thực thi.

Một dịch vụ có thể là một thành phần duy nhất, được bao quanh hoàn toàn bởi một ranh giới kiến trúc. Hoặc dịch vụ có thể gồm nhiều thành phần được ngăn cách bởi các ranh giới kiến trúc. Trong một số trường hợp hiếm hoi,[2] client và dịch vụ có thể phụ thuộc nhau đến mức hoàn toàn không còn ý nghĩa kiến trúc.

2. Chúng ta hy vọng những trường hợp đó hiếm. Thật không may, kinh nghiệm cho thấy điều ngược lại.

28

Ranh giới kiểm thử



Đúng vậy: kiểm thử là một phần của hệ thống và tham gia vào kiến trúc như mọi phần khác. Có khía cạnh mà sự tham gia này khá bình thường. Nhưng ở khía cạnh khác, nó có thể rất đặc thù.

Kiểm thử là thành phần của hệ thống

Có nhiều nhầm lẫn về kiểm thử. Chúng có thuộc hệ thống không? Chúng tách biệt khỏi hệ thống không? Có những loại kiểm thử nào? Kiểm thử đơn vị và kiểm thử tích hợp có khác nhau không? Còn kiểm thử chấp nhận, kiểm thử chức năng, kiểm thử Cucumber, kiểm thử TDD, kiểm thử BDD, kiểm thử thành phần, v.v. thì sao?

Cuốn sách này không nhằm sa vào cuộc tranh luận đó, và may mắn là cũng không cần thiết. Từ góc nhìn kiến trúc, mọi kiểm thử đều giống nhau. Dù là những kiểm thử nhỏ xíu do TDD tạo ra, hay các kiểm thử lớn bằng FitNesse, Cucumber, SpecFlow hoặc JBehave, chúng đều tương đương về mặt kiến trúc.

Bản chất của kiểm thử là tuân theo Quy tắc Phụ thuộc. Chúng rất chi tiết và cụ thể, đồng thời luôn phụ thuộc vào trong, về phía mã được kiểm thử. Thực tế, bạn có thể xem kiểm thử là vòng tròn ngoài cùng của kiến trúc. Không thành phần nào trong hệ thống phụ thuộc vào kiểm thử, còn kiểm thử luôn phụ thuộc vào trong, vào các thành phần của hệ thống.

Kiểm thử cũng có thể được triển khai độc lập. Thực tế, phần lớn thời gian chúng được triển khai trong các hệ thống kiểm thử chứ không phải hệ thống production. Vì vậy, ngay cả trong hệ thống không cần triển khai độc lập ở nơi khác, kiểm thử vẫn được triển khai độc lập.

Kiểm thử là thành phần bị cô lập nhất trong hệ thống. Chúng không cần thiết để hệ thống vận hành. Không người dùng nào phụ thuộc vào chúng. Vai trò của chúng là hỗ trợ phát triển, không phải vận hành. Dù vậy, chúng vẫn là thành phần của hệ thống như mọi thành phần khác. Thực tế, ở nhiều khía cạnh, chúng là hình mẫu mà các thành phần khác nên noi theo.

Thiết kế để dễ kiểm thử

Kiểm thử bị cô lập rất mạnh và thường không được triển khai cùng hệ thống. Điều này khiến các nhà phát triển nghĩ rằng kiểm thử nằm ngoài thiết kế hệ thống. Đây là một quan điểm tai hại. Kiểm thử không được tích hợp tốt vào thiết kế thường trở nên mong manh, đồng thời khiến hệ thống cứng nhắc và khó thay đổi.

Vấn đề nằm ở sự phụ thuộc. Kiểm thử phụ thuộc chặt chẽ vào hệ thống thì phải thay đổi cùng hệ thống. Ngay cả thay đổi nhỏ nhất trong một thành phần hệ thống cũng có thể làm nhiều kiểm thử phụ thuộc bị hỏng hoặc buộc phải sửa.

Tình trạng này có thể trở nên nghiêm trọng. Thay đổi ở một thành phần hệ thống dùng chung có thể làm hỏng hàng trăm, thậm chí hàng nghìn kiểm thử. Đây được gọi là Vấn đề Kiểm thử Mong manh (Fragile Tests Problem).

Không khó để thấy điều này xảy ra như thế nào. Chẳng hạn, hãy tưởng tượng một bộ kiểm thử dùng giao diện đồ họa để xác minh quy tắc nghiệp vụ. Kiểm thử có thể bắt đầu từ màn hình đăng nhập rồi điều hướng qua cấu trúc trang cho đến khi kiểm tra được một quy tắc nghiệp vụ cụ thể. Bất kỳ thay đổi nào ở trang đăng nhập hoặc cấu trúc điều hướng đều có thể làm hỏng vô số kiểm thử.

Kiểm thử mong manh thường khiến hệ thống trở nên cứng nhắc một cách trái ngược

với mục đích ban đầu. Khi nhận ra rằng thay đổi đơn giản trong hệ thống có thể gây ra hàng loạt lỗi kiểm thử, nhà phát triển có thể ngần ngại thực hiện thay đổi. Chẳng hạn, hãy tưởng tượng cuộc trao đổi giữa nhóm phát triển và nhóm tiếp thị, khi nhóm tiếp thị yêu cầu thay đổi đơn giản trong cấu trúc điều hướng khiến 1.000 kiểm thử bị hỏng.

Giải pháp là thiết kế để dễ kiểm thử. Quy tắc đầu tiên của thiết kế phần mềm, dù vì khả năng kiểm thử hay vì lý do nào khác, vẫn luôn như nhau: **Đừng phụ thuộc vào những thứ hay biến động.** Giao diện đồ họa hay biến động. Bộ kiểm thử thao tác hệ thống qua giao diện đồ họa sẽ mong manh. Vì vậy, hãy thiết kế hệ thống và kiểm thử sao cho có thể kiểm tra quy tắc nghiệp vụ mà không cần dùng giao diện đồ họa.

API kiểm thử

Cách đạt được mục tiêu này là tạo một API riêng để kiểm thử mọi quy tắc nghiệp vụ. API này cần có quyền hạn đặc biệt, cho phép kiểm thử vượt qua giới hạn bảo mật, bỏ qua tài nguyên tốn kém như cơ sở dữ liệu, và đưa hệ thống vào các trạng thái cụ thể để kiểm thử. API này sẽ là tập hợp mở rộng của các interactor và bộ chuyển đổi giao diện được giao diện người dùng sử dụng.

Mục đích của API kiểm thử là tách kiểm thử khỏi ứng dụng. Sự tách rời này không chỉ có nghĩa là bỏ phụ thuộc vào giao diện người dùng. Mục tiêu là tách cấu trúc của bộ kiểm thử khỏi cấu trúc của ứng dụng.

Phụ thuộc cấu trúc

Phụ thuộc cấu trúc là một trong những dạng phụ thuộc kiểm thử mạnh nhất và khó nhận biết nhất. Hãy tưởng tượng một bộ kiểm thử có một lớp kiểm thử cho từng lớp production và một tập phương thức kiểm thử cho từng phương thức production. Bộ kiểm thử như vậy phụ thuộc sâu sắc vào cấu trúc ứng dụng.

Khi một phương thức hoặc lớp production thay đổi, nhiều kiểm thử cũng phải thay đổi. Do đó, kiểm thử trở nên mong manh và khiến mã production cứng nhắc.

Vai trò của API kiểm thử là che giấu cấu trúc ứng dụng khỏi các kiểm thử. Điều này cho phép tái cấu trúc và phát triển mã production mà không ảnh hưởng đến kiểm thử. Nó cũng cho phép tái cấu trúc và phát triển kiểm thử mà không ảnh hưởng đến mã production.

Sự tách biệt trong quá trình phát triển này rất cần thiết vì theo thời gian, kiểm thử thường ngày càng cụ thể và chi tiết. Ngược lại, mã production ngày càng trừu tượng và tổng quát. Phụ thuộc cấu trúc chặt chẽ ngăn cản, hoặc ít nhất cản trở, sự phát triển cần thiết này và khiến mã production kém tổng quát, kém linh hoạt hơn mức có thể.

Bảo mật

Các quyền hạn đặc biệt của API kiểm thử có thể nguy hiểm nếu được triển khai trong hệ thống production. Nếu đây là mối lo ngại, hãy giữ API kiểm thử và những phần triển khai nguy hiểm của nó trong một thành phần riêng, có thể triển khai độc lập.

Kết luận

Kiểm thử không nằm ngoài hệ thống. Chúng là một phần của hệ thống và cần được thiết kế tốt để đem lại lợi ích về tính ổn định và hồi quy. Kiểm thử không được thiết kế như một phần của hệ thống thường trở nên mong manh và khó bảo trì. Cuối cùng, chúng có thể bị bỏ xó trong phòng bảo trì vì quá khó duy trì.

29

Kiến trúc nhúng sạch



James Grenning

Một thời gian trước, tôi đọc bài viết "The Growing Importance of Sustaining Software for the DoD"[1] trên blog của Doug Schmidt. Doug đưa ra nhận định sau:

> *"Phần mềm không hao mòn, nhưng firmware và phần cứng sẽ lỗi thời, khiến phần mềm phải được sửa đổi."*

Đó là khoảnh khắc giúp tôi hiểu rõ vấn đề. Doug nhắc đến hai thuật ngữ mà tôi tưởng là hiển nhiên, nhưng có lẽ không phải vậy. Phần mềm có thể hữu ích trong thời gian dài, còn

1. `https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-thedod.html`

firmware sẽ lỗi thời khi phần cứng phát triển. Nếu từng tham gia phát triển hệ thống nhúng, bạn biết phần cứng luôn được cải tiến. Đồng thời, các tính năng mới được thêm vào "phần mềm", khiến nó ngày càng phức tạp.

Tôi muốn bổ sung vào nhận định của Doug:

> *Dù phần mềm không hao mòn, các phụ thuộc không được quản lý vào firmware và phần cứng có thể phá hủy nó từ bên trong.*

Phần mềm nhúng không hiếm khi bị tước mất khả năng tồn tại lâu dài vì bị nhiễm các phụ thuộc vào phần cứng.

Tôi thích định nghĩa firmware của Doug, nhưng hãy xem còn định nghĩa nào khác. Tôi tìm thấy những cách diễn đạt sau:

- "Firmware được lưu trong các thiết bị nhớ không mất dữ liệu như ROM, EPROM hoặc bộ nhớ flash." (`https://en.wikipedia.org/wiki/Firmware`)
- "Firmware là một chương trình phần mềm hoặc tập hợp chỉ thị được lập trình vào một thiết bị phần cứng." (`https://techterms.com/definition/firmware`)
- "Firmware là phần mềm được nhúng trong một thiết bị phần cứng." (`https://www.lifewire.com/what-is-firmware-2625881`)
- Firmware là "phần mềm, gồm chương trình hoặc dữ liệu, được ghi vào bộ nhớ chỉ đọc (ROM)." (`http://www.webopedia.com/TERM/F/firmware.html`)

Nhận định của Doug khiến tôi nhận ra rằng các định nghĩa firmware được chấp nhận này sai, hoặc ít nhất đã lỗi thời. Firmware không có nghĩa là mã nằm trong ROM. Mã không phải firmware vì nơi lưu trữ của nó, mà vì nó phụ thuộc vào điều gì và khó thay đổi ra sao khi phần cứng phát triển. Phần cứng thực sự phát triển, chỉ cần dừng lại nhìn chiếc điện thoại của bạn là thấy. Vì vậy, ta nên cấu trúc mã nhúng dựa trên thực tế đó.

Tôi không có gì phản đối firmware hay các kỹ sư firmware. Chính tôi cũng từng viết firmware. Nhưng điều chúng ta thực sự cần là ít firmware hơn và nhiều phần mềm hơn. Thực ra, tôi thất vọng khi thấy các kỹ sư firmware viết quá nhiều firmware!

Các kỹ sư không làm hệ thống nhúng cũng viết firmware! Lập trình viên không chuyên nhúng vẫn viết firmware mỗi khi chôn câu lệnh SQL vào mã hoặc rải phụ thuộc nền tảng khắp nơi. Lập trình viên ứng dụng Android viết firmware khi không tách logic nghiệp vụ khỏi API Android.

Tôi từng tham gia nhiều dự án mà ranh giới giữa mã sản phẩm, tức phần mềm, và mã giao tiếp với phần cứng sản phẩm, tức firmware, mờ đến mức gần như không tồn tại. Chẳng hạn, cuối những năm 1990, tôi tham gia thiết kế lại một hệ thống truyền thông đang chuyển từ ghép kênh phân chia theo thời gian (TDM) sang thoại qua IP (VOIP). VOIP là cách làm phổ biến hiện nay, còn TDM từng được xem là công nghệ tiên tiến trong những năm 1950 và 1960, rồi được triển khai rộng rãi trong thập niên 1980 và 1990.

Mỗi khi hỏi kỹ sư hệ thống một cuộc gọi nên phản ứng thế nào trong một tình huống cụ thể, anh ấy lại biến mất rồi lát sau trở ra với câu trả lời rất chi tiết. "Anh lấy câu trả lời đó ở đâu?" chúng tôi hỏi. "Từ mã của sản phẩm hiện tại," anh ấy đáp. Mã cũ rối rắm đã trở thành bản đặc tả cho sản phẩm mới! Phần triển khai hiện tại không hề tách TDM khỏi

logic nghiệp vụ xử lý cuộc gọi. Toàn bộ sản phẩm phụ thuộc vào phần cứng và công nghệ từ đầu đến cuối, không thể tháo gỡ. Về cơ bản, toàn bộ sản phẩm đã trở thành firmware.

Xét một ví dụ khác. Thông điệp lệnh được gửi đến hệ thống qua cổng nối tiếp. Dĩ nhiên, có một bộ xử lý và điều phối thông điệp. Nó biết định dạng thông điệp, có thể phân tích cú pháp, rồi chuyển thông điệp đến mã xử lý yêu cầu. Không có gì bất ngờ, ngoại trừ việc bộ xử lý và điều phối thông điệp nằm cùng tệp với mã giao tiếp với phần cứng UART.[2] Bộ xử lý thông điệp bị lẫn chi tiết UART. Nó vốn có thể là phần mềm hữu ích lâu dài, nhưng thay vào đó lại trở thành firmware. Bộ xử lý thông điệp mất cơ hội trở thành phần mềm, điều đó thật không ổn!

Tôi đã hiểu từ lâu rằng cần tách phần mềm khỏi phần cứng, nhưng lời Doug giúp tôi làm rõ cách dùng hai thuật ngữ phần mềm và firmware khi nói về mối quan hệ giữa chúng.

Thông điệp dành cho kỹ sư và lập trình viên rất rõ: hãy ngừng viết quá nhiều firmware và cho mã của bạn cơ hội hữu ích lâu dài. Dĩ nhiên, chỉ yêu cầu như vậy không khiến nó xảy ra. Hãy xem cách giữ kiến trúc phần mềm nhúng sạch để phần mềm có cơ hội tồn tại lâu dài và hữu ích.

Bài kiểm tra App-titude

Vì sao phần lớn phần mềm nhúng có tiềm năng lại trở thành firmware? Có vẻ như phần lớn nỗ lực tập trung vào việc làm cho mã nhúng hoạt động, chứ không chú trọng cấu trúc để nó hữu ích lâu dài. Kent Beck mô tả ba hoạt động khi xây dựng phần mềm. Phần trích dẫn là lời của Kent, còn chữ in nghiêng là bình luận của tôi:

1. “Trước tiên, làm cho nó hoạt động.” Nếu không chạy được thì bạn hết cơ hội.
2. “Sau đó, làm cho nó đúng.” Tái cấu trúc mã để bạn và người khác hiểu được, đồng thời có thể phát triển nó khi nhu cầu thay đổi hoặc được hiểu rõ hơn.
3. “Cuối cùng, làm cho nó nhanh.” Tái cấu trúc mã để đạt hiệu năng “cần thiết”.

Phần lớn phần mềm nhúng tôi thấy ngoài thực tế dường như chỉ được viết với mục tiêu “Làm cho nó hoạt động”, và có lẽ còn bị ám ảnh bởi mục tiêu “Làm cho nó nhanh”, đạt được bằng cách bổ sung tối ưu hóa vi mô ở mọi cơ hội. Trong *The Mythical Man-Month*, Fred Brooks khuyên chúng ta “hãy dự tính sẽ vứt bỏ một bản”. Kent và Fred gần như đưa ra cùng một lời khuyên: tìm hiểu điều gì hiệu quả rồi tạo ra giải pháp tốt hơn.

Phần mềm nhúng cũng không đặc biệt hơn các phần mềm khác khi gặp những vấn đề này. Phần lớn ứng dụng không nhúng chỉ được xây dựng để chạy, mà ít chú ý đến việc làm cho mã đúng đắn để dùng lâu dài.

Tôi gọi việc làm cho ứng dụng hoạt động là bài kiểm tra App-titude dành cho lập trình viên. Lập trình viên, dù làm hệ thống nhúng hay không, nếu chỉ quan tâm đến việc ứng dụng chạy được thì đang gây bất lợi cho sản phẩm và người sử dụng lao động. Lập trình còn nhiều hơn việc làm cho ứng dụng chạy được.

Ví dụ về mã được tạo ra khi chỉ vượt qua bài kiểm tra App-titude là các hàm sau, nằm

2. Thiết bị phần cứng điều khiển cổng nối tiếp.

chung trong một tệp của hệ thống nhúng nhỏ:

```
ISR(TIMER1_vect) { ... }
ISR(INT2_vect) { ... }
void btn_Handler(void) { ... }
float calc_RPM(void) { ... }
static char Read_RawData(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
void Dev_Control(char Activation) { ... }
char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }
void Sensor_init(void) { ... }
void uC_Sleep(void) { ... }
```

Đây là thứ tự các hàm tôi tìm thấy trong mã nguồn. Bây giờ tôi sẽ tách chúng ra và nhóm theo mối quan tâm:

- **Các hàm chứa logic miền:**

```
float calc_RPM(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
```

- **Các hàm thiết lập nền tảng phần cứng:**

```
ISR(TIMER1_vect) { ... }*
ISR(INT2_vect) { ... }
void uC_Sleep(void) { ... }
```

- **Các hàm phản ứng khi nhấn nút bật/tắt:**

```
void btn_Handler(void) { ... }
void Dev_Control(char Activation) { ... }
```

- **Hàm đọc đầu vào A/D từ phần cứng:**

```
static char Read_RawData(void) { ... }
```

- **Các hàm lưu giá trị vào bộ nhớ bền vững:**

```
char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
float bytes2float(char bytes[4]) { ... }
```

```
void Recall_DataSet(void) { ... }
```

- **Hàm không làm điều mà tên của nó ngụ ý:**

```
void Sensor_init(void) { ... }
```

Khi xem các tệp khác trong ứng dụng này, tôi thấy nhiều thứ cản trở việc hiểu mã. Tôi cũng thấy cấu trúc tệp ngụ ý rằng cách duy nhất để kiểm thử mã là trên thiết bị nhúng. Hầu như mọi đoạn mã đều biết rằng nó chạy trên một kiến trúc vi xử lý đặc biệt, dùng các cấu trúc C "mở rộng"[3] gắn mã với một chuỗi công cụ và vi xử lý cụ thể. Mã trông như C nhưng không còn là C nữa. Mã này không thể hữu ích lâu dài nếu sản phẩm không bao giờ cần chuyển sang môi trường phần cứng khác.

Ứng dụng này hoạt động, người kỹ sư đã vượt qua bài kiểm tra App-titude. Nhưng không thể gọi đây là kiến trúc nhúng sạch.

Nút thắt phần cứng đích

Có nhiều vấn đề đặc thù mà nhà phát triển hệ thống nhúng phải xử lý, còn người không làm nhúng thì không, chẳng hạn bộ nhớ hạn chế, ràng buộc thời gian thực và hạn chót, IO hạn chế, giao diện người dùng khác thường, cảm biến và kết nối với thế giới thực. Phần cứng thường được phát triển đồng thời với phần mềm và firmware. Khi viết mã cho loại hệ thống này, có thể bạn không có nơi nào để chạy mã. Nếu chưa đủ tệ, khi nhận được phần cứng, nhiều khả năng nó còn có lỗi riêng, khiến tiến độ phát triển phần mềm chậm hơn bình thường.

Đúng là hệ thống nhúng có những điểm đặc thù. Kỹ sư nhúng cũng có chuyên môn đặc thù. Nhưng phát triển nhúng không đặc biệt đến mức các nguyên tắc trong cuốn sách này không áp dụng được.

Một vấn đề riêng của hệ thống nhúng là nút thắt phần cứng đích. Nếu mã nhúng được cấu trúc mà không áp dụng các nguyên tắc và thực hành của Kiến trúc Sạch, bạn thường chỉ có thể kiểm thử mã trên thiết bị đích. Khi đó, thiết bị đích trở thành nút thắt làm chậm công việc.

Kiến trúc nhúng sạch có thể kiểm thử

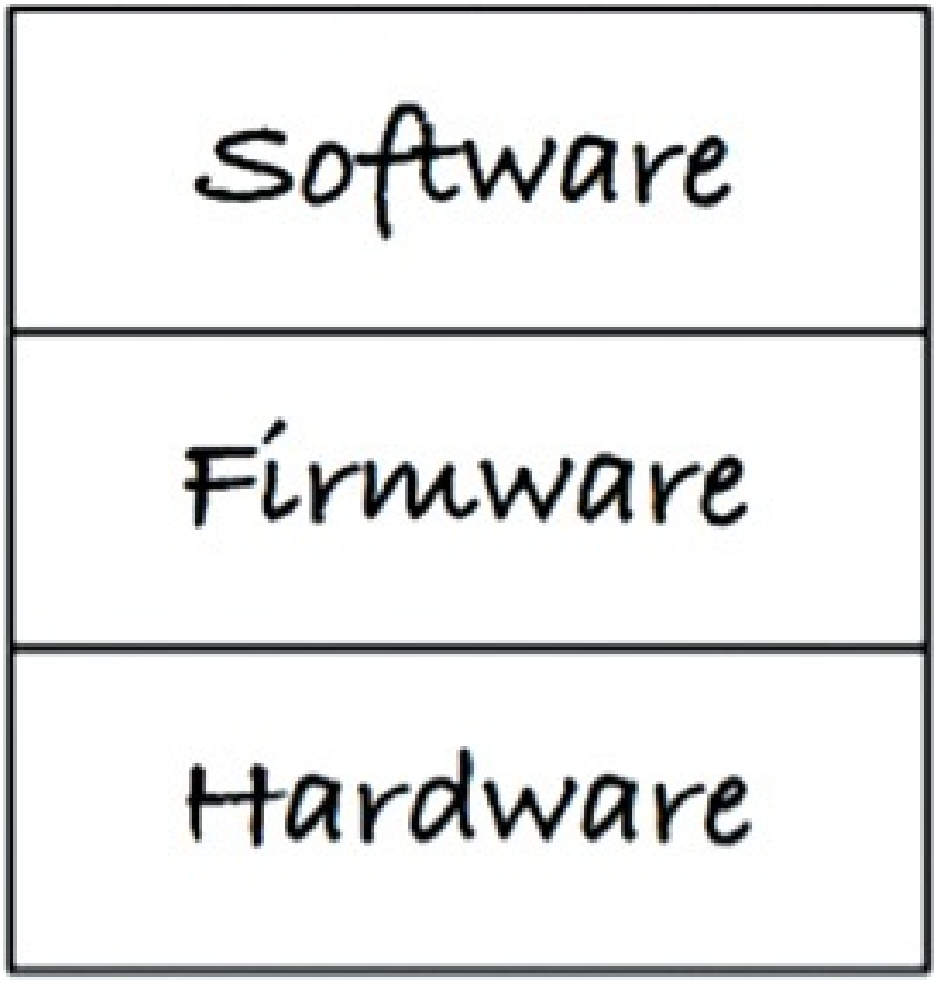
Hãy xem cách áp dụng một số nguyên tắc kiến trúc vào phần mềm và firmware nhúng để loại bỏ nút thắt phần cứng đích.

---

3. Một số nhà cung cấp chip thêm từ khóa vào ngôn ngữ C để dễ truy cập thanh ghi và cổng IO. Tuy nhiên, một khi làm vậy, mã không còn là C nữa.

Các lớp

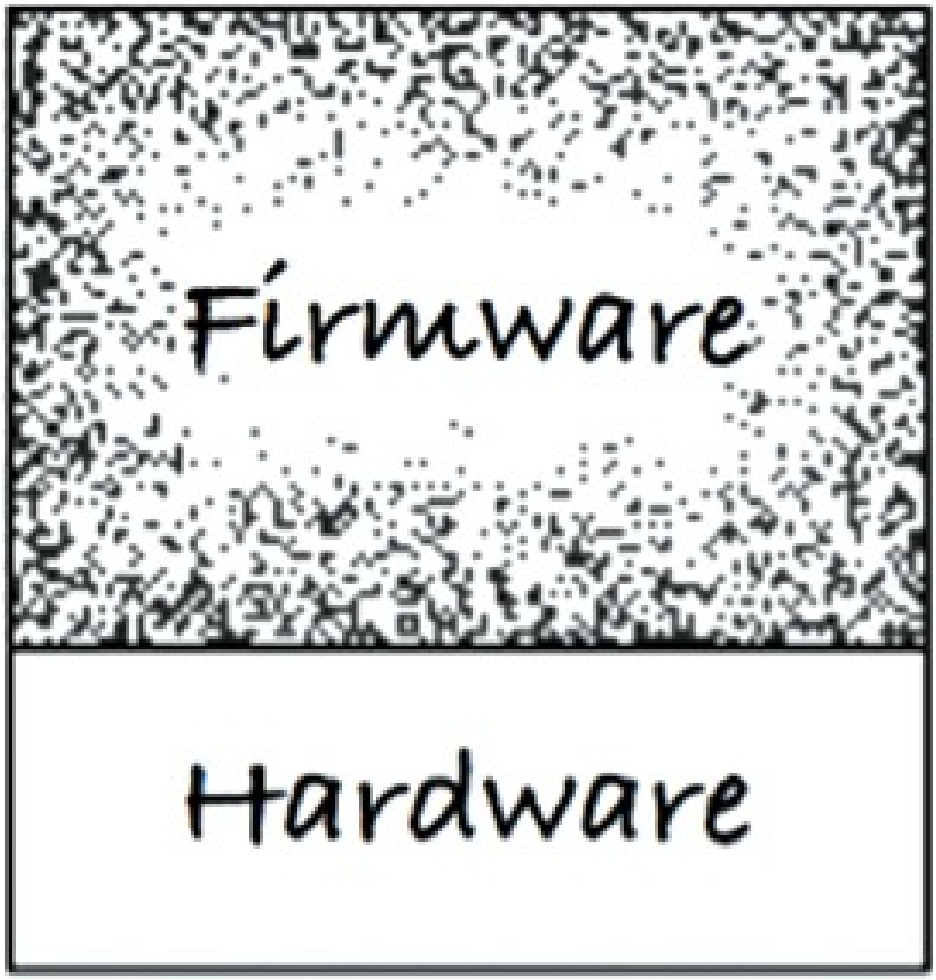
Có nhiều cách phân lớp. Hãy bắt đầu với ba lớp như trong Hình 29.1. Phần dưới cùng là phần cứng. Như Doug cảnh báo, do tiến bộ công nghệ và Định luật Moore, phần cứng sẽ thay đổi. Linh kiện sẽ lỗi thời, còn linh kiện mới sẽ tiêu thụ ít điện hơn, hoạt động tốt hơn hoặc rẻ hơn. Dù lý do là gì, với tư cách kỹ sư nhúng, tôi không muốn phải làm nhiều việc hơn mức cần thiết khi thay đổi phần cứng không thể tránh khỏi.



Hình 29.1
Ba lớp

Việc tách phần cứng khỏi phần còn lại của hệ thống là điều hiển nhiên, ít nhất sau khi phần cứng đã được xác định (Hình 29.2). Đây là nơi thường bắt đầu vấn đề khi cố vượt qua bài

kiểm tra App-titude. Không có gì ngăn hiểu biết về phần cứng làm ô nhiễm toàn bộ mã. Nếu không cẩn thận khi sắp xếp mã và quy định điều một mô-đun được phép biết về mô-đun khác, mã sẽ rất khó thay đổi. Tôi không chỉ nói đến trường hợp phần cứng thay đổi, mà cả khi người dùng yêu cầu thay đổi hoặc cần sửa lỗi.



Hình 29.2
Phần cứng phải được tách khỏi phần còn lại của hệ thống

Trộn lẫn phần mềm và firmware là một phản mẫu. Mã theo phản mẫu này chống lại thay đổi. Hơn nữa, thay đổi trở nên nguy hiểm và thường dẫn đến hậu quả ngoài ý muốn. Ngay cả thay đổi nhỏ cũng có thể đòi hỏi kiểm thử hồi quy toàn hệ thống. Nếu chưa tạo kiểm thử có thiết bị đo và quan sát bên ngoài, hãy chuẩn bị tinh thần chán ngấy các kiểm thử thủ

công, rồi chờ báo cáo lỗi mới.

Phần cứng là chi tiết

Ranh giới giữa phần mềm và firmware thường không rõ ràng bằng ranh giới giữa mã và phần cứng, như Hình 29.3 cho thấy.

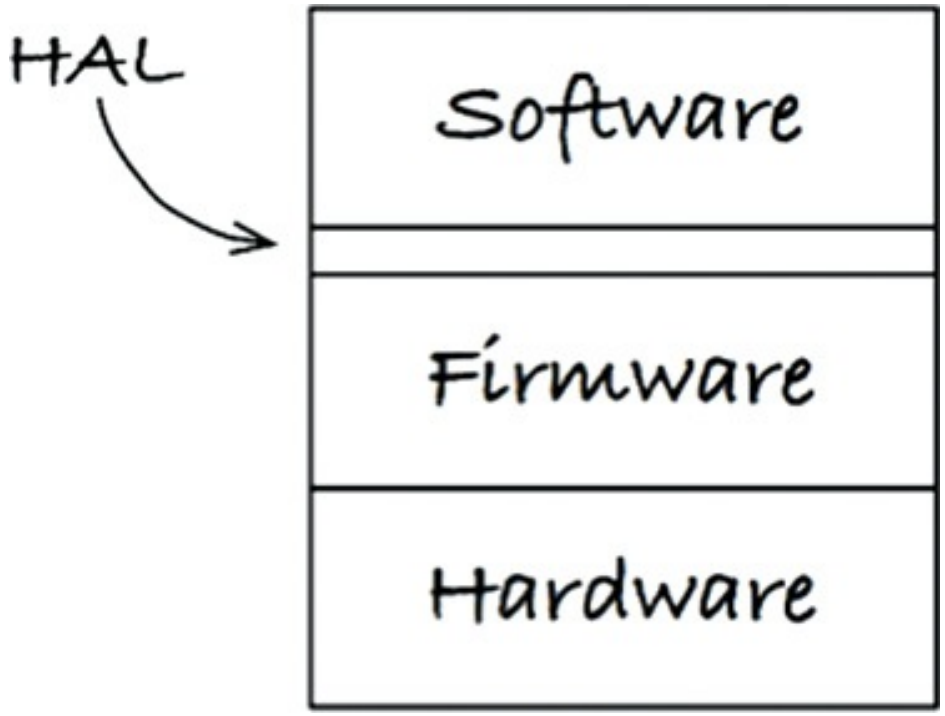


Hình 29.3

Ranh giới giữa phần mềm và firmware mờ hơn ranh giới giữa mã và phần cứng

Một trong những nhiệm vụ của nhà phát triển phần mềm nhúng là làm rõ ranh giới đó. Ranh giới giữa phần mềm và firmware gọi là lớp trừu tượng phần cứng (HAL), như Hình 29.4 cho thấy. Đây không phải ý tưởng mới. HAL đã có trong máy tính cá nhân từ trước thời

Windows.



Hình 29.4
Lớp trừu tượng phần cứng

HAL phục vụ phần mềm nằm bên trên nó, và API của HAL nên được thiết kế theo nhu cầu của phần mềm đó. Chẳng hạn, firmware có thể lưu byte và mảng byte vào bộ nhớ flash. Ngược lại, ứng dụng cần lưu và đọc các cặp tên/giá trị từ một cơ chế lưu trữ nào đó. Phần mềm không nên quan tâm các cặp tên/giá trị được lưu trong bộ nhớ flash, đĩa quay, đám mây hay bộ nhớ lõi. HAL cung cấp dịch vụ mà không tiết lộ cách thực hiện. Cách triển khai bằng flash là chi tiết cần được giấu khỏi phần mềm.

Ví dụ khác, đèn LED được nối với một bit GPIO. Firmware có thể cho phép truy cập trực tiếp các bit GPIO, trong khi HAL cung cấp hàm `Led_TurnOn(5)`. Đây là lớp trừu tượng phần cứng khá thấp. Hãy xem xét việc nâng mức trừu tượng từ góc nhìn phần cứng lên góc nhìn phần mềm và sản phẩm. Đèn LED biểu thị điều gì? Giả sử nó báo pin yếu. Ở một mức nào đó, firmware hoặc gói hỗ trợ bo mạch có thể cung cấp `Led_TurnOn(5)`, còn HAL cung cấp `Indicate_LowBattery()`. Có thể thấy HAL biểu đạt các dịch vụ mà ứng dụng cần. Cũng có thể thấy các lớp có thể chứa những lớp khác. Đây giống một mô hình fractal lặp lại hơn là một tập lớp được định nghĩa cố định. Các ánh xạ GPIO là chi tiết cần được giấu khỏi phần mềm.

Không để lộ chi tiết phần cứng cho người dùng HAL

Phần mềm trong kiến trúc nhúng sạch có thể được kiểm thử ngoài phần cứng đích. HAL thành công cung cấp đường nối hoặc tập điểm thay thế giúp kiểm thử ngoài thiết bị đích.

Bộ xử lý là chi tiết

Khi ứng dụng nhúng dùng chuỗi công cụ chuyên biệt, nó thường cung cấp các tệp tiêu đề để *giúp bạn*.[4] Những trình biên dịch này thường tự ý mở rộng ngôn ngữ C, thêm từ khóa mới để truy cập tính năng của bộ xử lý. Mã trông như C nhưng không còn là C.

Đôi khi, trình biên dịch C do nhà cung cấp đưa ra có những biến trông như biến toàn cục để truy cập trực tiếp thanh ghi bộ xử lý, cổng IO, bộ định thời xung nhịp, bit IO, bộ điều khiển ngắt và các chức năng khác của bộ xử lý. Truy cập dễ dàng vào những thứ này rất hữu ích, nhưng hãy nhớ rằng mã nào sử dụng các tiện ích này sẽ không còn là C nữa. Nó không thể biên dịch cho bộ xử lý khác, thậm chí có thể không biên dịch được với trình biên dịch khác dành cho cùng bộ xử lý.

Tôi không muốn nghĩ rằng nhà cung cấp chip hoặc công cụ đang cố ý ràng buộc sản phẩm của bạn với trình biên dịch. Hãy tin rằng họ thực sự muốn giúp. Nhưng giờ đến lượt bạn dùng sự trợ giúp đó sao cho không gây hại trong tương lai. Bạn phải giới hạn những tệp được phép biết về phần mở rộng của C.

Hãy xem tệp tiêu đề được thiết kế cho họ DSP ACME, đúng loại mà Wile E. Coyote sử dụng:

```
#ifndef _ACME_STD_TYPES
#define _ACME_STD_TYPES

#if defined(_ACME_X42)
typedef unsigned int Uint_32;
typedef unsigned short Uint_16;
typedef unsigned char Uint_8;

typedef int Int_32;
typedef short Int_16;
typedef char Int_8;

#elif defined(_ACME_A42)
typedef unsigned long Uint_32;
typedef unsigned int Uint_16;
typedef unsigned char Uint_8;

typedef long Int_32;
typedef int Int_16;
typedef char Int_8;
#else
#error <acmetypes.h> is not supported for this environment
#endif

#endif
```

4. Câu này cố ý dùng HTML.

Không nên dùng trực tiếp tệp tiêu đề `acmetypes.h`. Nếu làm vậy, mã của bạn sẽ bị ràng buộc với một trong các DSP ACME. Bạn nói mình đang dùng DSP ACME, vậy có vấn đề gì? Bạn không thể biên dịch mã nếu không đưa tệp tiêu đề này vào. Nếu dùng tệp đó và định nghĩa `_ACME_X42` hoặc `_ACME_A42`, các số nguyên sẽ sai kích thước khi kiểm thử ngoài thiết bị đích. Tệ hơn nữa, một ngày nào đó bạn sẽ muốn chuyển ứng dụng sang bộ xử lý khác. Việc không chọn khả năng chuyển đổi và không giới hạn những tệp biết về ACME sẽ khiến công việc đó khó hơn nhiều.

Thay vì dùng `acmetypes.h`, hãy theo hướng chuẩn hóa hơn và dùng `stdint.h`. Nhưng nếu trình biên dịch của thiết bị đích không cung cấp `stdint.h` thì sao? Bạn có thể tự viết tệp tiêu đề này. Phiên bản `stdint.h` dùng cho bản build đích sẽ dùng `acmetypes.h` như sau:

```
#ifndef _STDINT_H_
#define _STDINT_H_

#include <acmetypes.h>

typedef Uint_32 uint32_t;
typedef Uint_16 uint16_t;
typedef Uint_8 uint8_t;

typedef Int_32 int32_t;
typedef Int_16 int16_t;
typedef Int_8 int8_t;

#endif
```

Việc để phần mềm nhúng và firmware dùng `stdint.h` giúp mã sạch và dễ chuyển đổi. Tất nhiên, toàn bộ phần mềm nên độc lập với bộ xử lý, nhưng không phải firmware nào cũng có thể độc lập như vậy. Đoạn mã tiếp theo tận dụng các phần mở rộng đặc biệt của C để truy cập thiết bị ngoại vi trong vi điều khiển. Có khả năng sản phẩm của bạn dùng vi điều khiển này chính vì nó có các thiết bị ngoại vi tích hợp. Hàm sau gửi dòng chữ “hi” ra cổng nối tiếp. (Ví dụ này dựa trên mã thực tế.)

```
void say_hi()
{
IE = 0b11000000;
SBUF0 = (0x68);
while(TI_0 == 0);
TI_0 = 0;
SBUF0 = (0x69);
while(TI_0 == 0);
TI_0 = 0;
SBUF0 = (0x0a);
while(TI_0 == 0);
TI_0 = 0;
SBUF0 = (0x0d);
while(TI_0 == 0);
TI_0 = 0;
IE = 0b11010000;
}
```

Hàm nhỏ này có nhiều vấn đề. Một điều có thể khiến bạn chú ý là sự xuất hiện của `0b11000000`. Cú pháp nhị phân này khá hay, nhưng C có hỗ trợ không? Tiếc là không. Một số vấn đề khác liên quan trực tiếp đến các phần mở rộng C tùy chỉnh:

- `IE`: các bit cho phép ngắt.
- `SBUF0`: bộ đệm đầu ra nối tiếp.
- `TI_0`: ngắt báo bộ đệm truyền nối tiếp trống. Đọc giá trị 1 nghĩa là bộ đệm trống.

Các biến viết hoa thực sự truy cập thiết bị ngoại vi tích hợp trong vi điều khiển. Nếu muốn điều khiển ngắt và xuất ký tự, bạn phải dùng các thiết bị ngoại vi này. Đúng là tiện, nhưng đây không phải C.

Kiến trúc nhúng sạch chỉ dùng trực tiếp các thanh ghi truy cập thiết bị này ở một số ít nơi và giới hạn chúng hoàn toàn trong firmware. Mọi mã biết về các thanh ghi đó đều trở thành firmware, do đó bị ràng buộc với chip. Việc ràng buộc mã với bộ xử lý sẽ gây khó khăn khi bạn muốn làm cho mã chạy trước khi phần cứng ổn định. Nó cũng gây khó khăn khi chuyển ứng dụng nhúng sang bộ xử lý mới.

Nếu dùng vi điều khiển như vậy, firmware có thể cách ly các hàm cấp thấp này bằng một dạng lớp trừu tượng bộ xử lý (PAL). Firmware nằm phía trên PAL có thể được kiểm thử ngoài thiết bị đích, nhờ đó bớt "cứng".

Hệ điều hành là chi tiết

HAL là cần thiết, nhưng đã đủ chưa? Trong hệ thống nhúng chạy trực tiếp trên phần cứng, HAL có thể là tất cả những gì bạn cần để mã không quá phụ thuộc vào môi trường vận hành. Nhưng các hệ thống nhúng dùng hệ điều hành thời gian thực (RTOS), hoặc phiên bản nhúng của Linux hay Windows thì sao?

Muốn phần mềm nhúng có cơ hội hữu ích lâu dài, bạn phải xem hệ điều hành là chi tiết và bảo vệ phần mềm khỏi phụ thuộc vào hệ điều hành.

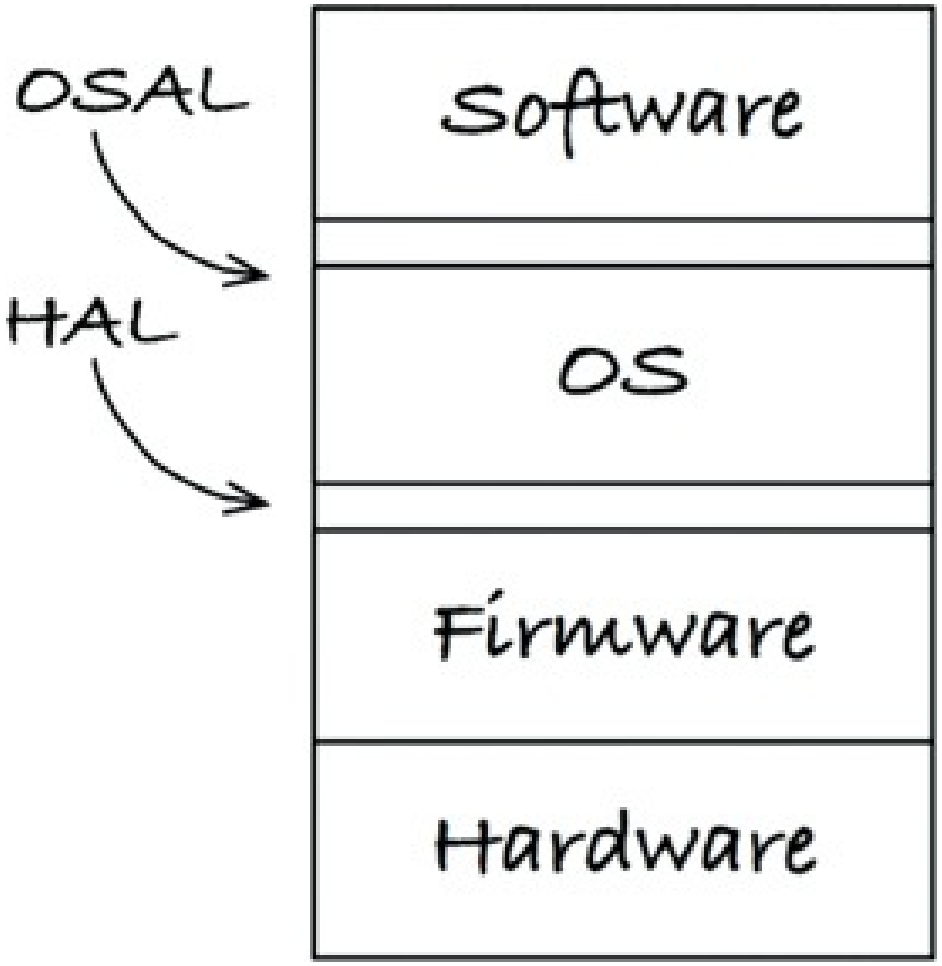
Phần mềm truy cập dịch vụ của môi trường vận hành thông qua hệ điều hành. Hệ điều hành là một lớp tách phần mềm khỏi firmware (Hình 29.5). Dùng trực tiếp hệ điều hành có thể gây vấn đề. Chẳng hạn, điều gì xảy ra nếu nhà cung cấp RTOS bị một công ty khác mua lại, phí bản quyền tăng hoặc chất lượng giảm? Điều gì xảy ra nếu nhu cầu thay đổi và RTOS không còn cung cấp khả năng bạn cần? Bạn sẽ phải thay đổi nhiều mã. Những thay đổi này không chỉ là chỉnh cú pháp cho phù hợp với API của hệ điều hành mới, mà còn có thể phải điều chỉnh ngữ nghĩa để phù hợp với khả năng và cơ chế khác của hệ điều hành đó.



Hình 29.5

Thêm hệ điều hành

Kiến trúc nhúng sạch cách ly phần mềm khỏi hệ điều hành thông qua lớp trừu tượng hệ điều hành (OSAL), như Hình 29.6 cho thấy. Trong một số trường hợp, triển khai lớp này có thể đơn giản như đổi tên một hàm. Trong trường hợp khác, nó có thể cần gói nhiều hàm lại với nhau.



Hình 29.6
Lớp trừu tượng hệ điều hành

Nếu từng chuyển phần mềm từ RTOS này sang RTOS khác, bạn biết việc đó rất đau đầu. Nếu phần mềm phụ thuộc vào OSAL thay vì phụ thuộc trực tiếp vào hệ điều hành, phần lớn công việc sẽ là viết OSAL mới tương thích với OSAL cũ. Bạn muốn làm việc nào hơn:

sửa một đống mã hiện có phức tạp, hay viết mã mới theo giao diện và hành vi đã xác định? Đây không phải câu hỏi đánh đố. Tôi chọn cách thứ hai.

Có thể đến đây bạn bắt đầu lo về mã phình to. Nhưng trên thực tế, lớp này trở thành nơi cách ly phần lớn mã trùng lặp liên quan đến việc dùng hệ điều hành. Sự trùng lặp này không nhất thiết gây nhiều chi phí phụ. Nếu định nghĩa OSAL, bạn cũng có thể khuyến khích ứng dụng có cấu trúc chung. Thay vì để mỗi luồng tự xây dựng mô hình đồng thời, bạn có thể cung cấp cơ chế truyền thông điệp.

OSAL có thể cung cấp các điểm kiểm thử để mã ứng dụng có giá trị trong lớp phần mềm được kiểm thử ngoài thiết bị và ngoài hệ điều hành. Phần mềm trong kiến trúc nhúng sạch có thể được kiểm thử mà không cần hệ điều hành đích. OSAL thành công cung cấp đường nối hoặc các điểm thay thế giúp kiểm thử ngoài thiết bị đích.

Lập trình theo giao diện và khả năng thay thế

Ngoài việc thêm HAL và có thể cả OSAL vào các lớp chính, gồm phần mềm, hệ điều hành, firmware và phần cứng, bạn có thể và nên áp dụng các nguyên tắc được mô tả trong suốt cuốn sách này. Những nguyên tắc đó khuyến khích phân tách mối quan tâm, lập trình theo giao diện và khả năng thay thế.

Ý tưởng kiến trúc phân lớp dựa trên việc lập trình theo giao diện. Khi một mô-đun tương tác với mô-đun khác thông qua giao diện, bạn có thể thay nhà cung cấp dịch vụ này bằng nhà cung cấp khác. Nhiều độc giả hẳn từng viết phiên bản `printf` nhỏ của riêng mình để triển khai trên thiết bị đích. Miễn là giao diện `printf` của bạn giống phiên bản chuẩn, bạn có thể thay dịch vụ này bằng dịch vụ kia.

Một quy tắc kinh nghiệm cơ bản là dùng tệp header để định nghĩa giao diện. Tuy nhiên, hãy cẩn thận với nội dung trong tệp header. Chỉ đưa vào đó các khai báo hàm, cùng những hằng số và tên struct mà các hàm cần.

Đừng làm lộn xộn tệp header giao diện bằng cấu trúc dữ liệu, hằng số và typedef chỉ cần cho phần triển khai. Đây không chỉ là vấn đề lộn xộn: những nội dung đó dẫn đến các phụ thuộc không mong muốn. Hãy giới hạn khả năng nhìn thấy chi tiết triển khai. Hãy dự kiến rằng các chi tiết đó sẽ thay đổi. Càng ít vị trí trong mã biết về các chi tiết này thì càng ít nơi cần tìm và sửa.

Kiến trúc nhúng sạch có thể được kiểm thử bên trong từng lớp vì các mô-đun tương tác thông qua giao diện. Mỗi giao diện tạo ra đường nối hoặc điểm thay thế giúp kiểm thử ngoài thiết bị đích.

Chỉ thị biên dịch có điều kiện DRY

Một cách áp dụng khả năng thay thế thường bị bỏ qua liên quan đến cách chương trình C và C++ nhúng xử lý các thiết bị đích hoặc hệ điều hành khác nhau. Người ta thường dùng biên dịch có điều kiện để bật hoặc tắt các đoạn mã. Tôi nhớ một trường hợp đặc biệt rắc rối, khi câu lệnh `#ifdef BOARD_V2` xuất hiện vài nghìn lần trong một ứng dụng viễn thông.

Việc lặp mã này vi phạm nguyên tắc Đừng Lặp Lại Chính Mình (DRY).[5] Nếu chỉ thấy `#ifdef BOARD_V2` một lần thì chưa thật sự là vấn đề. Sáu nghìn lần thì là vấn đề nghiêm trọng. Trong hệ thống nhúng, chỉ thị biên dịch có điều kiện xác định loại phần cứng đích thường bị lặp lại. Nhưng còn cách nào khác?

Nếu có lớp trừu tượng phần cứng thì sao? Loại phần cứng sẽ trở thành chi tiết được HAL che giấu. Nếu HAL cung cấp một tập giao diện, thay vì biên dịch có điều kiện, ta có thể dùng linker hoặc một hình thức liên kết lúc chạy để nối phần mềm với phần cứng.

Kết luận

Những người phát triển phần mềm nhúng có nhiều điều để học từ kinh nghiệm ngoài lĩnh vực phần mềm nhúng. Nếu bạn là nhà phát triển nhúng và đã cầm cuốn sách này lên đọc, bạn sẽ tìm thấy nhiều hiểu biết quý giá về phát triển phần mềm trong những lời nói và ý tưởng ở đây.

Để mọi mã trở thành firmware không tốt cho sức khỏe lâu dài của sản phẩm. Chỉ có thể kiểm thử trên phần cứng đích cũng không tốt cho sức khỏe lâu dài của sản phẩm. Kiến trúc nhúng sạch tốt cho sức khỏe lâu dài của sản phẩm.

---

5. Hunt và Thomas, *The Pragmatic Programmer*.

Phần VI: Chi tiết

30

Cơ sở dữ liệu chỉ là chi tiết



Từ góc nhìn kiến trúc, cơ sở dữ liệu không phải một thực thể kiến trúc. Nó chỉ là chi tiết, không đạt đến mức một yếu tố kiến trúc. Quan hệ của nó với kiến trúc hệ thống phần mềm cũng giống như quan hệ giữa núm cửa và kiến trúc ngôi nhà.

Tôi biết đây là lời dễ gây tranh cãi. Tin tôi đi, tôi từng tranh luận về chuyện này. Để

nói rõ: tôi không nói đến mô hình dữ liệu. Cấu trúc dữ liệu trong ứng dụng có ý nghĩa lớn đối với kiến trúc hệ thống. Nhưng cơ sở dữ liệu không phải mô hình dữ liệu. Cơ sở dữ liệu là phần mềm, một tiện ích giúp truy cập dữ liệu. Từ góc nhìn kiến trúc, tiện ích đó không liên quan vì nó là chi tiết cơ chế cấp thấp. Kiến trúc sư giỏi không để các cơ chế cấp thấp làm ô nhiễm kiến trúc hệ thống.

Cơ sở dữ liệu quan hệ

Edgar Codd xác định các nguyên tắc của cơ sở dữ liệu quan hệ vào năm 1970. Đến giữa những năm 1980, mô hình quan hệ đã trở thành hình thức lưu trữ dữ liệu thống trị. Sự phổ biến ấy có lý do chính đáng: mô hình quan hệ thanh lịch, có kỷ luật và vững chắc. Đây là công nghệ lưu trữ và truy cập dữ liệu rất tốt.

Nhưng dù công nghệ có xuất sắc, hữu ích và vững chắc về mặt toán học đến đâu, nó vẫn chỉ là công nghệ. Vì vậy, nó vẫn là chi tiết.

Bảng quan hệ có thể thuận tiện cho một số cách truy cập dữ liệu, nhưng việc sắp xếp dữ liệu thành các hàng trong bảng không có ý nghĩa kiến trúc. Ca sử dụng của ứng dụng không nên biết hay quan tâm đến những vấn đề đó. Thực tế, hiểu biết về cấu trúc bảng của dữ liệu phải được giới hạn trong các hàm tiện ích cấp thấp nhất ở những vòng tròn ngoài của kiến trúc.

Nhiều framework truy cập dữ liệu cho phép truyền các hàng và bảng cơ sở dữ liệu khắp hệ thống dưới dạng đối tượng. Đây là lỗi kiến trúc. Cách làm đó ràng buộc ca sử dụng, quy tắc nghiệp vụ và đôi khi cả giao diện người dùng vào cấu trúc quan hệ của dữ liệu.

Vì sao hệ thống cơ sở dữ liệu phổ biến đến vậy?

Vì sao hệ thống phần mềm và doanh nghiệp phần mềm lại bị các hệ thống cơ sở dữ liệu chi phối? Vì sao Oracle, MySQL và SQL Server có vị thế vượt trội? Chỉ một từ thôi: đĩa.

Đĩa từ quay là nền tảng lưu trữ dữ liệu chính suốt năm thập kỷ. Nhiều thế hệ lập trình viên không biết đến hình thức lưu trữ nào khác. Công nghệ đĩa đã phát triển từ những chồng đĩa khổng lồ có phiến đĩa đường kính 48 inch, nặng hàng nghìn pound và chứa được 20 megabyte, đến những đĩa tròn mỏng đường kính 3 inch, chỉ nặng vài gram nhưng chứa được một terabyte hoặc hơn. Đó là một chặng đường dài. Trong suốt thời gian ấy, lập trình viên luôn phải đối mặt với đặc tính tai hại của công nghệ đĩa: đĩa rất chậm.

Dữ liệu trên đĩa được lưu trong các rãnh tròn. Các rãnh được chia thành sector, mỗi sector chứa một lượng byte tiện dụng, thường là 4 KB. Mỗi phiến đĩa có thể có hàng trăm rãnh, và cả ổ có thể gồm hơn chục phiến. Muốn đọc một byte cụ thể, ta phải di chuyển đầu đọc đến đúng rãnh, chờ đĩa quay đến đúng sector, đọc toàn bộ 4 KB của sector đó vào RAM rồi truy cập vị trí cần tìm trong vùng đệm RAM. Tất cả việc đó mất thời gian, tính bằng mili giây.

Mili giây nghe có vẻ ngắn, nhưng dài hơn chu kỳ của hầu hết bộ xử lý đến một triệu lần. Nếu dữ liệu không nằm trên đĩa, ta có thể truy cập nó trong nano giây thay vì mili giây.

Để giảm độ trễ do đĩa gây ra, cần có chỉ mục, bộ nhớ đệm và các cách truy vấn được

tối ưu hóa. Cũng cần một cách biểu diễn dữ liệu thống nhất để các chỉ mục, bộ nhớ đệm và truy vấn biết mình đang xử lý dữ liệu nào. Nói ngắn gọn, cần một hệ thống truy cập và quản lý dữ liệu. Theo thời gian, các hệ thống này chia thành hai loại riêng biệt: hệ thống tệp và hệ quản trị cơ sở dữ liệu quan hệ (RDBMS).

Hệ thống tệp dựa trên tài liệu. Chúng cung cấp cách tự nhiên và tiện lợi để lưu trữ toàn bộ tài liệu. Chúng phù hợp khi cần lưu và truy xuất một tập tài liệu theo tên, nhưng không giúp nhiều khi cần tìm trong nội dung tài liệu. Dễ tìm tệp `login.c`, nhưng khó và chậm tìm mọi tệp `.c` có biến tên `x`.

Hệ thống cơ sở dữ liệu dựa trên nội dung. Chúng giúp tìm bản ghi theo nội dung một cách tự nhiên và thuận tiện. Chúng rất giỏi liên kết nhiều bản ghi dựa trên một phần nội dung chung. Tiếc là chúng khá kém trong việc lưu và truy xuất những tài liệu không có cấu trúc nội bộ mà cơ sở dữ liệu hiểu được.

Cả hai loại hệ thống đều tổ chức dữ liệu trên đĩa để lưu và truy xuất hiệu quả nhất theo nhu cầu truy cập riêng. Mỗi loại có cách lập chỉ mục và sắp xếp dữ liệu riêng. Ngoài ra, cuối cùng chúng đều đưa dữ liệu cần thiết vào RAM, nơi có thể thao tác nhanh.

Nếu không có đĩa thì sao?

Đĩa từng phổ biến đến đâu thì nay đang dần biến mất. Chẳng bao lâu nữa, chúng sẽ đi theo ổ băng từ, đĩa mềm và CD, nhường chỗ cho RAM.

Hãy tự hỏi: khi không còn đĩa và mọi dữ liệu đều được lưu trong RAM, bạn sẽ tổ chức dữ liệu thế nào? Bạn sẽ xếp nó thành bảng rồi truy cập bằng SQL? Bạn sẽ sắp xếp nó thành tệp rồi truy cập qua thư mục?

Dĩ nhiên là không. Bạn sẽ tổ chức dữ liệu thành danh sách liên kết, cây, bảng băm, ngăn xếp, hàng đợi hoặc vô số cấu trúc dữ liệu khác, rồi truy cập bằng con trỏ hoặc tham chiếu, vì đó là cách lập trình viên làm việc.

Thực tế, nếu suy nghĩ kỹ, bạn sẽ nhận ra mình vẫn luôn làm như vậy. Dù dữ liệu nằm trong cơ sở dữ liệu hay hệ thống tệp, bạn vẫn đọc nó vào RAM rồi sắp xếp lại cho thuận tiện thành danh sách, tập hợp, ngăn xếp, hàng đợi, cây hoặc cấu trúc dữ liệu nào bạn thích. Rất hiếm khi ta để dữ liệu nguyên dạng bảng hoặc tệp.

Chi tiết

Đây là lý do tôi nói cơ sở dữ liệu chỉ là chi tiết. Nó chỉ là cơ chế dùng để chuyển dữ liệu qua lại giữa bề mặt đĩa và RAM. Xét cho cùng, cơ sở dữ liệu không hơn gì một thùng chứa lớn chứa các bit để lưu dữ liệu lâu dài. Nhưng ta hiếm khi sử dụng dữ liệu ở dạng đó.

Vì vậy, từ góc nhìn kiến trúc, ta không nên quan tâm dữ liệu có dạng gì khi nằm trên bề mặt đĩa từ quay. Thực tế, ta không nên thừa nhận rằng đĩa tồn tại.

Còn hiệu năng thì sao?

Hiệu năng chẳng phải là mối quan tâm kiến trúc sao? Tất nhiên là có. Nhưng với lưu trữ dữ liệu, đây là mối quan tâm có thể được đóng gói và tách biệt hoàn toàn khỏi quy tắc nghiệp vụ. Đúng là cần đọc và ghi dữ liệu nhanh, nhưng đó là mối quan tâm cấp thấp. Ta có thể xử lý nó bằng các cơ chế truy cập dữ liệu cấp thấp. Nó hoàn toàn không liên quan đến kiến trúc tổng thể của hệ thống.

Câu chuyện

Cuối những năm 1980, tôi dẫn dắt một nhóm kỹ sư phần mềm ở công ty khởi nghiệp đang xây dựng và tiếp thị hệ thống quản lý mạng để đo độ toàn vẹn truyền thông của các đường dây viễn thông T1. Hệ thống truy xuất dữ liệu từ các thiết bị ở hai đầu đường dây, rồi chạy một loạt thuật toán dự đoán để phát hiện và báo cáo vấn đề.

Chúng tôi dùng nền tảng UNIX và lưu dữ liệu trong các tệp truy cập ngẫu nhiên đơn giản. Không cần cơ sở dữ liệu quan hệ vì dữ liệu có rất ít mối liên hệ dựa trên nội dung. Lưu dữ liệu trong cây và danh sách liên kết trong các tệp truy cập ngẫu nhiên phù hợp hơn. Nói ngắn gọn, chúng tôi giữ dữ liệu ở dạng tiện nhất để nạp vào RAM và thao tác.

Chúng tôi thuê một quản lý tiếp thị cho công ty khởi nghiệp, một người dễ mến và hiểu biết. Nhưng ngay lập tức, anh ta nói chúng tôi phải có cơ sở dữ liệu quan hệ trong hệ thống. Đây không phải lựa chọn hay vấn đề kỹ thuật, mà là yêu cầu tiếp thị.

Tôi không hiểu. Vì sao phải sắp xếp lại danh sách liên kết và cây thành hàng loạt hàng và bảng được truy cập qua SQL? Vì sao phải gánh toàn bộ chi phí và độ phức tạp của một RDBMS khổng lồ khi hệ thống tệp truy cập ngẫu nhiên đơn giản đã quá đủ? Tôi kiên quyết phản đối.

Một kỹ sư phần cứng trong công ty cũng hùa theo quan điểm cần RDBMS. Anh ta tin rằng hệ thống phần mềm cần RDBMS vì lý do kỹ thuật. Anh ta tổ chức các cuộc họp sau lưng tôi với ban lãnh đạo công ty, vẽ lên bảng những hình người que đứng trên mái nhà đặt trên một cây cột, rồi hỏi: “Bạn có xây nhà trên một cây cột không?” Hàm ý của anh ta là RDBMS lưu các bảng trong những tệp truy cập ngẫu nhiên bằng cách nào đó đáng tin cậy hơn các tệp truy cập ngẫu nhiên chúng tôi đang dùng.

Tôi đã tranh luận với anh ta. Tôi tranh luận với người làm tiếp thị. Tôi giữ vững nguyên tắc kỹ thuật trước sự thiếu hiểu biết đáng kinh ngạc. Tôi cứ tranh luận mãi.

Cuối cùng, kỹ sư phần cứng được thăng chức, vượt qua tôi để trở thành quản lý phần mềm. Cuối cùng, họ đưa RDBMS vào hệ thống tội nghiệp đó. Và cuối cùng, họ hoàn toàn đúng còn tôi sai.

Không phải vì lý do kỹ thuật, tôi nói rõ điều đó. Về mặt kỹ thuật, tôi đúng. Tôi đã đúng khi phản đối đưa RDBMS vào lõi kiến trúc hệ thống. Tôi sai vì khách hàng kỳ vọng sản phẩm phải có cơ sở dữ liệu quan hệ. Họ không biết sẽ dùng nó làm gì. Họ cũng không có cách thực tế nào để dùng dữ liệu quan hệ trong hệ thống của chúng tôi. Nhưng điều đó không quan trọng: khách hàng hoàn toàn mong đợi một RDBMS. Nó đã trở thành một ô đánh dấu trong danh sách yêu cầu của người mua phần mềm. Không có lý do kỹ thuật nào cả, tính hợp lý không liên quan. Nhu cầu ấy phi lý, đến từ bên ngoài và hoàn toàn không

có cơ sở, nhưng vẫn là nhu cầu có thật.

Nhu cầu đó đến từ đâu? Nó bắt nguồn từ các chiến dịch tiếp thị rất hiệu quả của những nhà cung cấp cơ sở dữ liệu thời ấy. Họ đã thuyết phục được các lãnh đạo cấp cao rằng "tài sản dữ liệu" của doanh nghiệp cần được bảo vệ, và hệ thống cơ sở dữ liệu của họ là phương tiện lý tưởng để làm việc đó.

Ngày nay, ta vẫn thấy những chiến dịch tiếp thị tương tự. Từ "doanh nghiệp" và khái niệm "Kiến trúc hướng dịch vụ" liên quan đến tiếp thị nhiều hơn thực tế.

Tôi nên làm gì trong tình huống ngày xưa đó? Tôi nên gắn RDBMS ở bên cạnh hệ thống và cung cấp một kênh truy cập dữ liệu hẹp, an toàn, đồng thời giữ các tệp truy cập ngẫu nhiên trong lõi hệ thống. Tôi đã làm gì? Tôi nghỉ việc và trở thành chuyên gia tư vấn.

Kết luận

Cách tổ chức dữ liệu, tức mô hình dữ liệu, có ý nghĩa kiến trúc. Các công nghệ và hệ thống chuyển dữ liệu ra vào bề mặt từ quay thì không. Hệ quản trị cơ sở dữ liệu quan hệ buộc dữ liệu phải được tổ chức thành bảng và truy cập bằng SQL liên quan đến nhóm thứ hai nhiều hơn nhóm thứ nhất. Dữ liệu có ý nghĩa. Cơ sở dữ liệu chỉ là chi tiết.

31

Web chỉ là chi tiết



Bạn có phải là lập trình viên trong thập niên 1990 không? Bạn có nhớ web đã làm thay đổi mọi thứ ra sao không? Bạn có nhớ chúng ta đã nhìn các kiến trúc máy khách-máy chủ cũ bằng ánh mắt khinh miệt như thế nào trước công nghệ mới sáng bóng mang tên Web không?

Thực ra, web chẳng thay đổi điều gì cả. Hoặc ít nhất, nó không nên thay đổi. Web chỉ là dao động mới nhất trong chuỗi dao động của ngành từ thập niên 1960. Những dao động này luân phiên giữa việc tập trung toàn bộ sức mạnh tính toán vào máy chủ trung tâm và phân tán toàn bộ sức mạnh tính toán ra các thiết bị đầu cuối.

Chỉ trong khoảng một thập kỷ kể từ khi web trở nên phổ biến, chúng ta đã chứng kiến vài lần dao động như vậy. Ban đầu, ta nghĩ toàn bộ sức mạnh tính toán sẽ nằm trong các cụm máy chủ, còn trình duyệt chỉ làm nhiệm vụ đơn giản. Sau đó, ta bắt đầu đưa applet vào trình duyệt. Nhưng rồi ta không thích cách đó, nên chuyển nội dung động trở lại máy chủ. Sau đó lại không thích như vậy, nên phát minh ra Web 2.0 và đưa nhiều xử lý trở lại trình duyệt bằng Ajax và JavaScript. Ta đi xa đến mức tạo ra những ứng dụng khổng lồ chạy hoàn toàn trong trình duyệt. Giờ đây, mọi người lại hào hứng chuyển JavaScript về máy chủ bằng Node.

(Thở dài.)

Con lắc vô tận

Dĩ nhiên, sẽ sai nếu nghĩ những dao động ấy bắt đầu cùng với web. Trước web là kiến trúc máy khách-máy chủ. Trước đó là các máy tính mini trung tâm với nhiều thiết bị đầu cuối đơn giản. Trước nữa là máy tính lớn với các thiết bị đầu cuối màn hình xanh thông minh, khá giống trình duyệt ngày nay. Trước đó nữa là phòng máy tính và những chồng thẻ đục lỗ…

Câu chuyện cứ thế tiếp diễn. Chúng ta dường như không thể quyết định sức mạnh tính toán nên đặt ở đâu. Ta liên tục chuyển qua lại giữa tập trung và phân tán. Tôi nghĩ những dao động ấy sẽ còn tiếp tục trong một thời gian dài.

Xét trong toàn bộ lịch sử công nghệ thông tin, web chẳng thay đổi điều gì cả. Web chỉ là một trong nhiều dao động của cuộc giằng co bắt đầu trước khi hầu hết chúng ta ra đời và sẽ tiếp tục rất lâu sau khi phần lớn chúng ta nghỉ hưu.

Tuy nhiên, với tư cách kiến trúc sư, chúng ta phải nhìn dài hạn. Những dao động đó chỉ là vấn đề ngắn hạn mà ta muốn đẩy ra xa khỏi lõi quy tắc nghiệp vụ.

Hãy nghe câu chuyện về công ty Q. Công ty Q xây dựng một hệ thống tài chính cá nhân rất phổ biến. Đó là ứng dụng máy tính để bàn có giao diện đồ họa hữu ích. Tôi rất thích dùng nó.

Rồi web xuất hiện. Trong phiên bản tiếp theo, công ty Q thay đổi giao diện để trông và hoạt động giống trình duyệt. Tôi sửng sốt! Thiên tài tiếp thị nào lại nghĩ rằng phần mềm tài chính cá nhân chạy trên máy tính để bàn nên mang hình thức và cách hoạt động của trình duyệt web?

Dĩ nhiên tôi ghét giao diện mới. Có vẻ mọi người khác cũng vậy, vì sau vài phiên bản, công ty Q dần loại bỏ dáng vẻ giống trình duyệt và đưa hệ thống tài chính cá nhân trở lại

giao diện máy tính để bàn thông thường.

Giờ hãy tưởng tượng bạn là kiến trúc sư phần mềm của Q. Một thiên tài tiếp thị thuyết phục ban lãnh đạo rằng toàn bộ giao diện người dùng phải thay đổi để giống web hơn. Bạn sẽ làm gì? Hay đúng hơn, trước đó bạn nên làm gì để bảo vệ ứng dụng khỏi thiên tài tiếp thị ấy?

Bạn nên tách quy tắc nghiệp vụ khỏi giao diện người dùng. Tôi không biết các kiến trúc sư của Q đã làm vậy chưa. Một ngày nào đó tôi rất muốn nghe câu chuyện của họ. Nếu ở đó vào thời điểm ấy, chắc chắn tôi đã cố gắng hết sức để cô lập quy tắc nghiệp vụ khỏi giao diện đồ họa, vì chẳng ai biết các thiên tài tiếp thị sẽ làm gì tiếp theo.

Giờ hãy xét công ty A, nhà sản xuất một chiếc điện thoại thông minh tuyệt đẹp. Gần đây, họ phát hành phiên bản nâng cấp "hệ điều hành" của điện thoại. Thật kỳ lạ khi ta nói về hệ điều hành bên trong điện thoại. Trong số những thay đổi khác, bản nâng cấp "hệ điều hành" đó thay đổi hoàn toàn hình thức và cách hoạt động của mọi ứng dụng. Vì sao? Tôi đoán là do một thiên tài tiếp thị nào đó.

Tôi không phải chuyên gia về phần mềm trong thiết bị đó, nên không biết thay đổi này có gây khó khăn đáng kể cho lập trình viên ứng dụng trên điện thoại của công ty A hay không. Tôi hy vọng kiến trúc sư của A và kiến trúc sư các ứng dụng đã giữ giao diện người dùng tách biệt khỏi quy tắc nghiệp vụ, vì luôn có những thiên tài tiếp thị rình chờ lợi dụng mối phụ thuộc nhỏ tiếp theo mà bạn tạo ra.

Kết luận

Kết luận rất đơn giản: giao diện đồ họa chỉ là chi tiết. Web là một giao diện đồ họa, nên web cũng chỉ là chi tiết. Với tư cách kiến trúc sư, bạn nên đặt những chi tiết như vậy phía sau các ranh giới để tách chúng khỏi logic nghiệp vụ cốt lõi.

Hãy nghĩ theo cách này: WEB là một thiết bị IO. Từ thập niên 1960, chúng ta đã hiểu giá trị của việc viết ứng dụng độc lập với thiết bị. Động lực để giữ tính độc lập đó chưa hề thay đổi. Web không phải ngoại lệ.

Hay có phải không? Có thể lập luận rằng giao diện đồ họa, cũng như web, độc đáo và phong phú đến mức theo đuổi kiến trúc độc lập thiết bị là vô lý. Khi nghĩ đến sự phức tạp của việc xác thực bằng JavaScript, các lời gọi Ajax kéo-thả hay vô số tiện ích có thể đặt lên trang web, người ta dễ cho rằng độc lập thiết bị là không thực tế.

Ở một mức độ nào đó, điều này đúng. Tương tác giữa ứng dụng và giao diện đồ họa diễn ra dồn dập theo những cách rất riêng của từng loại giao diện. Cách trình duyệt và ứng dụng web phối hợp khác với cách giao diện đồ họa máy tính để bàn và ứng dụng phối hợp. Có vẻ khó mà trừu tượng hóa sự phối hợp ấy theo cách UNIX trừu tượng hóa thiết bị.

Tuy nhiên, có thể trừu tượng hóa một ranh giới khác giữa giao diện người dùng và ứng dụng. Ta có thể xem logic nghiệp vụ là một tập hợp ca sử dụng, mỗi ca sử dụng thực hiện một chức năng thay mặt người dùng. Có thể mô tả từng ca sử dụng bằng dữ liệu đầu vào, quá trình xử lý và dữ liệu đầu ra.

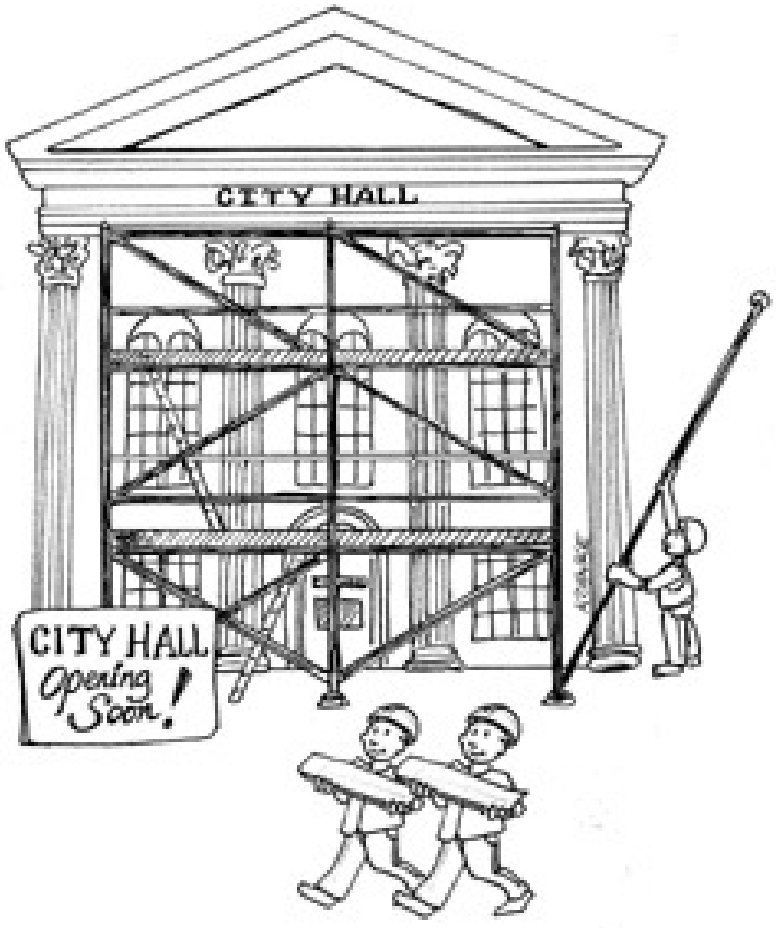
Tại một thời điểm nào đó trong quá trình tương tác giữa giao diện người dùng và ứng dụng, dữ liệu đầu vào sẽ hoàn tất và ca sử dụng có thể được thực thi. Khi hoàn tất, dữ liệu

kết quả có thể được đưa trở lại quá trình tương tác giữa giao diện và ứng dụng.

Có thể đặt toàn bộ dữ liệu đầu vào và dữ liệu đầu ra kết quả vào các cấu trúc dữ liệu, rồi dùng chúng làm giá trị đầu vào và đầu ra cho tiến trình thực thi ca sử dụng. Với cách tiếp cận này, có thể xem mỗi ca sử dụng đang điều khiển thiết bị IO của giao diện người dùng theo cách độc lập với thiết bị.

32

Framework chỉ là chi tiết



Framework trở nên rất phổ biến. Nhìn chung, đây là điều tốt. Có nhiều framework miễn phí, mạnh mẽ và hữu ích.

Tuy nhiên, framework không phải kiến trúc, dù một số framework cố gắng làm như vậy.

Tác giả framework

Hầu hết tác giả cung cấp framework miễn phí vì muốn giúp đỡ cộng đồng và đóng góp lại. Đó là điều đáng quý. Tuy nhiên, dù động cơ của họ có cao đẹp đến đâu, họ không đặt lợi ích tốt nhất của bạn lên hàng đầu. Họ không thể làm vậy vì không biết bạn là ai và không biết vấn đề của bạn.

Tác giả framework hiểu vấn đề của chính họ, đồng nghiệp và bạn bè. Họ viết framework để giải quyết những vấn đề đó, không phải vấn đề của bạn.

Dĩ nhiên, vấn đề của bạn có thể trùng lặp khá nhiều với vấn đề của họ. Nếu không, framework đã chẳng phổ biến đến vậy. Trong phạm vi có sự trùng lặp, framework quả thực có thể rất hữu ích.

Mối ràng buộc bất cân xứng

Mối quan hệ giữa bạn và tác giả framework bất cân xứng một cách đáng kinh ngạc. Bạn phải cam kết rất nhiều với framework, nhưng tác giả không cam kết với bạn chút nào.

Hãy suy nghĩ kỹ về điều này. Khi dùng framework, bạn đọc tài liệu do tác giả cung cấp. Trong tài liệu đó, tác giả và những người dùng framework khác khuyên bạn cách tích hợp phần mềm với framework. Thông thường, điều đó có nghĩa là bọc kiến trúc của bạn quanh framework. Tác giả khuyên bạn kế thừa các lớp cơ sở của framework và đưa tiện ích của framework vào đối tượng nghiệp vụ. Họ thúc giục bạn gắn ứng dụng với framework chặt nhất có thể.

Với tác giả framework, gắn phần mềm với chính framework của họ không phải rủi ro. Họ muốn gắn với framework đó vì họ hoàn toàn kiểm soát nó.

Hơn nữa, tác giả muốn bạn gắn ứng dụng với framework vì một khi đã gắn như vậy, việc thoát ra sẽ rất khó. Không gì khiến tác giả framework cảm thấy được công nhận hơn một nhóm người dùng sẵn sàng kế thừa không thể tách rời từ các lớp cơ sở của họ.

Thực chất, tác giả đang đề nghị bạn kết hôn với framework, tức cam kết lớn và lâu dài. Thế nhưng, trong bất kỳ trường hợp nào, tác giả cũng không đưa ra cam kết tương ứng với bạn. Đây là cuộc hôn nhân một chiều. Bạn gánh mọi rủi ro và trách nhiệm, còn tác giả framework không gánh gì cả.

Những rủi ro

Bạn cần cân nhắc một số rủi ro sau:

- Kiến trúc của framework thường không sạch. Framework thường vi phạm Quy tắc Phụ thuộc. Chúng muốn bạn kế thừa mã của chúng vào đối tượng nghiệp vụ, tức Entity của bạn. Chúng muốn framework được gắn vào vòng tròn trong cùng. Một khi đã vào đó, framework sẽ không rời đi. Chiếc nhẫn cưới đã ở trên tay bạn và sẽ còn ở đó.
- Framework có thể giúp bạn triển khai một số tính năng ban đầu. Nhưng khi sản phẩm trưởng thành, nó có thể vượt quá khả năng của framework. Nếu đã đeo chiếc nhẫn ấy, bạn sẽ thấy framework ngày càng cản trở mình.
- Framework có thể phát triển theo hướng không hữu ích với bạn. Bạn có thể buộc phải nâng cấp lên phiên bản mới không giúp ích gì. Thậm chí, các tính năng cũ mà bạn dùng có thể biến mất hoặc thay đổi theo cách khó theo kịp.
- Có thể xuất hiện framework mới và tốt hơn mà bạn muốn chuyển sang dùng.

Giải pháp

Giải pháp là gì?

> *Đừng kết hôn với framework!*

Bạn vẫn có thể dùng framework, chỉ đừng gắn chặt với nó. Hãy giữ nó ở khoảng cách an toàn. Xem framework như chi tiết thuộc một trong các vòng tròn ngoài của kiến trúc. Đừng để nó lọt vào các vòng tròn trong.

Nếu framework muốn bạn kế thừa các lớp cơ sở của nó cho đối tượng nghiệp vụ, hãy từ chối. Thay vào đó, hãy tạo proxy và đặt chúng vào các thành phần plugin của quy tắc nghiệp vụ.

Đừng để framework lọt vào mã lõi. Thay vào đó, hãy tích hợp framework vào các thành phần cắm vào mã lõi, tuân theo Quy tắc Phụ thuộc.

Chẳng hạn, có thể bạn thích Spring. Spring là framework tiêm phụ thuộc tốt. Có thể bạn dùng Spring để tự động nối các phụ thuộc. Điều đó ổn, nhưng đừng rải chú thích `@Autowired` khắp các đối tượng nghiệp vụ. Đối tượng nghiệp vụ không nên biết về Spring.

Thay vào đó, bạn có thể dùng Spring để tiêm phụ thuộc vào thành phần `Main`. `Main` được phép biết về Spring vì nó là thành phần bẩn nhất và cấp thấp nhất trong kiến trúc.

Bây giờ tôi tuyên bố hai bạn thành vợ chồng…

Có một số framework mà bạn buộc phải kết hôn. Chẳng hạn, nếu dùng C++, có lẽ bạn phải kết hôn với STL, rất khó tránh. Nếu dùng Java, gần như chắc chắn bạn phải gắn với thư viện chuẩn.

Điều đó bình thường, nhưng vẫn nên là một quyết định có cân nhắc. Bạn phải hiểu rằng khi kết hôn framework với ứng dụng, bạn sẽ gắn bó với framework đó trong suốt vòng đời ứng dụng. Dù tốt hay xấu, khi khỏe mạnh hay lúc đau ốm, khi giàu sang hay nghèo khó, dù từ bỏ mọi framework khác, bạn vẫn sẽ dùng framework này. Đây không phải là cam kết nên đưa ra nhẹ nhàng.

Kết luận

Khi gặp một framework, hãy cố đừng kết hôn với nó ngay. Hãy tìm cách "hẹn hò" một thời gian trước khi quyết định gắn bó. Nếu có thể, hãy giữ framework sau một ranh giới kiến trúc càng lâu càng tốt.

Có lẽ bạn tìm được cách lấy sữa mà không cần mua cả con bò.

33

Nghiên cứu tình huống: Bán video



Đã đến lúc tập hợp các quy tắc và ý tưởng về kiến trúc vào một nghiên cứu tình huống. Nghiên cứu này ngắn và đơn giản, nhưng minh họa quy trình mà kiến trúc sư giỏi sử dụng cùng những quyết định họ đưa ra.

Sản phẩm

Trong nghiên cứu tình huống này, tôi chọn sản phẩm mà mình khá am hiểu: phần mềm cho một trang web bán video. Tất nhiên, nó gợi nhớ đến cleancoders.com, trang web nơi tôi bán các video hướng dẫn phần mềm.

Ý tưởng cơ bản rất đơn giản. Chúng ta có một bộ video muốn bán trên web cho cả cá nhân và doanh nghiệp. Cá nhân có thể trả một mức giá để xem trực tuyến và mức giá cao hơn để tải video về và sở hữu vĩnh viễn. Giấy phép doanh nghiệp chỉ cho phép xem trực tuyến, được mua theo lô và có chiết khấu theo số lượng.

Thông thường, cá nhân vừa là người xem vừa là người mua. Ngược lại, trong doanh nghiệp, người mua video thường khác với người sẽ xem chúng.

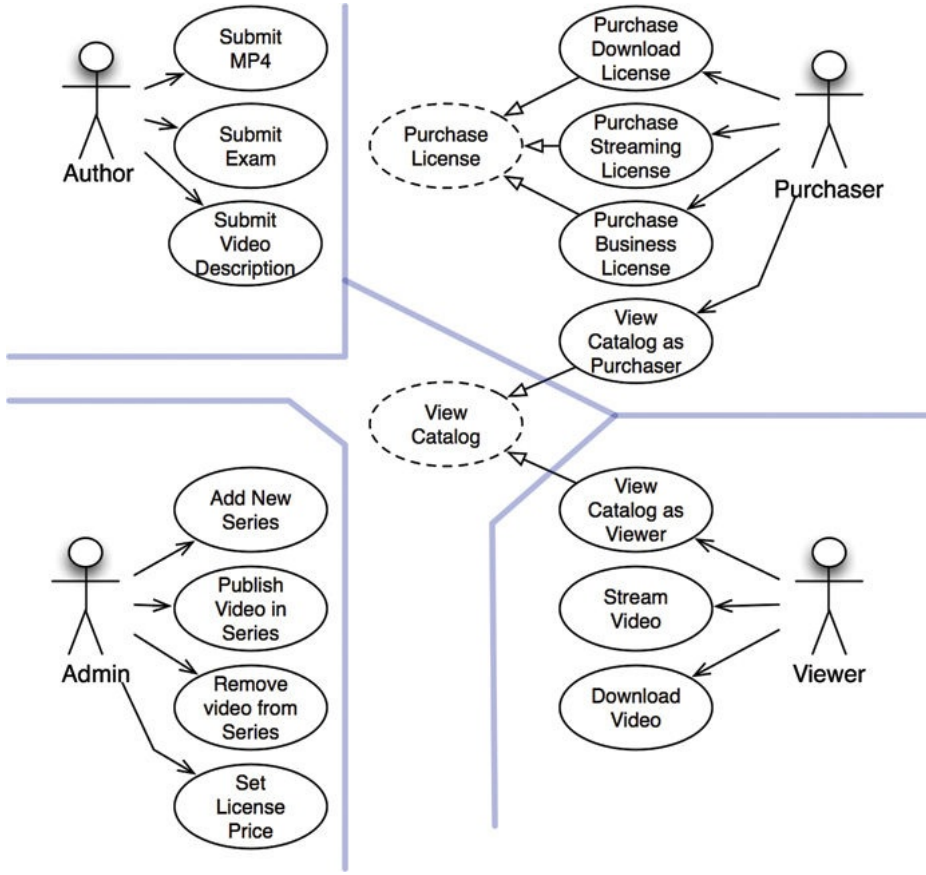
Tác giả video cần cung cấp tệp video, phần mô tả bằng văn bản và các tệp bổ trợ gồm bài thi, bài tập, lời giải, mã nguồn cùng những tài liệu khác.

Quản trị viên cần thêm loạt video mới, thêm hoặc xóa video khỏi loạt và thiết lập giá cho các loại giấy phép khác nhau.

Bước đầu tiên để xác định kiến trúc ban đầu của hệ thống là nhận diện các tác nhân và ca sử dụng.

Phân tích ca sử dụng

Hình 33.1 cho thấy một phân tích ca sử dụng điển hình.



Hình 33.1

Một phân tích ca sử dụng điển hình

Có thể nhận ra bốn tác nhân chính. Theo Nguyên tắc Trách nhiệm Đơn nhất, bốn tác nhân này sẽ là bốn nguồn thay đổi chính của hệ thống. Mỗi khi thêm tính năng mới hoặc thay đổi tính năng hiện có, thay đổi đó sẽ nhằm phục vụ một trong các tác nhân. Vì vậy, ta muốn phân chia hệ thống sao cho thay đổi phục vụ một tác nhân không ảnh hưởng đến các tác nhân khác.

Các ca sử dụng trong Hình 33.1 không phải danh sách đầy đủ. Chẳng hạn, bạn sẽ không thấy ca sử dụng đăng nhập hay đăng xuất. Lý do bỏ chúng chỉ đơn giản là để giữ quy mô vấn đề trong phạm vi cuốn sách. Nếu đưa vào mọi ca sử dụng khác nhau, riêng chương này đã phải thành một cuốn sách.

Hãy chú ý các ca sử dụng nét đứt ở giữa Hình 33.1. Đây là các ca sử dụng trừu tượng.[1] Ca sử dụng trừu tượng xác định một chính sách chung mà một ca sử dụng khác sẽ cụ thể

1. Đây là ký hiệu riêng của tôi cho ca sử dụng "trừu tượng". Cách chuẩn hơn là dùng một stereotype UML như

hóa. Như bạn thấy, các ca sử dụng *View Catalog as Viewer* và *View Catalog as Purchaser* đều kế thừa từ ca sử dụng trừu tượng *View Catalog*.

Một mặt, tôi không nhất thiết phải tạo sự trừu tượng này. Tôi có thể bỏ ca sử dụng trừu tượng khỏi sơ đồ mà không ảnh hưởng đến tính năng của toàn sản phẩm. Mặt khác, hai ca sử dụng này rất giống nhau, nên tôi nghĩ tốt hơn là nhận ra điểm chung và thống nhất chúng ngay từ đầu trong quá trình phân tích.

Kiến trúc thành phần

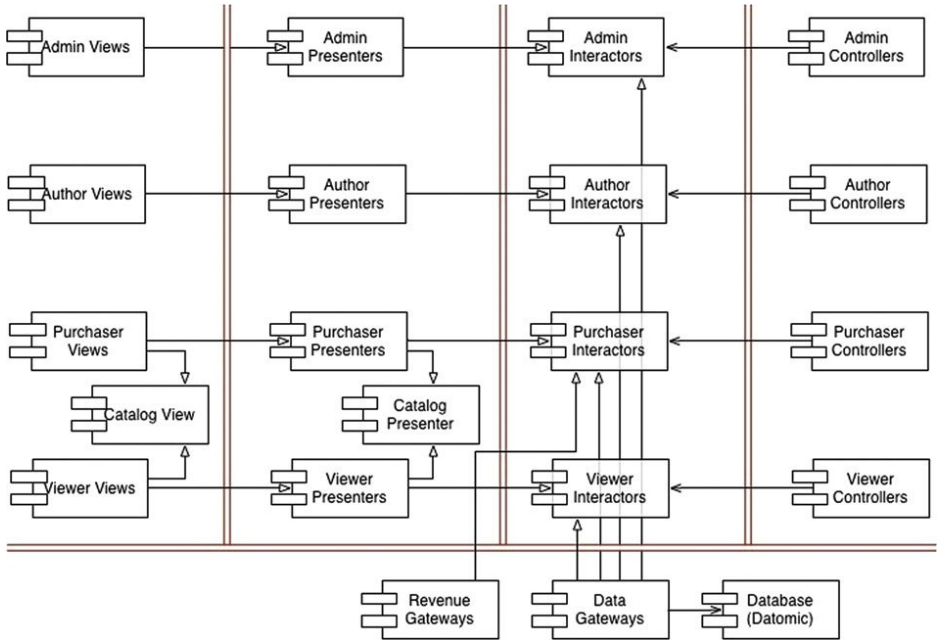
Giờ đã biết các tác nhân và ca sử dụng, ta có thể tạo kiến trúc thành phần sơ bộ (Hình 33.2).

Các đường đôi trong sơ đồ biểu thị ranh giới kiến trúc như thường lệ. Bạn có thể thấy cách phân chia điển hình giữa view, presenter, interactor và controller. Bạn cũng thấy tôi chia mỗi nhóm đó theo các tác nhân tương ứng.

Mỗi thành phần trong Hình 33.2 có thể được đóng gói thành tệp `.jar` hoặc `.dll` tiềm năng. Mỗi thành phần chứa view, presenter, interactor và controller được phân bổ cho nó.

Hãy chú ý các thành phần đặc biệt dành cho `Catalog View` và `Catalog Presenter`. Đây là cách tôi xử lý ca sử dụng trừu tượng `View Catalog`. Tôi giả định view và presenter đó sẽ được viết thành các lớp trừu tượng trong những thành phần này, còn thành phần kế thừa sẽ chứa các lớp view và presenter kế thừa từ các lớp trừu tượng ấy.

`<<abstract>>`, nhưng ngày nay tôi không thấy việc tuân thủ những tiêu chuẩn như vậy có nhiều ích lợi.



Hình 33.2
Kiến trúc thành phần sơ bộ

Liệu tôi có thực sự chia hệ thống thành tất cả các thành phần này rồi phân phối chúng dưới dạng tệp `.jar` hoặc `.dll` không? Có và không. Tôi chắc chắn sẽ chia môi trường biên dịch và build theo cách này để có thể tạo các sản phẩm bàn giao độc lập như vậy. Tôi cũng sẽ giữ quyền gộp chúng thành số lượng sản phẩm bàn giao nhỏ hơn nếu cần. Chẳng hạn, với cách phân chia trong Hình 33.2, có thể dễ dàng gộp chúng thành năm tệp `.jar`, lần lượt cho view, presenter, interactor, controller và tiện ích. Khi đó, tôi có thể triển khai độc lập các thành phần có khả năng thay đổi độc lập với nhau cao nhất.

Một cách nhóm khác là đặt view và presenter chung trong một tệp `.jar`, còn interactor, controller và tiện ích nằm trong tệp `.jar` riêng. Cách nhóm thô sơ hơn nữa là tạo hai tệp `.jar`: một tệp chứa view và presenter, tệp còn lại chứa mọi thứ khác.

Giữ các lựa chọn này mở giúp chúng ta điều chỉnh cách triển khai theo sự thay đổi của hệ thống theo thời gian.

Quản lý phụ thuộc

Luồng điều khiển trong Hình 33.2 đi từ phải sang trái. Đầu vào đi vào controller, rồi interactor xử lý đầu vào để tạo kết quả. Sau đó presenter định dạng kết quả, còn view hiển thị nội dung đã trình bày.

Hãy chú ý rằng không phải mọi mũi tên đều đi từ phải sang trái. Thực tế, phần lớn mũi

tên hướng từ trái sang phải vì kiến trúc tuân theo Quy tắc Phụ thuộc. Mọi phụ thuộc đều đi qua ranh giới theo cùng một hướng và luôn hướng về các thành phần chứa chính sách cấp cao hơn.

Cũng lưu ý rằng quan hệ sử dụng, được biểu diễn bằng mũi tên rỗng, cùng hướng với luồng điều khiển, còn quan hệ kế thừa, được biểu diễn bằng mũi tên kín, đi ngược luồng điều khiển. Điều này thể hiện cách chúng ta dùng Nguyên tắc Đóng-Mở để bảo đảm phụ thuộc đi đúng hướng và thay đổi ở chi tiết cấp thấp không lan lên ảnh hưởng đến chính sách cấp cao.

Kết luận

Sơ đồ kiến trúc ở Hình 33.2 có hai chiều phân tách. Thứ nhất là phân tách tác nhân theo Nguyên tắc Trách nhiệm Đơn nhất. Thứ hai là Quy tắc Phụ thuộc. Cả hai đều nhằm tách các thành phần thay đổi vì những lý do khác nhau và với tốc độ khác nhau. Lý do khác nhau tương ứng với các tác nhân; tốc độ khác nhau tương ứng với các cấp độ chính sách.

Khi cấu trúc mã theo cách này, bạn có thể tùy ý kết hợp các thành phần khi triển khai hệ thống. Bạn có thể nhóm chúng thành các sản phẩm bàn giao có thể triển khai theo bất kỳ cách nào hợp lý, rồi dễ dàng thay đổi cách nhóm khi điều kiện thay đổi.

34

Chương còn thiếu



Simon Brown

Tất cả lời khuyên bạn đã đọc đến đây chắc chắn sẽ giúp thiết kế phần mềm tốt hơn, với các lớp và thành phần có ranh giới rõ ràng, trách nhiệm minh bạch và phụ thuộc được kiểm soát. Nhưng hóa ra ma quỷ lại ẩn trong chi tiết triển khai, và rất dễ thất bại ở chặng cuối nếu không suy nghĩ kỹ về chúng.

Hãy tưởng tượng chúng ta đang xây dựng một hiệu sách trực tuyến. Một trong những ca sử dụng cần triển khai cho phép khách hàng xem trạng thái đơn hàng. Dù ví dụ này dùng Java, các nguyên tắc cũng áp dụng cho ngôn ngữ lập trình khác. Tạm gác Kiến trúc Sạch sang một bên và xem một số cách thiết kế, tổ chức mã.

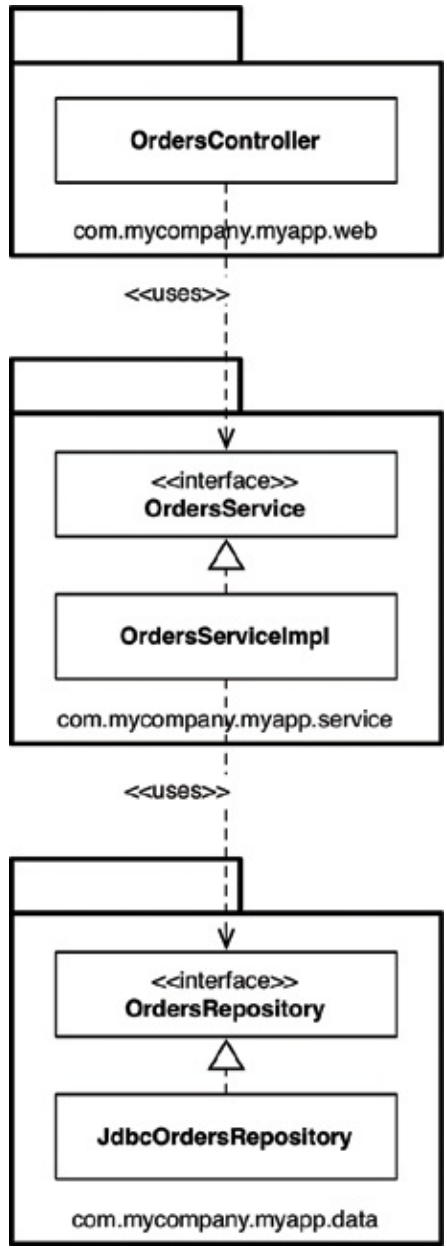
Chia gói theo lớp

Cách tiếp cận đầu tiên, và có lẽ đơn giản nhất, là kiến trúc phân lớp ngang truyền thống, trong đó ta tách mã dựa trên chức năng kỹ thuật của nó. Cách này thường được gọi là "chia gói theo lớp". Hình 34.1 cho thấy một ví dụ dưới dạng sơ đồ lớp UML.

Trong kiến trúc phân lớp điển hình này, có một lớp cho mã web, một lớp cho "logic nghiệp vụ" và một lớp cho lưu trữ. Nói cách khác, mã được chia ngang thành các lớp để nhóm những loại thành phần tương tự. Trong "kiến trúc phân lớp nghiêm ngặt", mỗi lớp chỉ nên phụ thuộc vào lớp liền kề bên dưới. Trong Java, các lớp thường được triển khai thành các package. Như Hình 34.1 cho thấy, mọi phụ thuộc giữa các lớp (package) đều hướng xuống dưới. Ví dụ này có các kiểu Java sau:

- `OrdersController`: controller web, tương tự controller Spring MVC, xử lý yêu cầu từ web.
- `OrdersService`: giao diện xác định "logic nghiệp vụ" liên quan đến đơn hàng.
- `OrdersServiceImpl`: phần triển khai dịch vụ đơn hàng.[1]
- `OrdersRepository`: giao diện xác định cách truy cập thông tin đơn hàng đã lưu.
- `JdbcOrdersRepository`: phần triển khai giao diện repository.

1. Đây có thể là cách đặt tên lớp rất tệ, nhưng như ta sẽ thấy sau, có lẽ điều đó không quan trọng.


OrdersController
com.mycompany.myapp.web
<<uses>>
<<interface>>
OrdersService
OrdersServiceImpl
com.mycompany.myapp.service
<<uses>>
<<interface>>
OrdersRepository
JdbcOrdersRepository
com.mycompany.myapp.data

Hình 34.1

Chia gói theo lớp

Trong bài "Presentation Domain Data Layering",[2] Martin Fowler nói rằng chọn kiến trúc phân lớp như vậy là cách khởi đầu tốt. Ông không phải người duy nhất nghĩ vậy. Nhiều sách, hướng dẫn, khóa đào tạo và mã ví dụ bạn tìm thấy cũng khuyên tạo kiến trúc phân lớp. Đây là cách nhanh để khởi động hệ thống mà không cần nhiều phức tạp. Tuy nhiên, như Martin chỉ ra, khi phần mềm mở rộng và phức tạp hơn, bạn sẽ sớm thấy ba "xô" mã lớn là không đủ và cần nghĩ đến việc mô-đun hóa sâu hơn.

Một vấn đề khác, như Uncle Bob đã nói, là kiến trúc phân lớp không thể hiện rõ miền nghiệp vụ. Đặt cạnh nhau mã của hai kiến trúc phân lớp thuộc hai miền nghiệp vụ rất khác nhau, chúng có thể trông giống nhau đến kỳ lạ: web, dịch vụ và repository. Còn một vấn đề lớn khác với kiến trúc phân lớp, nhưng chúng ta sẽ nói đến sau.

Chia gói theo tính năng

Một cách khác để tổ chức mã là dùng phong cách "chia gói theo tính năng". Đây là cách chia dọc dựa trên các tính năng liên quan, khái niệm miền hoặc aggregate root, theo thuật ngữ thiết kế hướng miền. Trong những cách triển khai điển hình mà tôi từng thấy, mọi kiểu được đặt trong cùng một Java package, tên package phản ánh khái niệm được nhóm lại.

Với cách này, như Hình 34.2 cho thấy, ta vẫn có các giao diện và lớp như trước, nhưng tất cả được đặt trong một Java package thay vì chia thành ba package. Đây là tái cấu trúc rất đơn giản so với cách "chia gói theo lớp", nhưng cấu trúc cấp cao nhất của mã giờ thể hiện miền nghiệp vụ. Ta thấy mã này liên quan đến đơn hàng chứ không chỉ là web, dịch vụ và repository.

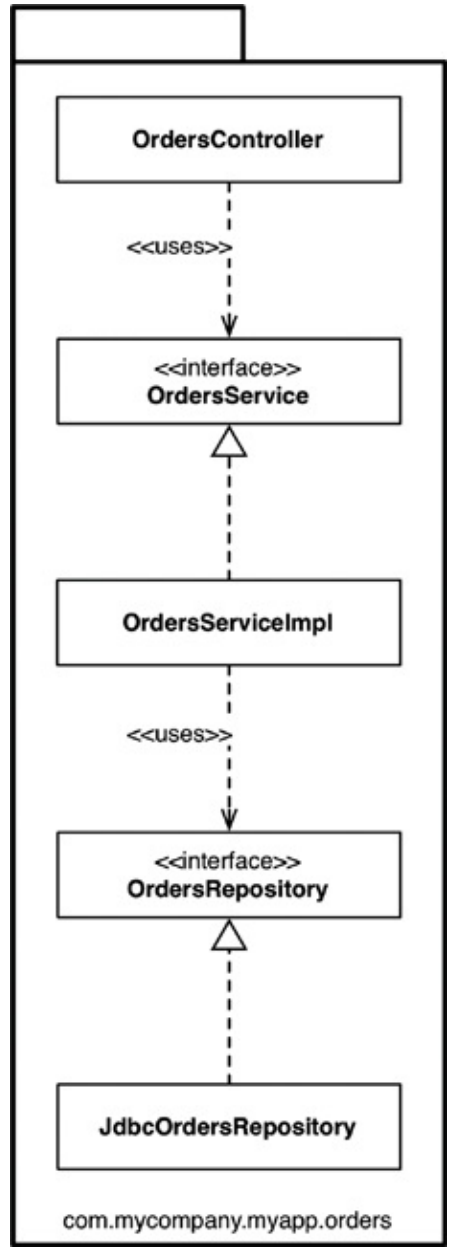
Một lợi ích khác là có thể dễ tìm hơn toàn bộ mã cần sửa khi ca sử dụng "xem đơn hàng" thay đổi. Mọi thứ nằm trong cùng một Java package thay vì rải khắp nơi.[3]

Tôi thường thấy các nhóm phát triển nhận ra mình gặp vấn đề với phân lớp ngang ("chia gói theo lớp"), rồi chuyển sang phân lớp dọc ("chia gói theo tính năng"). Theo tôi, cả hai cách đều chưa tối ưu. Nếu đã đọc đến đây, có thể bạn đang nghĩ rằng còn cách tốt hơn, và bạn nghĩ đúng.

---

2. `https://martinfowler.com/bliki/PresentationDomainDataLayering.html`

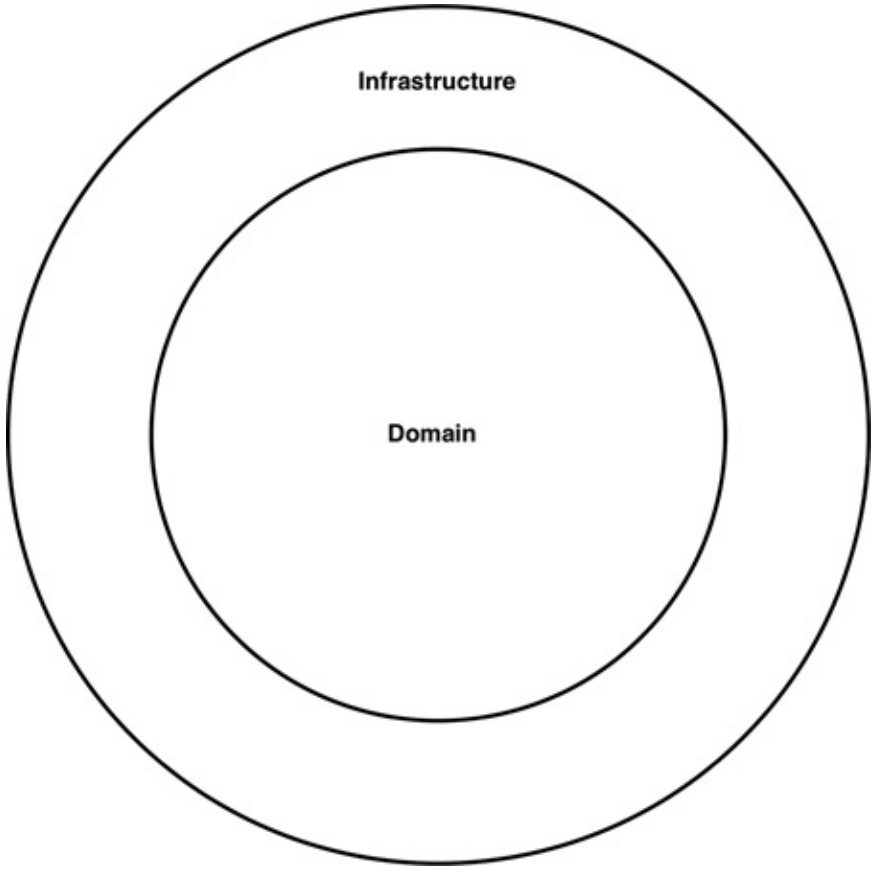
3. Lợi ích này ít quan trọng hơn khi các IDE hiện đại có khả năng điều hướng tốt. Tuy nhiên, gần đây người ta lại quay về dùng trình soạn thảo văn bản nhẹ, vì những lý do mà có lẽ tôi đã quá già để hiểu.



Hình 34.2
Chia gói theo tính năng

Cổng và bộ chuyển đổi

Như Uncle Bob đã nói, các cách tiếp cận như "cổng và bộ chuyển đổi", "kiến trúc lục giác", "ranh giới, controller, entity" và những cách tương tự nhằm tạo kiến trúc trong đó mã tập trung vào nghiệp vụ hoặc miền được tách biệt và độc lập với chi tiết triển khai kỹ thuật như framework và cơ sở dữ liệu. Tóm lại, ta thường thấy các codebase được chia thành phần "bên trong" (miền) và phần "bên ngoài" (hạ tầng), như Hình 34.3 gợi ý.

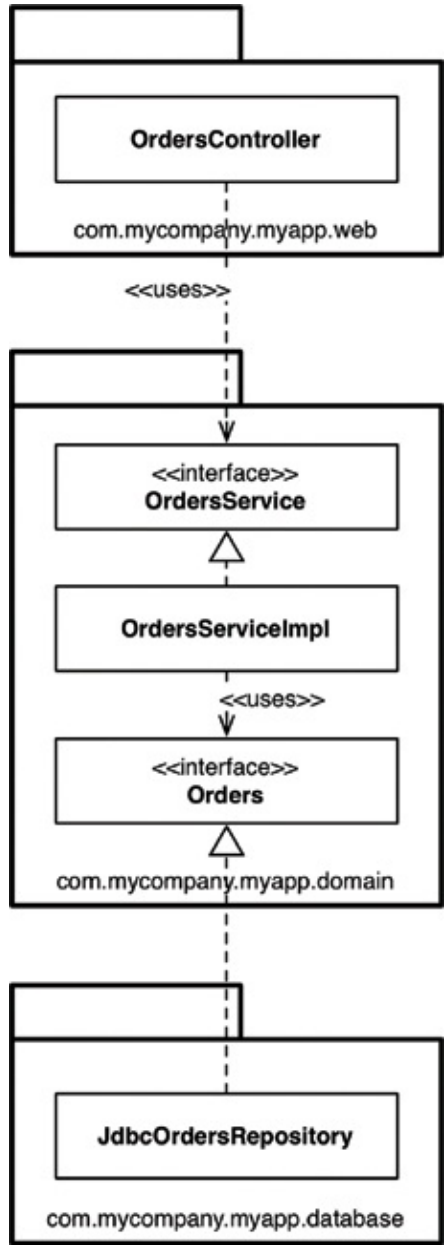


Hình 34.3
Codebase có phần bên trong và bên ngoài

Vùng "bên trong" chứa mọi khái niệm miền, còn vùng "bên ngoài" chứa các tương tác với thế giới bên ngoài, chẳng hạn giao diện người dùng, cơ sở dữ liệu và tích hợp bên thứ ba. Quy tắc chính là phần "bên ngoài" phụ thuộc vào phần "bên trong", không bao giờ ngược lại. Hình 34.4 cho thấy một phiên bản triển khai ca sử dụng "xem đơn hàng".

Package `com.mycompany.myapp.domain` là phần "bên trong"; các package khác là

phần “bên ngoài”. Hãy chú ý phụ thuộc đều hướng về phía “bên trong”. Độc giả tinh ý sẽ nhận thấy `OrdersRepository` trong các sơ đồ trước đã được đổi tên thành `Orders`. Cách đặt tên này xuất phát từ thiết kế hướng miền, trong đó mọi thứ ở “bên trong” nên được gọi theo “ngôn ngữ phổ quát của miền”. Nói cách khác, khi thảo luận về miền, ta nói về “đơn hàng”, không phải “repository đơn hàng”.



Hình 34.4

Ca sử dụng xem đơn hàng

Cũng cần lưu ý đây là phiên bản đơn giản hóa của sơ đồ lớp UML, vì nó thiếu các thành

phần như interactor và đối tượng dùng để tổ chức dữ liệu qua ranh giới phụ thuộc.

Chia gói theo thành phần

Dù hoàn toàn đồng ý với các thảo luận về SOLID, REP, CCP và CRP cũng như phần lớn lời khuyên trong cuốn sách này, tôi đi đến kết luận hơi khác về cách tổ chức mã. Vì vậy, tôi sẽ trình bày một lựa chọn khác, gọi là "chia gói theo thành phần". Để cung cấp bối cảnh, phần lớn sự nghiệp của tôi là xây dựng phần mềm doanh nghiệp, chủ yếu bằng Java, cho nhiều miền nghiệp vụ khác nhau. Các hệ thống ấy cũng rất đa dạng. Phần lớn chạy trên web, nhưng một số dùng kiến trúc máy khách-máy chủ,[4] phân tán, dựa trên thông điệp hoặc mô hình khác. Công nghệ có khác nhau, nhưng điểm chung là kiến trúc của hầu hết hệ thống dựa trên kiến trúc phân lớp truyền thống.

Tôi đã nêu vài lý do nên xem kiến trúc phân lớp là không tốt, nhưng đó chưa phải toàn bộ câu chuyện. Mục đích của kiến trúc phân lớp là tách mã có cùng chức năng. Phần web được tách khỏi logic nghiệp vụ, rồi logic nghiệp vụ được tách khỏi truy cập dữ liệu. Như ta thấy trong sơ đồ lớp UML, khi triển khai, một lớp thường tương ứng với một Java package. Để `OrdersController` có thể phụ thuộc vào giao diện `OrdersService`, giao diện này phải được đánh dấu `public` vì chúng nằm trong các package khác nhau. Tương tự, giao diện `OrdersRepository` phải là `public` để lớp `OrdersServiceImpl` có thể truy cập từ ngoài package repository.

Trong kiến trúc phân lớp nghiêm ngặt, các mũi tên phụ thuộc luôn phải hướng xuống, mỗi lớp chỉ phụ thuộc vào lớp liền kề bên dưới. Điều này giúp tạo đồ thị phụ thuộc không chu trình gọn gàng, đạt được bằng cách đặt quy tắc về cách các thành phần trong codebase được phép phụ thuộc nhau. Vấn đề lớn là ta có thể gian lận bằng cách tạo phụ thuộc không mong muốn mà vẫn giữ được một đồ thị phụ thuộc không chu trình.

Giả sử bạn tuyển một người mới vào nhóm và giao cho họ triển khai một ca sử dụng khác liên quan đến đơn hàng. Vì mới vào nhóm, người đó muốn gây ấn tượng và hoàn thành ca sử dụng thật nhanh. Sau vài phút ngồi uống cà phê, họ tìm thấy lớp `OrdersController` hiện có và quyết định đặt mã cho trang web mới liên quan đến đơn hàng vào đó. Nhưng trang cần dữ liệu đơn hàng từ cơ sở dữ liệu. Người mới chợt nghĩ ra: "Ồ, đã có giao diện `OrdersRepository` rồi. Mình chỉ cần tiêm phần triển khai vào controller là xong!" Vài phút sau, trang web hoạt động. Nhưng sơ đồ UML thu được trông như Hình 34.5.

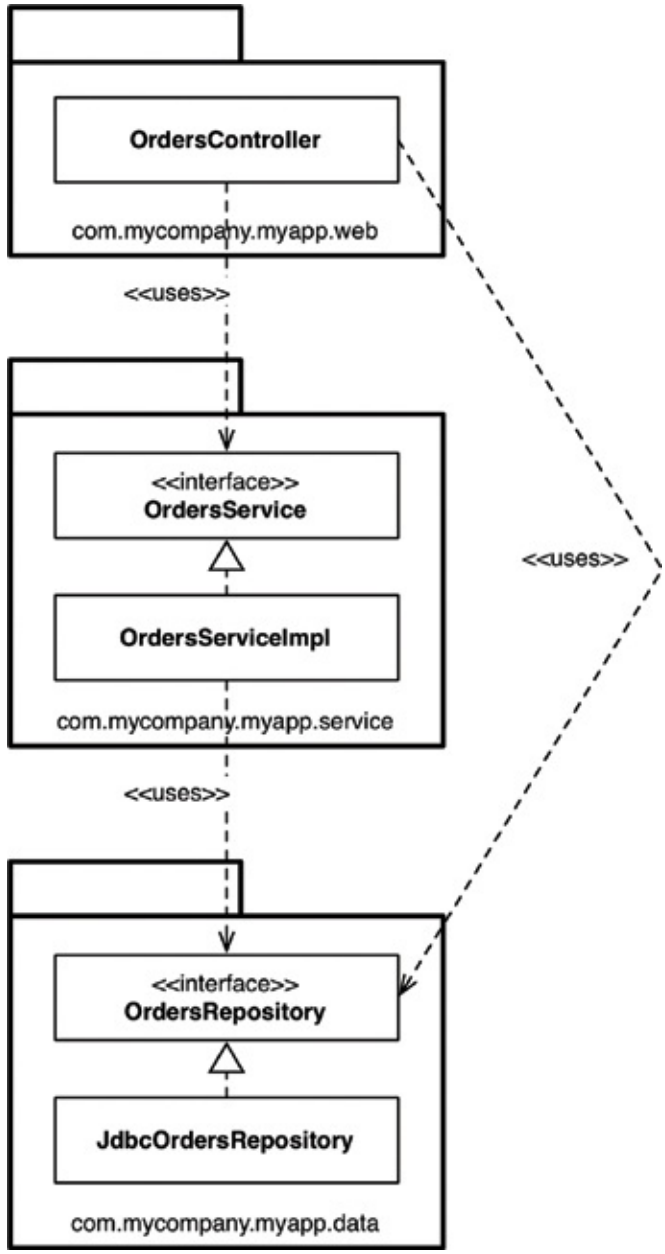
Các mũi tên phụ thuộc vẫn hướng xuống, nhưng giờ `OrdersController` bỏ qua `OrdersService` trong một số ca sử dụng. Cách tổ chức này thường gọi là kiến trúc phân lớp lỏng, trong đó một lớp được phép bỏ qua các lớp liền kề. Trong một số tình huống đây là kết quả có chủ ý, chẳng hạn khi áp dụng mẫu CQRS.[5] Trong nhiều trường hợp khác, bỏ

4. Công việc đầu tiên của tôi sau khi tốt nghiệp đại học năm 1996 là xây dựng ứng dụng máy khách-máy chủ trên máy tính để bàn bằng PowerBuilder, một ngôn ngữ thế hệ thứ tư rất năng suất để phát triển ứng dụng dùng cơ sở dữ liệu. Vài năm sau, tôi xây dựng ứng dụng máy khách-máy chủ bằng Java, khi đó chúng tôi phải tự viết phần kết nối cơ sở dữ liệu (trước JDBC) và bộ công cụ giao diện đồ họa riêng dựa trên AWT. Đó gọi là "tiến bộ" đấy!

5. Trong mẫu Tách biệt Trách nhiệm Lệnh và Truy vấn (CQRS), có các mẫu riêng để cập nhật và đọc dữ liệu.

qua lớp logic nghiệp vụ là điều không mong muốn, nhất là khi lớp này chịu trách nhiệm bảo đảm quyền truy cập vào từng bản ghi.

Ca sử dụng mới hoạt động, nhưng có lẽ không được triển khai như ta kỳ vọng. Tôi thường thấy điều này ở các nhóm tư vấn, và thường chỉ lộ ra khi nhóm lần đầu hình dung được cấu trúc thực sự của codebase.



Hình 34.5

Kiến trúc phân lớp lỏng

Ở đây cần một hướng dẫn, một nguyên tắc kiến trúc, đại loại như: “Controller web không

bao giờ được truy cập repository trực tiếp". Dĩ nhiên, câu hỏi là làm sao thực thi quy tắc đó. Nhiều nhóm tôi gặp chỉ nói: "Chúng tôi thực thi nguyên tắc này bằng kỷ luật tốt và đánh giá mã, vì tin tưởng các nhà phát triển." Sự tự tin đó thật đáng mừng, nhưng ai cũng biết chuyện gì xảy ra khi ngân sách và hạn chót ngày càng đến gần.

Một số ít nhóm khác cho biết họ dùng công cụ phân tích tĩnh, chẳng hạn NDepend, Structure101 và Checkstyle, để phát hiện và tự động thực thi các vi phạm kiến trúc trong lúc build. Có thể bạn từng thấy các quy tắc như vậy. Chúng thường là biểu thức chính quy hoặc chuỗi ký tự đại diện, quy định rằng "kiểu trong package `**/web` không được truy cập kiểu trong `**/data`", rồi được chạy sau bước biên dịch.

Cách tiếp cận này hơi thô sơ nhưng có thể hiệu quả. Nó báo cáo các vi phạm nguyên tắc kiến trúc do nhóm xác định và, hy vọng, làm build thất bại. Vấn đề của cả hai cách là chúng có thể sai sót, và vòng phản hồi dài hơn mức cần thiết. Nếu không kiểm soát, codebase có thể biến thành "quả cầu bùn lớn".[6] Nếu có thể, cá nhân tôi muốn dùng trình biên dịch để thực thi kiến trúc.

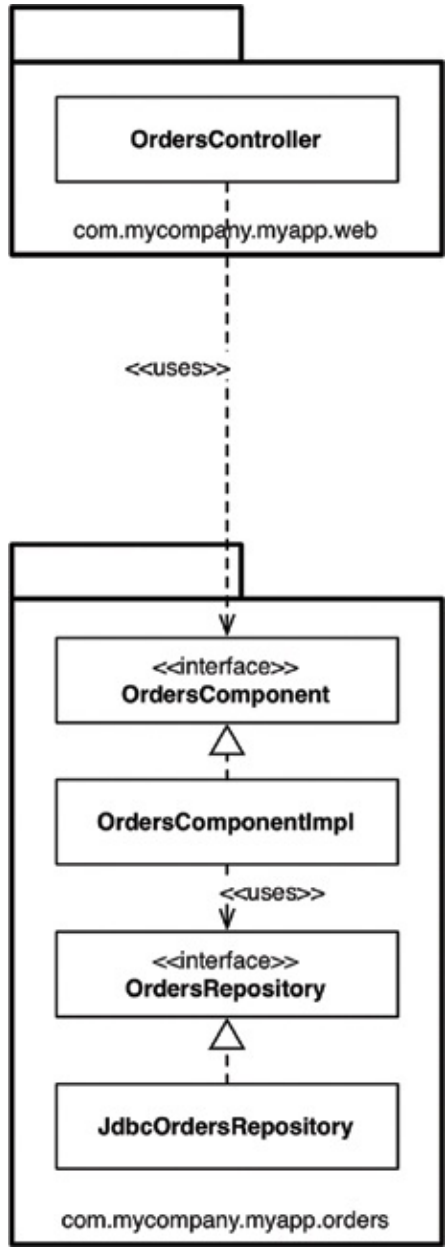
Đó là lúc đến với lựa chọn "chia gói theo thành phần". Đây là cách tiếp cận lai, kết hợp những gì đã thấy, nhằm gom mọi trách nhiệm liên quan đến một thành phần có độ hạt lớn vào một Java package. Cách này nhìn hệ thống theo hướng lấy dịch vụ làm trung tâm, cũng là điều ta thấy trong kiến trúc vi dịch vụ. Giống như cổng và bộ chuyển đổi xem web chỉ là một cơ chế phân phối khác, "chia gói theo thành phần" vẫn tách giao diện người dùng khỏi các thành phần có độ hạt lớn đó. Hình 34.6 cho thấy một cách triển khai ca sử dụng "xem đơn hàng".

Về bản chất, cách tiếp cận này gom "logic nghiệp vụ" và mã lưu trữ dữ liệu vào một đơn vị mà tôi gọi là "thành phần". Trước đó trong sách, Uncle Bob định nghĩa "thành phần" như sau:

> *Thành phần là đơn vị triển khai. Chúng là thực thể nhỏ nhất có thể được triển khai trong một hệ thống. Trong Java, chúng là các tệp jar.*

---

6. `http://www.laputan.org/mud/`



Hình 34.6

Ca sử dụng xem đơn hàng

Định nghĩa thành phần của tôi hơi khác: “Một nhóm chức năng liên quan được đặt sau một

giao diện rõ ràng, nằm trong môi trường thực thi như ứng dụng". Định nghĩa này xuất phát từ "mô hình kiến trúc phần mềm C4" của tôi,[7] một cách phân cấp đơn giản để hình dung cấu trúc tĩnh của hệ thống phần mềm bằng container, thành phần và lớp (hoặc mã nguồn). Theo mô hình này, hệ thống phần mềm gồm một hoặc nhiều container, chẳng hạn ứng dụng web, ứng dụng di động, ứng dụng độc lập, cơ sở dữ liệu hoặc hệ thống tệp. Mỗi container chứa một hoặc nhiều thành phần, mỗi thành phần được triển khai bằng một hoặc nhiều lớp (hoặc đoạn mã). Việc mỗi thành phần có nằm trong tệp jar riêng hay không là mối quan tâm độc lập với những điều đó.

Lợi ích chính của cách "chia gói theo thành phần" là nếu viết mã cần làm việc với đơn hàng thì chỉ có một nơi cần tìm: `OrdersComponent`. Bên trong thành phần, sự phân tách mối quan tâm vẫn được duy trì, nên logic nghiệp vụ vẫn tách khỏi lưu trữ dữ liệu. Nhưng đó là chi tiết triển khai mà người dùng thành phần không cần biết. Cách này tương tự kết quả khi dùng kiến trúc vi dịch vụ hoặc Kiến trúc Hướng Dịch vụ: một `OrdersService` riêng đóng gói mọi thứ liên quan đến xử lý đơn hàng. Điểm khác biệt chính là chế độ tách rời. Có thể xem các thành phần được định nghĩa rõ trong ứng dụng nguyên khối như bước đệm hướng đến kiến trúc vi dịch vụ.

Ma quỷ ẩn trong chi tiết triển khai

Thoạt nhìn, cả bốn cách tiếp cận đều có vẻ là những cách tổ chức mã khác nhau, nên có thể được xem là các kiểu kiến trúc khác nhau. Nhưng nhận định này nhanh chóng sụp đổ nếu triển khai sai chi tiết.

Tôi thường xuyên thấy người ta dùng quá thoải mái từ bổ nghĩa truy cập `public` trong các ngôn ngữ như Java. Có vẻ như lập trình viên dùng từ khóa `public` theo bản năng, không cần suy nghĩ. Nó đã thành phản xạ. Nếu không tin, hãy xem các mẫu mã trong sách, hướng dẫn và framework mã nguồn mở trên GitHub. Xu hướng này xuất hiện bất kể codebase hướng đến kiểu kiến trúc nào: lớp ngang, lớp dọc, cổng và bộ chuyển đổi hay kiểu khác. Đánh dấu mọi kiểu là `public` nghĩa là bạn không tận dụng các cơ chế đóng gói mà ngôn ngữ lập trình cung cấp. Trong một số trường hợp, không có gì ngăn ai đó viết mã khởi tạo trực tiếp một lớp triển khai cụ thể, vi phạm kiến trúc dự định.

Tổ chức và đóng gói

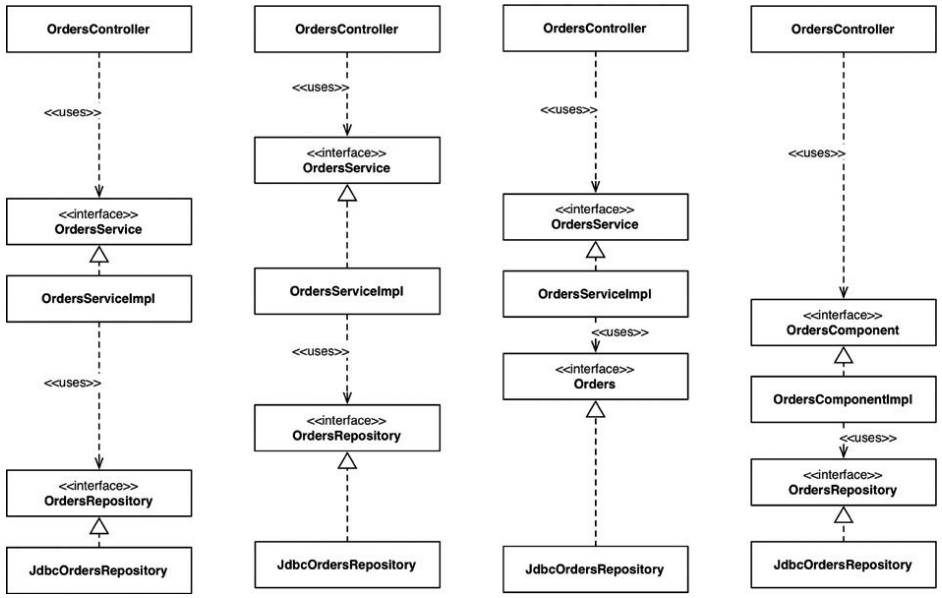
Nhìn vấn đề theo cách khác, nếu mọi kiểu trong ứng dụng Java đều là `public`, package chỉ còn là cơ chế tổ chức, một cách nhóm như thư mục, chứ không còn dùng để đóng gói. Vì kiểu `public` có thể được dùng ở bất kỳ đâu trong codebase, bạn có thể bỏ qua package vì chúng hầu như không đem lại giá trị thực. Kết quả là nếu bỏ qua package vì chúng không cung cấp cách đóng gói và che giấu, thì kiểu kiến trúc bạn muốn tạo ra cũng không còn quan trọng. Nhìn lại các sơ đồ UML, package Java trở thành chi tiết không liên quan nếu mọi kiểu đều được đánh dấu `public`. Về bản chất, cả bốn cách tiếp cận kiến trúc ở trên

7. Xem `https://www.structurizr.com/help/c4` để biết thêm thông tin.

đều giống nhau khi lạm dụng từ bổ nghĩa này (Hình 34.7).

Hãy xem kỹ các mũi tên giữa các kiểu trong Hình 34.7. Chúng đều giống nhau bất kể bạn đang theo kiểu kiến trúc nào. Về khái niệm, các cách tiếp cận rất khác nhau, nhưng về cú pháp lại giống hệt. Hơn nữa, có thể lập luận rằng nếu mọi kiểu đều `public`, thì thực chất bạn chỉ có bốn cách mô tả kiến trúc phân lớp ngang truyền thống. Đây là một thủ thuật thú vị. Dĩ nhiên, không ai lại đánh dấu mọi kiểu Java là `public`. Trừ những lúc họ làm vậy. Tôi đã thấy chuyện đó.

Từ bổ nghĩa truy cập trong Java không hoàn hảo,[8] nhưng bỏ qua chúng là tự chuốc rắc rối. Vị trí đặt kiểu Java trong package có thể tạo khác biệt lớn về khả năng truy cập, nếu dùng đúng các từ bổ nghĩa truy cập. Nếu đưa package trở lại sơ đồ và làm mờ các kiểu có thể áp dụng từ bổ nghĩa hạn chế hơn, ta sẽ thấy hình ảnh khá thú vị (Hình 34.8).



Hình 34.7

Cả bốn cách tiếp cận kiến trúc đều giống nhau

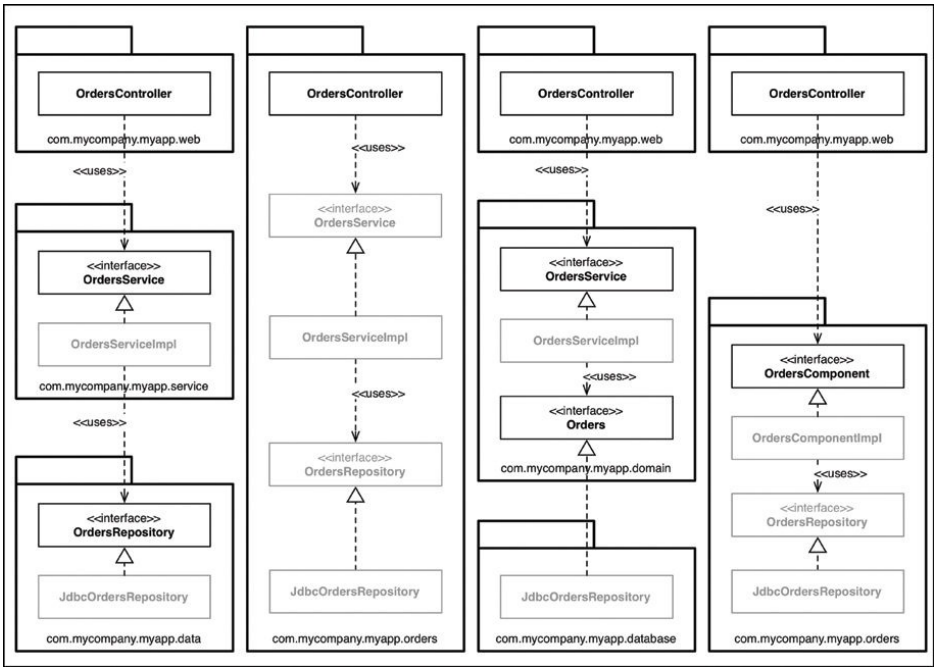
Chuyển từ trái sang phải, trong cách "chia gói theo lớp", các giao diện `OrdersService` và `OrdersRepository` phải là `public` vì lớp ở package khác phụ thuộc vào chúng. Ngược lại, các lớp triển khai (`OrdersServiceImpl` và `JdbcOrdersRepository`) có thể hạn chế truy cập trong package. Không ai cần biết đến chúng, vì chúng chỉ là chi tiết triển khai.

Trong cách "chia gói theo tính năng", `OrdersController` là điểm vào duy nhất của package, nên có thể hạn chế truy cập mọi thứ khác trong package. Điều kiện quan trọng là

8. Chẳng hạn trong Java, dù ta thường nghĩ package có cấu trúc phân cấp, nhưng không thể đặt giới hạn truy cập dựa trên quan hệ package-cha và package-con. Mọi cấu trúc phân cấp bạn tạo chỉ nằm trong tên package và cấu trúc thư mục trên đĩa.

không thành phần nào bên ngoài package được truy cập thông tin về đơn hàng nếu không đi qua controller. Điều này có thể mong muốn hoặc không.

Trong cách cổng và bộ chuyển đổi, các giao diện `OrdersService` và `Orders` có phụ thuộc đi vào từ package khác nên phải là `public`. Một lần nữa, các lớp triển khai có thể bị giới hạn trong package và được tiêm phụ thuộc lúc chạy.



Hình 34.8

Các kiểu màu xám là những kiểu có thể dùng từ bổ nghĩa truy cập hạn chế hơn

Cuối cùng, trong cách "chia gói theo thành phần", giao diện `OrdersComponent` có phụ thuộc đi vào từ controller, còn mọi thứ khác có thể bị giới hạn trong package. Càng ít kiểu `public` thì số phụ thuộc tiềm năng càng nhỏ. Giờ không có cách nào[9] để mã bên ngoài package dùng trực tiếp giao diện hoặc phần triển khai `OrdersRepository`, nên có thể dựa vào trình biên dịch để thực thi nguyên tắc kiến trúc này. Bạn có thể làm tương tự trong .NET bằng từ khóa `internal`, nhưng cần tạo assembly riêng cho từng thành phần.

Để nói thật rõ, điều tôi vừa mô tả áp dụng cho ứng dụng nguyên khối, trong đó toàn bộ mã nằm trong một cây mã nguồn. Nếu bạn xây dựng loại ứng dụng này, như rất nhiều người đang làm, tôi khuyến khích dùng trình biên dịch để thực thi nguyên tắc kiến trúc thay vì dựa vào kỷ luật cá nhân và công cụ hậu biên dịch.

9. Trừ khi gian lận bằng cơ chế reflection của Java, nhưng xin đừng làm vậy.

Các chế độ tách rời khác

Ngoài ngôn ngữ lập trình đang dùng, thường còn có những cách khác để tách phụ thuộc mã nguồn. Trong Java có các framework mô-đun như OSGi và hệ thống mô-đun Java 9 mới. Nếu dùng đúng cách, các hệ thống mô-đun cho phép phân biệt kiểu `public` với kiểu được công bố ra bên ngoài. Chẳng hạn, bạn có thể tạo một mô-đun Orders trong đó mọi kiểu đều được đánh dấu `public`, nhưng chỉ công bố một nhóm nhỏ cho bên ngoài sử dụng. Việc này đã đến muộn, nhưng tôi hào hứng vì hệ thống mô-đun Java 9 sẽ cho ta thêm một công cụ để xây dựng phần mềm tốt hơn và khơi lại sự quan tâm đến tư duy thiết kế.

Một lựa chọn khác là tách phụ thuộc mã nguồn bằng cách chia mã thành các cây mã nguồn riêng. Với ví dụ cổng và bộ chuyển đổi, ta có thể có ba cây mã nguồn:

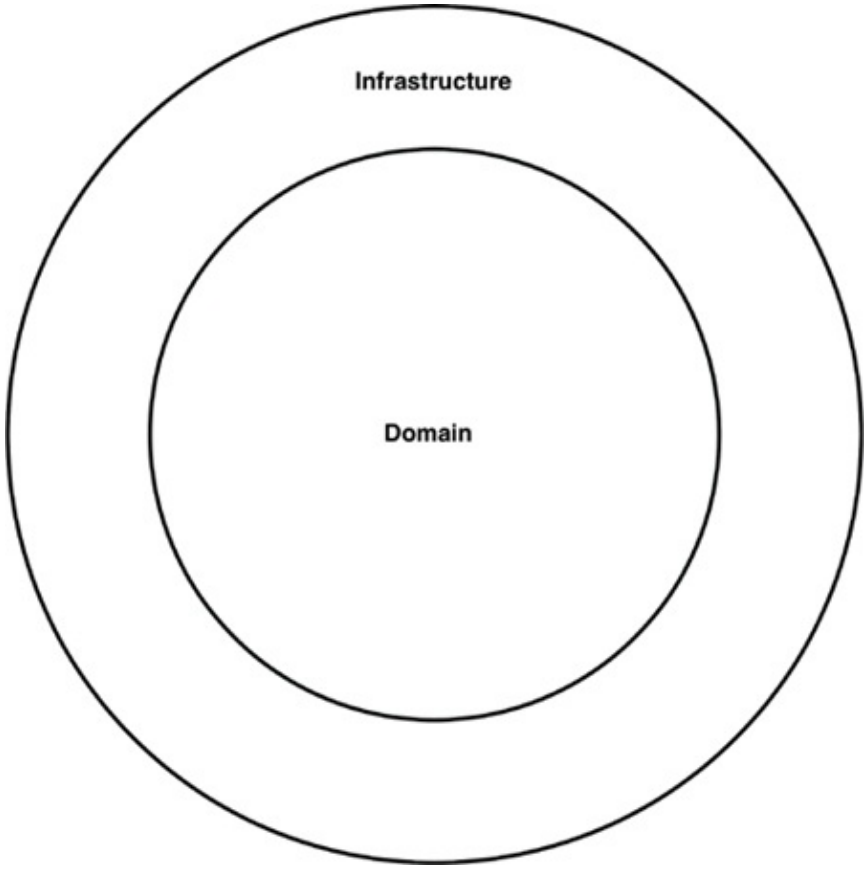
- Mã nguồn cho nghiệp vụ và miền, tức mọi thứ độc lập với công nghệ và framework: `OrdersService`, `OrdersServiceImpl` và `Orders`.
- Mã nguồn cho web: `OrdersController`.
- Mã nguồn cho lưu trữ dữ liệu: `JdbcOrdersRepository`.

Hai cây mã nguồn sau có phụ thuộc lúc biên dịch vào mã nghiệp vụ và miền. Bản thân phần nghiệp vụ không biết gì về web hay mã lưu trữ dữ liệu. Khi triển khai, có thể cấu hình các mô-đun hoặc dự án riêng trong công cụ build như Maven, Gradle hay MSBuild. Lý tưởng nhất là lặp lại cách làm này, tạo cây mã nguồn riêng cho từng thành phần trong ứng dụng. Tuy nhiên, đây là giải pháp lý tưởng hóa, vì việc chia mã nguồn theo cách này kéo theo các vấn đề thực tế về hiệu năng, độ phức tạp và bảo trì.

Một cách đơn giản hơn mà một số người dùng cho mã cổng và bộ chuyển đổi là chỉ tạo hai cây mã nguồn:

- Mã miền, tức phần "bên trong".
- Mã hạ tầng, tức phần "bên ngoài".

Cách này khớp đẹp với sơ đồ ở Hình 34.9, sơ đồ mà nhiều người dùng để tóm tắt kiến trúc cổng và bộ chuyển đổi. Phần hạ tầng có phụ thuộc lúc biên dịch vào phần miền.



Hình 34.9
Mã miền và mã hạ tầng

Cách tổ chức mã nguồn này cũng hiệu quả, nhưng cần lưu ý sự đánh đổi tiềm tàng. Tôi gọi đây là "phản mẫu Vành đai Périphérique của cổng và bộ chuyển đổi". Thành phố Paris có một đường vành đai mang tên Boulevard Périphérique, cho phép đi vòng quanh Paris mà không phải vào khu vực phức tạp bên trong. Nếu đặt toàn bộ mã hạ tầng vào một cây mã nguồn, mã hạ tầng ở một khu vực ứng dụng, chẳng hạn controller web, có thể gọi trực tiếp mã ở khu vực khác, chẳng hạn repository cơ sở dữ liệu, mà không đi qua miền. Điều này đặc biệt dễ xảy ra nếu quên dùng từ bổ nghĩa truy cập thích hợp.

Kết luận: lời khuyên còn thiếu

Mục đích chính của chương này là nhấn mạnh rằng ý định thiết kế tốt nhất cũng có thể bị phá hỏng trong chớp mắt nếu không tính đến những chi tiết của chiến lược triển khai. Hãy

nghĩ cách ánh xạ thiết kế mong muốn vào cấu trúc mã, cách tổ chức mã đó và chế độ tách rời cần dùng lúc chạy cũng như lúc biên dịch. Giữ các lựa chọn mở khi có thể, nhưng hãy thực tế, cân nhắc quy mô nhóm, trình độ kỹ năng và độ phức tạp của giải pháp cùng với giới hạn thời gian và ngân sách. Cũng nên dùng trình biên dịch để thực thi kiểu kiến trúc đã chọn, đồng thời chú ý đến phụ thuộc trong các khu vực khác như mô hình dữ liệu. Ma quỷ nằm trong chi tiết triển khai.

Phần VII: Phụ lục

Phụ lục A

Khảo cổ kiến trúc



Để khai quật các nguyên tắc của kiến trúc tốt, hãy cùng nhìn lại 45 năm dự án tôi đã tham gia kể từ năm 1970. Một số dự án đáng chú ý từ góc độ kiến trúc. Những dự án khác đáng nhớ vì bài học rút ra và ảnh hưởng của chúng đến các dự án sau.

Phụ lục này có phần tự truyện. Tôi đã cố giữ nội dung liên quan đến chủ đề kiến trúc, nhưng trong một bài tự truyện thì đôi khi những yếu tố khác vẫn xen vào. ;-)

Hệ thống kế toán công đoàn

Cuối những năm 1960, một công ty tên ASC Tabulating ký hợp đồng với Chi nhánh 705 của Công đoàn Teamsters để cung cấp hệ thống kế toán. Máy tính ASC chọn cho hệ thống là GE Datanet 30, như Hình A.1 cho thấy.



Hình A.1

GE Datanet 30 Ảnh do Ed Thelen cung cấp, ed-thelen.org

Như bạn thấy trong ảnh, đây là một cỗ máy khổng lồ.[1] Nó chiếm cả một căn phòng, và căn phòng cần được kiểm soát môi trường nghiêm ngặt.

Máy tính này được chế tạo trước thời mạch tích hợp, bằng các transistor rời. Bên trong còn có một số đèn điện tử, nhưng chỉ dùng trong bộ khuếch đại tín hiệu của ổ băng từ.

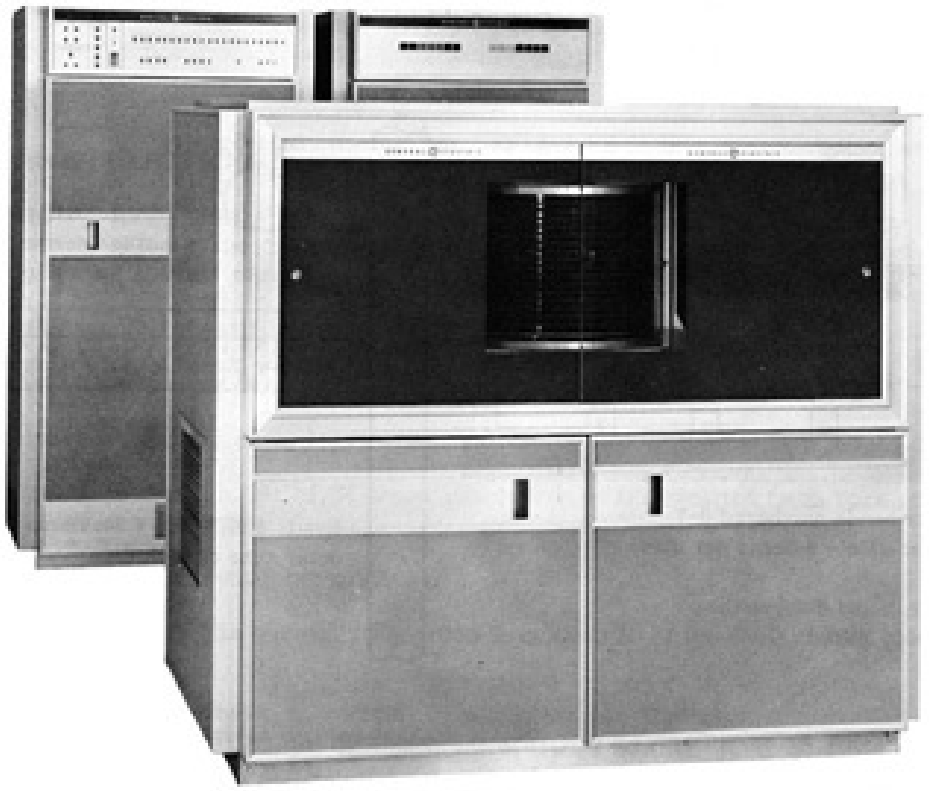
Theo tiêu chuẩn ngày nay, cỗ máy vừa khổng lồ, vừa chậm, nhỏ và thô sơ. Nó có 16K × 18 bit bộ nhớ lõi, chu kỳ khoảng 7 micro giây.[2] Máy chiếm một căn phòng lớn được kiểm soát môi trường. Nó có ổ băng từ 7 track và một ổ đĩa dung lượng khoảng 20 megabyte.

---

1. Một câu chuyện chúng tôi nghe về chiếc máy đặc biệt ở ASC là nó được chở bằng xe tải bán rơ-moóc lớn cùng với đồ nội thất gia đình. Trên đường đi, xe đâm vào cầu ở tốc độ cao. Máy tính vẫn nguyên vẹn, nhưng trượt về phía trước và nghiền nát đồ đạc thành mảnh vụn.

2. Ngày nay, ta sẽ nói nó có tần số xung nhịp 142 kHz.

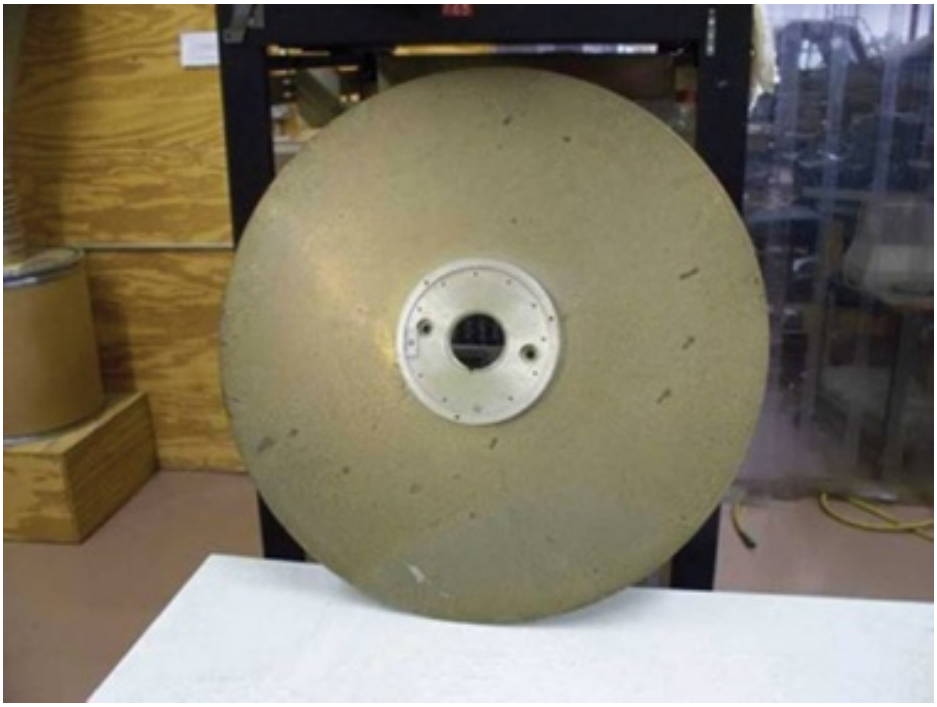
Ổ đĩa ấy là một con quái vật. Bạn có thể thấy nó trong Hình A.2, nhưng ảnh chưa thể hiện hết kích thước. Đỉnh tủ cao hơn đầu tôi. Các phiến đĩa đường kính 36 inch, dày 3/8 inch. Hình A.3 cho thấy một phiến đĩa.



MASS RANDOM ACCESS DATA STORAGE UNIT

Hình A.2

Bộ lưu trữ dữ liệu cùng các phiến đĩa Ảnh do Ed Thelen cung cấp, ed-thelen.org



Hình A.3
Một phiến đĩa: dày 3/8 inch, đường kính 36 inch

Bây giờ hãy đếm số phiến đĩa trong ảnh đầu tiên. Có hơn một tá. Mỗi phiến có một tay đọc riêng, được dẫn động bằng bộ truyền động khí nén. Bạn có thể nhìn các đầu đọc di chuyển ngang qua phiến đĩa. Thời gian tìm kiếm có lẽ từ nửa giây đến một giây.

Khi cỗ máy quái vật này khởi động, tiếng động giống động cơ phản lực. Sàn nhà rung lên cho đến khi máy đạt tốc độ.[3]

Thành tựu nổi bật của Datanet 30 là khả năng điều khiển nhiều thiết bị đầu cuối không đồng bộ với tốc độ khá cao. Đó chính xác là điều ASC cần.

ASC đặt tại Lake Bluff, Illinois, cách Chicago 30 dặm về phía bắc. Văn phòng Local 705 ở trung tâm Chicago. Công đoàn muốn khoảng một tá nhân viên nhập liệu dùng thiết bị đầu cuối CRT[4] để nhập dữ liệu vào hệ thống (Hình A.4). Báo cáo được in bằng máy điện báo ASR35 (Hình A.5).

---

3. Hãy tưởng tượng khối lượng của chiếc đĩa và động năng của nó! Một ngày nọ, chúng tôi đến nơi và thấy mạt kim loại nhỏ rơi ra từ đáy tủ. Chúng tôi gọi nhân viên bảo trì. Ông khuyên tắt máy. Khi đến sửa, ông nói một vòng bi đã mòn. Rồi ông kể rằng nếu không sửa, những chiếc đĩa này có thể tuột khỏi trục, xuyên qua tường gạch bê tông và cắm vào ô tô trong bãi đỗ.

4. Đèn hình tia âm cực: màn hình đơn sắc nền xanh, hiển thị ASCII.



Hình A.4

Thiết bị đầu cuối CRT Datapoint Ảnh do Bill Degnan cung cấp, vintagecomputer.net



Hình A.5

Máy điện báo ASR35 Ảnh do Joe Mabel cung cấp, được phép sử dụng

Các thiết bị CRT chạy ở tốc độ 30 ký tự mỗi giây. Cuối những năm 1960, đó là tốc độ khá tốt vì modem còn tương đối thô sơ.

ASC thuê từ công ty điện thoại khoảng một tá đường dây riêng và gấp đôi số modem 300 baud để nối Datanet 30 với các thiết bị đầu cuối này.

Những máy tính ấy không có hệ điều hành. Chúng thậm chí không có hệ thống tệp. Thứ bạn có chỉ là trình hợp dịch.

Nếu cần lưu dữ liệu trên đĩa, bạn ghi dữ liệu trực tiếp lên đĩa. Không có tệp. Không có thư mục. Bạn phải xác định track, phiến đĩa và sector nào sẽ chứa dữ liệu, rồi điều khiển ổ đĩa để ghi vào đó. Đúng vậy, chúng tôi tự viết trình điều khiển đĩa.

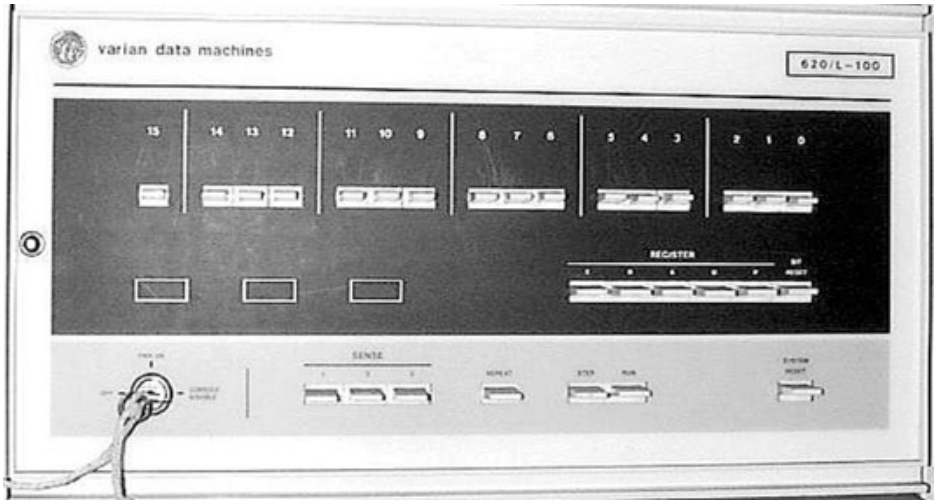
Hệ thống kế toán công đoàn có ba loại bản ghi: Đại lý, Nhà tuyển dụng và Thành viên. Hệ thống hỗ trợ các thao tác CRUD cho những bản ghi này, đồng thời có các chức năng ghi nhận phí đoàn, tính toán thay đổi trong sổ cái và những việc tương tự.

Hệ thống ban đầu được một chuyên gia tư vấn viết bằng hợp dịch. Bằng cách nào đó, ông đã nhồi được toàn bộ chương trình vào 16K.

Có thể hình dung cỗ Datanet 30 lớn ấy rất tốn kém để vận hành và bảo trì. Chuyên gia tư vấn phần mềm giữ cho hệ thống hoạt động cũng rất đắt. Hơn nữa, máy tính mini đang dần phổ biến và rẻ hơn nhiều.

Năm 1971, khi tôi 18 tuổi, ASC thuê tôi cùng hai người bạn mọt máy tính để thay thế toàn bộ hệ thống kế toán công đoàn bằng một hệ thống chạy trên máy tính mini Varian 620/f

(Hình A.6). Máy tính rẻ. Chúng tôi cũng rẻ. Vì vậy, đối với ASC, đây có vẻ là một món hời.



Hình A.6
Máy tính mini Varian 620/f

Máy Varian có bus 16 bit và bộ nhớ lõi 32K × 16. Chu kỳ khoảng 1 micro giây. Nó mạnh hơn Datanet 30 nhiều. Máy dùng công nghệ đĩa IBM 2314 rất thành công, cho phép lưu 30 megabyte trên các phiến đĩa đường kính chỉ 14 inch, không thể xuyên thủng tường gạch bê tông khi nổ tung!

Dĩ nhiên, chúng tôi vẫn không có hệ điều hành, hệ thống tệp hay ngôn ngữ cấp cao. Chúng tôi chỉ có trình hợp dịch. Nhưng chúng tôi vẫn xoay xở được.

Trại trẻ mồ côi máy tính mini

Thay vì cố nhồi toàn bộ hệ thống vào 32K, chúng tôi tạo một hệ thống overlay. Các ứng dụng được nạp từ đĩa vào một vùng bộ nhớ dành riêng cho overlay. Chúng chạy trong vùng nhớ đó, rồi bị hoán đổi trở lại đĩa cùng RAM cục bộ để nhường chỗ cho chương trình khác.

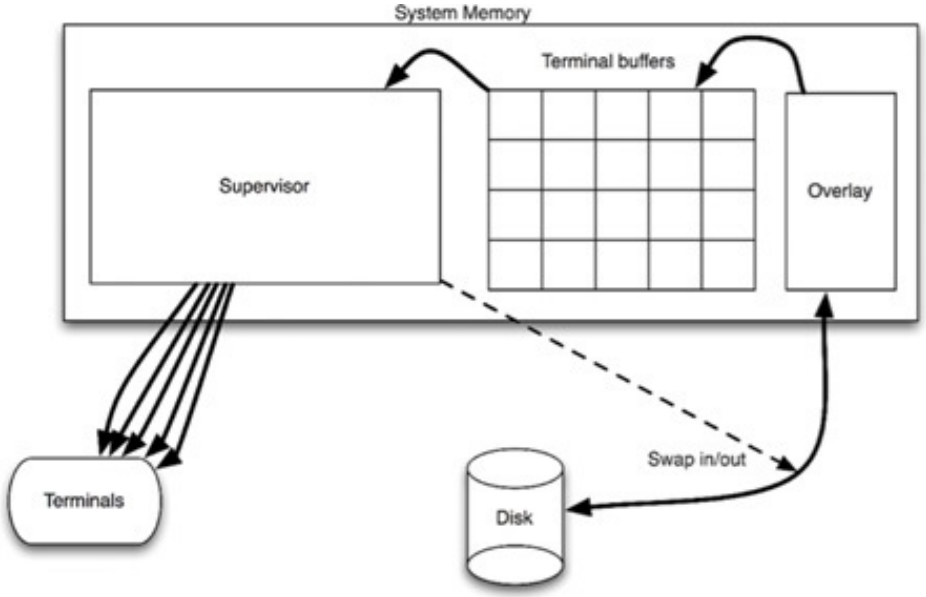
Chương trình được hoán đổi vào vùng overlay, chạy đủ lâu để làm đầy bộ đệm đầu ra, rồi bị hoán đổi ra để chương trình khác được nạp vào.

Dĩ nhiên, khi giao diện người dùng chỉ chạy ở 30 ký tự mỗi giây, chương trình dành rất nhiều thời gian chờ. Chúng tôi có nhiều thời gian để hoán đổi chương trình vào và ra khỏi đĩa, giữ cho mọi thiết bị đầu cuối chạy nhanh nhất có thể. Không ai phàn nàn về thời gian phản hồi.

Chúng tôi viết một bộ giám sát có khả năng ưu tiên, quản lý ngắt và IO. Chúng tôi viết ứng dụng, trình điều khiển đĩa, trình điều khiển thiết bị đầu cuối, trình điều khiển băng từ và mọi thứ khác trong hệ thống. Không có phần nào mà chúng tôi không tự viết. Dù phải vật lộn với quá nhiều tuần làm việc 80 giờ, chúng tôi cũng đưa được cỗ máy quái vật vào

hoạt động chỉ trong 8 hoặc 9 tháng.

Kiến trúc hệ thống rất đơn giản (Hình A.7). Khi một ứng dụng khởi chạy, nó tạo đầu ra cho đến khi bộ đệm đầu cuối tương ứng đầy. Sau đó, bộ giám sát hoán đổi ứng dụng đó ra và nạp ứng dụng mới vào. Bộ giám sát tiếp tục gửi nội dung trong bộ đệm đến thiết bị đầu cuối với tốc độ 30 ký tự mỗi giây cho đến khi bộ đệm gần cạn. Khi đó, nó hoán đổi ứng dụng trở lại để nạp đầy bộ đệm.



Hình A.7
Kiến trúc hệ thống

Trong hệ thống này có hai ranh giới. Ranh giới thứ nhất là ranh giới đầu ra ký tự. Ứng dụng không biết đầu ra sẽ đến thiết bị đầu cuối 30 ký tự mỗi giây. Thực tế, từ góc nhìn ứng dụng, đầu ra ký tự hoàn toàn trừu tượng. Ứng dụng chỉ chuyển chuỗi cho bộ giám sát. Bộ giám sát chịu trách nhiệm nạp bộ đệm, gửi ký tự đến thiết bị đầu cuối và hoán đổi ứng dụng vào, ra khỏi bộ nhớ.

Ranh giới này có phụ thuộc xuôi, tức phụ thuộc cùng hướng với luồng điều khiển. Ứng dụng phụ thuộc lúc biên dịch vào bộ giám sát, và luồng điều khiển đi từ ứng dụng đến bộ giám sát. Ranh giới ngăn ứng dụng biết đầu ra sẽ được gửi đến loại thiết bị nào.

Ranh giới thứ hai đảo ngược phụ thuộc. Bộ giám sát có thể khởi chạy ứng dụng, nhưng không có phụ thuộc lúc biên dịch vào chúng. Luồng điều khiển đi từ bộ giám sát đến ứng dụng. Giao diện đa hình đảo ngược phụ thuộc rất đơn giản: mọi ứng dụng đều được khởi chạy bằng cách nhảy đến cùng một địa chỉ bộ nhớ chính xác trong vùng overlay. Ranh giới ngăn bộ giám sát biết bất kỳ điều gì về ứng dụng, ngoại trừ điểm khởi chạy.

Cắt laser

Năm 1973, tôi gia nhập một công ty ở Chicago tên là Teradyne Applied Systems (TAS). Đây là một bộ phận của Teradyne Inc., có trụ sở chính tại Boston. Sản phẩm của chúng tôi là hệ thống dùng laser công suất tương đối cao để cắt tỉa linh kiện điện tử với độ chính xác rất cao.

Thời đó, nhà sản xuất in lụa các linh kiện điện tử lên đế gốm. Các đế này cỡ khoảng một inch vuông. Linh kiện thường là điện trở, thiết bị cản dòng điện.

Điện trở phụ thuộc vào nhiều yếu tố, trong đó có thành phần vật liệu và hình dạng. Điện trở càng rộng thì điện trở suất càng thấp.

Hệ thống đặt đế gốm vào một giá đỡ có đầu dò tiếp xúc với điện trở. Hệ thống đo điện trở, rồi dùng laser đốt bỏ một phần để làm điện trở mỏng dần cho đến khi đạt giá trị mong muốn, với sai số khoảng một phần mười phần trăm.

Chúng tôi bán các hệ thống này cho nhà sản xuất. Chúng tôi cũng dùng một số hệ thống nội bộ để cắt tỉa các lô tương đối nhỏ cho nhà sản xuất nhỏ.

Máy tính là M365. Thời đó, nhiều công ty tự chế tạo máy tính của mình. Teradyne chế tạo M365 và cung cấp cho mọi bộ phận. M365 là phiên bản cải tiến của PDP-8, một máy tính mini phổ biến lúc bấy giờ.

M365 điều khiển bàn định vị, di chuyển đế gốm bên dưới đầu dò. Nó điều khiển hệ thống đo và laser. Laser được định vị bằng gương X-Y có thể xoay theo chương trình. Máy tính cũng điều khiển được công suất laser.

Môi trường phát triển trên M365 tương đối thô sơ. Không có đĩa. Bộ nhớ lưu trữ lớn là các băng từ dạng hộp giống băng âm thanh 8-track cũ. Băng và ổ đĩa do Tri-Data sản xuất.

Giống băng âm thanh 8-track thời đó, băng được nối thành vòng. Ổ chỉ chạy băng theo một hướng, không có chức năng tua lại! Muốn đưa băng về đầu, bạn phải cho nó chạy đến "điểm nạp".

Băng chạy với tốc độ khoảng một foot mỗi giây. Vì vậy, nếu vòng băng dài 25 foot, có thể mất đến 25 giây để đưa nó về điểm nạp. Do đó, Tri-Data sản xuất hộp băng với nhiều độ dài, từ 10 đến 100 foot.

M365 có một nút ở mặt trước. Nhấn nút sẽ nạp một chương trình bootstrap nhỏ vào bộ nhớ và thực thi nó. Chương trình đọc khối dữ liệu đầu tiên từ băng rồi chạy khối đó. Thông thường, khối này chứa trình nạp, dùng để nạp hệ điều hành nằm trên phần còn lại của băng.

Hệ điều hành sẽ yêu cầu người dùng nhập tên chương trình cần chạy. Các chương trình được lưu trên băng ngay sau hệ điều hành. Ta nhập tên chương trình, chẳng hạn trình soạn thảo ED-402. Hệ điều hành tìm chương trình đó trên băng, nạp vào bộ nhớ rồi chạy.

Bảng điều khiển là CRT ASCII dùng phosphor xanh, rộng 72 ký tự[5] và cao 24 dòng. Tất cả ký tự đều viết hoa.

Để sửa chương trình, bạn nạp trình soạn thảo ED-402 rồi đưa vào băng chứa mã nguồn. Bạn đọc một khối mã nguồn từ băng vào bộ nhớ, nội dung được hiển thị trên màn hình. Một khối băng có thể chứa 50 dòng mã. Bạn sửa bằng cách di chuyển con trỏ trên màn hình và

5. Con số 72 bắt nguồn từ thẻ đục lỗ Hollerith, mỗi thẻ có 80 ký tự. Tám ký tự cuối được "dành riêng" cho số thứ tự phòng khi bạn làm rơi bộ thẻ.

gõ theo cách tương tự vi. Khi xong, bạn ghi khối đó sang một băng khác rồi đọc khối tiếp theo từ băng nguồn. Cứ thế cho đến khi hoàn tất.

Không thể cuộn ngược về các khối trước đó. Bạn sửa chương trình theo đường thẳng từ đầu đến cuối. Muốn quay lại đầu, bạn phải chép xong mã nguồn sang băng đầu ra rồi bắt đầu một phiên sửa mới trên băng đó. Có lẽ không ngạc nhiên khi chúng tôi in chương trình ra giấy, đánh dấu mọi chỉnh sửa bằng mực đỏ, rồi sửa mã từng khối dựa vào bản in đã đánh dấu. Khi chỉnh sửa xong, chúng tôi quay lại hệ điều hành và chạy trình hợp dịch. Trình hợp dịch đọc băng mã nguồn và ghi băng nhị phân, đồng thời in danh sách mã ra máy in dòng.

Băng không đáng tin cậy tuyệt đối, nên chúng tôi luôn ghi đồng thời hai băng. Như vậy, ít nhất một băng có xác suất cao không bị lỗi.

Chương trình của chúng tôi dài khoảng 20.000 dòng mã và mất gần 30 phút để biên dịch. Khả năng gặp lỗi đọc băng trong thời gian đó vào khoảng 1 trên 10. Nếu trình hợp dịch gặp lỗi băng, nó sẽ rung chuông ở bảng điều khiển rồi in một loạt lỗi. Bạn có thể nghe tiếng chuông điên dại khắp phòng thí nghiệm, cùng tiếng chửi rủa của lập trình viên vừa biết mình phải bắt đầu lại lượt biên dịch 30 phút.

Kiến trúc chương trình điển hình cho thời ấy. Có một Chương trình Điều hành Chính, gọi tắt là MOP. Nó quản lý các chức năng IO cơ bản và cung cấp những tiện ích sơ khai của một “shell” trên bảng điều khiển. Nhiều bộ phận của Teradyne dùng chung mã nguồn MOP, nhưng mỗi bộ phận đều rẽ nhánh để dùng riêng. Vì vậy, chúng tôi gửi cập nhật mã nguồn cho nhau dưới dạng bản in đánh dấu, rồi tích hợp thủ công một cách rất cẩn thận.

Một lớp tiện ích chuyên dụng điều khiển phần cứng đo, bàn định vị và laser. Ranh giới giữa lớp này và MOP tốt nhất cũng chỉ mơ hồ. Lớp tiện ích gọi MOP, nhưng MOP đã được sửa riêng cho lớp đó và thường gọi ngược trở lại. Thực tế, chúng tôi không xem chúng là hai lớp riêng biệt. Với chúng tôi, đó chỉ là mã được thêm vào MOP theo cách gắn kết chặt chẽ.

Tiếp theo là lớp cách ly. Lớp này cung cấp giao diện máy ảo cho các chương trình ứng dụng, được viết bằng một ngôn ngữ đặc thù miền hoàn toàn khác, dựa trên dữ liệu. Ngôn ngữ có các phép toán để di chuyển laser, di chuyển bàn, cắt, đo và những việc tương tự. Khách hàng viết chương trình cắt tỉa bằng ngôn ngữ này, còn lớp cách ly thực thi chúng.

Mục tiêu không phải tạo một ngôn ngữ cắt laser độc lập với máy. Thực tế, ngôn ngữ có nhiều đặc điểm riêng gắn chặt với các lớp bên dưới. Thay vào đó, nó cung cấp cho lập trình viên ứng dụng một ngôn ngữ “đơn giản hơn” hợp dịch M356 để lập trình công việc cắt tỉa.

Công việc cắt tỉa được nạp từ băng và thực thi bởi hệ thống. Về cơ bản, hệ thống của chúng tôi là một hệ điều hành cho các ứng dụng cắt tỉa.

Hệ thống được viết bằng hợp dịch M365 và biên dịch thành một đơn vị duy nhất, tạo ra mã nhị phân tuyệt đối.

Các ranh giới trong ứng dụng này tốt nhất cũng chỉ lỏng lẻo. Ngay cả ranh giới giữa mã hệ thống và ứng dụng viết bằng DSL cũng không được thực thi tốt. Phụ thuộc có ở khắp nơi.

Nhưng đó là tình trạng phổ biến của phần mềm vào đầu thập niên 1970.

Giám sát đúc nhôm

Giữa thập niên 1970, khi OPEC cấm vận dầu mỏ và tình trạng thiếu xăng khiến các tài xế giận dữ đánh nhau ở trạm xăng, tôi bắt đầu làm việc tại Outboard Marine Corporation (OMC). Đây là công ty mẹ của Johnson Motors và máy cắt cỏ Lawnboy.

OMC có một cơ sở khổng lồ ở Waukegan, Illinois, chuyên đúc các bộ phận nhôm cho động cơ và sản phẩm của công ty. Nhôm được nung chảy trong những lò lớn, rồi chở bằng các thùng lớn đến hàng chục máy đúc áp lực nhôm vận hành riêng lẻ. Mỗi máy có một người vận hành chịu trách nhiệm lắp khuôn, chạy máy và lấy chi tiết vừa đúc ra. Lương của họ được tính theo số lượng chi tiết sản xuất.

Tôi được thuê tham gia dự án tự động hóa xưởng sản xuất. OMC mua một máy tính IBM System/7, câu trả lời của IBM cho dòng máy tính mini. Họ nối máy tính này với mọi máy đúc trên sàn để đếm và đo thời gian chu kỳ của từng máy. Vai trò của chúng tôi là thu thập thông tin rồi hiển thị trên màn hình xanh IBM 3270.

Ngôn ngữ lập trình là hợp dịch. Một lần nữa, mọi mã chạy trên máy tính này đều do chúng tôi viết. Không có hệ điều hành, thư viện chương trình con hay framework. Chỉ có mã thô.

Đây cũng là mã thời gian thực, điều khiển bằng ngắt. Mỗi khi máy đúc hoàn tất một chu kỳ, chúng tôi phải cập nhật một loạt thống kê và gửi thông điệp đến một máy IBM 370 khổng lồ ở xa, chạy chương trình CICS-COBOL để hiển thị thống kê trên màn hình xanh.

Tôi ghét công việc này. Trời ơi, ghét lắm. Công việc thì vui! Nhưng văn hóa công ty thì… Chỉ cần nói rằng tôi bị yêu cầu phải đeo cà vạt.

Ồ, tôi đã cố gắng. Thật sự đã cố. Nhưng rõ ràng tôi không vui khi làm ở đó, và đồng nghiệp đều biết. Họ biết vì tôi không nhớ được những ngày quan trọng hoặc không dậy đủ sớm để dự các cuộc họp cần thiết. Đây là công việc lập trình duy nhất mà tôi từng bị sa thải, và tôi đáng bị như vậy.

Từ góc độ kiến trúc, ở đây không có nhiều điều để học, ngoài một điểm. System/7 có một chỉ thị thú vị tên là set program interrupt (SPI). Nó cho phép kích hoạt ngắt bộ xử lý để xử lý các ngắt ưu tiên thấp hơn đang chờ. Ngày nay, trong Java, ta gọi việc đó là `Thread.yield()`.

4-TEL

Tháng 10 năm 1976, sau khi bị OMC sa thải, tôi quay lại một bộ phận khác của Teradyne, nơi tôi gắn bó suốt 12 năm. Sản phẩm tôi tham gia mang tên 4-TEL. Mục đích là kiểm tra mọi đường dây điện thoại trong một khu vực dịch vụ mỗi đêm, rồi lập báo cáo về các đường dây cần sửa chữa. Hệ thống cũng cho phép nhân viên kiểm tra chi tiết từng đường dây cụ thể.

Ban đầu hệ thống có kiến trúc giống hệ thống cắt laser. Đó là ứng dụng nguyên khối viết bằng hợp dịch, không có ranh giới đáng kể. Nhưng vào lúc tôi gia nhập công ty, điều đó sắp thay đổi.

Hệ thống được các nhân viên kiểm tra trong trung tâm dịch vụ (SC) sử dụng. Một trung tâm dịch vụ bao phủ nhiều tổng đài trung tâm (CO), mỗi tổng đài có thể phục vụ đến 10.000 đường dây điện thoại. Phần cứng quay số và đo lường phải được đặt bên trong CO, nên máy tính M365 được đặt ở đó. Chúng tôi gọi chúng là thiết bị kiểm tra đường dây tổng đài (COLT). Một máy M365 khác được đặt tại SC và gọi là máy tính khu vực dịch vụ (SAC). SAC có một số modem để gọi đến COLT và giao tiếp ở tốc độ 300 baud, tức 30 ký tự mỗi giây.

Ban đầu, máy tính COLT làm mọi việc, kể cả giao tiếp với bảng điều khiển, hiển thị menu và tạo báo cáo. SAC chỉ là bộ ghép kênh đơn giản, nhận đầu ra từ COLT rồi hiển thị lên màn hình.

Vấn đề là 30 ký tự mỗi giây rất chậm. Nhân viên không thích nhìn ký tự lần lượt hiện ra trên màn hình, nhất là khi họ chỉ quan tâm đến một vài thông tin quan trọng. Hơn nữa, lúc đó bộ nhớ lõi của M365 rất đắt và chương trình lại lớn.

Giải pháp là tách phần mềm quay số và đo đường dây khỏi phần phân tích kết quả và in báo cáo. Phần sau được chuyển sang SAC, còn phần trước vẫn ở COLT. Nhờ vậy, COLT có thể nhỏ hơn và cần ít bộ nhớ hơn. Phản hồi trên thiết bị đầu cuối cũng nhanh hơn nhiều vì SAC tạo báo cáo.

Kết quả rất thành công. Màn hình cập nhật nhanh, sau khi quay số đến COLT thích hợp, và dung lượng bộ nhớ của COLT giảm đáng kể.

Ranh giới được tạo ra rất rõ ràng và tách rời chặt chẽ. SAC và COLT trao đổi các gói dữ liệu rất ngắn. Những gói này là một dạng DSL rất đơn giản, biểu diễn các lệnh cơ bản như “DIAL XXXX” hoặc “MEASURE”.

M365 được nạp từ băng từ. Các ổ băng này đắt tiền và không đáng tin cậy, nhất là trong môi trường công nghiệp của tổng đài điện thoại. Ngoài ra, M365 đắt so với các thiết bị điện tử khác trong COLT. Vì vậy, chúng tôi bắt đầu dự án thay M365 bằng một máy tính vi mô dùng bộ xử lý 8085.

Máy tính mới gồm một bo mạch xử lý chứa 8085, một bo RAM chứa 32K RAM và ba bo ROM, mỗi bo chứa 12K bộ nhớ chỉ đọc. Tất cả các bo vừa vào cùng khung máy với phần cứng đo lường, nhờ đó không cần khung máy cồng kềnh riêng cho M365.

Các bo ROM chứa 12 chip EPROM Intel 2708, tức bộ nhớ chỉ đọc có thể xóa và lập trình.[6] Hình A.8 cho thấy một chip như vậy. Chúng tôi nạp phần mềm vào chip bằng thiết

6. Đúng, tôi hiểu đây là một cách diễn đạt tự mâu thuẫn.

bị đặc biệt gọi là bộ ghi PROM, được điều khiển từ môi trường phát triển. Có thể xóa chip bằng cách chiếu tia cực tím cường độ cao.[7]



Hình A.8
Chip EPROM

Bạn tôi CK và tôi dịch chương trình hợp dịch M365 của COLT sang hợp dịch 8085. Chúng tôi dịch thủ công và mất khoảng sáu tháng. Kết quả cuối cùng dài khoảng 30K mã 8085.

Môi trường phát triển của chúng tôi có 64K RAM và không có ROM, nên có thể nhanh chóng tải các tệp nhị phân đã biên dịch vào RAM để kiểm thử.

Khi chương trình hoạt động, chúng tôi chuyển sang dùng EPROM. Chúng tôi ghi

7. Chip có một cửa sổ nhựa trong nhỏ để nhìn thấy silicon bên trong, đồng thời cho tia cực tím đi vào xóa dữ liệu.

chương trình vào 30 chip rồi lắp từng chip vào đúng khe trên ba bo ROM. Mỗi chip được dán nhãn để biết nó nằm ở khe nào.

Chương trình 30K là một tệp nhị phân duy nhất, dài 30K. Để ghi chip, chúng tôi chia ảnh nhị phân đó thành 30 đoạn, mỗi đoạn 1K, rồi ghi từng đoạn vào chip có nhãn tương ứng.

Cách này hoạt động rất tốt. Chúng tôi bắt đầu sản xuất hàng loạt phần cứng và triển khai hệ thống ngoài thực tế.

Nhưng phần mềm thì mềm.[8] Mọi người cần thêm tính năng và sửa lỗi. Khi số hệ thống được lắp đặt tăng lên, việc cập nhật phần mềm bằng cách ghi 30 chip cho mỗi lần triển khai, rồi cử nhân viên dịch vụ hiện trường thay cả 30 chip tại từng địa điểm, trở thành cơn ác mộng.

Có đủ loại vấn đề. Đôi khi chip bị dán nhầm nhãn hoặc nhãn bong ra. Đôi khi kỹ sư dịch vụ hiện trường thay nhầm chip. Có lúc họ vô tình làm gãy một chân của chip mới. Vì vậy, kỹ sư hiện trường phải mang theo chip dự phòng cho cả 30 vị trí.

Vì sao phải thay cả 30 chip? Mỗi khi thêm hoặc xóa mã trong tệp thực thi 30K, địa chỉ nạp từng lệnh đều thay đổi. Địa chỉ các chương trình con và hàm được gọi cũng thay đổi. Vì thế, mọi chip đều bị ảnh hưởng, cho dù thay đổi nhỏ đến đâu.

Một ngày nọ, sếp đến hỏi tôi cách giải quyết vấn đề. Ông nói cần một cách sửa firmware mà không phải thay cả 30 chip mỗi lần. Chúng tôi cùng suy nghĩ một lúc rồi bắt đầu dự án "Vectorization". Dự án kéo dài ba tháng.

Ý tưởng rất đơn giản. Chúng tôi chia chương trình 30K thành 32 tệp mã nguồn có thể biên dịch độc lập, mỗi tệp dưới 1K. Ở đầu mỗi tệp, chúng tôi chỉ dẫn cho trình biên dịch địa chỉ nạp chương trình tạo ra, chẳng hạn `ORG C400` cho chip sẽ được lắp ở vị trí C4.

Ở đầu mỗi tệp mã nguồn, chúng tôi cũng tạo một cấu trúc dữ liệu có kích thước cố định, chứa địa chỉ mọi chương trình con trên chip đó. Cấu trúc dài 40 byte, đủ chứa tối đa 20 địa chỉ. Điều đó nghĩa là mỗi chip không thể có quá 20 chương trình con.

Tiếp theo, chúng tôi tạo một vùng đặc biệt trong RAM gọi là vector. Vùng này chứa 32 bảng, mỗi bảng dài 40 byte, vừa đủ để lưu các con trỏ ở đầu mỗi chip.

Cuối cùng, chúng tôi đổi mọi lời gọi chương trình con trên mọi chip thành lời gọi gián tiếp qua vector RAM tương ứng.

Khi bộ xử lý khởi động, nó quét từng chip và nạp bảng vector ở đầu mỗi chip vào vùng vector trong RAM. Sau đó, nó nhảy vào chương trình chính.

Cách này hoạt động rất tốt. Khi sửa lỗi hoặc thêm tính năng, giờ đây chúng tôi chỉ cần biên dịch lại một hoặc hai chip rồi gửi riêng những chip đó cho kỹ sư dịch vụ hiện trường.

Chúng tôi đã làm cho các chip có thể triển khai độc lập. Chúng tôi đã phát minh ra điều phối đa hình. Chúng tôi đã phát minh ra đối tượng.

Đây thực sự là một kiến trúc plugin. Chúng tôi cắm các chip vào hệ thống. Cuối cùng, chúng tôi thiết kế để có thể cài một tính năng vào sản phẩm bằng cách cắm chip chứa tính năng đó vào khe còn trống. Menu điều khiển tự động xuất hiện và phần liên kết với ứng dụng chính cũng được thiết lập tự động.

---

8. Đúng, tôi biết khi phần mềm được ghi vào ROM thì nó được gọi là firmware, nhưng ngay cả firmware thực chất vẫn "mềm".

Dĩ nhiên, lúc đó chúng tôi chưa biết các nguyên tắc hướng đối tượng, cũng không biết gì về việc tách giao diện người dùng khỏi quy tắc nghiệp vụ. Nhưng những yếu tố căn bản đã có mặt, và chúng rất mạnh mẽ.

Một lợi ích bất ngờ khác là chúng tôi có thể vá firmware qua kết nối quay số. Nếu tìm thấy lỗi trong firmware, chúng tôi gọi đến thiết bị và dùng chương trình giám sát tích hợp để đổi vector RAM của chương trình con lỗi sang một vùng RAM trống. Sau đó, chúng tôi nhập chương trình con đã sửa vào vùng RAM đó bằng mã máy hệ thập lục phân.

Điều này giúp ích rất nhiều cho hoạt động dịch vụ hiện trường và khách hàng. Khi có vấn đề, họ không cần chờ chúng tôi gửi chip mới và xếp lịch một chuyến bảo trì khẩn cấp. Hệ thống có thể được vá ngay, còn chip mới được lắp trong lần bảo trì định kỳ tiếp theo.

Máy tính khu vực dịch vụ

Máy tính khu vực dịch vụ (SAC) của 4-TEL dựa trên máy tính mini M365. Hệ thống này giao tiếp với tất cả COLT ngoài hiện trường, qua modem riêng hoặc modem quay số. Nó yêu cầu COLT đo đường dây điện thoại, nhận kết quả thô rồi phân tích phức tạp để xác định và định vị lỗi.

Xác định nơi điều phối

Một nền tảng kinh tế của hệ thống là phân bổ đúng thợ sửa chữa. Theo quy định công đoàn, thợ được chia thành ba nhóm: thợ tổng đài trung tâm, thợ cáp và thợ đường dây thuê bao. Thợ tổng đài sửa lỗi bên trong tổng đài trung tâm. Thợ cáp sửa lỗi trên mạng cáp nối tổng đài với khách hàng. Thợ đường dây thuê bao sửa lỗi bên trong cơ sở khách hàng và trên đường dây nối cáp bên ngoài với cơ sở đó, tức đoạn "drop".

Khi khách hàng báo sự cố, hệ thống chẩn đoán và xác định cần điều phối nhóm thợ nào. Điều này giúp công ty điện thoại tiết kiệm rất nhiều vì điều phối sai khiến khách hàng phải chờ và thợ phải đi lại vô ích.

Mã quyết định điều phối được thiết kế và viết bởi một người rất thông minh nhưng giao tiếp rất kém. Quá trình viết mã được mô tả là: "Ba tuần nhìn chằm chằm lên trần nhà, rồi hai ngày mã tuôn ra từ mọi lỗ trên người anh ấy, sau đó anh ấy nghỉ việc."

Không ai hiểu mã đó. Mỗi lần thêm tính năng hoặc sửa lỗi, chúng tôi đều làm hỏng nó theo cách nào đó. Vì một trong những lợi ích kinh tế chính của hệ thống dựa trên mã này, mỗi lỗi mới đều khiến công ty vô cùng xấu hổ.

Cuối cùng, ban quản lý yêu cầu chúng tôi khóa chặt mã đó và không bao giờ sửa nữa. Mã chính thức trở nên cứng nhắc.

Trải nghiệm này khiến tôi hiểu giá trị của mã sạch, tốt.

Kiến trúc

Hệ thống được viết năm 1976 bằng hợp dịch M365. Đó là một chương trình nguyên khối duy nhất, khoảng 60.000 dòng. Hệ điều hành do công ty tự viết, không có ưu tiên và chuyển tác vụ dựa trên polling. Chúng tôi gọi nó là MPS, viết tắt của hệ thống đa xử lý. Máy tính M365 không có stack tích hợp, nên biến riêng của từng tác vụ được lưu trong vùng nhớ đặc biệt rồi hoán đổi ra vào khi chuyển ngữ cảnh. Biến dùng chung được quản lý bằng khóa và semaphore. Vấn đề tái nhập và điều kiện tranh chấp xảy ra liên tục.

Không có sự cách ly giữa logic điều khiển thiết bị, giao diện người dùng và quy tắc nghiệp vụ. Chẳng hạn, mã điều khiển modem bị rải khắp phần lớn mã quy tắc nghiệp vụ và giao diện người dùng. Không ai cố gom nó thành mô-đun hay trừu tượng hóa giao diện. Modem được điều khiển ở mức bit bằng mã phân tán khắp hệ thống.

Điều tương tự cũng đúng với giao diện thiết bị đầu cuối. Mã điều khiển thông điệp và định dạng không được cách ly. Chúng rải khắp codebase 60.000 dòng.

Các mô-đun modem chúng tôi dùng được thiết kế để gắn trên bo mạch in. Chúng tôi mua thiết bị từ bên thứ ba rồi tích hợp với các mạch khác trên một bo mạch lắp vào backplane tùy chỉnh. Các mô-đun này đắt tiền. Vài năm sau, chúng tôi quyết định tự thiết kế modem. Chúng tôi trong nhóm phần mềm van nài kỹ sư phần cứng giữ nguyên các định dạng bit điều khiển modem cũ. Chúng tôi giải thích rằng mã điều khiển modem bị rải khắp nơi, và trong tương lai hệ thống phải hỗ trợ cả hai loại modem. Chúng tôi nài nỉ, thuyết phục: “Xin hãy làm cho modem mới trông giống modem cũ từ góc nhìn điều khiển phần mềm.”

Nhưng khi nhận modem mới, cấu trúc điều khiển hoàn toàn khác. Không chỉ khác một chút, mà khác hoàn toàn.

Cảm ơn nhé, kỹ sư phần cứng.

Chúng tôi phải làm gì? Chúng tôi không chỉ thay toàn bộ modem cũ bằng modem mới. Thay vào đó, chúng tôi kết hợp modem cũ và mới trong cùng hệ thống. Phần mềm phải xử lý đồng thời cả hai loại. Chúng tôi có buộc phải bao quanh mọi chỗ điều khiển modem bằng cờ và trường hợp đặc biệt không? Có hàng trăm chỗ như vậy!

Cuối cùng, chúng tôi chọn một giải pháp còn tệ hơn. Một chương trình con cụ thể ghi dữ liệu lên bus giao tiếp nối tiếp dùng để điều khiển mọi thiết bị, kể cả modem. Chúng tôi sửa chương trình con đó để nhận diện các mẫu bit riêng của modem cũ rồi chuyển chúng thành mẫu bit mà modem mới cần.

Việc này không hề đơn giản. Lệnh modem gồm chuỗi thao tác ghi vào các địa chỉ IO khác nhau trên bus nối tiếp. Bản vá phải diễn giải tuần tự các lệnh đó rồi chuyển thành chuỗi khác, với địa chỉ IO, thời gian và vị trí bit khác.

Chúng tôi làm cho nó hoạt động được, nhưng đó là bản vá tệ nhất có thể tưởng tượng. Chính thất bại này giúp tôi hiểu giá trị của việc cách ly phần cứng khỏi quy tắc nghiệp vụ và trừu tượng hóa giao diện.

Cuộc đại thiết kế lại trong mơ

Đến thập niên 1980, ý tưởng tự sản xuất máy tính mini và kiến trúc máy tính riêng dần lỗi thời. Trên thị trường có nhiều máy tính vi mô, rẻ và tiêu chuẩn hóa hơn việc tiếp tục dựa vào kiến trúc độc quyền từ cuối thập niên 1960. Cùng với kiến trúc tồi tệ của phần mềm SAC, điều đó khiến ban quản lý kỹ thuật bắt đầu tái kiến trúc toàn bộ hệ thống SAC.

Hệ thống mới sẽ được viết bằng C, dùng hệ điều hành UNIX trên đĩa và chạy trên máy vi tính Intel 8086. Nhóm phần cứng bắt đầu làm máy tính mới, còn một nhóm nhỏ lập trình viên, "Đội Hổ" (Tiger Team), được giao viết lại phần mềm.

Tôi sẽ không làm bạn chán với chi tiết về thất bại ban đầu. Chỉ cần nói rằng Đội Hổ đầu tiên thất bại hoàn toàn sau khi tiêu tốn hai hoặc ba năm công sức cho dự án phần mềm không tạo ra được sản phẩm nào.

Một hoặc hai năm sau, có lẽ vào năm 1982, họ khởi động lại quy trình. Mục tiêu là thiết kế lại toàn bộ SAC bằng C và UNIX trên phần cứng 80286 mới do chúng tôi tự thiết kế, rất mạnh mẽ. Chúng tôi gọi máy tính đó là "Deep Thought".

Quá trình kéo dài nhiều năm, rồi thêm nhiều năm nữa. Tôi không biết SAC chạy UNIX đầu tiên cuối cùng được triển khai khi nào. Tôi nghĩ lúc đó tôi đã rời công ty, vào năm 1988. Thực tế, tôi không chắc nó có bao giờ được triển khai hay không.

Vì sao trì hoãn? Nói ngắn gọn, nhóm thiết kế lại rất khó bắt kịp đội ngũ lập trình viên lớn đang tích cực duy trì hệ thống cũ. Đây chỉ là một ví dụ về khó khăn họ gặp phải.

Châu Âu

Gần thời điểm SAC được viết lại bằng C, công ty bắt đầu mở rộng doanh số sang châu Âu. Họ không thể chờ phần mềm thiết kế lại hoàn tất, nên tất nhiên đã triển khai hệ thống M365 cũ ở châu Âu.

Vấn đề là hệ thống điện thoại châu Âu rất khác hệ thống ở Hoa Kỳ. Cách tổ chức nghề nghiệp và bộ máy hành chính cũng khác. Vì vậy, một trong những lập trình viên giỏi nhất của chúng tôi được cử sang Vương quốc Anh để dẫn dắt nhóm phát triển địa phương, sửa đổi phần mềm SAC cho phù hợp với các vấn đề ở châu Âu.

Dĩ nhiên, không có nỗ lực nghiêm túc nào được thực hiện để tích hợp những thay đổi đó vào phần mềm ở Hoa Kỳ. Đây là thời kỳ trước khi mạng có thể truyền những codebase lớn qua đại dương. Các lập trình viên ở Anh chỉ đơn giản rẽ nhánh mã Hoa Kỳ và sửa theo nhu cầu. Điều đó gây ra nhiều khó khăn. Ở cả hai bờ Đại Tây Dương đều phát hiện lỗi cần được sửa từ phía bên kia. Nhưng các mô-đun đã thay đổi đáng kể, nên rất khó xác định bản sửa ở Hoa Kỳ có hoạt động ở Anh hay không. Sau vài năm đau đầu và khi đường truyền tốc độ cao nối văn phòng Hoa Kỳ với Anh được lắp đặt, một nỗ lực nghiêm túc đã được tiến hành để hợp nhất hai nhánh mã, biến khác biệt thành vấn đề cấu hình. Nỗ lực này thất bại trong cả ba lần thử. Hai codebase rất giống nhau, nhưng vẫn quá khác biệt để hợp nhất, nhất là trong môi trường thị trường thay đổi nhanh lúc đó. Trong khi đó, "Đội Hổ", đang cố viết lại mọi thứ bằng C và UNIX, nhận ra họ cũng phải xử lý sự khác biệt giữa châu Âu và Hoa Kỳ. Điều đó dĩ nhiên không đẩy nhanh tiến độ.

Kết luận về SAC

Còn nhiều câu chuyện khác tôi có thể kể về hệ thống này, nhưng chúng quá chán nản để tiếp tục. Chỉ cần nói rằng nhiều bài học khó khăn nhất trong sự nghiệp phần mềm của tôi đến từ việc đắm mình trong mã hợp dịch kinh khủng của SAC.

Ngôn ngữ C

Phần cứng máy tính 8085 mà chúng tôi dùng trong dự án 4-Tel Micro cung cấp nền tảng tính toán chi phí thấp cho nhiều dự án có thể nhúng vào môi trường công nghiệp. Chúng tôi có thể trang bị cho nó 32K RAM và thêm 32K ROM, cùng một cơ chế rất linh hoạt và mạnh mẽ để điều khiển thiết bị ngoại vi. Nhưng chúng tôi không có một ngôn ngữ linh hoạt và tiện dụng để lập trình máy. Viết mã bằng hợp dịch 8085 thật chẳng vui chút nào.

Hơn nữa, trình hợp dịch chúng tôi dùng do chính các lập trình viên trong công ty viết. Nó chạy trên máy M365, dùng hệ điều hành băng từ hộp được mô tả trong mục "Cắt laser".

Số phận đã sắp đặt để kỹ sư phần cứng trưởng thuyết phục giám đốc điều hành rằng chúng tôi cần một máy tính thật. Ông ấy không thực sự biết sẽ dùng nó làm gì, nhưng có ảnh hưởng chính trị lớn. Vì vậy, chúng tôi mua một máy PDP-11/60.

Khi đó tôi chỉ là một lập trình viên phần mềm cấp thấp, nhưng vô cùng phấn khích. Tôi biết chính xác mình muốn làm gì với máy tính ấy. Tôi quyết tâm biến nó thành máy của mình.

Hướng dẫn sử dụng được gửi đến nhiều tháng trước khi máy giao. Tôi mang chúng về nhà đọc ngấu nghiến. Đến khi máy được giao, tôi đã hiểu rất sâu về cả phần cứng lẫn phần mềm, ít nhất là sâu nhất có thể nhờ tự học ở nhà.

Tôi tham gia soạn đơn đặt hàng. Đặc biệt, tôi chỉ định loại lưu trữ đĩa cho máy tính mới. Tôi quyết định mua hai ổ đĩa có thể dùng các bộ đĩa tháo rời, mỗi bộ chứa 25 megabyte.[9]

Năm mươi megabyte! Con số ấy dường như vô tận! Tôi nhớ mình đi dọc hành lang văn phòng vào đêm khuya, cười khúc khích như Phù thủy Độc ác phương Tây: "Năm mươi megabyte! Hahahahahahahahah!"

Tôi nhờ người quản lý cơ sở vật chất xây một căn phòng nhỏ có sáu thiết bị đầu cuối VT100. Tôi trang trí phòng bằng ảnh chụp ngoài không gian. Các nhà phát triển phần mềm sẽ dùng phòng này để viết và biên dịch mã.

Khi máy được giao, tôi mất vài ngày lắp đặt, nối dây cho các thiết bị đầu cuối và làm cho mọi thứ hoạt động. Đó là niềm vui, một công việc xuất phát từ đam mê.

Chúng tôi mua các trình hợp dịch chuẩn cho 8085 từ Boston Systems Office và chuyển mã 4-Tel Micro sang cú pháp đó. Chúng tôi xây dựng hệ thống biên dịch chéo cho phép tải các tệp nhị phân đã biên dịch từ PDP-11 xuống môi trường phát triển 8085 và thiết bị ghi ROM. Và thế là mọi thứ hoạt động ngon lành.

---

9. RKO7.

C

Nhưng vấn đề hợp dịch 8085 vẫn còn đó, và tôi không hài lòng. Tôi nghe nói có một ngôn ngữ "mới" được dùng rộng rãi ở Bell Labs. Họ gọi nó là "C". Vì vậy, tôi mua cuốn *The C Programming Language* của Kernighan và Ritchie. Giống như sách hướng dẫn PDP-11 vài tháng trước, tôi đọc ngấu nghiến cuốn này.

Tôi kinh ngạc trước vẻ thanh lịch giản dị của ngôn ngữ này. Nó không hy sinh sức mạnh nào của hợp dịch, nhưng cho phép truy cập sức mạnh đó bằng cú pháp tiện dụng hơn nhiều. Tôi hoàn toàn bị thuyết phục.

Tôi mua trình biên dịch C của Whitesmiths và cho chạy trên PDP-11. Đầu ra của trình biên dịch là cú pháp hợp dịch tương thích với trình biên dịch 8085 của Boston Systems Office. Nhờ vậy, chúng tôi có con đường từ C đến phần cứng 8085! Chúng tôi đã sẵn sàng.

Vấn đề duy nhất còn lại là thuyết phục nhóm lập trình viên nhúng vốn chỉ viết hợp dịch chuyển sang dùng C. Nhưng đó là một câu chuyện ác mộng để kể vào lúc khác…

BOSS

Nền tảng 8085 của chúng tôi không có hệ điều hành. Trải nghiệm với hệ thống MPS trên M365 và cơ chế ngắt thô sơ của IBM System/7 khiến tôi tin rằng 8085 cần một bộ chuyển tác vụ đơn giản. Vì vậy, tôi nghĩ ra BOSS: Hệ điều hành và Bộ lập lịch Cơ bản (Basic Operating System and Scheduler).[10]

Phần lớn BOSS được viết bằng C. Nó cho phép tạo các tác vụ đồng thời. Các tác vụ không bị ngắt ưu tiên. Việc chuyển tác vụ không dựa trên ngắt, mà giống hệ thống MPS trên M365, dựa trên cơ chế polling đơn giản. Polling xảy ra mỗi khi một tác vụ chờ sự kiện.

Lời gọi BOSS để tạm dừng một tác vụ như sau:

```
block(eventCheckFunction);
```

Lời gọi này tạm dừng tác vụ hiện tại, đưa `eventCheckFunction` vào danh sách polling và gắn hàm đó với tác vụ vừa bị chặn. Sau đó, hệ thống chờ trong vòng lặp polling, gọi từng hàm trong danh sách cho đến khi một hàm trả về `true`. Tác vụ gắn với hàm đó sẽ được phép chạy.

Nói cách khác, như tôi đã nói, đó là một bộ chuyển tác vụ đơn giản, không có ưu tiên.

Phần mềm này trở thành nền tảng cho vô số dự án trong những năm tiếp theo. Một trong những dự án đầu tiên là pCCU.

pCCU

Cuối thập niên 1970 và đầu thập niên 1980 là giai đoạn biến động đối với các công ty điện thoại. Một nguyên nhân là cuộc cách mạng số.

Trong suốt thế kỷ trước đó, tổng đài trung tâm kết nối với điện thoại khách hàng bằng một cặp dây đồng. Các dây được bó thành cáp trải rộng khắp vùng nông thôn. Có dây được

10. Sau này tên này được đổi thành "Bob's Only Successful Software".

treo trên cột, có dây được chôn dưới đất.

Đồng là kim loại quý, và công ty điện thoại có hàng tấn, đúng nghĩa là hàng tấn, dây đồng phủ khắp cả nước. Khoản đầu tư ban đầu khổng lồ đó có thể được thu hồi phần nào bằng cách truyền cuộc gọi qua kết nối số. Một cặp dây đồng có thể truyền hàng trăm cuộc gọi dưới dạng số.

Để đáp ứng xu hướng này, các công ty điện thoại bắt đầu thay thiết bị chuyển mạch trung tâm analog cũ bằng tổng đài số hiện đại.

Sản phẩm 4-Tel của chúng tôi kiểm tra dây đồng, chứ không phải kết nối số. Trong môi trường số vẫn có nhiều dây đồng, nhưng chúng ngắn hơn trước và nằm gần điện thoại khách hàng. Tín hiệu được truyền dưới dạng số từ tổng đài trung tâm đến điểm phân phối cục bộ, tại đó nó được chuyển lại thành tín hiệu analog rồi truyền đến khách hàng qua dây đồng thông thường. Điều đó có nghĩa thiết bị đo phải đặt ở nơi bắt đầu các dây đồng, còn thiết bị quay số phải ở lại tổng đài trung tâm. Vấn đề là mọi COLT gộp cả chức năng quay số lẫn đo lường trong cùng một thiết bị. Nếu nhận ra ranh giới kiến trúc hiển nhiên này sớm hơn vài năm, chúng tôi đã tiết kiệm được cả một gia tài!

Vì vậy, chúng tôi nghĩ ra kiến trúc sản phẩm mới: CCU/CMU, tức bộ điều khiển COLT và bộ đo COLT. CCU đặt tại tổng đài trung tâm và quay số đến các đường dây cần kiểm tra. CMU đặt tại điểm phân phối cục bộ và đo dây đồng dẫn đến điện thoại khách hàng.

Vấn đề là mỗi CCU phục vụ nhiều CMU. Thông tin cho biết cần dùng CMU nào với từng số điện thoại nằm trong tổng đài số. Vì vậy, CCU phải truy vấn tổng đài số để xác định cần giao tiếp và điều khiển CMU nào.

Chúng tôi hứa với các công ty điện thoại sẽ hoàn thành kiến trúc mới đúng thời điểm họ chuyển đổi. Chúng tôi biết họ còn cách nhiều tháng, thậm chí nhiều năm, nên không cảm thấy vội. Chúng tôi cũng biết cần vài năm công sức để phát triển phần cứng và phần mềm CCU/CMU mới.

Cái bẫy lịch trình

Thời gian trôi qua, chúng tôi luôn có việc khẩn cấp khiến phải hoãn phát triển kiến trúc CCU/CMU. Chúng tôi thấy an toàn khi hoãn vì các công ty điện thoại liên tục trì hoãn triển khai tổng đài số. Xem lịch trình của họ, chúng tôi tin chắc mình còn nhiều thời gian và liên tục hoãn dự án.

Rồi một ngày, sếp gọi tôi vào văn phòng và nói: “Một khách hàng sẽ triển khai tổng đài số vào tháng tới. Chúng ta phải có CCU/CMU hoạt động trước lúc đó.”

Tôi bàng hoàng! Làm sao có thể hoàn thành nhiều năm công sức phát triển chỉ trong một tháng? Nhưng sếp tôi đã có kế hoạch…

Thực tế, chúng tôi không cần kiến trúc CCU/CMU hoàn chỉnh. Công ty điện thoại triển khai tổng đài số rất nhỏ. Họ chỉ có một tổng đài trung tâm và hai điểm phân phối cục bộ. Quan trọng hơn, các điểm phân phối “cục bộ” đó không gần lắm. Chúng thực ra có các tổng đài analog thông thường, kết nối đến vài trăm khách hàng. Tốt hơn nữa, đó là loại tổng đài có thể được COLT thông thường quay số đến. Còn tốt hơn nữa, số điện thoại khách hàng chứa mọi thông tin cần thiết để quyết định dùng điểm phân phối nào. Nếu số điện thoại có chữ số 5, 6 hoặc 7 ở vị trí nhất định thì gọi đến điểm phân phối 1, nếu không thì gọi đến

điểm phân phối 2.

Vì vậy, như sếp giải thích, chúng tôi thực ra không cần CCU/CMU. Điều cần thiết là một máy tính đơn giản ở tổng đài trung tâm, nối qua modem với hai COLT tiêu chuẩn tại các điểm phân phối. SAC giao tiếp với máy tính ở tổng đài trung tâm. Máy tính đó giải mã số điện thoại rồi chuyển lệnh quay số và đo lường đến COLT tại điểm phân phối thích hợp.

Thế là pCCU ra đời.

Đây là sản phẩm đầu tiên được triển khai cho khách hàng, viết bằng C và dùng BOSS. Tôi mất khoảng một tuần phát triển. Câu chuyện này không có ý nghĩa kiến trúc sâu sắc, nhưng là lời dẫn hay cho dự án tiếp theo.

DLU/DRU

Đầu thập niên 1980, một trong các khách hàng của chúng tôi là công ty điện thoại ở Texas. Họ phải phục vụ một khu vực địa lý rộng lớn. Thực tế, khu vực đó rộng đến mức một vùng dịch vụ cần nhiều văn phòng khác nhau để điều phối thợ. Những văn phòng ấy có nhân viên kỹ thuật cần thiết bị đầu cuối kết nối đến SAC.

Có thể bạn nghĩ đây là vấn đề đơn giản, nhưng hãy nhớ câu chuyện xảy ra đầu thập niên 1980. Thiết bị đầu cuối từ xa không phổ biến. Tệ hơn nữa, phần cứng SAC giả định rằng mọi thiết bị đầu cuối đều ở gần. Thiết bị của chúng tôi nối với nhau bằng bus nối tiếp tốc độ cao độc quyền.

Chúng tôi có khả năng kết nối thiết bị đầu cuối từ xa, nhưng cách đó dựa trên modem, mà đầu thập niên 1980 modem thường chỉ đạt 300 bit mỗi giây. Khách hàng không hài lòng với tốc độ chậm đó.

Modem tốc độ cao có bán trên thị trường nhưng rất đắt, và cần đường truyền riêng "đã xử lý". Chất lượng đường dây quay số chắc chắn không đủ tốt.

Khách hàng yêu cầu giải pháp. Chúng tôi đưa ra DLU/DRU.

DLU/DRU là viết tắt của "Display Local Unit" và "Display Remote Unit", tức Bộ Hiển thị Cục bộ và Bộ Hiển thị Từ xa. DLU là bo máy tính cắm vào khung máy SAC và giả làm bo quản lý thiết bị đầu cuối. Tuy nhiên, thay vì điều khiển bus nối tiếp cho thiết bị cục bộ, nó nhận luồng ký tự rồi ghép kênh qua một đường modem riêng 9600 bps.

DRU là một hộp đặt tại địa điểm từ xa của khách hàng. Nó nối với đầu kia của đường 9600 bps và có phần cứng điều khiển các thiết bị đầu cuối cục bộ trên bus nối tiếp độc quyền. Nó tách kênh các ký tự nhận qua đường 9600 bps rồi gửi đến đúng thiết bị đầu cuối cục bộ.

Lạ thật phải không? Chúng tôi phải chế tạo một giải pháp mà ngày nay phổ biến đến mức chẳng ai nghĩ đến. Nhưng thời đó…

Chúng tôi còn phải tự phát minh giao thức truyền thông vì khi ấy các giao thức chuẩn không phải phần mềm chia sẻ mã nguồn mở. Thực tế, lúc đó chúng tôi chưa có bất kỳ kết nối Internet nào.

Kiến trúc

Kiến trúc hệ thống này rất đơn giản, nhưng có vài điểm thú vị tôi muốn nêu. Trước hết, cả hai thiết bị đều dùng công nghệ 8085, được viết bằng C và dùng BOSS. Nhưng đó là điểm tương đồng duy nhất.

Dự án có hai người. Tôi là trưởng dự án, Mike Carew là cộng sự thân thiết. Tôi thiết kế và viết mã cho DLU; Mike phụ trách DRU.

Kiến trúc DLU dựa trên mô hình luồng dữ liệu. Mỗi tác vụ làm một việc nhỏ, tập trung, rồi truyền đầu ra sang tác vụ kế tiếp qua hàng đợi. Hãy nghĩ đến mô hình ống và bộ lọc trong UNIX. Kiến trúc khá phức tạp. Một tác vụ có thể ghi vào hàng đợi do nhiều tác vụ khác xử lý. Một số tác vụ khác ghi vào hàng đợi chỉ có một tác vụ xử lý.

Hãy nghĩ đến dây chuyền lắp ráp. Mỗi vị trí có một nhiệm vụ duy nhất, đơn giản và tập trung. Sau đó sản phẩm chuyển sang vị trí kế tiếp. Đôi khi dây chuyền tách thành nhiều nhánh, đôi khi các nhánh lại hợp nhất. Đó chính là DLU.

DRU của Mike dùng một cách tiếp cận khác hẳn. Anh ấy tạo một tác vụ cho mỗi thiết bị đầu cuối, rồi trong tác vụ đó thực hiện toàn bộ công việc cho thiết bị ấy. Không có hàng đợi. Không có luồng dữ liệu. Chỉ có nhiều tác vụ lớn giống nhau, mỗi tác vụ quản lý thiết bị đầu cuối riêng.

Cách này đối lập với dây chuyền lắp ráp. Ở đây, hình ảnh thích hợp là nhiều thợ lành nghề, mỗi người tự xây dựng toàn bộ sản phẩm.

Lúc đó tôi nghĩ kiến trúc của mình tốt hơn. Mike dĩ nhiên cho rằng cách của anh ấy tốt hơn. Chúng tôi có nhiều cuộc tranh luận vui vẻ về chuyện này. Cuối cùng, cả hai đều hoạt động khá tốt. Tôi nhận ra rằng kiến trúc phần mềm có thể rất khác nhau nhưng hiệu quả ngang nhau.

VRS

Trong thập niên 1980, nhiều công nghệ mới liên tục xuất hiện. Một trong số đó là điều khiển bằng giọng nói qua máy tính.

Một tính năng của hệ thống 4-Tel là giúp thợ xác định vị trí lỗi trong cáp. Quy trình như sau:

- Nhân viên kiểm tra tại tổng đài trung tâm dùng hệ thống để ước lượng khoảng cách, tính bằng foot, đến vị trí lỗi. Độ chính xác khoảng 20%. Sau đó, nhân viên điều phối thợ sửa cáp đến một điểm tiếp cận gần vị trí đó.
- Khi đến nơi, thợ sửa cáp gọi cho nhân viên kiểm tra và đề nghị bắt đầu quy trình xác định lỗi. Nhân viên kích hoạt chức năng xác định lỗi của 4-Tel. Hệ thống bắt đầu đo đặc tính điện tử của đường dây hỏng và in thông báo trên màn hình, yêu cầu thực hiện một số thao tác như mở cáp hoặc nối tắt cáp.
- Nhân viên cho thợ biết hệ thống yêu cầu thao tác nào, còn thợ thông báo khi hoàn tất. Nhân viên báo lại cho hệ thống rồi hệ thống tiếp tục kiểm tra.
- Sau hai hoặc ba lượt trao đổi, hệ thống tính lại khoảng cách đến lỗi. Thợ sửa cáp lái xe đến vị trí mới rồi bắt đầu lại quy trình.

Hãy tưởng tượng sẽ tiện lợi thế nào nếu thợ sửa cáp đang trên cột điện hoặc đứng bên hộp đấu dây có thể tự điều khiển hệ thống. Đó chính xác là điều công nghệ giọng nói mới cho phép. Thợ sửa cáp có thể gọi trực tiếp vào hệ thống, điều khiển bằng phím bấm và nghe kết quả được đọc lại bằng giọng nói dễ chịu.

Đặt tên

Công ty tổ chức một cuộc thi nhỏ để chọn tên hệ thống mới. Một trong những tên sáng tạo nhất là SAM CARP, viết tắt của "Still Another Manifestation of Capitalist Avarice Repressing the Proletariat" (Một biểu hiện nữa của lòng tham tư bản đàn áp giai cấp vô sản).

Dĩ nhiên tên đó không được chọn.

Một đề xuất khác là Teradyne Interactive Test System, cũng không được chọn.

Một đề xuất nữa là Service Area Test Access Network. Tên này cũng không được chọn.

Cuối cùng, cái tên chiến thắng là VRS, viết tắt của Voice Response System.

Kiến trúc

Tôi không làm việc trên hệ thống này, nhưng đã nghe chuyện xảy ra. Câu chuyện tôi sắp kể là chuyện nghe lại từ người khác, nhưng phần lớn tôi tin là chính xác.

Đó là thời kỳ hứng khởi của máy tính vi mô, hệ điều hành UNIX, C và cơ sở dữ liệu SQL. Chúng tôi quyết tâm dùng tất cả.

Trong số nhiều nhà cung cấp cơ sở dữ liệu, cuối cùng chúng tôi chọn UNIFY. UNIFY chạy trên UNIX nên rất phù hợp với chúng tôi.

UNIFY cũng hỗ trợ công nghệ mới gọi là SQL nhúng. Công nghệ này cho phép nhúng câu lệnh SQL dưới dạng chuỗi trực tiếp vào mã C. Và chúng tôi đã làm vậy, ở khắp nơi.

Thật tuyệt khi có thể đặt SQL trực tiếp vào mã ở bất cứ đâu. Và chúng tôi muốn đặt ở đâu? Ở khắp mọi nơi! Vì vậy, SQL bị rải khắp thân mã.

Dĩ nhiên, thời đó SQL chưa phải tiêu chuẩn vững chắc. Mỗi nhà cung cấp có nhiều đặc điểm riêng. Vì vậy, các câu lệnh SQL riêng và lời gọi API đặc thù của UNIFY cũng bị rải khắp mã.

Cách này hoạt động rất tốt! Hệ thống thành công. Thợ sử dụng nó, công ty điện thoại yêu thích nó. Mọi người đều vui vẻ.

Rồi sản phẩm UNIFY chúng tôi dùng bị ngừng cung cấp.

Ôi. Ôi không.

Vì vậy, chúng tôi quyết định chuyển sang SyBase. Hay là Ingress nhỉ? Tôi không nhớ. Chỉ biết chúng tôi phải tìm trong toàn bộ mã C, tìm mọi câu SQL nhúng và lời gọi API riêng rồi thay bằng thao tác tương ứng của nhà cung cấp mới.

Sau khoảng ba tháng, chúng tôi bỏ cuộc. Không thể làm cho nó hoạt động. Chúng tôi phụ thuộc vào UNIFY chặt đến mức không có hy vọng thực tế nào tái cấu trúc mã với chi phí hợp lý.

Vì vậy, chúng tôi thuê một bên thứ ba bảo trì UNIFY theo hợp đồng. Dĩ nhiên, phí bảo trì tăng năm này qua năm khác.

Kết luận về VRS

Đây là một trong những cách tôi hiểu rằng cơ sở dữ liệu là chi tiết cần được cách ly khỏi mục đích nghiệp vụ tổng thể của hệ thống. Đây cũng là một lý do khiến tôi không thích gắn kết chặt với hệ thống phần mềm bên thứ ba.

Lễ tân điện tử

Năm 1983, công ty chúng tôi đứng ở giao điểm của hệ thống máy tính, viễn thông và thoại. Giám đốc điều hành nghĩ rằng đây có thể là vị trí thuận lợi để phát triển sản phẩm mới. Để thực hiện ý tưởng này, ông lập một nhóm ba người, trong đó có tôi, giao nhiệm vụ lên ý tưởng, thiết kế và triển khai sản phẩm mới cho công ty.

Chúng tôi nhanh chóng nghĩ ra Lễ tân Điện tử (ER).

Ý tưởng rất đơn giản. Khi bạn gọi đến một công ty, ER sẽ trả lời và hỏi bạn muốn nói chuyện với ai. Bạn dùng phím bấm để đánh vần tên người đó, rồi ER kết nối cuộc gọi. Người dùng ER có thể gọi vào hệ thống và dùng các lệnh đơn giản bằng phím bấm để cho biết số điện thoại có thể liên lạc với người cần tìm, ở bất kỳ đâu trên thế giới. Thực tế, hệ thống có thể lưu nhiều số thay thế.

Khi bạn gọi ER và nhập RMART, mã của tôi, ER sẽ gọi số đầu tiên trong danh sách. Nếu tôi không trả lời và xác nhận danh tính, hệ thống sẽ gọi số tiếp theo, rồi số nữa. Nếu vẫn không liên lạc được với tôi, ER sẽ ghi lại lời nhắn của người gọi.

Sau đó, ER định kỳ tìm cách liên lạc với tôi để chuyển lời nhắn đó cùng mọi lời nhắn khác người khác để lại.

Đây là hệ thống thư thoại đầu tiên trên thế giới, và chúng tôi nắm bằng sáng chế về nó.[11]

Chúng tôi tự chế tạo toàn bộ phần cứng cho hệ thống này: bo máy tính, bo bộ nhớ, bo thoại/viễn thông và mọi thứ khác. Bo máy tính chính là Deep Thought, bộ xử lý Intel 80286 mà tôi đã nhắc đến trước đó.

Mỗi bo thoại hỗ trợ một đường dây điện thoại. Bo gồm giao diện điện thoại, bộ mã hóa/giải mã thoại, một ít bộ nhớ và máy tính vi mô Intel 80186.

Phần mềm cho bo máy tính chính được viết bằng C. Hệ điều hành là MP/M-86, một hệ điều hành đa xử lý, chạy trên đĩa và dùng dòng lệnh sơ khai. MP/M là phiên bản UNIX dành cho người ít tiền.

Phần mềm trên các bo thoại được viết bằng hợp dịch và không có hệ điều hành. Deep Thought giao tiếp với các bo thoại qua bộ nhớ dùng chung.

Ngày nay, kiến trúc của hệ thống này được gọi là hướng dịch vụ. Mỗi đường dây điện thoại được giám sát bởi một tiến trình lắng nghe chạy dưới MP/M. Khi có cuộc gọi, hệ thống khởi động tiến trình xử lý ban đầu rồi chuyển cuộc gọi cho nó. Khi cuộc gọi chuyển từ trạng thái này sang trạng thái khác, tiến trình xử lý thích hợp được khởi động và tiếp quản.

---

11. Công ty chúng tôi nắm bằng sáng chế. Hợp đồng lao động quy định mọi phát minh thuộc về công ty. Sếp nói với tôi: “Anh bán nó cho chúng tôi với giá một đô la, mà chúng tôi còn chẳng trả anh đồng đô la đó.”

Thông điệp giữa các dịch vụ được truyền qua tệp trên đĩa. Dịch vụ đang chạy xác định dịch vụ kế tiếp, ghi thông tin trạng thái cần thiết vào tệp, chạy lệnh để khởi động dịch vụ đó rồi thoát.

Đây là lần đầu tôi xây dựng một hệ thống như vậy. Thực tế, đây cũng là lần đầu tôi làm kiến trúc sư chính cho toàn bộ sản phẩm. Mọi việc liên quan đến phần mềm đều do tôi làm, và hệ thống chạy rất tốt.

Tôi không nói kiến trúc này "sạch" theo nghĩa của cuốn sách này. Nó không phải kiến trúc plugin. Tuy nhiên, các dấu hiệu của ranh giới thực sự đã xuất hiện. Các dịch vụ có thể triển khai độc lập và nằm trong phạm vi trách nhiệm riêng. Có tiến trình cấp cao lẫn cấp thấp, và nhiều phụ thuộc đi đúng hướng.

Sự sụp đổ của ER

Tiếc là việc tiếp thị sản phẩm này không thành công. Teradyne là công ty bán thiết bị kiểm thử. Chúng tôi không biết cách thâm nhập thị trường thiết bị văn phòng.

Sau nhiều lần thử trong hai năm, giám đốc điều hành bỏ cuộc và, thật đáng tiếc, rút đơn xin cấp bằng sáng chế. Bằng sáng chế thuộc về công ty đã nộp đơn chỉ ba tháng sau chúng tôi. Vì vậy, chúng tôi nhường toàn bộ thị trường thư thoại và chuyển tiếp cuộc gọi điện tử cho họ.

Đau thật!

Mặt khác, bạn không thể trách tôi vì những chiếc máy phiền phức giờ đang quấy nhiễu cuộc sống của chúng ta.

Hệ thống điều phối thợ

ER thất bại như một sản phẩm, nhưng chúng tôi vẫn còn phần cứng và phần mềm để cải tiến các dòng sản phẩm hiện có. Hơn nữa, thành công tiếp thị của VRS thuyết phục chúng tôi rằng nên cung cấp hệ thống phản hồi bằng giọng nói để giao tiếp với thợ điện thoại, không phụ thuộc vào hệ thống kiểm thử.

Thế là CDS, Hệ thống Điều phối Thợ (Craft Dispatch System), ra đời. Về cơ bản, CDS là ER nhưng tập trung vào một miền rất hẹp: quản lý việc phân công thợ sửa điện thoại ngoài hiện trường.

Khi phát hiện sự cố trên đường dây điện thoại, trung tâm dịch vụ tạo phiếu báo lỗi. Các phiếu được lưu trong một hệ thống tự động. Khi thợ ngoài hiện trường hoàn thành công việc, họ gọi đến trung tâm dịch vụ để nhận nhiệm vụ tiếp theo. Nhân viên trung tâm mở phiếu kế tiếp và đọc nội dung cho thợ.

Chúng tôi tìm cách tự động hóa quy trình đó. Mục tiêu là để thợ ngoài hiện trường gọi vào CDS và yêu cầu nhiệm vụ tiếp theo. CDS sẽ truy vấn hệ thống phiếu báo lỗi và đọc kết quả. CDS theo dõi thợ nào được giao phiếu nào, rồi thông báo trạng thái sửa chữa cho hệ thống phiếu.

Hệ thống này có một số tính năng thú vị liên quan đến việc giao tiếp với hệ thống phiếu báo lỗi, hệ thống quản lý mạng lưới và các hệ thống kiểm thử tự động.

Trải nghiệm với kiến trúc hướng dịch vụ của ER khiến tôi muốn thử ý tưởng này mạnh tay hơn. Máy trạng thái của phiếu báo lỗi phức tạp hơn nhiều so với máy trạng thái xử lý cuộc gọi của ER. Tôi bắt đầu tạo thứ ngày nay được gọi là kiến trúc vi dịch vụ.

Mọi chuyển trạng thái trong bất kỳ cuộc gọi nào, dù nhỏ đến đâu, đều khiến hệ thống khởi động một dịch vụ mới. Thực tế, máy trạng thái được đưa ra ngoài thành một tệp văn bản để hệ thống đọc. Mỗi sự kiện đến từ đường dây điện thoại trở thành một chuyển trạng thái trong máy trạng thái hữu hạn đó. Tiến trình hiện tại khởi động tiến trình mới do máy trạng thái chỉ định để xử lý sự kiện, sau đó tiến trình hiện tại thoát hoặc chờ trong hàng đợi.

Máy trạng thái bên ngoài này cho phép thay đổi luồng ứng dụng mà không cần đổi mã, đúng theo Nguyên tắc Đóng-Mở. Chúng tôi có thể dễ dàng thêm dịch vụ mới, độc lập với mọi dịch vụ khác, rồi nối nó vào luồng bằng cách sửa tệp văn bản chứa máy trạng thái. Thậm chí có thể làm vậy khi hệ thống đang chạy. Nói cách khác, chúng tôi có khả năng hoán đổi nóng và một dạng ngôn ngữ thực thi quy trình nghiệp vụ hiệu quả.

Cách ER dùng tệp đĩa để giao tiếp giữa các dịch vụ quá chậm đối với việc chuyển dịch vụ nhanh hơn nhiều này. Vì vậy, chúng tôi phát minh ra cơ chế bộ nhớ dùng chung gọi là 3DBB.[12] 3DBB cho phép truy cập dữ liệu theo tên. Các tên được dùng là tên gán cho từng thực thể máy trạng thái.

3DBB rất phù hợp để lưu chuỗi và hằng số, nhưng không thể chứa cấu trúc dữ liệu phức tạp. Lý do liên quan đến kỹ thuật nhưng dễ hiểu. Mỗi tiến trình trong MP/M chạy trong phân vùng bộ nhớ riêng. Con trỏ đến dữ liệu trong phân vùng này không có ý nghĩa trong phân vùng khác. Vì vậy, dữ liệu trong 3DBB không thể chứa con trỏ. Chuỗi thì được, nhưng cây, danh sách liên kết hay cấu trúc dữ liệu nào có con trỏ đều không dùng được.

Phiếu báo lỗi đến từ nhiều nguồn. Một số được tạo tự động, số khác do con người nhập. Nhân viên tạo phiếu thủ công khi trao đổi với khách hàng về sự cố. Khi khách hàng mô tả vấn đề, nhân viên nhập lời phàn nàn và quan sát vào một luồng văn bản có cấu trúc. Nội dung trông như sau:

```
/pno 8475551212 /noise /dropped-calls
```

Bạn hiểu ý rồi đấy. Ký tự `/` bắt đầu một chủ đề mới. Sau dấu gạch chéo là mã, tiếp theo là các tham số. Có hàng nghìn mã, và một phiếu báo lỗi có thể chứa hàng chục mã trong phần mô tả. Tệ hơn, vì được nhập thủ công, chúng thường bị viết sai chính tả hoặc định dạng không đúng. Chúng dành cho con người diễn giải, không phải để máy tính xử lý.

Vấn đề của chúng tôi là giải mã các chuỗi nửa tự do này, diễn giải và sửa lỗi, rồi chuyển chúng thành giọng nói để đọc cho thợ đang đứng trên cột điện, nghe bằng ống nghe. Việc đó đòi hỏi, trong số những thứ khác, kỹ thuật phân tích cú pháp và biểu diễn dữ liệu rất linh hoạt. Dữ liệu còn phải truyền qua 3DBB, vốn chỉ xử lý được chuỗi.

Trên máy bay, giữa các chuyến thăm khách hàng, tôi nghĩ ra một lược đồ gọi là FLD, Dữ liệu Gắn nhãn Trường (Field Labeled Data). Ngày nay, ta gọi những thứ tương tự là XML hoặc JSON. Định dạng khác nhau, nhưng ý tưởng giống nhau. FLD là cây nhị phân

12. Bảng đen ba chiều. Nếu sinh vào thập niên 1950, có lẽ bạn hiểu câu đùa này: Drizzle, Drazzle, Druzzle, Drone.

liên kết tên với dữ liệu theo cấu trúc đệ quy. Có thể truy vấn FLD bằng API đơn giản và chuyển đổi qua lại với định dạng chuỗi tiện dụng, phù hợp với 3DBB.

Vậy là năm 1985 chúng tôi đã có vi dịch vụ giao tiếp qua cơ chế giống socket trong bộ nhớ dùng chung, sử dụng thứ tương tự XML.

Dưới ánh mặt trời chẳng có gì mới.

Clear Communications

Năm 1988, một nhóm nhân viên Teradyne rời công ty để lập công ty khởi nghiệp Clear Communications. Vài tháng sau, tôi gia nhập họ. Nhiệm vụ của chúng tôi là viết phần mềm cho hệ thống giám sát chất lượng truyền thông của đường dây T1, các đường dây số dùng để truyền thông tin đường dài khắp đất nước. Tầm nhìn là một màn hình lớn có bản đồ Hoa Kỳ, các đường T1 chạy chằng chịt và nhấp nháy màu đỏ khi chất lượng suy giảm.

Hãy nhớ rằng giao diện đồ họa mới xuất hiện năm 1988. Apple Macintosh mới chỉ năm tuổi. Windows lúc đó là chuyện đùa. Nhưng Sun Microsystems đang sản xuất các máy trạm Sparc có giao diện X Window đáng tin cậy. Vì vậy, chúng tôi chọn Sun, đồng nghĩa với C và UNIX.

Đây là công ty khởi nghiệp. Chúng tôi làm việc 70 đến 80 giờ mỗi tuần. Chúng tôi có tầm nhìn, động lực, ý chí, năng lượng và chuyên môn. Chúng tôi có cổ phần. Chúng tôi mơ làm triệu phú. Chúng tôi đầy ảo tưởng.

Mã C tuôn ra từ mọi lỗ trên người chúng tôi. Chúng tôi nhét nó vào đây, đẩy nó vào kia. Chúng tôi xây những lâu đài trên không khổng lồ. Chúng tôi tạo tiến trình, hàng đợi thông điệp và kiến trúc hoành tráng. Chúng tôi tự viết toàn bộ ngăn xếp truyền thông ISO bảy lớp, đến tận lớp liên kết dữ liệu.

Chúng tôi viết mã giao diện đồ họa. MÃ GOOEY! Trời ơi! Chúng tôi viết mã GOOOOOEY.

Cá nhân tôi viết một hàm C dài 3.000 dòng tên là `gi()`, viết tắt của Graphic Interpreter. Đó là kiệt tác bầy nhầy. Tôi còn viết nhiều đoạn mã bầy nhầy khác ở Clear, nhưng đoạn này tai tiếng nhất.

Kiến trúc ư? Đùa à? Đây là công ty khởi nghiệp. Chúng tôi không có thời gian cho kiến trúc. Chỉ cần viết mã, chết tiệt! Viết mã để sống còn!

Vì vậy, chúng tôi viết mã. Rồi tiếp tục viết mã. Và lại viết mã. Nhưng sau ba năm, điều chúng tôi không làm được là bán hàng. Đúng là có một hoặc hai hệ thống được lắp đặt. Nhưng thị trường không mấy quan tâm đến tầm nhìn hoành tráng của chúng tôi, còn các nhà đầu tư mạo hiểm thì bắt đầu chán nản.

Lúc đó tôi ghét cuộc sống của mình. Tôi thấy mọi công sức và ước mơ sụp đổ. Tôi mâu thuẫn ở nơi làm việc, ở nhà vì công việc và với chính mình.

Rồi tôi nhận được một cuộc điện thoại làm thay đổi mọi thứ.

Chuẩn bị

Hai năm trước cuộc điện thoại đó, có hai việc quan trọng xảy ra.

Thứ nhất, tôi thiết lập được kết nối UUCP đến một công ty gần đó. Công ty đó có kết nối UUCP đến một cơ sở khác nối với Internet. Tất nhiên, đây là những kết nối quay số. Máy Sparcstation chính, đặt trên bàn tôi, dùng modem 1200 bps để gọi máy chủ UUCP hai lần mỗi ngày. Nhờ vậy, chúng tôi có email và Netnews, một mạng xã hội sơ khai nơi mọi người thảo luận về các vấn đề thú vị.

Thứ hai, Sun phát hành trình biên dịch C++. Tôi đã quan tâm đến C++ và hướng đối tượng từ năm 1983, nhưng rất khó kiếm trình biên dịch. Khi có cơ hội, tôi lập tức đổi ngôn ngữ. Tôi bỏ lại các hàm C dài 3.000 dòng và bắt đầu viết C++ ở Clear. Rồi tôi học hỏi…

Tôi đọc sách. Dĩ nhiên tôi đọc *The C++ Programming Language* và *The Annotated C++ Reference Manual* (The ARM) của Bjarne Stroustrup. Tôi đọc cuốn sách tuyệt vời về thiết kế hướng trách nhiệm của Rebecca Wirfs-Brock, *Designing Object Oriented Software*. Tôi đọc *OOA and OOD and OOP* của Peter Coad. Tôi đọc *Smalltalk-80* của Adele Goldberg. Tôi đọc *Advanced C++ Programming Styles and Idioms* của James O. Coplien. Nhưng có lẽ quan trọng nhất là tôi đọc *Object Oriented Design with Applications* của Grady Booch.

Tên hay thật! Grady Booch. Làm sao quên được cái tên đó? Hơn nữa, ông là nhà khoa học trưởng ở một công ty tên Rational! Tôi cũng muốn làm nhà khoa học trưởng! Vì vậy, tôi đọc sách của ông, rồi học hỏi, học hỏi và tiếp tục học…

Trong khi học, tôi cũng bắt đầu tranh luận trên Netnews, giống cách mọi người tranh luận trên Facebook ngày nay. Các cuộc tranh luận của tôi xoay quanh C++ và hướng đối tượng. Suốt hai năm, tôi giải tỏa nỗi bực bội tích tụ ở nơi làm việc bằng cách tranh luận với hàng trăm người trên Usenet về tính năng ngôn ngữ và nguyên tắc thiết kế tốt nhất. Sau một thời gian, tôi bắt đầu nói được vài điều có lý.

Chính trong một cuộc tranh luận như vậy, nền tảng của các nguyên tắc SOLID đã được đặt ra.

Và mọi cuộc tranh luận đó, có lẽ cả một vài điều có lý, đã khiến người ta chú ý đến tôi…

Uncle Bob

Một trong các kỹ sư ở Clear là chàng trai trẻ tên Billy Vogel. Billy đặt biệt danh cho mọi người. Anh gọi tôi là Uncle Bob. Dù tên tôi là Bob, tôi nghi rằng anh ấy đang ám chỉ J. R. “Bob” Dobbs (xem `https://en.wikipedia.org/wiki/File:Bobdobbs.png`).

Lúc đầu tôi mặc kệ. Nhưng qua nhiều tháng, việc anh liên tục nói “Uncle Bob… Uncle Bob” giữa áp lực và thất vọng của công ty khởi nghiệp bắt đầu khiến tôi phát chán.

Rồi một ngày, điện thoại reo.

Cuộc điện thoại

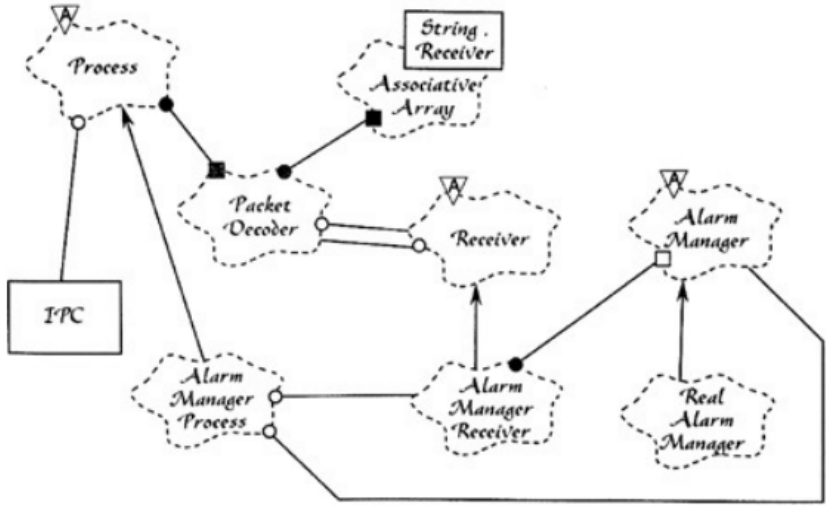
Đó là một nhà tuyển dụng. Anh ta biết tên tôi vì có người nói tôi biết C++ và thiết kế hướng đối tượng. Tôi không chắc bằng cách nào, nhưng nghi rằng điều đó liên quan đến sự hiện diện của tôi trên Netnews.

Anh ta nói có một cơ hội ở Thung lũng Silicon, tại một công ty tên Rational. Họ cần người giúp xây dựng công cụ CASE.[13]

Mặt tôi tái mét. Tôi biết đây là chuyện gì. Tôi không biết bằng cách nào mình biết, nhưng tôi biết. Đây là công ty của Grady Booch. Tôi thấy trước mắt cơ hội được hợp tác với Grady Booch!

ROSE

Năm 1990, tôi gia nhập Rational với tư cách lập trình viên hợp đồng. Tôi làm việc trên sản phẩm ROSE, một công cụ cho phép lập trình viên vẽ sơ đồ Booch, những sơ đồ Grady đã viết về trong *Object-Oriented Analysis and Design with Applications* (Hình A.9 cho thấy một ví dụ).



Hình A.9
Sơ đồ Booch

Ký pháp Booch rất mạnh mẽ. Nó báo trước những ký pháp như UML.

ROSE có một kiến trúc thực sự. Nó được cấu trúc thành các lớp đúng nghĩa, và phụ

13. Computer Aided Software Engineering.

thuộc giữa các lớp được kiểm soát hợp lý. Kiến trúc giúp sản phẩm có thể phát hành, phát triển và triển khai độc lập.

Ồ, nó không hoàn hảo. Có nhiều nguyên tắc kiến trúc mà chúng tôi vẫn chưa hiểu. Chẳng hạn, chúng tôi không tạo được cấu trúc plugin thực sự.

Chúng tôi cũng mắc phải một trong những trào lưu đáng tiếc nhất thời đó, dùng cái gọi là cơ sở dữ liệu hướng đối tượng.

Nhưng nhìn chung, đây là trải nghiệm tuyệt vời. Tôi dành một năm rưỡi làm việc với nhóm Rational trên ROSE. Đây là một trong những trải nghiệm kích thích trí tuệ nhất trong sự nghiệp của tôi.

Các cuộc tranh luận tiếp diễn

Dĩ nhiên, tôi không ngừng tranh luận trên Netnews. Thực tế, tôi tăng đáng kể hoạt động trên mạng. Tôi bắt đầu viết bài cho *C++ Report*. Với sự giúp đỡ của Grady, tôi bắt đầu viết cuốn sách đầu tiên: *Designing Object-Oriented C++ Applications Using the Booch Method*.

Có một điều khiến tôi bận tâm. Nghe kỳ quặc nhưng đúng là vậy: chẳng ai gọi tôi là "Uncle Bob" nữa. Tôi nhận ra mình nhớ cái tên đó. Vì vậy, tôi phạm sai lầm khi đưa "Uncle Bob" vào chữ ký email và Netnews. Cái tên ấy cứ thế gắn với tôi. Cuối cùng, tôi nhận ra đó là một thương hiệu khá tốt.

… Dù mang tên nào khác

ROSE là một ứng dụng C++ khổng lồ. Nó được cấu trúc thành các lớp, với một quy tắc phụ thuộc được thực thi nghiêm ngặt. Đó không phải Quy tắc Phụ thuộc tôi mô tả trong cuốn sách này. Chúng tôi không hướng phụ thuộc về phía chính sách cấp cao. Thay vào đó, chúng tôi hướng phụ thuộc theo hướng luồng điều khiển truyền thống hơn. Giao diện đồ họa phụ thuộc vào phần biểu diễn, phần biểu diễn phụ thuộc vào quy tắc thao tác, rồi quy tắc thao tác phụ thuộc vào cơ sở dữ liệu. Cuối cùng, việc không hướng phụ thuộc về phía chính sách đã góp phần khiến sản phẩm suy tàn.

Kiến trúc ROSE giống kiến trúc của một trình biên dịch tốt. Ký pháp đồ họa được "phân tích cú pháp" thành biểu diễn nội bộ; biểu diễn đó được xử lý bằng các quy tắc rồi lưu vào cơ sở dữ liệu hướng đối tượng.

Cơ sở dữ liệu hướng đối tượng là ý tưởng tương đối mới, khiến giới lập trình hướng đối tượng xôn xao về những hệ quả của nó. Lập trình viên hướng đối tượng nào cũng muốn dùng loại cơ sở dữ liệu này trong hệ thống của mình. Ý tưởng tương đối đơn giản và đầy lý tưởng: cơ sở dữ liệu lưu đối tượng chứ không lưu bảng. Cơ sở dữ liệu được cho là trông như RAM. Khi truy cập một đối tượng, nó tự xuất hiện trong bộ nhớ. Nếu đối tượng trỏ đến một đối tượng khác, đối tượng kia sẽ xuất hiện trong bộ nhớ ngay khi được truy cập. Cứ như phép thuật.

Cơ sở dữ liệu đó có lẽ là sai lầm thực tế lớn nhất của chúng tôi. Chúng tôi muốn có phép thuật, nhưng nhận được một framework bên thứ ba cồng kềnh, chậm, xâm nhập và

đắt đỏ, khiến tiến độ của chúng tôi bị cản trở ở gần như mọi khía cạnh.

Đó không phải sai lầm duy nhất. Thực tế, sai lầm lớn nhất là thiết kế quá mức. Có nhiều lớp hơn những gì tôi mô tả ở đây, và mỗi lớp tạo thêm chi phí giao tiếp riêng. Điều đó làm năng suất của nhóm giảm đáng kể.

Sau nhiều năm công sức, vô số khó khăn và hai lần phát hành nguội nhạt, toàn bộ công cụ bị loại bỏ và thay bằng một ứng dụng nhỏ xinh do một nhóm nhỏ ở Wisconsin viết.

Và tôi học được rằng kiến trúc vĩ đại đôi khi dẫn đến thất bại lớn. Kiến trúc phải đủ linh hoạt để phù hợp với quy mô vấn đề. Thiết kế kiến trúc cấp doanh nghiệp trong khi thực sự chỉ cần một công cụ máy tính để bàn nhỏ xinh là công thức dẫn đến thất bại.

Kỳ thi đăng ký kiến trúc sư

Đầu thập niên 1990, tôi trở thành chuyên gia tư vấn thực thụ. Tôi đi khắp thế giới dạy mọi người về khái niệm hướng đối tượng mới mẻ này. Công việc tư vấn của tôi tập trung mạnh vào thiết kế và kiến trúc của hệ thống hướng đối tượng.

Một trong những khách hàng tư vấn đầu tiên là Educational Testing Service (ETS). Tổ chức này ký hợp đồng với Hội đồng Quốc gia các Ban Đăng ký Kiến trúc sư (NCARB) để tổ chức kỳ thi đăng ký cho ứng viên kiến trúc sư mới.

Ai muốn trở thành kiến trúc sư được đăng ký hành nghề, tức người thiết kế công trình, ở Hoa Kỳ hoặc Canada đều phải vượt qua kỳ thi. Kỳ thi yêu cầu ứng viên giải quyết một số bài toán kiến trúc liên quan đến thiết kế công trình. Ứng viên có thể nhận yêu cầu thiết kế thư viện công cộng, nhà hàng hoặc nhà thờ, rồi được yêu cầu vẽ các sơ đồ kiến trúc thích hợp.

Kết quả được thu thập và lưu lại cho đến khi có thể tập hợp một nhóm kiến trúc sư cấp cao làm giám khảo chấm bài. Các buổi chấm bài này rất lớn, tốn kém và là nguồn gốc của nhiều sự mơ hồ, chậm trễ.

NCARB muốn tự động hóa quy trình: ứng viên làm bài trên máy tính, rồi một máy tính khác đánh giá và chấm điểm. NCARB yêu cầu ETS phát triển phần mềm, còn ETS thuê tôi tập hợp nhóm lập trình viên để xây dựng sản phẩm.

ETS chia bài toán thành 18 tình huống kiểm tra riêng biệt. Mỗi tình huống cần một ứng dụng giao diện đồ họa kiểu CAD để ứng viên trình bày lời giải. Một ứng dụng chấm điểm riêng sẽ nhận lời giải và tạo điểm số.

Cộng sự của tôi, Jim Newkirk, và tôi nhận ra 36 ứng dụng này có rất nhiều điểm chung. 18 ứng dụng giao diện đồ họa dùng các thao tác và cơ chế tương tự. 18 ứng dụng chấm điểm dùng cùng kỹ thuật toán học. Vì có những yếu tố dùng chung này, chúng tôi quyết tâm xây dựng một framework có thể tái sử dụng cho cả 36 ứng dụng. Chúng tôi còn bán ý tưởng này cho ETS bằng cách nói sẽ dành nhiều thời gian cho ứng dụng đầu tiên, nhưng sau đó những ứng dụng còn lại sẽ ra lò vài tuần một lần.

Đến đây có lẽ bạn đang đập tay lên trán hoặc gõ đầu vào cuốn sách. Những ai đủ lớn tuổi có thể nhớ lời hứa "tái sử dụng" của hướng đối tượng. Khi đó, chúng tôi đều tin rằng chỉ cần viết mã C++ hướng đối tượng sạch sẽ thì tự nhiên sẽ tạo ra thật nhiều mã có thể tái sử dụng.

Vì vậy, chúng tôi bắt tay viết ứng dụng đầu tiên, phức tạp nhất trong loạt này. Nó có tên Vignette Grande. Hai chúng tôi làm toàn thời gian trên Vignette Grande, đồng thời hướng đến việc tạo framework có thể tái sử dụng. Việc đó mất một năm. Cuối năm ấy, chúng tôi có 45.000 dòng mã framework và 6.000 dòng mã ứng dụng. Chúng tôi bàn giao sản phẩm cho ETS, rồi họ ký hợp đồng để chúng tôi viết thật nhanh 17 ứng dụng còn lại.

Jim và tôi tuyển thêm ba lập trình viên, rồi bắt đầu làm các tình huống kế tiếp.

Nhưng có chuyện không ổn. Framework có thể tái sử dụng mà chúng tôi tạo ra không thực sự dễ tái sử dụng. Nó không phù hợp với các ứng dụng mới. Có những ma sát tinh vi khiến mọi thứ không hoạt động.

Điều đó khiến chúng tôi nản lòng, nhưng tin rằng biết cách xử lý. Chúng tôi đến ETS và báo rằng dự án sẽ chậm tiến độ, framework 45.000 dòng cần được viết lại hoặc ít nhất điều chỉnh. Chúng tôi nói sẽ mất thêm thời gian.

Không cần nói cũng biết ETS không vui với tin này.

Thế là chúng tôi bắt đầu lại. Chúng tôi gác framework cũ sang một bên và bắt đầu viết đồng thời bốn tình huống mới. Chúng tôi mượn ý tưởng và mã từ framework cũ, nhưng sửa lại để phù hợp với cả bốn ứng dụng mà không cần thay đổi. Nỗ lực này mất thêm một năm. Kết quả là framework 45.000 dòng mới cùng bốn tình huống, mỗi ứng dụng dài khoảng 3.000 đến 6.000 dòng.

Không cần nói cũng biết quan hệ giữa ứng dụng giao diện đồ họa và framework tuân theo Quy tắc Phụ thuộc. Các tình huống là plugin của framework. Mọi chính sách giao diện đồ họa cấp cao đều nằm trong framework. Mã tình huống chỉ là keo kết nối.

Quan hệ giữa ứng dụng chấm điểm và framework phức tạp hơn một chút. Chính sách chấm điểm cấp cao nằm trong tình huống. Framework chấm điểm cắm vào tình huống đó.

Dĩ nhiên, cả hai ứng dụng đều được liên kết tĩnh thành chương trình C++, nên khi đó chúng tôi không hề nghĩ đến khái niệm plugin. Tuy nhiên, phụ thuộc vẫn tuân theo Quy tắc Phụ thuộc.

Sau khi bàn giao bốn ứng dụng đó, chúng tôi bắt đầu làm bốn ứng dụng tiếp theo. Lần này, chúng bắt đầu xuất hiện ở đầu ra cứ vài tuần một lần, đúng như dự đoán. Sự chậm trễ đã khiến lịch trình lùi gần một năm, nên chúng tôi tuyển thêm một lập trình viên để đẩy nhanh tiến độ.

Chúng tôi kịp thời hạn và hoàn thành cam kết. Khách hàng vui. Chúng tôi cũng vui. Mọi việc đều tốt đẹp.

Nhưng chúng tôi đã học được một bài học quan trọng: không thể tạo framework có thể tái sử dụng trước khi tạo framework có thể sử dụng được. Muốn framework có thể tái sử dụng, phải phát triển nó cùng với một số ứng dụng thực sự sử dụng lại nó.

Kết luận

Như tôi đã nói từ đầu, phụ lục này phần nào mang tính tự truyện. Tôi đã nêu những điểm nổi bật trong các dự án mà tôi cho là có ảnh hưởng đến kiến trúc. Dĩ nhiên, tôi cũng kể vài sự việc không hoàn toàn liên quan đến nội dung kỹ thuật của cuốn sách nhưng vẫn đáng kể.

Đây chỉ là một phần lịch sử. Tôi đã tham gia nhiều dự án khác trong suốt những thập kỷ qua. Tôi cũng cố ý dừng câu chuyện ở đầu thập niên 1990 vì còn phải viết một cuốn sách khác về các sự kiện cuối thập niên 1990.

Hy vọng bạn thích chuyến đi ngắn qua ký ức của tôi và học được vài điều trên đường đi.